LAWPIKING

BOUTIQUE DE T-SHIRTS · DAKAR, SÉNÉGAL


```
{
  "status": 200,
  "shop": "LAWPIKING",
  "products": [
    { "id": 5,
      "name": "NOIR INTENSE",
      "price": 10000,
      "stock": 15 }
  ]
}
```


MAÎTRISER LE BACKEND

DE A À Z

Guide complet de Node.js, Express.js, PostgreSQL et Prisma appliqué au projet LAWPIKING

Mohamed Bakhoum

Manuel de formation · Édition d'octobre 2026

Node.js 24 · Express 5 · PostgreSQL 18 · Prisma 7

Sommaire

AVANT-PROPOS — Avant de commencer . . . 9

À qui s'adresse ce guide . . . 9
Comment lire ce livre . . . 9
Les versions utilisées dans ce guide . . . 10
Ce que contient réellement le projet LAWPIKING aujourd'hui . . . 11
La feuille de route du backend . . . 12

PARTIE 1 — Comprendre le backend . . . 14

1.1 Frontend et backend : la différence . . . 14
1.2 Client et serveur . . . 15
1.3 Le protocole HTTP . . . 16
1.4 Anatomie d'une requête et d'une réponse . . . 16
1.5 Les méthodes HTTP . . . 18
1.6 Les codes de statut HTTP . . . 19
1.7 Le format JSON . . . 20
1.8 Qu'est-ce qu'une API REST ? . . . 21
1.9 L'architecture client-serveur de LAWPIKING . . . 22
1.10 Le cycle complet d'une requête . . . 23
1.11 Exercices de la partie 1 . . . 24

PARTIE 2 — Node.js de A à Z . . . 25

2.1 Qu'est-ce que Node.js ? . . . 25
2.2 Pourquoi utiliser Node.js pour LAWPIKING ? . . . 25
2.3 Node.js et JavaScript dans le navigateur . . . 26
2.4 Installer Node.js sur Windows . . . 26
2.5 Le terminal Windows : les commandes dont tu as besoin . . . 27
2.6 Ton premier script Node.js . . . 28
2.7 npm et le fichier package.json . . . 29
2.8 Les modules JavaScript : import et export . . . 31
2.9 Synchrone et asynchrone . . . 32
2.10 Callbacks, Promises et async/await . . . 33
2.11 La boucle d'événements (Event Loop) . . . 35
2.12 Les variables d'environnement . . . 36
2.13 La gestion des erreurs . . . 37
2.14 Créer un serveur HTTP avec Node.js seul . . . 39

2.15 Les scripts npm . . . 40
2.16 Exercices de la partie 2 . . . 42

PARTIE 3 — Express.js . . . 43

3.1 Qu'est-ce qu'Express.js ? . . . 43
3.2 Pourquoi Express avec Node.js ? . . . 43
3.3 Installation et premier serveur . . . 44
3.4 Les routes . . . 45
3.5 Lire la requête : req.params, req.query, req.body . . . 46
3.6 Envoyer la réponse : res . . . 47
3.7 Les middlewares . . . 47
3.8 Organiser les routes avec un Router . . . 48
3.9 Une API REST complète : les produits . . . 49
3.10 Valider les données . . . 51
3.11 La gestion centralisée des erreurs . . . 53
3.12 Filtrer et paginer une liste . . . 55
3.13 Tester l'API . . . 55
3.14 Récapitulatif : les 10 règles d'une bonne API Express . . . 56
3.15 Exercices de la partie 3 . . . 57

PARTIE 4 — PostgreSQL . . . 58

4.1 Qu'est-ce qu'une base de données ? . . . 58
4.2 Pourquoi PostgreSQL ? . . . 58
4.3 Installer PostgreSQL sous Windows . . . 59
4.4 Les outils : psql et pgAdmin . . . 59
4.5 Créer la base et un utilisateur dédié . . . 60
4.6 Les types de données . . . 61
4.7 Créer des tables : CREATE TABLE . . . 61
4.8 Insérer des données : INSERT . . . 63
4.9 Lire des données : SELECT . . . 63
4.10 Modifier et supprimer : UPDATE et DELETE . . . 64
4.11 Les relations . . . 65
4.12 Les jointures : JOIN . . . 66
4.13 Agrégats : compter, additionner, regrouper . . . 67
4.14 Les index . . . 67
4.15 Les transactions . . . 68
4.16 Sauvegarder et restaurer . . . 68
4.17 Les migrations . . . 69

4.18 Récapitulatif . . . 69
4.19 Exercices de la partie 4 . . . 70

PARTIE 5 — Conception de la base LAWPIKING . . . 71

5.1 Partir de ce que le projet fait vraiment . . . 71
5.2 Les principes de conception . . . 71
5.3 Les deux diagrammes de la base . . . 72
5.4 Tailles, couleurs et stock : les variantes . . . 73
5.5 La suppression douce et les produits désactivés . . . 74
5.6 Les commandes figent le prix et le nom . . . 75
5.7 Les rôles : ADMIN et CUSTOMER . . . 75
5.8 Les commandes WhatsApp . . . 75
5.9 Le SQL complet : les 11 tables, étape par étape . . . 75
5.10 Vérifier la cohérence . . . 79
5.11 Ce que le modèle ne prévoit pas (volontairement) . . . 80
5.12 Exercices de la partie 5 . . . 81

PARTIE 6 — Prisma ORM . . . 82

6.1 Qu'est-ce que Prisma ? . . . 82
6.2 Installation . . . 83
6.3 Le fichier schema.prisma . . . 84
6.4 Commandes essentielles de Prisma . . . 89
6.5 Le client Prisma . . . 90
6.6 Peupler la base : le script seed . . . 91
6.7 Les opérations CRUD avec Prisma . . . 92
6.8 Les transactions Prisma . . . 94
6.9 Les erreurs Prisma . . . 95
6.10 Prisma Studio . . . 96
6.11 Étendre le client : $extends . . . 96
6.12 SQL ou Prisma : que choisir ? . . . 97
6.13 Les bonnes pratiques . . . 97
6.14 Exercices de la partie 6 . . . 98

PARTIE 7 — Architecture en couches . . . 99

7.1 Pourquoi organiser le code ? . . . 99
7.2 La structure des dossiers . . . 100
7.3 Le trajet complet d'une requête : créer un produit . . . 101
7.4 Brancher les modules dans app.js . . . 106
7.5 Services entre modules . . . 106

7.6 Les erreurs traversent toutes les couches . . . 106
7.7 Les erreurs d'architecture les plus fréquentes . . . 106
7.8 Quand est-ce trop ? . . . 107
7.9 Exercices de la partie 7 . . . 107

PARTIE 8 — Authentification et sécurité . . . 109

8.1 Authentification et autorisation . . . 109
8.2 Ne jamais stocker un mot de passe en clair . . . 109
8.3 Les jetons JWT . . . 110
8.4 Comparer les approches . . . 111
8.5 Variables d'environnement à ajouter . . . 112
8.6 Créer l'administrateur sans inscription publique . . . 113
8.7 Le module auth . . . 114
8.8 Les middlewares authenticate et requireAdmin . . . 117
8.9 Les protections de base dans app.js . . . 117
8.10 Les injections SQL . . . 118
8.11 Les autres attaques à connaître . . . 119
8.12 Les secrets . . . 119
8.13 Tester l'authentification . . . 119
8.14 Liste de contrôle sécurité . . . 120
8.15 Exercices de la partie 8 . . . 120

PARTIE 9 — Images et fichiers . . . 122

9.1 Le problème à résoudre . . . 122
9.2 Pourquoi ne pas mettre les images dans la base ? . . . 122
9.3 Comment un fichier voyage : multipart/form-data . . . 122
9.4 Installer Multer . . . 123
9.5 Local ou distant ? . . . 123
9.6 Le contrôle des fichiers . . . 124
9.7 Les routes d'images . . . 126
9.8 Côté React : envoyer le fichier . . . 127
9.9 Les bonnes pratiques . . . 128
9.10 Exercices de la partie 9 . . . 128

PARTIE 10 — Connecter React au backend . . . 130

10.1 La bonne nouvelle : le frontend est déjà prêt . . . 130
10.2 Ce qui change selon la page . . . 131
10.3 Configurer l'adresse de l'API . . . 131
10.4 fetch ou Axios ? . . . 132

10.5 Le client API . . . 132
10.6 Le service d'authentification . . . 133
10.7 Les états : chargement, erreur, vide . . . 135
10.8 Avant / après : créer un produit . . . 135
10.9 Synchroniser produits, catégories et stock . . . 137
10.10 Les commandes réelles : enregistrer, puis ouvrir WhatsApp . . . 137
10.11 Les erreurs de connexion les plus fréquentes . . . 138
10.12 Ordre de branchement conseillé . . . 138
10.13 Exercices de la partie 10 . . . 139

PARTIE 11 — Les modules LAWPIKING . . . 141

11.1 Comment lire cette partie . . . 141
11.2 Le module auth . . . 141
11.3 Le module users . . . 142
11.4 Le module categories . . . 143
11.5 Le module products . . . 144
11.6 Le module images . . . 145
11.7 Le module stock et variantes . . . 146
11.8 Le module orders . . . 147
11.9 Le module reviews . . . 150
11.10 Le module promotions . . . 151
11.11 Le module settings . . . 152
11.12 Brancher tous les modules . . . 153
11.13 Exercices de la partie 11 . . . 154

PARTIE 12 — Tests et débogage . . . 155

12.1 Pourquoi tester ? . . . 155
12.2 Tester à la main avec Postman ou Insomnia . . . 155
12.3 Que faut-il tester sur chaque route ? . . . 156
12.4 Les tests unitaires avec le test runner de Node.js . . . 157
12.5 Les tests d'intégration avec Supertest . . . 158
12.6 Déboguer . . . 161
12.7 Les journaux (logs) . . . 162
12.8 Liste de contrôle avant de dire « ça marche » . . . 163
12.9 Exercices de la partie 12 . . . 163

PARTIE 13 — Déploiement . . . 165

13.1 Local et production : deux mondes différents . . . 165
13.2 Les trois éléments à héberger . . . 166

13.3 Préparer le backend . . . 166
13.4 Les variables d'environnement de production . . . 168
13.5 La base PostgreSQL distante . . . 168
13.6 Le frontend en production . . . 169
13.7 Les images en production . . . 170
13.8 Durcir le serveur . . . 170
13.9 Les journaux et la surveillance . . . 171
13.10 Les sauvegardes . . . 171
13.11 Maintenance . . . 171
13.12 Le déploiement pas à pas : résumé . . . 172
13.13 Liste de contrôle avant la mise en ligne . . . 172
13.14 Exercices de la partie 13 . . . 173

PARTIE 14 — Projet pratique . . . 174

14.1 Le principe . . . 174
14.2 Vue d'ensemble . . . 174
14.3 Étape 1 : préparer l'environnement . . . 175
14.4 Étape 2 : le serveur minimal . . . 175
14.5 Étape 3 : erreurs, journal, validation . . . 175
14.6 Étape 4 : la base PostgreSQL . . . 176
14.7 Étape 5 : Prisma . . . 176
14.8 Étape 6 : le seed . . . 176
14.9 Étape 7 : l'authentification . . . 177
14.10 Étape 8 : les catégories (le premier module complet) . . . 177
14.11 Étape 9 : les produits . . . 177
14.12 Étape 10 : images et stock . . . 177
14.13 Étape 11 : commandes, avis, promotions, réglages . . . 178
14.14 Étape 12 : tests, sécurité, déploiement . . . 178
14.15 Récapitulatif des fichiers finaux . . . 178
14.16 Les pièges des débutants sur ce projet . . . 178
14.17 Après le projet : pour aller plus loin . . . 179
14.18 Exercices de la partie 14 . . . 180

ANNEXE A — Aide-mémoire et glossaire . . . 181

Les commandes essentielles . . . 181
psql . . . 181
Prisma . . . 182
Les codes HTTP . . . 182

Les codes d'erreur à connaître 183
Glossaire 183

ANNEXE B — Corrigés des exercices 185

Corrigés de la partie 1 185
Corrigés de la partie 2 186
Corrigés de la partie 3 187
Corrigés de la partie 4 188
Corrigés de la partie 5 189
Corrigés de la partie 6 190
Corrigés de la partie 7 192
Corrigés de la partie 8 193
Corrigés de la partie 9 194
Corrigés de la partie 10 195
Corrigés de la partie 11 197
Corrigés de la partie 12 198
Corrigés de la partie 13 199
Corrigés de la partie 14 200

AVANT-PROPOS

Avant de commencer

Comment utiliser ce guide, ce que contient réellement le projet LAWPIKING, et la feuille de route du backend.

À qui s'adresse ce guide

Ce livre est écrit pour **toi** : un développeur web en formation, qui sait déjà construire une interface avec React, Vite et Tailwind CSS, mais qui n'a jamais construit un backend. Rien n'est supposé acquis côté serveur. Chaque terme technique est expliqué la première fois qu'il apparaît, chaque commande est accompagnée de son utilité et de son résultat attendu, et chaque morceau de code important est commenté ligne par ligne.

Tous les exemples viennent du projet **LAWPIKING**, la boutique de T-shirts dont tu as déjà réalisé le frontend. Quand on parle de produits, ce sont tes 9 T-shirts à 10 000 FCFA. Quand on parle de commandes, c'est le message WhatsApp que ton panier envoie. Quand on parle d'administration, ce sont les pages que tu as déjà construites : Dashboard, Produits, Catégories, Commandes, Stock, Promotions, Avis et Paramètres.

> **À RETENIR — Le but de ce livre**
> À la fin, tu ne dois pas seulement avoir « un backend qui marche » : tu dois pouvoir **expliquer** chaque dossier, chaque requête, chaque table, et **reproduire** chaque opération toi-même, sans copier-coller aveuglément.

Comment lire ce livre

Le livre suit l'ordre naturel d'un apprentissage : d'abord comprendre ce qu'est un serveur (partie 1), puis les outils un par un (Node.js, Express, PostgreSQL, Prisma), puis la façon de les assembler (architecture, sécurité, images), et enfin la connexion avec ton frontend React, les modules LAWPIKING, les tests et le déploiement.

Les encadrés colorés reviennent partout, toujours avec le même sens :

> **DÉFINITION SIMPLE**
> Une **définition simple** : le mot expliqué comme à quelqu'un qui n'a jamais entendu ce terme.

> **EXEMPLE LAWPIKING**
> Un **exemple LAWPIKING** : la notion appliquée à ta boutique, avec tes vrais produits et tes vraies pages.

> **ATTENTION**
> Un **danger** : une erreur de sécurité, une perte de données ou un comportement qui peut te coûter cher.

À RETENIR
Ce qu'il faut **absolument retenir** avant de passer à la suite.

ERREUR FRÉQUENTE
Une **erreur fréquente** des débutants, avec le message que tu verras et sa solution.

BONNE PRATIQUE
Une **bonne pratique** professionnelle, ce que font les équipes expérimentées.

CHANGEMENT DE VERSION
Un **changement important entre versions**, par exemple Prisma 6 contre Prisma 7.

OBLIGATOIRE
Ce qui est **obligatoire** pour que le projet fonctionne.

FACULTATIF
Ce qui est **facultatif** : utile, mais tu peux le faire plus tard.

Chaque grande partie se termine par des **exercices progressifs**. Essaie-les seul avant d'ouvrir les corrigés, qui sont regroupés à la fin du livre (annexe B). Un exercice dont tu as lu la correction sans avoir cherché ne t'apprend presque rien.

Conventions de notation

Notation	Signification
`PS C:\...>`	Une commande à taper dans **PowerShell** sous Windows (le terminal). On ne tape pas le début `PS C:\...>`, il est affiché par le terminal.
Bloc noir	Un échange dans le terminal : commandes (en vert) et résultat attendu (en gris).
Bloc clair avec numéros	Du code à écrire dans un fichier, avec le nom du fichier en en-tête.
`<mot>`	Une valeur à remplacer par la tienne, par exemple `<ton-mot-de-passe>`.
↪	Une ligne de code trop longue, repliée sur la ligne suivante pour tenir dans la page.

Les versions utilisées dans ce guide

Les outils évoluent vite. Les commandes et le code de ce livre ont été écrits pour les versions ci-dessous (vérifiées en octobre 2026). Quand une version change un comportement important, un encadré violet le signale.

Outil	Version retenue	Remarque
Node.js	**24 (LTS)**	Node 22 est encore maintenu ; Node 26 deviendra LTS plus tard. Vérifie toujours avec `node --version`.
npm	fourni avec Node.js	Pas d'installation séparée.
Express.js	**5.x**	Version installée par défaut par `npm install express`. Différente d'Express 4 sur les erreurs asynchrones.
PostgreSQL	**18**	Les exemples SQL fonctionnent aussi en 16 et 17.
pgAdmin	fourni avec l'installeur Windows	Interface graphique pour PostgreSQL.
Prisma ORM	**7.x**	Changements majeurs par rapport à Prisma 6 : voir la partie 6. Prisma 8 existe en version candidate, il n'est pas utilisé ici.
React / Vite	ceux de ton frontend actuel	`package.json` fixe React 19.3.1 et Vite 8.3.5.

ATTENTION — Vérifie toujours les versions
Un tutoriel vieux de deux ans peut contenir des commandes qui ne marchent plus. Quand une commande de ce livre échoue, la première chose à faire est de comparer ta version (`node --version`, `npx prisma --version`) avec le tableau ci-dessus, puis de lire la documentation officielle de l'outil.

Ce que contient réellement le projet LAWPIKING aujourd'hui

Avant d'écrire un backend, il faut savoir **exactement** ce que le frontend fait et ne fait pas. Voici l'état réel du projet, tel qu'il existe dans ton dossier `Lawpikine`. Ce livre n'invente aucune fonctionnalité qui n'existe pas déjà.

La boutique publique

- **9 produits** (tous des T-shirts, `productType: "tshirts"`), définis dans `src/data/products.js`, chacun à **10 000 FCFA**, en tailles S, M, L, XL et XXL. Chaque produit a un `id`, un `slug` (par exemple `tshirt-noir-intense`), un nom, une image, une description et un dégradé de fond Tailwind.
- **Favoris** et **panier** : enregistrés dans le navigateur du visiteur (`localStorage`, clés `lawpikingFavorites` et `lawpikingCart`).
- **Commande par WhatsApp** : la fenêtre `CheckoutModal` collecte nom, téléphone, ville et adresse, puis ouvre un lien `https://wa.me/221776771009?text=…` avec le message pré-rempli (produits, tailles, quantités, total, coordonnées). **Aucune commande n'est enregistrée nulle part** : après l'envoi, le panier est simplement vidé.
- Pages : accueil, fiche produit (`/produit/:slug`), favoris, compte, contact et pied de page.

L'administration

Elle utilise des **services** (dossier `src/services/`) qui simulent une API : chaque fonction est asynchrone et ressemble déjà à un futur appel réseau. Les données sont conservées dans `localStorage`.

Page admin	Service actuel	Données	Équivalent futur (API REST)
Connexion	`authService.js`	Identifiants de démonstration lus dans le code (aucune sécurité réelle)	`POST /api/auth/login`
Produits	`adminProductService.js`	9 produits du catalogue + stock de démo	`GET/POST /api/products`, `PUT/DELETE /api/products/:id`
Stock	`adminProductService.js`	Quantités et seuil d'alerte (10 par défaut)	`PATCH /api/products/:id/stock`
Catégories	`adminCategoryService.js`	T-shirts, Pulls, Bonnets	`GET/POST /api/categories`, …
Commandes	`adminOrderService.js`	**9 commandes de démonstration** (statuts : en cours, livrée, annulée)	`GET /api/orders`, `PATCH /api/orders/:id/status`
Promotions	`adminPromotionService.js`	Aucune au départ ; pourcentage ou montant fixe, dates, produits	`GET/POST /api/promotions`, …
Avis	`adminReviewService.js`	**32 avis de démonstration** (approuvé, en attente, refusé)	`GET /api/reviews`, `PATCH …/status`
Paramètres	`adminSettingsService.js`	Général, réseaux sociaux, commandes, personnalisation	`GET/PUT /api/settings`

ATTENTION — Ce que les données de démonstration ne sont PAS

Les 9 commandes et les 32 avis de l'administration sont des **exemples marqués** `demo: true`. Ils ne viennent d'aucun vrai client. Les vraies commandes WhatsApp, elles, n'existent que dans la messagerie du vendeur. Le backend sera la première fois où ton site pourra **enregistrer** une commande.

Ce que le frontend ne fait pas encore

- Aucun compte client : la page `/compte` est une page d'information.
- La boutique publique lit un catalogue **statique** (`data/products.js`) : un produit ajouté dans l'administration n'apparaît pas encore dans la boutique.
- Les promotions ne modifient aucun prix, et les réglages ne changent rien à l'apparence publique.
- Le numéro WhatsApp est écrit **en dur** dans `CheckoutModal.jsx` et dans le bouton flottant de `App.jsx`.

C'est précisément ce que le backend va unifier : **une seule source de vérité** pour tout le monde.

La feuille de route du backend

Voici le chemin complet, en 14 parties, du premier concept jusqu'à la mise en ligne.

Partie	Sujet	Ce que tu sauras faire à la fin
1	Comprendre le backend	Expliquer HTTP, une API REST et le voyage d'une requête.
2	Node.js de A à Z	Installer Node, écrire un serveur, maîtriser l'asynchrone.
3	Express.js	Construire une API REST avec routes, middlewares et erreurs.
4	PostgreSQL de A à Z	Créer la base LAWPIKING en SQL et l'interroger.
5	Concevoir la base	Modéliser produits, variantes, commandes, avis, promotions.

Partie	Sujet	Ce que tu sauras faire à la fin
6	Prisma ORM	Décrire la base en schéma et la manipuler en JavaScript.
7	Architecture en couches	Organiser le projet en routes, contrôleurs, services, repositories.
8	Authentification et sécurité	Connexion admin, JWT, protection des routes, CORS, Helmet.
9	Images et fichiers	Envoyer une image depuis React et la stocker proprement.
10	Connecter React	Remplacer `localStorage` par l'API, étape par étape.
11	Les modules LAWPIKING	Développer les 10 modules de l'API.
12	Tests et débogage	Tester l'API et comprendre les messages d'erreur.
13	Déploiement	Passer de ton ordinateur à Internet.
14	Projet pratique	Construire le backend complet, étape par étape.

BONNE PRATIQUE — Ordre de lecture conseillé

Lis les parties 1 à 4 **sans te presser** : elles sont le socle. Les parties 5 à 9 se lisent avec ton éditeur de code ouvert, en tapant chaque exemple. Les parties 10 à 14 se font en construisant réellement le projet.

NOTE — Ce livre ne contient pas le backend terminé

Tu n'y trouveras pas un dossier « tout fait » à copier. Les codes montrés sont **les briques** et les exemples complets des opérations importantes. Le projet de la partie 14 te fait assembler le tout, progressivement, pour que tu comprennes ce que tu as construit.

PARTIE 1

Comprendre le backend

Frontend, serveur, HTTP, JSON, API REST : le voyage complet d'une requête, expliqué avec la boutique LAWPIKING.

1.1 Frontend et backend : la différence

DÉFINITION SIMPLE

Le **frontend** est la partie d'une application que l'utilisateur voit et touche, et qui s'exécute dans son navigateur. Le **backend** est la partie cachée, qui s'exécute sur un serveur, et qui détient les données et les règles.

Imagine un restaurant. La **salle** (les tables, la carte, la décoration) est le frontend : c'est ce que le client voit. La **cuisine** (le stock, les recettes, la caisse) est le backend : le client ne la voit jamais, mais sans elle rien ne sort. Le **serveur** qui fait le lien entre la salle et la cuisine, c'est l'API.

	Frontend	Backend
Où s'exécute-t-il ?	Dans le navigateur du visiteur	Sur un serveur, sur Internet
Qui le contrôle ?	**Le visiteur** : il peut lire et modifier le code reçu	**Toi** : le visiteur n'y a pas accès
Langages	HTML, CSS, JavaScript, React	JavaScript (Node.js), SQL
Rôle	Afficher, réagir aux clics, demander des données	Stocker, vérifier, décider, protéger
Exemple LAWPIKING	La grille des 9 T-shirts, le panier, les pages admin	La liste officielle des produits, le stock, les commandes, les mots de passe

EXEMPLE LAWPIKING — Pourquoi ton frontend actuel ne suffit pas

Aujourd'hui, le stock d'un T-shirt est enregistré dans le `localStorage` **de ton navigateur**. Si tu ouvres l'administration sur un autre ordinateur, tu ne vois pas les mêmes quantités. Si un client commande, ton stock ne baisse pas. Et les identifiants de l'admin (`admin@lawpiking.sn`) sont écrits dans le code JavaScript : **n'importe qui peut les lire** dans les outils du navigateur. Un backend règle ces trois problèmes : une donnée unique et partagée, mise à jour par le serveur, protégée derrière une vraie connexion.

À RETENIR

Tout ce qui s'exécute dans le navigateur peut être lu, copié et modifié par l'utilisateur. **Ne mets jamais de secret ni de règle de sécurité dans le frontend.** Le frontend demande, le backend décide.

1.2 Client et serveur

DÉFINITION SIMPLE

Un **client** est un programme qui fait une demande. Un **serveur** est un programme qui attend des demandes et y répond. Ce sont des **rôles**, pas des types de machines.

Quand ton navigateur charge la boutique, il joue le rôle de client : il demande des fichiers (HTML, JavaScript, images) à un serveur. Un serveur est simplement un programme qui tourne en permanence, branché sur un **port** (une « porte » numérotée de l'ordinateur), et qui attend qu'on frappe.

Quelques termes à connaître dès maintenant :

Terme	Explication simple	Exemple LAWPIKING
Hôte (host)	L'adresse de la machine qui héberge le serveur	`localhost` (ton ordinateur) ou `api.lawpiking.sn`
Port	Le numéro de la porte d'entrée du programme	Vite : `5173` ; ton API : `3000` ; PostgreSQL : `5432`
URL	L'adresse complète d'une ressource	`http://localhost:3000/api/products`
Localhost	« Cet ordinateur-ci » (adresse `127.0.0.1`)	Ton backend en développement
Domaine	Le nom lisible qui remplace l'adresse numérique	`lawpiking.sn`

Un site de production est souvent constitué de **plusieurs serveurs** qui discutent entre eux. Le tien en aura trois :

SCHÉMA — ARCHITECTURE CLIENT-SERVEUR DE LAWPIKING



Le navigateur ne parle JAMAIS directement à la base de données : seul le serveur y est autorisé.

Le serveur vérifie, décide, enregistre, puis répond. Le navigateur affiche.

- Le navigateur affiche la boutique et l'administration (React).
- Le serveur d'API (Node.js + Express) reçoit les demandes, applique les règles et répond.
- Le serveur de base de données (PostgreSQL) stocke les données de façon durable.

1.3 Le protocole HTTP

> **DÉFINITION SIMPLE**
> **HTTP** (HyperText Transfer Protocol) est le « langage » commun que parlent les navigateurs et les serveurs. Il définit comment formuler une demande et comment y répondre.

Pour que deux programmes écrits par des personnes différentes se comprennent, ils doivent suivre les mêmes règles de grammaire. HTTP est cette grammaire. Il est **sans mémoire** (on dit *stateless*) : chaque demande est indépendante, et le serveur n'a pas gardé le souvenir de la précédente. C'est pour cela qu'on utilise des **jetons** (partie 8) pour prouver qui l'on est à chaque demande.

HTTP fonctionne en **question-réponse** : le client parle en premier, le serveur répond, et c'est fini. Le serveur ne peut pas « appeler » le navigateur de lui-même (sauf techniques avancées comme WebSocket, qu'on n'utilise pas ici).

> **NOTE — HTTP et HTTPS**
> **HTTPS** est HTTP chiffré. Sur Internet, ton API doit toujours utiliser HTTPS, sinon les mots de passe voyagent en clair. En développement, sur `localhost`, HTTP suffit.

1.4 Anatomie d'une requête et d'une réponse

Une requête HTTP est un simple **texte** envoyé au serveur. Voici celle que ton navigateur enverra pour lire la liste des produits :

Requête HTTP HTTP

```
GET /api/products?category=tshirts HTTP/1.1
Host: localhost:3000
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Et voici la réponse du serveur :

Réponse HTTP HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 142

{"total":1,"products":[{"id":5,"name":"NOIR INTENSE","price":10000}]}
```

Partie	Dans la requête	Dans la réponse
Ligne de départ	Méthode + chemin + version : `GET /api/products?category=tshirts HTTP/1.1`	Version + code de statut : `HTTP/1.1 200 OK`
En-têtes (headers)	Informations sur la demande : `Host`, `Accept`, `Authorization`, `Content-Type`	Informations sur la réponse : `Content-Type`, `Content-Length`, `Set-Cookie`
Ligne vide	Sépare les en-têtes du corps	Idem
Corps (body)	Les données envoyées (surtout pour POST, PUT, PATCH)	Les données renvoyées (le JSON)

Décortiquer une URL

```
http://localhost:3000/api/products/5?category=tshirts&page=2
└──┘   └────────┘└┘ └──────────┘ └──────────────────────┘
protocole   hôte  port  chemin (path)   paramètres de requête (query)
```

Morceau	Rôle	Dans Express (partie 3)
`http`	Le protocole	—
`localhost`	L'hôte	—
`3000`	Le port	`app.listen(3000)`
`/api/products/5`	Le chemin : identifie la ressource	`req.params.id` vaut `5`
`?category=tshirts&page=2`	La *query string* : des filtres	`req.query.category`, `req.query.page`

Votre premier code : appeler une API depuis React

Voici comment ton frontend demandera la liste des produits. Tu connais déjà `fetch` : il envoie une requête HTTP.

exemple-fetch.js JavaScript

```
1 async function loadProducts() {
2   const response = await fetch("http://localhost:3000/api/products");
3   if (!response.ok) {
4     throw new Error("Erreur HTTP " + response.status);
5   }
6   const data = await response.json();
7   console.log(data.total, "produits reçus");
8   return data.products;
9 }
```

Ligne	Explication
1	`async function` déclare une fonction qui peut **attendre** un résultat sans bloquer la page.
2	`fetch(url)` envoie une requête **GET** (méthode par défaut) et renvoie une promesse. `await` attend la réponse du serveur.
3-5	`response.ok` vaut `true` si le code de statut est entre 200 et 299. Sinon on lève une erreur qui sera attrapée plus haut.
6	`response.json()` lit le corps de la réponse et le convertit de texte JSON en objet JavaScript. C'est aussi asynchrone.
7	On affiche dans la console le nombre de produits, par exemple `9 produits reçus`.
8	On renvoie le tableau de produits à celui qui a appelé la fonction.

1.5 Les méthodes HTTP

DÉFINITION SIMPLE

La **méthode** (ou « verbe ») HTTP dit **ce que l'on veut faire** avec la ressource désignée par l'URL.

Méthode	Intention	Corps ?	Exemple LAWPIKING
GET	Lire	Non	`GET /api/products` : liste des produits
POST	Créer	Oui	`POST /api/products` : ajouter un produit
PUT	Remplacer en entier	Oui	`PUT /api/products/5` : renvoyer tous les champs du produit 5
PATCH	Modifier une partie	Oui	`PATCH /api/products/5/stock` : changer seulement le stock
DELETE	Supprimer	Non	`DELETE /api/products/5` : supprimer le produit 5

La différence entre PUT et PATCH est subtile : **PUT** envoie l'objet complet (les champs oubliés seraient effacés ou remis à zéro), **PATCH** envoie seulement ce qui change. Ton frontend utilise déjà cette distinction : `updateAdminProduct` correspond à un PUT, `updateAdminProductStock` à un PATCH.

EXEMPLE LAWPIKING — Les cinq opérations sur un produit, vues de React

Les exemples ci-dessous sont ceux que tu écriras plus tard. Pour l'instant, retiens simplement la forme : la méthode, l'URL, et un corps JSON quand il y a des données à envoyer.

operations-produit.js JavaScript

```javascript
const API = "http://localhost:3000/api";
const json = { "Content-Type": "application/json" };

// GET : lire
await fetch(`${API}/products`);

// POST : créer
await fetch(`${API}/products`, { method: "POST", headers: json,
  body: JSON.stringify({ name: "BORDEAUX", price: 10000, categoryId: "..." }) });

// PUT : remplacer
await fetch(`${API}/products/5`, { method: "PUT", headers: json,
  body: JSON.stringify({ name: "NOIR INTENSE", price: 12000 }) });

// PATCH : modifier une partie
await fetch(`${API}/products/5/stock`, { method: "PATCH", headers: json,
  body: JSON.stringify({ stock: 20 }) });

// DELETE : supprimer
await fetch(`${API}/products/5`, { method: "DELETE" });
```

Ligne	Explication
1	Une constante pour ne pas répéter l'adresse de base de l'API. Elle changera en production.
2	L'en-tête `Content-Type: application/json` prévient le serveur que le corps est du JSON. Indispensable dès qu'on envoie un corps.
5	GET est la méthode par défaut : pas besoin de la préciser.
8-9	`method: "POST"` indique la création. `JSON.stringify` transforme l'objet JavaScript en texte JSON, car le corps d'une requête est du texte.
12-13	PUT vers `/products/5` : le `5` identifie le produit à remplacer.
16-17	PATCH vers une sous-ressource `/stock` : seule la quantité est envoyée.
20	DELETE n'a pas de corps : l'URL suffit à désigner la cible.

ATTENTION — Une méthode GET ne doit jamais modifier de données

GET doit être « sans danger » : on peut l'appeler dix fois sans rien changer. Les moteurs de recherche, les navigateurs et les outils de cache supposent que GET est inoffensif. Une suppression via GET est une faille de conception.

1.6 Les codes de statut HTTP

DÉFINITION SIMPLE

Le **code de statut** est un nombre à trois chiffres, dans la réponse, qui résume ce qui s'est passé. Le premier chiffre donne la famille.

Famille	Signification
2xx	Succès : la demande a été comprise et réussie.
3xx	Redirection : va chercher ailleurs.
4xx	Erreur **du client** : la demande est incorrecte ou interdite. Corrige la demande.
5xx	Erreur **du serveur** : la demande était valable, mais le serveur a échoué. Corrige le serveur.

Les neuf codes que tu utiliseras tous les jours :

Code	Nom	Quand l'utiliser	Exemple LAWPIKING
200	OK	Lecture ou modification réussie	`GET /api/products` renvoie la liste
201	Created	Une ressource vient d'être créée	`POST /api/products` : produit ajouté
400	Bad Request	La demande est mal formée	JSON illisible, identifiant `abc` au lieu d'un nombre
401	Unauthorized	Pas authentifié (ou jeton expiré)	`GET /api/orders` sans connexion admin
403	Forbidden	Authentifié, mais pas autorisé	Un client essaie de supprimer un produit
404	Not Found	La ressource n'existe pas	`GET /api/products/9999`
409	Conflict	Conflit avec l'état existant	Créer un produit dont le slug existe déjà
422	Unprocessable Entity	Demande bien formée, mais données invalides	Prix négatif, stock décimal, note de 7 sur 5
500	Internal Server Error	Bug ou panne côté serveur	La base de données ne répond pas

ERREUR FRÉQUENTE — 401 contre 403

Les débutants confondent souvent les deux. **401** veut dire « je ne sais pas qui tu es » (connecte-toi). **403** veut dire « je sais qui tu es, mais tu n'as pas le droit ». Un administrateur connecté qui touche à une ressource interdite reçoit 403, pas 401.

ERREUR FRÉQUENTE — 400 contre 422

Une règle simple : **400** pour une demande que le serveur ne peut même pas lire (JSON cassé), **422** pour une demande lisible dont le contenu ne respecte pas les règles du métier (prix négatif). Certaines équipes utilisent 400 pour les deux ; l'important est d'être **cohérent** dans tout le projet. Ce livre utilise 400 pour le format, 422 pour la validation.

BONNE PRATIQUE — Ne renvoie jamais 200 avec une erreur dedans

Évite `res.status(200).json({ error: "introuvable" })`. Le frontend (et les outils de supervision) se fient au code pour savoir si la demande a réussi. Renvoie le **bon code** : un 404 pour « introuvable ».

1.7 Le format JSON

DÉFINITION SIMPLE

JSON (JavaScript Object Notation) est un format texte pour représenter des données structurées. C'est la langue commune entre ton frontend et ton backend.

Un fichier JSON ressemble à un objet JavaScript, avec des règles plus strictes : les clés sont entre guillemets doubles, pas de commentaires, pas de virgule finale, pas de `undefined`.

un-produit.json JSON

```json
{
  "id": 5,
  "slug": "tshirt-noir-intense",
  "name": "NOIR INTENSE",
  "price": 10000,
  "sizes": ["S", "M", "L", "XL", "XXL"],
```

```
  "active": true,
  "category": { "id": 1, "name": "T-shirts" },
  "promotion": null
}
```

Ligne	Explication
1 et 10	Les accolades `{ }` délimitent un **objet** : un ensemble de paires clé-valeur.
2-4	Une valeur peut être un nombre (`5`) ou une chaîne de texte (entre guillemets doubles).
5	Un nombre sans guillemets : `10000`. Entre guillemets, ce serait du texte `"10000"`, ce qui poserait problème.
6	Les crochets `[ ]` délimitent un **tableau** (une liste ordonnée).
7	`true` et `false` : des booléens (sans guillemets).
8	Un objet peut contenir un autre objet : `category` a ses propres champs.
9	`null` signifie « aucune valeur » (ici, pas de promotion en cours).

Deux fonctions JavaScript convertissent entre texte JSON et objets :

```
const texte = JSON.stringify({ name: "NOIR INTENSE", price: 10000 });
// texte vaut '{"name":"NOIR INTENSE","price":10000}'  (une chaîne de caractères)

const objet = JSON.parse(texte);
console.log(objet.price); // 10000  (un nombre)
```

`JSON.stringify` **sérialise** (objet vers texte, pour envoyer ou stocker), `JSON.parse` **désérialise** (texte vers objet, pour utiliser). Ton frontend le fait déjà : le `localStorage` ne stocke que du texte, donc `lawpikingCart` est enregistré avec `JSON.stringify`.

1.8 Qu'est-ce qu'une API REST ?

> **DÉFINITION SIMPLE**
> Une **API** (Application Programming Interface) est une porte d'entrée par laquelle un programme parle à un autre. Une **API REST** est une API qui utilise HTTP de façon organisée : chaque **ressource** (produit, commande…) a une adresse, et les méthodes HTTP disent quelle action effectuer.

Dans une API REST, on pense en **ressources** (des noms), pas en actions (des verbes). Au lieu d'inventer des adresses comme `/getAllProducts` ou `/deleteProduct`, on a une adresse par ressource et on choisit l'action avec la méthode.

À éviter	REST propre	Explication
`GET /getAllProducts`	`GET /api/products`	Le verbe est dans la méthode, pas dans l'URL.
`POST /createProduct`	`POST /api/products`	Même URL, méthode différente.
`GET /deleteProduct?id=5`	`DELETE /api/products/5`	Une suppression n'est jamais un GET.
`POST /product/5/update`	`PUT /api/products/5`	Les noms de ressources sont au **pluriel**.

Les règles REST que ce livre applique :

1. **Les URL sont des noms au pluriel** : `/products`, `/categories`, `/orders`.
2. **Un identifiant désigne un élément** : `/products/5`.
3. **La méthode HTTP exprime l'action** : GET, POST, PUT, PATCH, DELETE.
4. **Le format d'échange est JSON** dans les deux sens.
5. **Le code de statut dit le résultat** : 200, 201, 404, etc.
6. **Le serveur ne garde pas de mémoire entre deux requêtes** (stateless).

EXEMPLE LAWPIKING — La carte REST de LAWPIKING

Voici l'API des modules principaux. Tu la reconnais : elle découle directement des fonctions de tes services actuels.

Ressource	Routes	Rôle
Produits	`GET /api/products` · `GET /api/products/:id` · `POST` · `PUT /:id` · `PATCH /:id/stock` · `DELETE /:id`	Catalogue et stock
Catégories	`GET/POST /api/categories` · `PUT/DELETE /api/categories/:id`	Rangement du catalogue
Commandes	`GET /api/orders` · `GET /:id` · `POST` · `PATCH /:id/status`	Commandes WhatsApp et suivi
Avis	`GET /api/reviews` · `POST` · `PATCH /:id/status` · `DELETE /:id`	Modération des avis
Promotions	`GET/POST /api/promotions` · `PUT/DELETE /:id`	Réductions
Authentification	`POST /api/auth/login` · `POST /api/auth/refresh` · `POST /api/auth/logout`	Connexion admin
Paramètres	`GET/PUT /api/settings`	Réglages de la boutique

1.9 L'architecture client-serveur de LAWPIKING

En assemblant ce que nous avons vu, voici l'ensemble des composants du futur LAWPIKING :

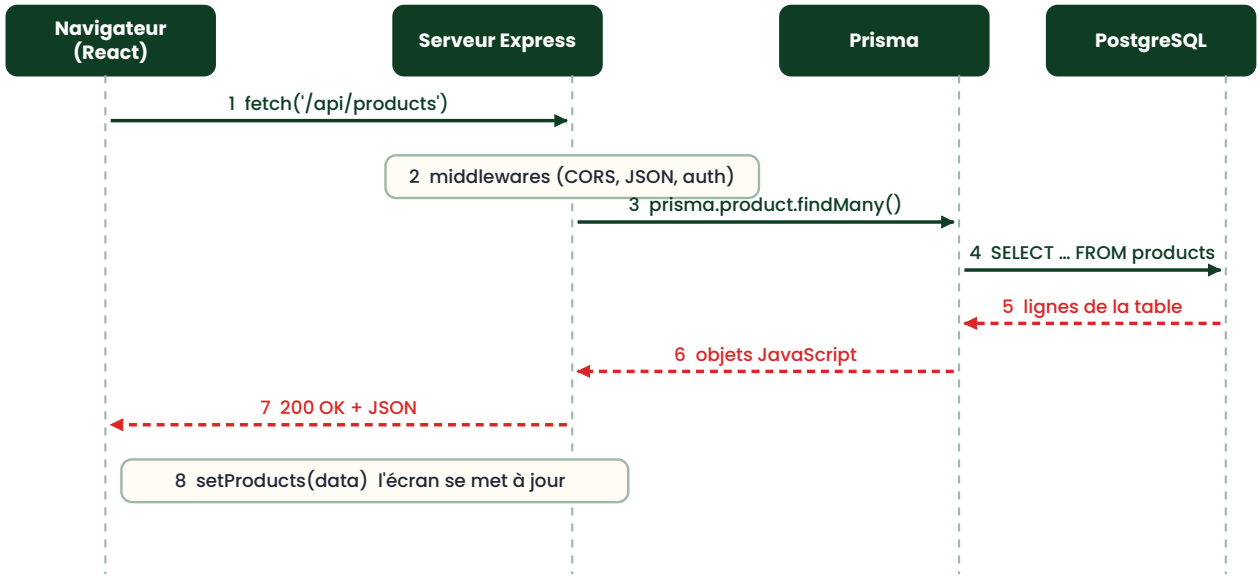
Composant	Technologie	Rôle	Où il s'exécute
Boutique et administration	React + Vite + Tailwind	Interface, formulaires, affichage	Navigateur du visiteur
API	Node.js + Express	Reçoit les requêtes, applique les règles	Serveur d'application
Accès aux données	Prisma ORM	Traduit JavaScript en SQL	Dans le serveur d'application
Base de données	PostgreSQL	Stocke tout de façon durable	Serveur de base de données
Fichiers images	Dossier `uploads/` puis stockage cloud	Conserve les photos produits	Serveur de fichiers
Messagerie	WhatsApp (`wa.me`)	Reçoit les messages de commande	Chez WhatsApp

> **NOTE — WhatsApp reste le canal de commande**
> Le backend **n'enlève pas** la commande par WhatsApp. Il ajoute un enregistrement : quand le client confirme, ton frontend envoie la commande à l'API (qui la sauvegarde et la numérote), puis ouvre WhatsApp comme aujourd'hui. Ainsi tu gardes l'expérience actuelle **et** tu as enfin un historique.

1.10 Le cycle complet d'une requête

Suivons une requête de bout en bout : l'administrateur ouvre la page Produits. Le schéma ci-dessous montre les huit étapes.

SCHÉMA — CYCLE COMPLET D'UNE REQUÊTE HTTP (GET /API/PRODUCTS)



Trait plein : la demande. Trait pointillé rouge : le retour.

1. **L'interface** React appelle `fetch("/api/products")`.
2. Le navigateur résout l'adresse et **envoie** la requête HTTP au serveur.
3. Le serveur Express la **reçoit** et lui fait traverser ses *middlewares* (CORS, lecture du JSON, vérification de la connexion).
4. La **route** `GET /api/products` correspond : elle appelle le contrôleur.
5. Le contrôleur demande les produits à la couche qui accède à la base (Prisma).
6. Prisma envoie un **SQL** à PostgreSQL, qui renvoie des lignes.
7. Le serveur transforme les lignes en JSON, ajoute `200 OK`, et **répond**.
8. React reçoit les données, met à jour son état, et **l'écran se redessine**.

> **À RETENIR**
> Chaque requête est **indépendante** : le serveur oublie tout dès qu'il a répondu. C'est l'application qui doit renvoyer, à chaque fois, tout ce qui est nécessaire (par exemple le jeton qui prouve que l'admin est connecté).

À RETENIR — En résumé — Partie 1

- Le **frontend** affiche et collecte ; le **backend** décide, protège et stocke. Tout ce qui est sensible se vérifie côté serveur.
- Une requête HTTP = **méthode** + **URL** + en-têtes + corps ; la réponse = **code de statut** + en-têtes + corps JSON.
- `GET` lit, `POST` crée, `PUT`/`PATCH` modifient, `DELETE` supprime. Les codes à connaître : 200, 201, 400, 401, 403, 404, 409, 422, 500.
- Une API **REST** expose des ressources (`/api/products`) manipulées par les méthodes HTTP.
- 401 = « qui es-tu ? », 403 = « tu n'as pas le droit », 400 = requête illisible, 422 = données invalides.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Expliquer la différence entre frontend et backend avec un exemple LAWPIKING	Frontend et backend
Lire une requête et une réponse HTTP	Anatomie d'une requête
Choisir la bonne méthode HTTP pour une action	Les méthodes HTTP
Associer un code de statut à une situation	Les codes de statut
Décrire une route REST de l'API	REST

1.11 Exercices de la partie 1

EXEMPLE LAWPIKING — Essaie d'abord seul, les corrigés sont en annexe B

QUESTIONS DE COMPRÉHENSION

1. Explique en deux phrases pourquoi les identifiants de l'admin ne doivent pas être vérifiés dans le frontend.
2. Quelle méthode HTTP utiliserais-tu pour : (a) afficher la liste des avis, (b) passer une commande à « livrée », (c) supprimer une promotion, (d) créer une catégorie « Casquettes » ?
3. Quel code de statut pour : (a) un produit créé, (b) un prix de `-500`, (c) une commande #LWP999 qui n'existe pas, (d) un visiteur non connecté qui demande `GET /api/orders`, (e) un slug déjà utilisé ?
4. Donne la différence entre `req.params` et `req.query` à partir de l'URL `/api/products/5?category=tshirts`.

EXERCICE DE LECTURE DE CODE

5. Dans `loadProducts` (voir plus haut), que se passe-t-il si le serveur répond `404` ? Que se passe-t-il si le serveur est éteint ? Les deux cas produisent-ils la même erreur ?

EXERCICE D'ÉCRITURE

6. Écris le JSON qui représente la catégorie « Bonnets » de LAWPIKING (id 3, valeur `bonnets`, description « Les bonnets de la collection LAWPIKING. », image vide, activée), puis écris l'appel `fetch` qui l'enverrait en `POST`.

MINI-PROJET

7. Dessine sur papier le voyage de la requête `PATCH /api/products/5/stock` avec le corps `{ "stock": 20 }` : qui l'envoie, quels sont les composants traversés, quelle réponse revient, avec quel code.

PARTIE 2

Node.js de A à Z

Installer Node.js, comprendre npm, les modules, l'asynchrone et écrire ton premier serveur HTTP.

2.1 Qu'est-ce que Node.js ?

> **DÉFINITION SIMPLE**
> **Node.js** est un programme qui permet d'exécuter du JavaScript **en dehors du navigateur**, directement sur un ordinateur ou un serveur.

Pendant longtemps, le JavaScript ne pouvait vivre que dans une page web : c'est le navigateur qui le lisait. En 2009, des ingénieurs ont pris le moteur JavaScript de Google Chrome, appelé **V8**, et l'ont emballé dans un programme autonome qui peut lire des fichiers, ouvrir des connexions réseau et parler à des bases de données. Ce programme, c'est Node.js.

Node.js n'est **ni un langage, ni un framework**. Le langage reste JavaScript. Node.js est l'**environnement d'exécution** (*runtime*) : ce qui lit ton code JavaScript et lui donne accès aux ressources de la machine.

Notion	Rôle	Exemple
JavaScript	Le langage, avec sa grammaire	`const prix = 10000;`
V8	Le moteur qui comprend et exécute le JavaScript (inventé pour Chrome)	Compile le code en langage machine
Node.js	V8 + les outils pour parler au système : fichiers, réseau, processus	`import fs from "node:fs"`
npm	Le gestionnaire de paquets livré avec Node.js	`npm install express`

2.2 Pourquoi utiliser Node.js pour LAWPIKING ?

- **Un seul langage partout.** Ton frontend est en JavaScript. Tu n'as pas besoin d'apprendre PHP, Java ou Python pour écrire le serveur : la syntaxe, `async/await`, les objets, tout ce que tu connais s'applique.
- **Un écosystème immense.** npm contient des centaines de milliers de bibliothèques : Express, Prisma, bcrypt, Multer… prêtes à l'emploi.
- **Idéal pour une API.** Node.js excelle quand il y a beaucoup d'attentes (base de données, réseau). Une boutique comme LAWPIKING passe 95 % de son temps à attendre PostgreSQL : c'est exactement son point fort.
- **Des types et des validations partagés.** Les règles de ton frontend (prix entier supérieur à 0, nom de 80 caractères maximum) peuvent être reprises quasiment telles quelles côté serveur.

EXEMPLE LAWPIKING — Tes règles de validation existent déjà
Dans `src/services/productValidation.js`, ton frontend vérifie : nom obligatoire (80 caractères maximum), prix entier supérieur à 0 (10 000 000 maximum), stock entier supérieur ou égal à 0 (100 000 maximum). Le backend refera **les mêmes contrôles**, car on ne doit jamais faire confiance à ce qui arrive du navigateur. Tu pourras copier la logique presque à l'identique.

2.3 Node.js et JavaScript dans le navigateur

C'est le même langage, mais pas les mêmes « outils autour ». Dans le navigateur tu as `window`, `document`, le DOM, `localStorage`. Dans Node.js, rien de tout cela n'existe : à la place tu as `process`, le système de fichiers et le réseau.

	Navigateur (React)	Node.js (backend)
Objet global	`window`	`globalThis`, `process`
Interagir avec la page	`document.getElementById(…)`	Impossible : il n'y a pas de page
Stockage	`localStorage`, `sessionStorage`	Fichiers (`fs`), base de données
Réseau	`fetch` (envoyer une requête)	`fetch` aussi, **et** créer un serveur (`http`)
Modules	`<script type="module">`, Vite	`import` / `export` (ES Modules) ou `require`
Sécurité	Bac à sable : accès limité	Accès **complet** au système : à manier avec soin

ATTENTION — Node.js a tous les droits
Un programme Node.js peut lire et effacer n'importe quel fichier de ta machine. N'exécute jamais un script d'origine inconnue, et lis ce que tu installes avec npm.

2.4 Installer Node.js sur Windows

OBLIGATOIRE
Sans Node.js, aucune commande des parties suivantes ne fonctionnera. C'est la toute première étape.

Méthode 1 : l'installeur officiel (recommandée au début).

1. Va sur le site officiel **nodejs.org**.
2. Télécharge la version marquée **LTS** (*Long Term Support* : support à long terme). C'est la version stable pour les projets réels. La version « Current » contient des nouveautés mais change plus vite.
3. Lance le fichier `.msi`, clique sur *Next* et accepte les options par défaut. **Laisse cochée l'option d'ajout de Node.js au PATH** : c'est elle qui permet de taper `node` dans n'importe quel terminal.
4. Ferme tous les terminaux ouverts, puis rouvre-en un nouveau (le PATH n'est lu qu'au démarrage d'un terminal).

Méthode 2 : en ligne de commande avec winget (le gestionnaire de paquets de Windows) :

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> winget install OpenJS.NodeJS.LTS
Found Node.js [OpenJS.NodeJS.LTS] Version 24.22.1
Successfully installed
```

Vérifier l'installation

Ouvre **PowerShell** (touche Windows, tape « PowerShell », Entrée) puis tape :

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> node --version
v24.22.1
PS C:\Users\mohamed> npm --version
11.6.2
```

Commande	Utilité	Résultat attendu
`node --version`	Affiche la version de Node.js installée	`v24.` suivi de numéros (ton numéro exact peut être plus récent)
`npm --version`	Affiche la version de npm	Un numéro de version, par exemple `11.x.x`
`node -p "process.release.lts"`	Indique le nom de la ligne LTS	Un nom de code (par exemple `Krypton`) : c'est la preuve que tu as une version LTS

ERREUR FRÉQUENTE — « npm.ps1 ne peut pas être chargé » (script désactivé)

Sur certaines installations de Windows, `npm install` affiche une erreur rouge contenant « l'exécution de scripts est désactivée sur ce système ». PowerShell bloque par défaut les scripts. Solution : une seule fois, tape `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`, réponds `O` (oui), puis relance ta commande. Cette règle ne s'applique qu'à **ton** compte utilisateur.

ERREUR FRÉQUENTE — « 'node' n'est pas reconnu en tant que commande »

Le terminal ne trouve pas Node.js. Causes fréquentes : (1) le terminal était déjà ouvert pendant l'installation, ferme-le et rouvre-le ; (2) l'option « ajouter au PATH » n'a pas été cochée, relance l'installeur ; (3) tu as installé Node.js dans un autre utilisateur Windows.

2.5 Le terminal Windows : les commandes dont tu as besoin

Le terminal est une fenêtre où l'on écrit des commandes au lieu de cliquer. Dans PowerShell, voici l'essentiel :

Commande	Utilité	Exemple
`pwd`	Affiche le dossier actuel	`pwd`
`ls`	Liste le contenu du dossier	`ls`
`cd <dossier>`	Entre dans un dossier	`cd Desktop`
`cd ..`	Remonte d'un dossier	`cd ..`
`mkdir <nom>`	Crée un dossier	`mkdir lawpiking-backend`
`code .`	Ouvre le dossier actuel dans VS Code	`code .`
`Ctrl + C`	**Arrête** le programme en cours (par exemple un serveur)	—
Flèche haut	Rappelle la commande précédente	—

Crée maintenant le dossier de travail du backend. Place-le **à côté** de ton frontend, pas dedans :

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> cd Desktop
PS C:\Users\mohamed\Desktop> mkdir lawpiking-backend

    Directory: C:\Users\mohamed\Desktop

Mode    LastWriteTime     Length Name
----    -------------     ------ ----
d-----  31/10/2026 10:12         lawpiking-backend

PS C:\Users\mohamed\Desktop> cd lawpiking-backend
PS ...\lawpiking-backend> code .
```

BONNE PRATIQUE — Frontend et backend : deux projets séparés
Garde deux dossiers distincts : `Lawpikine` (React) et `lawpiking-backend` (Node.js). Chacun a son propre `package.json`, ses propres dépendances et son propre serveur. Ils ne communiquent que par HTTP, exactement comme ils le feront en production.

2.6 Ton premier script Node.js

Dans VS Code, crée un fichier `hello.js` à la racine de `lawpiking-backend` :

hello.js JavaScript

```javascript
const shop = "LAWPIKING";
const products = [
  { name: "NOIR INTENSE", price: 10000 },
  { name: "ROUGE CARDINAL", price: 10000 },
];

const total = products.reduce((sum, product) => sum + product.price, 0);
console.log(`Boutique ${shop} : ${products.length} produits`);
console.log(`Valeur du catalogue : ${total} FCFA`);
```

Exécute-le avec Node.js :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> node hello.js
Boutique LAWPIKING : 2 produits
Valeur du catalogue : 20000 FCFA
```

Ligne	Explication
1	Une constante texte. Rien de nouveau : c'est du JavaScript.
2-5	Un tableau de deux objets représentant des produits.
7	`reduce` additionne les prix : `sum` est le cumul, qui part de `0`.
8-9	`console.log` écrit dans le **terminal** (et non dans la console du navigateur). Les marqueurs `${…}` insèrent des valeurs dans le texte.

La commande `node hello.js` lit le fichier, l'exécute de haut en bas, puis s'arrête. Un serveur, lui, ne s'arrête pas : il reste à l'écoute (nous le verrons plus loin).

2.7 npm et le fichier package.json

DÉFINITION SIMPLE

npm (Node Package Manager) est l'outil qui télécharge et gère les **paquets** : des bibliothèques de code écrites par d'autres. **package.json** est la fiche d'identité de ton projet : son nom, ses scripts et la liste de ce dont il dépend.

Dans `lawpiking-backend`, initialise le projet :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm init -y
Wrote to C:\Users\mohamed\Desktop\lawpiking-backend\package.json:
```

`npm init -y` crée un `package.json` avec des valeurs par défaut (le `-y` répond « oui » à toutes les questions). Ouvre-le et remplace son contenu par :

package.json JSON

```json
{
  "name": "lawpiking-backend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "node --watch src/server.js",
    "start": "node src/server.js"
  }
}
```

Champ	Rôle	Obligatoire ?
`name`, `version`	Identité du projet	Oui (npm les attend)
`private: true`	Empêche de publier le projet par erreur sur npm	Recommandé
`type: "module"`	Indique que les fichiers `.js` utilisent `import`/`export` (ES Modules)	**Oui** dans ce guide
`scripts`	Raccourcis de commandes lancés par `npm run <nom>`	Oui, pratique
`dependencies`	Paquets nécessaires **en production**	Ajouté automatiquement
`devDependencies`	Paquets utiles **seulement en développement**	Ajouté automatiquement

Installer un paquet

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install express
added 66 packages, and audited 67 packages in 3s
found 0 vulnerabilities
```

Cette commande fait trois choses : elle télécharge Express **et toutes ses dépendances** dans un dossier `node_modules/`, elle ajoute Express dans `dependencies` du `package.json`, et elle crée ou met à jour `package-lock.json`.

Élément	Contenu	À versionner (Git) ?
`package.json`	La liste de tes dépendances, avec des versions « souples »	**Oui**
`package-lock.json`	La liste **exacte** de tout ce qui est installé (versions précises)	**Oui** : il garantit que ton collègue installe les mêmes versions que toi
`node_modules/`	Les paquets téléchargés (parfois des dizaines de milliers de fichiers)	**Non** : on le recrée avec `npm install`

Pour un outil qui ne sert qu'en développement (par exemple un outil de test), ajoute `-D` :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install --save-dev vitest
```

Lire un numéro de version (semver)

Une version s'écrit `MAJEUR.MINEUR.CORRECTIF`, par exemple `5.1.0`. Dans `package.json`, tu verras `"express": "^5.1.0"`.

Écriture	Signification	Versions acceptées
`5.1.0`	Exactement cette version	5.1.0 uniquement
`^5.1.0`	Compatible : le **majeur** ne change pas	5.1.0 à 5.x.x, mais pas 6.0.0
`~5.1.0`	Seulement les corrections	5.1.0 à 5.1.x

Un changement de **majeur** peut casser ton code (Express 4 vers 5, Prisma 6 vers 7). Un changement de **mineur** ajoute des fonctions, un changement de **correctif** répare des bugs.

Les commandes npm à connaître

Commande	Utilité
`npm install`	Installe tout ce qui est listé dans `package.json` (à faire après avoir cloné un projet)
`npm ci`	Installation **stricte** à partir du `package-lock.json` : utilisée en déploiement et en test automatisé
`npm install <paquet>`	Ajoute un paquet aux dépendances
`npm install -D <paquet>`	Ajoute un paquet aux dépendances de développement
`npm uninstall <paquet>`	Retire un paquet
`npm run <script>`	Lance un script de `package.json` (`npm run dev`)
`npm start`	Raccourci pour `npm run start`
`npm outdated`	Liste les paquets qui ont une version plus récente
`npx <outil>`	Exécute un outil installé dans le projet **sans l'installer globalement** (par exemple `npx prisma`)

ERREUR FRÉQUENTE — Ne jamais envoyer node_modules sur GitHub
Crée un fichier `.gitignore` contenant au minimum `node_modules/`, `.env` et `uploads/`. Sans cela, tu publierais des milliers de fichiers inutiles, et surtout **tes secrets**.

2.8 Les modules JavaScript : import et export

DÉFINITION SIMPLE
Un **module** est un fichier JavaScript qui peut exposer (*exporter*) certaines de ses fonctions et en utiliser (*importer*) d'autres fichiers. C'est ce qui permet de découper un gros projet en petits fichiers lisibles.

Il existe deux systèmes, hérités de l'histoire de Node.js :

	CommonJS (ancien)	ES Modules (moderne, utilisé ici)
Importer	`const express = require("express");`	`import express from "express";`
Exporter	`module.exports = { add };`	`export { add };` ou `export default add;`
Activation	Par défaut dans un projet sans `"type"`	`"type": "module"` dans `package.json`
Comme dans React ?	Non	**Oui** : c'est la syntaxe que tu utilises déjà dans Vite

À RETENIR
Ce guide utilise exclusivement les **ES Modules** : `import` / `export`, comme dans ton frontend. Tu retrouveras `require` dans de très nombreux tutoriels anciens : c'est le même rôle avec une autre écriture.

src/utils/price.js JavaScript

```
1 export function formatPrice(amount) {
2   return `${new Intl.NumberFormat("fr-FR").format(amount)} FCFA`;
3 }
4
```

```
export const CURRENCY = "FCFA";

export default function discounted(price, percent) {
  return Math.round(price * (1 - percent / 100));
}
```

src/test-modules.js JavaScript

```
import discounted, { formatPrice, CURRENCY } from "./utils/price.js";

console.log(formatPrice(discounted(10000, 20)), CURRENCY);
```

Élément	Explication
`export function formatPrice`	Un **export nommé** : on l'importe avec des accolades `{ formatPrice }` et son nom exact. Un fichier peut en avoir plusieurs.
`export default function discounted`	Un **export par défaut** : un seul par fichier. On l'importe **sans accolades** et on peut le renommer.
`from "./utils/price.js"`	Chemin relatif au fichier courant. **Dans Node.js, l'extension `.js` est obligatoire** pour les fichiers locaux (contrairement à Vite).
Résultat	`8 000 FCFA FCFA` (affichage de `8 000 FCFA` puis de la constante `FCFA`).

ERREUR FRÉQUENTE — « Cannot use import statement outside a module »
Tu as écrit `import` dans un projet qui n'est pas en mode module. Ajoute `"type": "module"` dans `package.json`. À l'inverse, « require is not defined in ES module scope » signifie que tu as écrit `require` dans un projet en mode module : remplace-le par `import`.

ERREUR FRÉQUENTE — « Cannot find module … imported from … »
Tu as oublié l'extension `.js` dans un import relatif (`./utils/price` au lieu de `./utils/price.js`), ou le chemin est faux. Node.js ne la devine pas.

BONNE PRATIQUE — Les modules intégrés : le préfixe node:
Node.js fournit des modules prêts à l'emploi (`fs`, `path`, `http`, `crypto`…). On les importe avec le préfixe `node:`, par exemple `import { readFile } from "node:fs/promises";`. Le préfixe indique clairement « ceci vient de Node.js, pas de npm ».

2.9 Synchrone et asynchrone

DÉFINITION SIMPLE
Un code **synchrone** exécute une instruction après l'autre et **attend** que chacune soit finie. Un code **asynchrone** lance une opération longue (lire un fichier, interroger la base) et **continue**, puis reprend quand le résultat est prêt.

Dans un restaurant : un cuisinier **synchrone** prend une commande, la prépare entièrement en regardant le four, et seulement ensuite prend la commande suivante. Un cuisinier **asynchrone** lance le plat au four, puis prend la commande suivante pendant que le plat cuit, et revient quand le four sonne.

Pour un serveur, c'est vital. Imagine 50 clients qui chargent la boutique en même temps : si chaque requête bloquait Node.js pendant 100 ms à attendre PostgreSQL, le dernier client attendrait 5 secondes.

sync-vs-async.js JavaScript

```
import { readFileSync } from "node:fs";
import { readFile } from "node:fs/promises";

console.log("1. début");

const texte = readFileSync("hello.js", "utf8");   // synchrone : bloque
console.log("2. lu en synchrone :", texte.length, "caractères");

readFile("hello.js", "utf8").then((contenu) => {   // asynchrone : ne bloque pas
  console.log("4. lu en asynchrone :", contenu.length, "caractères");
});

console.log("3. fin du script");
```

RÉSULTAT résultat attendu

```
1. début
2. lu en synchrone : 323 caractères
3. fin du script
4. lu en asynchrone : 323 caractères
```

Observe l'ordre : « 3 » s'affiche **avant** « 4 ». La lecture asynchrone est lancée, Node.js passe à la suite, puis revient exécuter le `.then(...)` quand le fichier est lu. (La longueur exacte dépend de ton fichier.)

ATTENTION — N'utilise jamais les versions « Sync » dans un serveur
Les fonctions qui finissent par `Sync` (`readFileSync`…) **figent tout le serveur** pendant leur exécution. Elles sont acceptables dans un petit script de démarrage, jamais dans le traitement d'une requête.

2.10 Callbacks, Promises et async/await

Il y a trois générations de syntaxe pour gérer l'asynchrone. Tu dois les connaître toutes les trois, car tu les croiseras dans les bibliothèques et les tutoriels.

1. Les callbacks (la méthode historique)

DÉFINITION SIMPLE
Un **callback** est une fonction qu'on passe en argument, pour qu'elle soit appelée plus tard, quand l'opération est terminée.

callbacks.js JavaScript

```
import { readFile } from "node:fs";

readFile("hello.js", "utf8", (error, contenu) => {
  if (error) {
    console.error("Lecture impossible :", error.message);
    return;
  }
  console.log("Contenu lu :", contenu.length, "caractères");
});
```

Le callback reçoit toujours **l'erreur en premier** (`error`), puis le résultat (`contenu`). C'est une convention de Node.js. Le problème : quand on enchaîne plusieurs opérations (lire un fichier, puis interroger la base, puis écrire un autre fichier), les callbacks s'imbriquent les uns dans les autres, ce qu'on appelle le **callback hell** (l'enfer des callbacks).

2. Les Promises

> **DÉFINITION SIMPLE**
> Une **Promise** est un objet qui représente un résultat qui n'est **pas encore là** : il sera soit réussi (*resolved*), soit en échec (*rejected*).

promises.js JavaScript

```
import { readFile } from "node:fs/promises";

readFile("hello.js", "utf8")
  .then((contenu) => {
    console.log("Lu :", contenu.length, "caractères");
    return contenu.toUpperCase();
  })
  .then((majuscules) => console.log(majuscules.slice(0, 20)))
  .catch((error) => console.error("Échec :", error.message))
  .finally(() => console.log("Terminé"));
```

`.then()` s'exécute en cas de succès, `.catch()` en cas d'erreur (n'importe où dans la chaîne), `.finally()` dans tous les cas. Les `.then()` s'enchaînent à plat, sans imbrication.

3. async / await (la méthode moderne)

> **DÉFINITION SIMPLE**
> `async` marque une fonction qui renvoie toujours une Promise. `await` met la fonction en **pause** jusqu'à ce que la Promise soit terminée, puis renvoie son résultat, comme si le code était synchrone.

async-await.js JavaScript

```
import { readFile } from "node:fs/promises";

async function lireCatalogue() {
  try {
    const contenu = await readFile("hello.js", "utf8");
    console.log("Lu :", contenu.length, "caractères");
    return contenu;
  } catch (error) {
    console.error("Échec :", error.message);
    return null;
  } finally {
    console.log("Terminé");
  }
}

const resultat = await lireCatalogue();
console.log(resultat === null ? "échec" : "succès");
```

Ligne	Explication
3	`async function` : la fonction peut utiliser `await` et renvoie une Promise.
4	`try { … }` : on **essaie** le code ci-dedans.
5	`await readFile(...)` : on attend le fichier. La ligne suivante ne s'exécute qu'après. C'est un code qui se lit de haut en bas.
8	`catch (error)` : si une opération échoue (fichier introuvable…), on arrive ici avec l'erreur. Le programme **ne plante pas**.
11	`finally` : exécuté dans tous les cas (nettoyage, fermeture de connexion).
16	`await` au niveau du fichier (*top-level await*) est autorisé dans les ES Modules.

	Callback	Promise (`.then`)	async / await
Lisibilité	Mauvaise dès qu'on enchaîne	Bonne	**Excellente**
Gestion d'erreur	Paramètre `error` à tester partout	`.catch()`	`try / catch`
Usage dans ce guide	Compréhension seulement	Compréhension	**Partout**

ERREUR FRÉQUENTE — Oublier le await
Si tu écris `const produits = prisma.product.findMany();` sans `await`, `produits` n'est **pas** la liste mais une Promise : tu verras `Promise { <pending> }` dans le terminal. Le réflexe : dès qu'une fonction touche à la base, au disque ou au réseau, c'est probablement `await`.

ERREUR FRÉQUENTE — await dans une fonction non async
« await is only valid in async functions and the top level bodies of modules ». Ajoute `async` devant la fonction qui contient le `await`.

Faire plusieurs choses en même temps

Si deux opérations sont indépendantes, ne les attends pas l'une après l'autre : lance-les ensemble avec `Promise.all`.

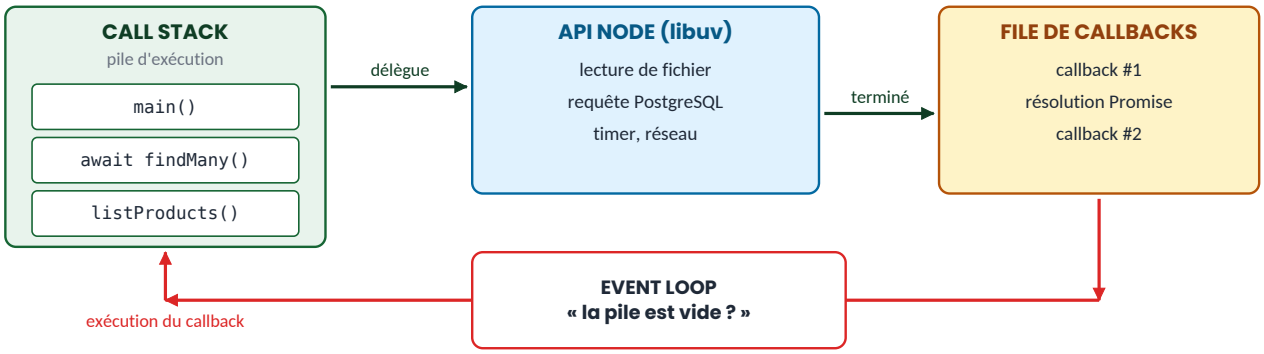
```
const [produits, categories] = await Promise.all([
  listProducts(),     // les deux requêtes partent en même temps
  listCategories(),
]);
```

C'est exactement ce que fera le futur Dashboard : compter produits, commandes et avis **en parallèle** au lieu de les enchaîner.

2.11 La boucle d'événements (Event Loop)

Comment Node.js peut-il gérer des milliers de connexions avec **un seul fil** d'exécution ? Grâce à la boucle d'événements.

SCHÉMA — LA BOUCLE D'ÉVÉNEMENTS (EVENT LOOP) DE NODE.JS



Node.js n'a qu'UN fil d'exécution JavaScript, mais il ne reste jamais bloqué à attendre la base de données :
il délègue l'attente, continue à servir d'autres requêtes, puis reprend quand le résultat arrive.

1. Ton code s'exécute dans la **pile d'appels** (*call stack*) : une ligne à la fois.
2. Quand il rencontre une opération longue (lire un fichier, interroger PostgreSQL), il la **délègue** au système (la couche libuv de Node.js) et continue.
3. Quand le système a fini, le résultat est placé dans une **file** avec la fonction à exécuter (le callback ou la suite de l'`await`).
4. La **boucle d'événements** surveille : « la pile est-elle vide ? » Si oui, elle prend la prochaine tâche de la file et l'exécute.

À RETENIR
Node.js ne fait qu'**une** chose JavaScript à la fois, mais il ne perd jamais de temps à attendre. Conséquence : un calcul lourd et synchrone (une boucle de plusieurs secondes) **bloque tout le monde**. Pour un serveur API, garde chaque fonction courte et délègue les attentes (`await`).

2.12 Les variables d'environnement

DÉFINITION SIMPLE
Une **variable d'environnement** est une valeur de configuration fournie au programme **depuis l'extérieur** du code : le numéro de port, l'adresse de la base de données, un mot de passe secret.

Pourquoi ne pas écrire ces valeurs dans le code ? Parce qu'elles **changent** selon l'endroit (ton PC, le serveur de production) et parce que certaines sont **secrètes**. Si tu écris le mot de passe PostgreSQL dans un fichier envoyé sur GitHub, il est public.

Le fichier `.env` (à la racine du projet) contient ces valeurs :

.env .env

```
PORT=3000
NODE_ENV=development
DATABASE_URL="postgresql://postgres:<mot-de-passe>@localhost:5432/lawpiking"
JWT_ACCESS_SECRET=<une-longue-chaine-aleatoire>
FRONTEND_URL=http://localhost:5173
```

Dans le code, on les lit avec `process.env` :

```
const port = Number(process.env.PORT) || 3000;
console.log(`Port choisi : ${port}`);
```

ATTENTION — Tout est du texte

`process.env.PORT` vaut `"3000"` (une chaîne), jamais le nombre `3000`. Convertis avec `Number(...)` quand c'est nécessaire.

Pour charger le fichier `.env` en développement, on installe le paquet `dotenv`, puis on l'importe en **première ligne** du programme :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install dotenv
```

src/config/env.js JavaScript

```
1 import "dotenv/config";
2
3 export const env = {
4   port: Number(process.env.PORT) || 3000,
5   nodeEnv: process.env.NODE_ENV || "development",
6   databaseUrl: process.env.DATABASE_URL,
7   frontendUrl: process.env.FRONTEND_URL || "http://localhost:5173",
8 };
```

`import "dotenv/config"` lit le fichier `.env` et copie ses lignes dans `process.env`. Ensuite, on regroupe les valeurs dans **un seul objet** `env` que le reste du projet importe : une seule porte d'entrée, des noms clairs, des valeurs par défaut.

BONNE PRATIQUE — Deux fichiers : .env et .env.example

Le `.env` (**secret**, jamais envoyé sur Git) contient les vraies valeurs. Le `.env.example` (**public**, envoyé sur Git) contient les mêmes clés avec des valeurs fictives : il sert de mode d'emploi à toute personne qui installe le projet.

FACULTATIF — Node.js sait lire un .env sans dotenv

Depuis Node.js 20.6, la commande `node --env-file=.env src/server.js` charge le fichier sans paquet supplémentaire. Ce guide garde `dotenv` car Prisma 7 l'utilise aussi (partie 6).

2.13 La gestion des erreurs

Une erreur n'est pas forcément un bug : « produit introuvable » est une situation **normale**. Il faut la distinguer d'un vrai plantage.

errors.js JavaScript

```
1 class AppError extends Error {
2   constructor(message, statusCode = 500, code = "INTERNAL_ERROR") {
3     super(message);
4     this.name = "AppError";
5     this.statusCode = statusCode;
6     this.code = code;
7   }
8 }
9
```

```
10 function findProduct(id) {
11   if (id !== 5) {
12     throw new AppError("Produit introuvable", 404, "PRODUCT_NOT_FOUND");
13   }
14   return { id: 5, name: "NOIR INTENSE" };
15 }
16
17 try {
18   findProduct(9999);
19 } catch (error) {
20   console.log(error.statusCode, error.code, "-", error.message);
21 }
```

RÉSULTAT résultat attendu

```
404 PRODUCT_NOT_FOUND - Produit introuvable
```

Ligne	Explication
1	On crée **notre propre type d'erreur** qui hérite de la classe `Error` de JavaScript.
2-3	Le constructeur reçoit le message, un code HTTP (404, 409…) et un code interne lisible par le frontend. `super(message)` appelle le constructeur de `Error`.
11-13	`throw` **lance** l'erreur : l'exécution s'arrête ici et remonte vers le premier `catch` qui l'attend.
18-20	`try/catch` attrape l'erreur et affiche ses propriétés.

Cette classe `AppError` sera réutilisée dans toute la partie Express : un service lance `AppError("Produit introuvable", 404)`, et un **gestionnaire d'erreurs central** la convertit en réponse HTTP.

Les filets de sécurité du processus

Deux événements attrapent ce que personne n'a géré. Ils sont là pour **journaliser puis arrêter proprement**, pas pour continuer comme si de rien n'était :

```
process.on("unhandledRejection", (reason) => {
  console.error("Promesse rejetée sans catch :", reason);
  process.exit(1);
});
process.on("uncaughtException", (error) => {
  console.error("Exception non attrapée :", error);
  process.exit(1);
});
```

ATTENTION — Après une exception inconnue, l'état du programme est incertain

Continuer à servir des clients après une erreur imprévue peut corrompre des données. Les bonnes pratiques : journaliser, quitter (`process.exit(1)`), et laisser un superviseur (le service d'hébergement) relancer le serveur.

2.14 Créer un serveur HTTP avec Node.js seul

Avant d'utiliser Express, construisons à la main le plus petit serveur possible. Tu comprendras ainsi **ce qu'Express simplifie**.

src/server.js JavaScript

```
1  import http from "node:http";
2
3  const products = [
4    { id: 5, slug: "tshirt-noir-intense", name: "NOIR INTENSE", price: 10000 },
5    { id: 6, slug: "tshirt-rouge-cardinal", name: "ROUGE CARDINAL", price: 10000 },
6  ];
7
8  const server = http.createServer((req, res) => {
9    const url = new URL(req.url, `http://${req.headers.host}`);
10
11   if (req.method === "GET" && url.pathname === "/api/products") {
12     res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
13     res.end(JSON.stringify({ total: products.length, products }));
14     return;
15   }
16
17   res.writeHead(404, { "Content-Type": "application/json; charset=utf-8" });
18   res.end(JSON.stringify({ error: "Route introuvable" }));
19 });
20
21 server.listen(3000, () => {
22   console.log("API LAWPIKING sur http://localhost:3000");
23 });
```

Ligne	Explication
1	On importe le module intégré `http` : pas d'installation nécessaire.
3-6	Des données en mémoire : deux produits (la vraie base viendra avec PostgreSQL).
8	`http.createServer(fonction)` crée le serveur. La fonction est appelée **à chaque requête** reçue, avec deux objets : `req` (la requête) et `res` (la réponse).
9	On reconstruit l'URL complète pour pouvoir lire le chemin (`pathname`) et les paramètres (`searchParams`).
11	On compare la méthode (`GET`) et le chemin : c'est un **routage fait main**.
12	`res.writeHead(200, …)` écrit le code de statut et les en-têtes. Ici on annonce du JSON.
13	`res.end(texte)` envoie le corps et **termine** la réponse. Sans `end`, le navigateur attend indéfiniment.
14	`return` empêche d'exécuter la suite (le 404) après avoir répondu.
17-18	Toute autre demande reçoit un 404 en JSON.
21	`server.listen(3000, …)` branche le serveur sur le port 3000 et fait qu'il **ne s'arrête plus**. Le callback s'exécute une fois prêt.

Lance-le, dans un terminal :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> node src/server.js
API LAWPIKING sur http://localhost:3000
```

Le terminal reste « occupé » : c'est normal, le serveur tourne. Ouvre un **second terminal** pour le tester (le premier doit rester ouvert) :

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> curl.exe http://localhost:3000/api/products
{"total":2,"products":[{"id":5,"slug":"tshirt-noir-intense","name":"NOIR INTENSE","price":10000},
↪
PS C:\Users\mohamed> curl.exe -i http://localhost:3000/inconnu
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Date: Sat, 31 Oct 2026 16:37:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Route introuvable"}
```

ERREUR FRÉQUENTE — « curl » dans PowerShell ne donne pas ce résultat
Dans PowerShell, `curl` est un **alias** d'une autre commande (`Invoke-WebRequest`) qui affiche un résultat très différent. Tape toujours `curl.exe` (avec `.exe`) pour utiliser le vrai programme curl, ou utilise `Invoke-RestMethod http://localhost:3000/api/products` qui affiche le JSON proprement.

ERREUR FRÉQUENTE — « EADDRINUSE : address already in use :::3000 »
Un autre programme (souvent **ton ancien serveur resté ouvert**) occupe déjà le port 3000. Retourne dans l'ancien terminal et appuie sur `Ctrl + C`. Sinon, change le numéro de port.

Tu peux aussi l'ouvrir directement dans ton navigateur à l'adresse `http://localhost:3000/api/products`. Pour arrêter le serveur : `Ctrl + C` dans son terminal.

À RETENIR — Ce que cet exemple t'apprend
Un serveur HTTP, c'est une **fonction appelée à chaque requête** : elle lit `req` (méthode, chemin, en-têtes, corps), puis écrit `res` (code, en-têtes, corps). Tout le reste (Express, middlewares, contrôleurs) n'est que de l'organisation autour de cette idée.

Un problème apparaît vite : pour gérer 30 routes, avec lecture du corps JSON, validation, authentification et erreurs, ce routage fait main devient un enchevêtrement de `if`. C'est la raison d'être d'Express (partie 3).

2.15 Les scripts npm

Les scripts de `package.json` évitent de retaper de longues commandes. Voici ceux que tu auras à la fin :

```
"scripts": {
  "dev": "node --watch src/server.js",
  "start": "node src/server.js",
  "db:migrate": "prisma migrate dev",
  "db:studio": "prisma studio",
  "test": "vitest run"
}
```

Script	Commande	Utilité
`npm run dev`	`node --watch src/server.js`	**Développement** : redémarre automatiquement le serveur à chaque modification d'un fichier
`npm start`	`node src/server.js`	**Production** : lance le serveur normalement
`npm run db:migrate`	`prisma migrate dev`	Applique les changements de schéma à ta base locale
`npm run db:studio`	`prisma studio`	Ouvre l'interface graphique de la base
`npm test`	`vitest run`	Lance tous les tests une fois

Le drapeau `--watch` de Node.js surveille tes fichiers : plus besoin de faire `Ctrl + C` puis de relancer à chaque changement.

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm run dev

> lawpiking-backend@1.0.0 dev
> node --watch src/server.js

API LAWPIKING sur http://localhost:3000
```

À RETENIR — En résumé — Partie 2

- **Node.js** = JavaScript hors du navigateur (moteur V8 + modules pour fichiers et réseau).
- `package.json` décrit le projet ; `package-lock.json` fixe les versions exactes ; `node_modules/` ne se versionne jamais.
- Utilise les **ES Modules** (`import`/`export`, `"type": "module"`) et `async`/`await` pour attendre sans bloquer.
- Les réglages passent par `.env` (jamais sur Git) et un objet `env` unique ; `AppError` représente les erreurs prévues.
- Dans un serveur, évite les fonctions `Sync` qui bloquent tous les visiteurs.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Installer Node.js et vérifier la version	Installer Node.js
Lire un package.json	npm et package.json
Écrire un module ES avec import et export	Modules
Utiliser async et await	Asynchrone
Lire une variable d'environnement avec dotenv	Variables d'environnement

2.16 Exercices de la partie 2

QUESTIONS DE COMPRÉHENSION

1. Quelle est la différence entre JavaScript, V8 et Node.js ?
2. Pourquoi `node_modules/` ne doit-il pas être versionné, alors que `package-lock.json` doit l'être ?
3. Que signifie `"^5.1.0"` ? Quelles versions peut installer npm ?
4. Pourquoi « Sync » est-il dangereux dans un serveur, mais acceptable dans un petit script ?

EXERCICES DE CODE

5. Écris un module `src/utils/stock.js` qui exporte une fonction `getStockStatus(stock, threshold = 10)` renvoyant `"out"`, `"low"` ou `"in"`, comme dans la page Stock de ton frontend (rupture à 0, faible si inférieur ou égal au seuil, sinon en stock). Importe-la depuis un autre fichier et teste-la avec les valeurs 0, 8 et 25.
6. Réécris `readFile("hello.js", …)` avec un callback, puis avec `.then`, puis avec `async/await`, et compare.
7. Écris une fonction `async function pause(ms)` qui attend `ms` millisecondes (indice : `new Promise` et `setTimeout`), puis affiche « bonjour » après 1 seconde.

DÉBOGAGE

8. Ce code affiche `Promise { <pending> }` au lieu des produits. Trouve l'erreur et corrige-la : `const produits = getProducts(); console.log(produits);` où `getProducts` est une fonction `async`.

MINI-PROJET

9. Ajoute à `src/server.js` la route `GET /api/products/:id` à la main (indice : découpe `url.pathname` avec `split("/")`), renvoie 404 si l'identifiant n'existe pas et 400 s'il n'est pas un nombre. Constate la lourdeur : c'est ce qu'Express va simplifier.

PARTIE 3

Express.js

Construire une API REST propre : routes, paramètres, middlewares, validation et gestion centralisée des erreurs.

3.1 Qu'est-ce qu'Express.js ?

> **DÉFINITION SIMPLE**
> **Express.js** est une petite bibliothèque pour Node.js qui simplifie la création de serveurs web et d'API : elle s'occupe du routage, de la lecture des requêtes et de l'envoi des réponses.

Dans la partie 2, nous avons écrit un serveur avec le module `http` seul. Il fonctionnait, mais chaque nouvelle route obligeait à écrire des `if (req.method === … && url.pathname === …)`, à découper l'URL à la main et à lire le corps de la requête morceau par morceau. Express fait tout cela à ta place, avec une écriture lisible :

Tâche	Avec node:http seul	Avec Express
Définir une route	`if (req.method === "GET" && url.pathname === "/api/products")`	`app.get("/api/products", handler)`
Lire un paramètre d'URL	Découper `pathname` avec `split("/")`	`req.params.id`
Lire le corps JSON	Écouter les événements `data` et `end`, puis `JSON.parse`	`app.use(express.json())` puis `req.body`
Envoyer du JSON	`writeHead` + `JSON.stringify` + `end`	`res.status(200).json(data)`
Ajouter une règle commune (journal, sécurité)	Copier-coller dans chaque route	Un **middleware** appliqué à toutes les routes
Gérer les erreurs	`try/catch` partout	Un seul gestionnaire central

3.2 Pourquoi Express avec Node.js ?

- **Minimaliste** : Express n'impose pas de structure. C'est à toi de l'organiser, ce que nous ferons avec l'architecture en couches (partie 7).
- **Le plus répandu** : énormément de documentation, de tutoriels et de bibliothèques compatibles (CORS, Helmet, Multer, etc. fonctionnent comme des middlewares Express).
- **Stable** : Express 5 est la version installée par défaut depuis 2025.

CHANGEMENT DE VERSION — Express 5 contre Express 4
Les tutoriels que tu trouveras en ligne ont longtemps été écrits pour Express 4. Les différences qui comptent pour toi : **(1)** dans Express 5, une erreur levée dans une fonction `async` ou une promesse rejetée est **automatiquement transmise** au gestionnaire d'erreurs (en Express 4, il fallait un `try/catch` ou une enveloppe). **(2)** `req.body` est `undefined` tant que tu n'as pas ajouté `express.json()`. **(3)** La syntaxe des chemins est plus stricte : un joker `*` doit être nommé (`/*splat`). **(4)** Le callback de `app.listen` reçoit l'erreur éventuelle (par exemple port déjà utilisé).

3.3 Installation et premier serveur

Dans `lawpiking-backend`, installe Express :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install express
added 66 packages, and audited 67 packages in 3s
```

Une bonne pratique est de séparer **la construction de l'application** (`app.js`) de **son démarrage** (`server.js`). `app.js` exporte l'application Express sans l'écouter sur un port ; `server.js` la lance. Cette séparation permet de tester l'API sans ouvrir de vrai port (partie 12).

src/app.js JavaScript

```
1  import express from "express";
2
3  const app = express();
4
5  app.use(express.json());
6
7  app.get("/api/health", (req, res) => {
8    res.status(200).json({ status: "ok", shop: "LAWPIKING" });
9  });
10
11 export default app;
```

src/server.js JavaScript

```
1  import { env } from "./config/env.js";
2  import app from "./app.js";
3
4  app.listen(env.port, (error) => {
5    if (error) {
6      console.error("Impossible de démarrer le serveur :", error.message);
7      process.exit(1);
8    }
9    console.log(`API LAWPIKING sur http://localhost:${env.port}`);
10 });
```

Fichier / ligne	Explication
`app.js` ligne 1	On importe Express (installé par npm).
`app.js` ligne 3	`express()` crée **l'application** : un objet qui sait recevoir des requêtes et les router.
`app.js` ligne 5	`app.use(express.json())` ajoute un **middleware** : il lit le corps des requêtes JSON et le place dans `req.body`.
`app.js` lignes 7-9	Une route `GET /api/health` (« le serveur est-il vivant ? »). `res.status(200).json({...})` renvoie du JSON avec le code 200.
`app.js` ligne 11	On **exporte** l'application sans la démarrer.
`server.js` ligne 4	`app.listen(port, callback)` démarre le serveur. En Express 5, le callback reçoit une erreur si le démarrage échoue (par exemple `EADDRINUSE`).

OBLIGATOIRE — Ajoute le démarrage au package.json

Le script `"dev": "node --watch src/server.js"` de la partie 2 lance désormais l'application Express. Teste :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm run dev
API LAWPIKING sur http://localhost:3000
```

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> curl.exe http://localhost:3000/api/health
{"status":"ok","shop":"LAWPIKING"}
```

3.4 Les routes

DÉFINITION SIMPLE

Une **route** associe une **méthode HTTP** et un **chemin** à une **fonction** (le *handler*) qui produit la réponse.

La forme générale est `app.METHODE(chemin, handler)`. Le handler reçoit trois arguments : `req` (la requête), `res` (la réponse) et, si besoin, `next` (passer au suivant).

```
app.get("/api/products", (req, res) => { /* lire la liste */ });
app.post("/api/products", (req, res) => { /* créer */ });
app.put("/api/products/:id", (req, res) => { /* remplacer */ });
app.patch("/api/products/:id/stock", (req, res) => { /* changer le stock */ });
app.delete("/api/products/:id", (req, res) => { /* supprimer */ });
```

Express parcourt tes routes **dans l'ordre où elles sont déclarées** et exécute la première qui correspond. Conséquence : les routes **précises** doivent être déclarées avant les routes **générales**.

ERREUR FRÉQUENTE — Une route générale qui avale une route précise

Si tu déclares `GET /api/products/:id` **avant** `GET /api/products/low-stock`, la requête `/api/products/low-stock` sera comprise comme `:id = "low-stock"` et ne sera jamais traitée par la deuxième route. Place toujours `/low-stock` avant `/:id`.

3.5 Lire la requête : req.params, req.query, req.body

C'est le point que les débutants confondent le plus. Trois endroits, trois usages :

Propriété	Origine de la valeur	Exemple de requête	Valeur lue
`req.params`	Un morceau **du chemin** (`:nom`). Sert à identifier **une** ressource.	`GET /api/products/5`	`"5"`
`req.query`	Après le `?`. Sert à **filtrer, trier, paginer**.	`GET /api/products?category=tshirts&page=2`	`"tshirts"`
`req.body`	Le **corps** de la requête. Sert à **envoyer** des données.	`POST /api/products` + JSON	`BORDEAUX`

ATTENTION — Tout arrive en texte

`req.params.id` vaut `"5"`, jamais le nombre 5. `req.query.page` vaut `"2"`. Convertis avec `Number(...)` et **vérifie le résultat** : `Number("abc")` donne `NaN`. Ne passe jamais un paramètre non vérifié à la base de données.

src/routes-demo.js — JavaScript

```
1  app.get("/api/products/:id", (req, res) => {
2    const id = Number(req.params.id);
3    const { category, page = "1" } = req.query;
4    res.json({ id, category: category ?? null, page: Number(page) });
5  });
6
7  app.post("/api/products", (req, res) => {
8    const { name, price } = req.body;
9    res.status(201).json({ received: { name, price } });
10 });
```

Ligne	Explication
1	`:id` est un **paramètre de chemin** : il capture ce qui se trouve à cette position de l'URL.
2	On convertit le texte en nombre.
3	On lit les filtres de la *query string*, avec `page` valant `"1"` par défaut. `category` est `undefined` si absente.
4	`category ?? null` remplace `undefined` par `null` (le JSON ne contient pas `undefined`).
7-8	On **déstructure** `req.body`. Sans `express.json()`, `req.body` serait `undefined` et cette ligne planterait.
9	`201 Created` : on a créé une ressource.

Test avec l'URL `http://localhost:3000/api/products/5?category=tshirts` (la requête GET) :

RÉSULTAT — résultat attendu

```
{"id":5,"category":"tshirts","page":1}
```

Et pour envoyer du JSON avec `curl.exe`, sous PowerShell, la manière la plus sûre est `Invoke-RestMethod` :

TERMINAL — Windows PowerShell

```
PS C:\Users\mohamed> Invoke-RestMethod -Method Post `
    -Uri http://localhost:3000/api/products `
    -ContentType "application/json" `
    -Body '{"name":"BORDEAUX","price":10000}'
```

```
received
--------
@{name=BORDEAUX; price=10000}
```

3.6 Envoyer la réponse : res

Méthode	Rôle	Exemple
`res.status(code)`	Définit le code de statut (chaînable)	`res.status(404)`
`res.json(objet)`	Envoie du JSON (et le bon `Content-Type`)	`res.json({ ok: true })`
`res.sendStatus(code)`	Envoie un code avec son texte standard	`res.sendStatus(204)`
`res.send(texte)`	Envoie du texte ou du HTML	`res.send("OK")`
`res.set(nom, valeur)`	Ajoute un en-tête	`res.set("X-Total-Count", "9")`

On combine souvent `status` et `json` : `res.status(201).json(produit)`.

ERREUR FRÉQUENTE — « Cannot set headers after they are sent to the client »
Tu as répondu **deux fois** à la même requête, typiquement un `res.json(...)` sans `return` suivi d'un autre `res.json(...)`. Une requête ne reçoit qu'une réponse. Écris `return res.status(404).json(...)` pour sortir de la fonction.

ERREUR FRÉQUENTE — La requête reste bloquée (« pending »)
Ton handler n'a jamais appelé `res.json`, `res.send` ni `res.end`, et n'a pas appelé `next()`. Le navigateur attend indéfiniment. Chaque chemin d'exécution d'une route doit se terminer par une réponse.

3.7 Les middlewares

DÉFINITION SIMPLE
Un **middleware** est une fonction qui s'exécute **entre** l'arrivée de la requête et la réponse finale. Elle peut lire ou modifier `req` et `res`, répondre elle-même, ou passer la main à la suivante avec `next()`.

Imagine l'entrée d'une salle de concert : le contrôle des billets, puis la fouille, puis le placement. Chaque étape peut refuser (la requête s'arrête) ou laisser passer (`next()`). C'est une **chaîne**.

La signature est `(req, res, next)` :

src/middlewares/requestLogger.js — JavaScript

```
export function requestLogger(req, res, next) {
  const start = Date.now();

  res.on("finish", () => {
    const duration = Date.now() - start;
    console.log(`${req.method} ${req.originalUrl} -> ${res.statusCode} (${duration} ms)`);
```

```
  });

  next();
}
```

Ligne	Explication
1	Un middleware est une simple fonction avec trois paramètres.
2	On note l'heure d'arrivée.
4	`res.on("finish", …)` : on s'abonne à l'événement « la réponse est partie ». Le journal s'écrit donc **après** la réponse, avec le code final.
6	`req.originalUrl` est l'URL complète demandée (même à l'intérieur d'un routeur).
10	`next()` : **indispensable**. Sans lui, la requête reste bloquée dans ce middleware.

Branche-le dans `app.js`, **avant** les routes (l'ordre de déclaration est l'ordre d'exécution) :

```
app.use(requestLogger);
app.use(express.json());
```

RÉSULTAT résultat attendu

```
GET /api/health -> 200 (3 ms)
POST /api/products -> 201 (5 ms)
GET /api/products/9999 -> 404 (2 ms)
```

Trois sortes de middlewares :

Type	Exemple	Où le placer
Globaux (toute l'application)	`app.use(express.json())`, journal, CORS, Helmet	Tout en haut de `app.js`
De route (une route précise)	`router.post("/", requireAdmin, createProduct)`	Dans la définition de la route
D'erreur (4 paramètres)	`(err, req, res, next)`	Tout en **bas**, après les routes

À RETENIR

Une requête traverse les middlewares **dans l'ordre**. Chacun fait soit `next()` (continuer), soit une réponse (terminer), soit `next(erreur)` (aller directement au gestionnaire d'erreurs). Quasi toutes les fonctions de sécurité du backend (partie 8) sont des middlewares.

3.8 Organiser les routes avec un Router

Mettre toutes les routes dans `app.js` devient illisible. Un **Router** est un « mini-Express » qui regroupe les routes d'une ressource. On le monte ensuite sur un préfixe.

src/modules/products/products.routes.js JavaScript

```
import { Router } from "express";
import * as controller from "./products.controller.js";

const router = Router();

```

```
router.get("/", controller.listProducts);
router.get("/:id", controller.getProduct);
router.post("/", controller.createProduct);
router.put("/:id", controller.updateProduct);
router.patch("/:id/stock", controller.updateStock);
router.delete("/:id", controller.deleteProduct);

export default router;
```

src/app.js (extrait) JavaScript

```
import productsRouter from "./modules/products/products.routes.js";

app.use("/api/products", productsRouter);
```

`app.use("/api/products", productsRouter)` **ajoute le préfixe** : la route `router.get("/:id")` répond donc à `GET /api/products/:id`. Dans le fichier de routes, on n'écrit jamais `/api/products` : c'est `app.js` qui décide du préfixe. Si un jour l'API passe en version 2, on ne change qu'une ligne.

3.9 Une API REST complète : les produits

Construisons les cinq endpoints demandés, avec des données **en mémoire** (la base PostgreSQL viendra aux parties 4 à 6). L'objectif ici est d'apprendre la mécanique.

Les données (3 des 9 produits réels de LAWPIKING) :

src/modules/products/products.data.js JavaScript

```
export const products = [
  { id: 1, slug: "tshirt-vert-menthe", name: "VERT MENTHE", price: 10000, stock: 25 },
  { id: 5, slug: "tshirt-noir-intense", name: "NOIR INTENSE", price: 10000, stock: 15 },
  { id: 6, slug: "tshirt-rouge-cardinal", name: "ROUGE CARDINAL", price: 10000, stock: 0 },
];
let nextId = 10;
export const takeId = () => nextId++;
```

Les contrôleurs : chaque fonction lit la requête, agit et répond.

src/modules/products/products.controller.js JavaScript

```
import { products, takeId } from "./products.data.js";

export function listProducts(req, res) {
  const { search = "", inStock } = req.query;
  let result = products.filter((p) =>
    p.name.toLowerCase().includes(search.toLowerCase()),
  );
  if (inStock === "true") result = result.filter((p) => p.stock > 0);
  res.status(200).json({ total: result.length, products: result });
}

export function getProduct(req, res) {
  const id = Number(req.params.id);
  const product = products.find((p) => p.id === id);
  if (!product) {
    return res.status(404).json({ error: "Produit introuvable" });
  }
  res.status(200).json(product);
}
```

```
20
21 export function createProduct(req, res) {
22   const { name, price, stock = 0 } = req.body;
23   const product = { id: takeId(), slug: slugify(name), name, price, stock };
24   products.push(product);
25   res.status(201).json(product);
26 }
27
28 export function updateProduct(req, res) {
29   const id = Number(req.params.id);
30   const index = products.findIndex((p) => p.id === id);
31   if (index === -1) {
32     return res.status(404).json({ error: "Produit introuvable" });
33   }
34   const { name, price, stock } = req.body;
35   products[index] = { ...products[index], name, price, stock };
36   res.status(200).json(products[index]);
37 }
38
39 export function updateStock(req, res) {
40   const product = products.find((p) => p.id === Number(req.params.id));
41   if (!product) {
42     return res.status(404).json({ error: "Produit introuvable" });
43   }
44   product.stock = req.body.stock;
45   res.status(200).json(product);
46 }
47
48 export function deleteProduct(req, res) {
49   const index = products.findIndex((p) => p.id === Number(req.params.id));
50   if (index === -1) {
51     return res.status(404).json({ error: "Produit introuvable" });
52   }
53   products.splice(index, 1);
54   res.sendStatus(204);
55 }
56
57 const slugify = (text) =>
58   `tshirt-${text.toLowerCase().normalize("NFD").replace(/[\u0300-\u036f]/g, "")
59     .replace(/[^a-z0-9]+/g, "-").replace(/(^-|-$)/g, "")}`;
```

Explication des cinq endpoints, un par un :

Endpoint	Ce qu'il fait	Réponse
`GET /api/products`	Lit `search` et `inStock` dans la query, filtre le tableau, renvoie la liste et le total. Aucun paramètre n'est obligatoire.	`200` + `{ total, products }`
`GET /api/products/:id`	Convertit `:id` en nombre, cherche avec `find`. Si rien n'est trouvé, **404** ; sinon le produit.	`200` ou `404`
`POST /api/products`	Lit `name`, `price`, `stock` dans `req.body`, génère un `id` et un `slug` (même logique que ta fonction `slugify` du frontend), ajoute à la liste.	`201` + le produit créé
`PUT /api/products/:id`	Cherche l'index du produit ; s'il existe, le **remplace** par les nouvelles valeurs (en gardant `id` et `slug`).	`200` + le produit modifié, ou `404`
`DELETE /api/products/:id`	Cherche l'index et retire l'élément avec `splice`.	`204 No Content` (pas de corps)

Et le `PATCH /api/products/:id/stock`, déclaré en plus : il ne change **que** le stock, comme ta page Stock.

ATTENTION — Rien n'est encore vérifié !
Dans ce code, un client peut envoyer `price: "gratuit"` ou `stock: -50`, et ces valeurs seront acceptées. Une API ne doit **jamais** faire confiance à ce qu'elle reçoit. La section suivante règle ce problème.

Le chemin parcouru par une requête

Suivons `GET /api/products/5` ligne par ligne :

1. Le navigateur envoie `GET /api/products/5`.
2. `app.use(requestLogger)` : le journal note l'heure de départ, puis `next()`.
3. `app.use(express.json())` : pas de corps JSON, donc il ne fait rien et appelle `next()`.
4. `app.use("/api/products", productsRouter)` : le préfixe correspond, Express entre dans le routeur avec le reste du chemin : `/5`.
5. `router.get("/:id", controller.getProduct)` correspond : `req.params.id` vaut `"5"`.
6. Le contrôleur trouve le produit et appelle `res.status(200).json(product)`.
7. La réponse repart. L'événement `finish` déclenche l'écriture du journal : `GET /api/products/5 -> 200 (2 ms)`.

3.10 Valider les données

DÉFINITION SIMPLE
Valider consiste à vérifier que les données reçues respectent les règles **avant** de s'en servir : champs présents, bons types, valeurs raisonnables.

La validation côté frontend (que tu as déjà) sert au confort de l'utilisateur. La validation côté backend sert à la **sécurité** : quelqu'un peut contourner ton formulaire avec `curl` et envoyer n'importe quoi. Les deux sont nécessaires.

Plutôt que d'écrire des dizaines de `if`, on utilise une bibliothèque de validation, **Zod**, qui décrit la forme attendue en un schéma.

TERMINAL — Windows PowerShell

```
PS ...\lawpiking-backend> npm install zod
```

src/modules/products/products.schema.js — JavaScript

```
import { z } from "zod";

export const createProductSchema = z.object({
  name: z.string().trim().min(1, "Le nom est obligatoire").max(80, "80 caractères maximum"),
  description: z.string().max(1000).optional().default(""),
  price: z.coerce.number()
    .int("Le prix doit être entier")
    .positive("Le prix doit être supérieur à 0"),
  stock: z.coerce.number().int().min(0, "Le stock ne peut pas être négatif").default(0),
  categoryId: z.string().min(1, "Choisis une catégorie"),
});

export const updateStockSchema = z.object({
  stock: z.coerce.number().int().min(0).max(100000),
});
```

Ligne	Explication
3	`z.object({...})` : on décrit un objet avec ses champs.
4	`z.string()` doit être du texte. `.trim()` retire les espaces autour, `.min(1, msg)` interdit le vide (message personnalisé), `.max(80)` fixe la limite, comme dans ton formulaire.
5	`.optional().default("")` : champ facultatif, valeur `""` s'il est absent.
6	`z.coerce.number()` **convertit** `"10000"` (texte) en nombre, `.int()` refuse `10.5`, `.positive()` refuse `0` et les négatifs.
7	`.default(0)` : si absent, le stock vaut 0.

Ensuite, un **middleware de validation réutilisable** :

src/middlewares/validate.js JavaScript

```
1  import { AppError } from "../utils/AppError.js";
2
3  export const validate = (schema, source = "body") => (req, res, next) => {
4    const result = schema.safeParse(req[source]);
5    if (!result.success) {
6      const fields = {};
7      for (const issue of result.error.issues) {
8        fields[issue.path.join(".")] = issue.message;
9      }
10     return next(new AppError("Données invalides", 422, "VALIDATION_ERROR", fields));
11   }
12   req[source] = result.data;
13   next();
14 };
```

Ligne	Explication
3	Une **fonction qui renvoie un middleware** : on lui donne un schéma, elle renvoie la fonction `(req, res, next)`. On pourra donc écrire `validate(createProductSchema)`.
4	`safeParse` vérifie sans lancer d'exception. Il renvoie `{ success: true, data }` ou `{ success: false, error }`.
6-9	On transforme la liste d'erreurs de Zod en objet `{ champ: message }`, par exemple `{ price: "Le prix doit être supérieur à 0" }`.
10	On transmet l'erreur **au gestionnaire central** avec `next(erreur)` : le code 422 sera renvoyé.
12	Si tout est valide, on **remplace** `req.body` par les données nettoyées et converties (le prix devient un vrai nombre).

Utilisation dans le routeur :

```
router.post("/", validate(createProductSchema), controller.createProduct);
router.patch("/:id/stock", validate(updateStockSchema), controller.updateStock);
```

Désormais, `controller.createProduct` n'est exécuté **que si** les données sont valides.

> **BONNE PRATIQUE — Une validation, une seule source de vérité**
> Les mêmes règles s'appliquent à la création et à la modification. Si tu les modifies un jour (par exemple prix maximum), tu ne changes qu'un schéma. C'est tout le contraire d'un `if` copié-collé dans cinq contrôleurs.

3.11 La gestion centralisée des erreurs

> **DÉFINITION SIMPLE**
> Un **gestionnaire d'erreurs central** est un middleware spécial à **quatre paramètres** `(err, req, res, next)` qui reçoit **toutes** les erreurs de l'application et les convertit en réponses HTTP cohérentes.

Sans lui, chaque contrôleur répondrait aux erreurs à sa façon : des messages différents, des codes incohérents, parfois des plantages. Avec lui, les contrôleurs et services **lancent** des erreurs, et un seul fichier décide de la réponse. La classe `AppError` de la partie 2 reçoit un quatrième paramètre, `details`, pour transporter les erreurs champ par champ. Remplace ton fichier par cette version :

src/utils/AppError.js JavaScript

```
export class AppError extends Error {
  constructor(message, statusCode = 500, code = "INTERNAL_ERROR", details = null) {
    super(message);
    this.name = "AppError";
    this.statusCode = statusCode;
    this.code = code;
    this.details = details;
  }
}
```

src/middlewares/errorHandler.js JavaScript

```
import { AppError } from "../utils/AppError.js";

export function notFoundHandler(req, res, next) {
  const message = `Route introuvable : ${req.method} ${req.originalUrl}`;
  next(new AppError(message, 404, "ROUTE_NOT_FOUND"));
}

export function errorHandler(err, req, res, next) {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      error: { code: err.code, message: err.message, details: err.details },
    });
  }

  if (err.type === "entity.parse.failed") {
    return res.status(400).json({
      error: { code: "INVALID_JSON", message: "JSON invalide dans la requête" },
    });
  }

  console.error("Erreur inattendue :", err);
  res.status(500).json({
    error: { code: "INTERNAL_ERROR", message: "Une erreur interne est survenue" },
  });
}
```

Ligne	Explication
3-5	`notFoundHandler` : s'il atteint ce middleware, **aucune route n'a répondu**. On fabrique un 404 uniforme.
7	Le gestionnaire a **quatre** paramètres. Express reconnaît un gestionnaire d'erreurs **uniquement** grâce à ce nombre. Même si `next` n'est pas utilisé, il doit figurer.
8-12	Une erreur **que nous avons prévue** (`AppError`) : on renvoie son code HTTP et son message au client.
14-18	`express.json()` signale un JSON cassé avec `err.type === "entity.parse.failed"` : on répond 400.
20-23	Toute autre erreur est un **bug imprévu** : on l'écrit dans les journaux (pour toi) et on renvoie un message **générique** au client (jamais la trace complète, qui révélerait des détails internes).

Branche-les **tout en bas** de `app.js`, après toutes les routes :

src/app.js (version finale) JavaScript

```
1  import express from "express";
2  import { requestLogger } from "./middlewares/requestLogger.js";
3  import { notFoundHandler, errorHandler } from "./middlewares/errorHandler.js";
4  import productsRouter from "./modules/products/products.routes.js";
5
6  const app = express();
7
8  app.use(requestLogger);
9  app.use(express.json());
10
11 app.get("/api/health", (req, res) => res.json({ status: "ok", shop: "LAWPIKING" }));
12 app.use("/api/products", productsRouter);
13
14 app.use(notFoundHandler);
15 app.use(errorHandler);
16
17 export default app;
```

Maintenant, les services et contrôleurs peuvent simplement lancer :

```
export function getProduct(req, res) {
  const product = products.find((p) => p.id === Number(req.params.id));
  if (!product) throw new AppError("Produit introuvable", 404, "PRODUCT_NOT_FOUND");
  res.status(200).json(product);
}
```

Plus besoin de `try/catch` ni de `next(erreur)` dans les fonctions `async` : **Express 5** capture l'erreur et la transmet tout seul au gestionnaire.

> **CHANGEMENT DE VERSION — Express 4 demandait un try/catch dans chaque route async**
> Dans Express 4, une erreur levée dans une fonction `async` ne passait pas par le gestionnaire d'erreurs : il fallait écrire `try { … } catch (e) { next(e); }` partout, ou utiliser un paquet d'aide. Express 5 supprime cette corvée. Si tu lis un tutoriel avec `asyncHandler`, tu n'en as plus besoin ici.

Les réponses d'erreur ont toujours la même forme

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Données invalides",
    "details": { "price": "Le prix doit être supérieur à 0" }
  }
}
```

Ton frontend (partie 10) pourra afficher `details.price` sous le champ du prix, exactement comme tes formulaires affichent aujourd'hui leurs erreurs de validation.

3.12 Filtrer et paginer une liste

Une boutique avec 5 000 produits ne renvoie pas tout d'un coup. On utilise la **pagination** : `GET /api/products?page=2&limit=10`.

src/utils/pagination.js JavaScript

```
1 export function getPagination(query, defaultLimit = 10, maxLimit = 50) {
2   const page = Math.max(1, Number.parseInt(query.page, 10) || 1);
3   const asked = Number.parseInt(query.limit, 10) || defaultLimit;
4   const limit = Math.min(maxLimit, Math.max(1, asked));
5   return { page, limit, skip: (page - 1) * limit };
6 }
```

Ligne	Explication	
2	`parseInt(valeur, 10)` convertit en entier ; si ce n'est pas un nombre, `	`1 donne 1.` Math.max(1, ...)` empêche une page 0 ou négative.
3	Pareil pour `limit`, **plafonné** à 50 : sinon quelqu'un demanderait `limit=1000000`.	
4	`skip` est le nombre d'éléments à sauter : page 3 de 10 éléments saute les 20 premiers.	

La réponse contient aussi les informations de pagination :

```
{ "total": 9, "page": 1, "limit": 10, "totalPages": 1, "products": [ ... ] }
```

3.13 Tester l'API

Tant que le frontend n'est pas connecté, tu testes avec un outil. Tu en as trois :

Outil	Avantage	Quand
`curl.exe` / `Invoke-RestMethod`	Déjà installés sous Windows	Vérifications rapides
Postman ou **Insomnia**	Interface graphique, collections sauvegardées	Travail quotidien (partie 12)
Extension VS Code **REST Client**	Fichier `.http` versionné avec le code	Documenter les exemples

Voici une session de test complète. Les sorties sont celles que tu dois obtenir :

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> curl.exe -i http://localhost:3000/api/products/9999
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{"error":{"code":"PRODUCT_NOT_FOUND","message":"Produit introuvable","details":null}}
PS C:\Users\mohamed> Invoke-RestMethod -Method Post `
    -Uri http://localhost:3000/api/products `
    -ContentType "application/json" -Body '{"name":"","price":-5}'
Invoke-RestMethod : Le serveur distant a retourné une erreur : (422)
```

Pour voir le corps de l'erreur 422 (que `Invoke-RestMethod` masque), utilise `curl.exe` :

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> curl.exe -s -X POST http://localhost:3000/api/products `
    -H "Content-Type: application/json" -d "{\"name\":\"\",\"price\":-5}"
{"error":{"code":"VALIDATION_ERROR","message":"Données invalides",
"details":{"name":"Le nom est obligatoire",
"price":"Le prix doit être supérieur à 0",
"categoryId":"Choisis une catégorie"}}}
```

3.14 Récapitulatif : les 10 règles d'une bonne API Express

1. Sépare `app.js` (construction) et `server.js` (démarrage).
2. Un **routeur par ressource**, monté sur un préfixe `/api/…`.
3. Les routes **déclarent**, les contrôleurs **orchestrent**, les services **décident** (partie 7).
4. Toujours `express.json()` avant les routes, et le gestionnaire d'erreurs **après**.
5. **Valide** toute donnée entrante (corps, paramètres, query).
6. Renvoie le **bon code** HTTP, avec le **même format** de réponse d'erreur.
7. Jamais de trace d'erreur ni de secret dans une réponse.
8. Plafonne toujours la pagination.
9. Chaque chemin d'une route se termine par une réponse.
10. Garde les messages d'erreur en français, mais les **codes** (`PRODUCT_NOT_FOUND`) en anglais stable : le frontend s'appuie dessus.

À RETENIR — En résumé — Partie 3

- **Express** relie une méthode + un chemin à une fonction ; les routes se déclarent du plus précis au plus général.
- `req.params` (chemin), `req.query` (après `?`) et `req.body` (corps JSON) sont toujours du **texte** à valider.
- Un **middleware** fait `next()`, répond, ou transmet une erreur ; l'ordre de déclaration est l'ordre d'exécution.
- **Zod** valide chaque entrée ; un gestionnaire d'erreurs central à 4 paramètres produit des réponses uniformes.
- Express 5 transmet automatiquement les erreurs des fonctions `async` : plus de `try/catch` obligatoire.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Créer une application Express et la démarrer	Installation
Lire params, query et body	Lire la requête
Écrire un middleware	Les middlewares
Valider avec Zod	Valider les données
Centraliser les erreurs	Gestion centralisée des erreurs

3.15 Exercices de la partie 3

QUESTIONS DE COMPRÉHENSION

1. Pourquoi sépare-t-on `app.js` et `server.js` ?
2. Quelle est la différence entre `req.params`, `req.query` et `req.body` ? Donne un exemple LAWPIKING pour chacun.
3. À quoi sert `next()` ? Que se passe-t-il si un middleware l'oublie ?
4. Comment Express reconnaît-il un gestionnaire d'erreurs ?

EXERCICES DE CODE

5. Ajoute une route `GET /api/products/low-stock?threshold=10` qui renvoie les produits dont le stock est supérieur à 0 mais inférieur ou égal au seuil. Où dois-tu la déclarer, et pourquoi ?
6. Écris un middleware `requireJson` qui répond 415 (*Unsupported Media Type*) si la méthode est POST, PUT ou PATCH et que l'en-tête `Content-Type` n'est pas du JSON.
7. Écris le schéma Zod `createCategorySchema` : nom obligatoire (60 caractères maximum), description facultative (200 caractères maximum), `enabled` booléen avec `true` par défaut.

DÉBOGAGE

8. La route `GET /api/products/:id` répond toujours 404, même pour `id = 5`. Le code compare `p.id === req.params.id`. Quelle est l'erreur ? Corrige-la.
9. Après un `POST`, le serveur ne répond jamais et le navigateur tourne sans fin. Le contrôleur contient `products.push(product);` mais aucune réponse. Corrige.

MINI-PROJET

10. Construis l'API `categories` en mémoire (T-shirts, Pulls, Bonnets) avec les cinq routes REST, un schéma Zod, le gestionnaire d'erreurs central, et refuse (409) la création d'une catégorie dont le nom existe déjà (sans tenir compte des majuscules ni des accents).

PARTIE 4

PostgreSQL

Installer PostgreSQL sous Windows et maîtriser le SQL : tables, contraintes, relations, jointures, transactions, sauvegardes.

4.1 Qu'est-ce qu'une base de données ?

> **DÉFINITION SIMPLE**
> Une **base de données** est un espace organisé où l'on range des informations de façon durable, avec des règles qui garantissent leur cohérence. **PostgreSQL** est un **système de gestion de base de données relationnelle** (SGBDR) : un programme qui stocke les données dans des **tables** reliées entre elles et qui se pilote avec le langage **SQL**.

Aujourd'hui, ton frontend LAWPIKING range ses produits, catégories, commandes et avis dans le `localStorage` du navigateur. Ce stockage a trois limites : il est **propre à un seul navigateur**, il est **modifiable par n'importe qui** depuis les outils du navigateur, et il est **limité en taille** (quelques mégaoctets). Une vraie base de données corrige ces trois points : une seule source de vérité, protégée derrière le serveur, sans limite pratique.

Vocabulaire	Équivalent simple	Exemple LAWPIKING
Base de données	Le classeur entier	`lawpiking`
Table	Un tableau (comme une feuille Excel)	`products`
Colonne	Une information, avec son type	`price`
Ligne (enregistrement)	Un élément	Le produit NOIR INTENSE
Clé primaire	L'identifiant unique d'une ligne	`id`
Clé étrangère	Une colonne qui désigne une ligne d'une autre table	`products.category_id`
Requête	Une question ou une instruction posée à la base	`SELECT * FROM products;`

4.2 Pourquoi PostgreSQL ?

- **Gratuit et open source**, utilisé par de très grandes entreprises.
- **Fiable** : il respecte les règles ACID (une opération réussit en entier ou pas du tout, voir *transactions*).
- **Riche** : types modernes (UUID, JSON, dates avec fuseau), contraintes puissantes, index performants.
- **Très bien supporté** par Prisma et par tous les hébergeurs (partie 13).

4.3 Installer PostgreSQL sous Windows

CHANGEMENT DE VERSION — PostgreSQL 18
Ce guide utilise **PostgreSQL 18**. Les commandes SQL de ce guide fonctionnent aussi en 16 et 17. Vérifie la version actuelle sur postgresql.org/download/windows avant d'installer.

1. Va sur **postgresql.org/download/windows** puis clique sur *Download the installer* (installateur EDB). Choisis la version 18 pour Windows x86-64.
2. Lance le fichier téléchargé. Garde les composants cochés : **PostgreSQL Server**, **pgAdmin 4**, **Command Line Tools**. *Stack Builder* est facultatif (tu peux le décocher).
3. Garde le dossier de données proposé.
4. **Choisis un mot de passe pour le super-utilisateur** `postgres` et **note-le** (gestionnaire de mots de passe). Tu en auras besoin à chaque connexion administrateur.
5. Garde le port **5432**. C'est le port standard de PostgreSQL.
6. Garde la locale par défaut, puis termine l'installation.

ATTENTION — Le mot de passe de postgres
Si tu l'oublies, tu ne pourras plus créer de base. Ne le mets **jamais** dans le code ni sur GitHub : il ira dans le fichier `.env` (partie 2).

Vérifier l'installation. Ouvre PowerShell :

TERMINAL — Windows PowerShell

```
PS C:\Users\mohamed> psql --version
psql (PostgreSQL) 18.0
```

ERREUR FRÉQUENTE — « psql : Le terme 'psql' n'est pas reconnu »
Windows ne trouve pas le programme car son dossier n'est pas dans le **PATH**. Ajoute `C:\Program Files\PostgreSQL\18\bin` aux variables d'environnement (Paramètres, *Modifier les variables d'environnement du système*, variable `Path`, *Nouveau*), puis **ferme et rouvre** PowerShell.

Vérifier que le service tourne : dans PowerShell, `Get-Service postgres` doit afficher l'état `Running`.

4.4 Les outils : psql et pgAdmin

Outil	Nature	Quand l'utiliser
psql	Terminal	Commandes rapides, scripts, sauvegardes
pgAdmin 4	Interface graphique	Explorer les tables, lire les données, écrire des requêtes
Prisma Studio	Interface de Prisma (partie 6)	Consulter et modifier les données du projet

Se connecter avec psql :

TERMINAL — Windows PowerShell

```
PS C:\Users\mohamed> psql -U postgres
Mot de passe pour l'utilisateur postgres :
postgres=#
```

L'invite `postgres=#` signifie que tu es connecté à la base `postgres` en super-utilisateur. Les **méta-commandes** psql commencent par un antislash, et **ne se terminent pas** par `;` :

Commande	Rôle
`\l`	Lister les bases
`\c lawpiking`	Se connecter à la base `lawpiking`
`\dt`	Lister les tables
`\d products`	Décrire la table `products` (colonnes, index, clés)
`\du`	Lister les utilisateurs (rôles)
`\x`	Basculer l'affichage vertical (pratique pour les lignes larges)
`\i fichier.sql`	Exécuter un fichier SQL
`\q`	Quitter

Au contraire, une **instruction SQL** se termine toujours par un **point-virgule** `;`. Si tu l'oublies, psql attend la suite (l'invite devient `postgres-#`) : tape `;` puis Entrée.

Dans pgAdmin : au premier lancement, définis un mot de passe maître (pour pgAdmin seulement), puis déplie *Servers*, *PostgreSQL 18* et saisis le mot de passe de `postgres`. Clic droit sur une table, *View/Edit Data*, *All Rows* pour voir son contenu ; l'outil **Query Tool** (clic droit sur la base) sert à exécuter du SQL.

4.5 Créer la base et un utilisateur dédié

En production comme en développement, l'application ne doit pas se connecter avec le super-utilisateur : on crée un utilisateur qui n'a de droits que sur **sa** base.

01_create_database.sql — SQL

```sql
CREATE USER lawpiking_user WITH PASSWORD 'change-moi-en-local';
CREATE DATABASE lawpiking OWNER lawpiking_user ENCODING 'UTF8';
```

Ligne	Explication
1	Crée un **rôle** de connexion, avec son mot de passe (entre apostrophes simples). Choisis le tien.
2	Crée la base `lawpiking`, dont `lawpiking_user` est **propriétaire**, en UTF-8 (accents français et caractères spéciaux).

TERMINAL — Windows PowerShell

```
postgres=# \l
postgres=# \c lawpiking lawpiking_user
Mot de passe pour l'utilisateur lawpiking_user :
Vous êtes maintenant connecté à la base « lawpiking » (utilisateur « lawpiking_user »).
lawpiking=>
```

Plus tard, l'URL de connexion de l'application sera :

.env — .env

```
DATABASE_URL="postgresql://lawpiking_user:ton_mot_de_passe@localhost:5432/lawpiking"
```

Lecture : `protocole://utilisateur:mot_de_passe@hôte:port/base`.

ATTENTION — Caractères spéciaux dans le mot de passe
Si le mot de passe contient `@`, `:` ou `/`, il doit être encodé en URL (`@` devient `%40`). Pour le développement, choisis un mot de passe simple en lettres et chiffres.

4.6 Les types de données

Chaque colonne a un **type**. Voici ceux dont LAWPIKING a besoin :

Type	Contenu	Exemple LAWPIKING
`INTEGER`	Entier (jusqu'à environ 2 milliards)	Stock : `25`
`NUMERIC(12,2)`	Nombre décimal exact	Montants d'argent si centimes
`TEXT`	Texte de longueur libre	Description d'un produit
`VARCHAR(n)`	Texte limité à `n` caractères	Nom du produit : `VARCHAR(80)`
`BOOLEAN`	`TRUE` ou `FALSE`	`is_active`
`UUID`	Identifiant universel de 36 caractères	`id` d'un produit
`TIMESTAMPTZ`	Date et heure **avec fuseau horaire**	`created_at`
`DATE`	Date seule	Début d'une promotion
`JSONB`	Données JSON interrogeables	Un réglage complexe

BONNE PRATIQUE — Pourquoi des entiers pour les prix ?
Le franc CFA (XOF) n'a pas de centimes : `10000` signifie 10 000 FCFA. Un `INTEGER` est exact, rapide et évite les erreurs d'arrondi des nombres flottants. Si tu devais gérer des centimes, tu stockerais des **centimes entiers** ou un `NUMERIC`, jamais un `FLOAT`.

BONNE PRATIQUE — Pourquoi des UUID ?
Avec un compteur (1, 2, 3…), n'importe qui peut deviner l'adresse d'une commande voisine (`/orders/2`). Un UUID comme `0b9c9b1e-...` est imprévisible. PostgreSQL sait en générer avec `gen_random_uuid()`.

4.7 Créer des tables : CREATE TABLE

Construisons d'abord un petit exemple complet, **les catégories**, puis **les produits**.

02_categories_products.sql — SQL

```sql
CREATE TABLE categories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        VARCHAR(40)  NOT NULL UNIQUE,
  name        VARCHAR(60)  NOT NULL,
  description TEXT         NOT NULL DEFAULT '',
  is_enabled  BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE products (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id       UUID           NOT NULL REFERENCES categories(id),
  slug              VARCHAR(100)   NOT NULL UNIQUE,
```

```
  name              VARCHAR(80)    NOT NULL,
  description       TEXT           NOT NULL DEFAULT '',
  price             INTEGER        NOT NULL CHECK (price > 0),
  is_active         BOOLEAN        NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ    NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ    NOT NULL DEFAULT now(),
  deleted_at        TIMESTAMPTZ
);
```

Élément	Explication
`CREATE TABLE categories (...)`	Crée la table. Chaque ligne entre parenthèses est une colonne : **nom**, **type**, **contraintes**.
`PRIMARY KEY`	Clé primaire : valeur **unique et jamais vide**. Elle identifie la ligne.
`DEFAULT gen_random_uuid()`	Si on n'indique pas d'`id`, PostgreSQL en fabrique un.
`NOT NULL`	La colonne **doit** avoir une valeur. Sans cette contrainte, elle peut valoir `NULL` (« inconnu »).
`UNIQUE`	Deux lignes ne peuvent pas avoir la même valeur (deux produits ne partagent pas le même `slug`).
`DEFAULT TRUE` / `DEFAULT now()`	Valeur automatique si elle n'est pas donnée. `now()` est l'instant actuel.
`REFERENCES categories(id)`	**Clé étrangère** : `category_id` doit correspondre à un `id` existant de `categories`. Impossible d'inventer une catégorie.
`CHECK (price > 0)`	Règle personnalisée : un prix à 0 ou négatif est refusé par la base elle-même.
`deleted_at TIMESTAMPTZ`	Sans `NOT NULL` : reste vide tant que le produit n'est pas supprimé (suppression « douce », voir plus bas).

Dans cet exemple, remarque que `products.category_id` **dépend** de `categories`. La table `categories` doit donc être créée **avant** `products`, sinon PostgreSQL répond « la relation categories n'existe pas ».

Les contraintes : le gardien de tes données

Une **contrainte** est une règle que la base fait respecter même si ton code a un bug :

Contrainte	Garantit que	Quand la violer donne
`NOT NULL`	La colonne est renseignée	Erreur `23502`
`UNIQUE`	Aucun doublon	Erreur `23505`
`PRIMARY KEY`	Unique + non nulle	Erreur `23505`
`FOREIGN KEY` (`REFERENCES`)	La ligne référencée existe	Erreur `23503`
`CHECK`	Une condition est vraie	Erreur `23514`
`DEFAULT`	Une valeur par défaut existe	(jamais d'erreur)

> **À RETENIR**
> Le backend valide les données (partie 3), **et** la base les protège (contraintes). Les deux sont complémentaires : si un autre programme ou un futur développeur écrit dans la base, les contraintes protègent quand même LAWPIKING. Ces codes d'erreur (`23505`…) seront convertis en réponses 409 ou 422 par le backend.

4.8 Insérer des données : INSERT

03_insert.sql SQL

```sql
INSERT INTO categories (slug, name, description)
VALUES ('tshirts', 'T-shirts', 'Les T-shirts LAWPIKING'),
       ('pulls',   'Pulls',    'Les pulls LAWPIKING'),
       ('bonnets', 'Bonnets',  'Les bonnets LAWPIKING');

INSERT INTO products (category_id, slug, name, description, price)
SELECT id, 'tshirt-noir-intense', 'NOIR INTENSE', 'T-shirt noir, coupe classique', 10000
FROM categories WHERE slug = 'tshirts';

INSERT INTO products (category_id, slug, name, price)
SELECT id, 'tshirt-vert-menthe', 'VERT MENTHE', 10000
FROM categories WHERE slug = 'tshirts';
```

Élément	Explication
`INSERT INTO table (colonnes)`	Indique où insérer et quelles colonnes sont renseignées.
`VALUES (...), (...)`	Plusieurs lignes d'un coup, séparées par des virgules. Les textes sont entre **apostrophes simples**.
`INSERT ... SELECT`	Au lieu de copier un UUID à la main, on **va chercher** l'`id` de la catégorie `tshirts`.
Colonnes omises	`id`, `created_at`... prennent leur valeur `DEFAULT`.

RÉSULTAT résultat attendu

```
INSERT 0 3
INSERT 0 1
INSERT 0 1
```

`INSERT 0 3` signifie : 3 lignes insérées (le `0` est historique). Pour récupérer les valeurs générées, ajoute `RETURNING id, slug` à la fin de l'insertion.

> **ERREUR FRÉQUENTE — ERREUR : une valeur NULL viole la contrainte NOT NULL**
> Tu as oublié une colonne obligatoire sans `DEFAULT`, ou tu as envoyé `NULL` dedans. PostgreSQL te donne le nom de la colonne : ajoute-la à l'`INSERT`.

> **ERREUR FRÉQUENTE — ERREUR : la valeur d'une clé dupliquée rompt la contrainte unique**
> Un produit avec ce `slug` existe déjà. Choisis un autre `slug` (le backend ajoutera `-2`, `-3`... comme le fait ton frontend aujourd'hui).

4.9 Lire des données : SELECT

`SELECT` est l'instruction que tu utiliseras le plus.

04_select.sql SQL

```sql
-- Tout lire
SELECT * FROM products;

-- Choisir les colonnes
SELECT name, price FROM products;
```

```sql

-- Filtrer, trier, limiter
SELECT name, price
FROM products
WHERE price >= 10000 AND is_active = TRUE
ORDER BY name ASC
LIMIT 10 OFFSET 0;

-- Recherche par texte (sans tenir compte des majuscules)
SELECT name FROM products WHERE name ILIKE '%noir%';
```

Mot-clé	Rôle
`SELECT *`	Toutes les colonnes. Pratique pour explorer, mauvais réflexe dans le code (on demande uniquement ce qu'il faut).
`WHERE`	Garde seulement les lignes qui respectent la condition. Opérateurs : `=`, `<>`, `<`, `>`, `AND`, `OR`, `IN (...)`, `IS NULL`.
`ORDER BY name ASC`	Trie. `ASC` croissant, `DESC` décroissant.
`LIMIT 10 OFFSET 0`	Prend 10 lignes en sautant 0 : c'est la **pagination**.
`ILIKE '%noir%'`	Recherche insensible à la casse ; `%` remplace n'importe quel texte.

ATTENTION — NULL ne se compare pas avec =

`WHERE deleted_at = NULL` ne renvoie **rien**. Écris `WHERE deleted_at IS NULL`. `NULL` signifie « inconnu » : on ne peut pas dire qu'il est égal à quoi que ce soit.

RÉSULTAT résultat attendu

```
        name   | price
---------------+-------
 NOIR INTENSE | 10000
 VERT MENTHE  | 10000
(2 lignes)
```

4.10 Modifier et supprimer : UPDATE et DELETE

05_update_delete.sql SQL

```sql
UPDATE products
SET price = 12000, updated_at = now()
WHERE slug = 'tshirt-noir-intense';

DELETE FROM products WHERE slug = 'tshirt-vert-menthe';
```

ATTENTION — Le WHERE est vital

`UPDATE products SET price = 0;` sans `WHERE` modifie **tous** les produits. `DELETE FROM products;` les supprime tous. Avant un `UPDATE` ou un `DELETE`, écris d'abord le même `WHERE` dans un `SELECT` pour vérifier les lignes concernées. En production, fais-le dans une transaction (voir plus bas).

RÉSULTAT résultat attendu

```
UPDATE 1
DELETE 1
```

Le nombre indique les lignes touchées. `UPDATE 0` signifie que **rien** ne correspondait au `WHERE` : c'est ce qui deviendra un « 404 produit introuvable » dans l'API.

Suppression douce (soft delete)

Dans LAWPIKING, un produit déjà commandé ne doit **pas** disparaître : les commandes passées y font référence. On le **masque** plutôt que de le supprimer :

```
UPDATE products SET deleted_at = now(), is_active = FALSE WHERE slug = 'tshirt-noir-intense';
SELECT name FROM products WHERE deleted_at IS NULL;
```

La boutique n'affiche que les produits dont `deleted_at IS NULL`. L'administrateur peut un jour le restaurer en remettant `deleted_at` à `NULL`.

4.11 Les relations

Entre deux tables, il existe trois sortes de relations :

Relation	Signification	Exemple LAWPIKING	Comment la représenter
1 à N	Un élément A a plusieurs B ; un B appartient à un seul A	Une catégorie a plusieurs produits	Clé étrangère dans la table « N » (`products.category_id`)
N à N	Plusieurs A pour plusieurs B	Une promotion concerne plusieurs produits, un produit a plusieurs promotions	Une **table de liaison** (`promotion_products`)
1 à 1	Un A pour un seul B	(rare) Un utilisateur et son profil	Clé étrangère `UNIQUE`

06_promotions.sql SQL

```sql
CREATE TABLE promotions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(80) NOT NULL,
  percent_off INTEGER     NOT NULL CHECK (percent_off BETWEEN 1 AND 90),
  starts_at   TIMESTAMPTZ NOT NULL,
  ends_at     TIMESTAMPTZ NOT NULL,
  CHECK (ends_at > starts_at)
);

CREATE TABLE promotion_products (
  promotion_id UUID NOT NULL REFERENCES promotions(id) ON DELETE CASCADE,
  product_id   UUID NOT NULL REFERENCES products(id)   ON DELETE CASCADE,
  PRIMARY KEY (promotion_id, product_id)
);
```

Élément	Explication
`percent_off BETWEEN 1 AND 90`	Un pourcentage de réduction raisonnable.
`CHECK (ends_at > starts_at)`	Contrainte sur **deux colonnes** : une promotion ne finit pas avant de commencer.
`promotion_products`	**Table de liaison** : chaque ligne signifie « ce produit participe à cette promotion ».
`PRIMARY KEY (promotion_id, product_id)`	Clé **composée** : le même produit ne peut pas être ajouté deux fois à la même promotion (anti-doublon).
`ON DELETE CASCADE`	Si la promotion (ou le produit) est supprimée, les lignes de liaison disparaissent automatiquement.

Le comportement `ON DELETE` choisit ce qui arrive aux lignes dépendantes :

Option	Effet quand on supprime la ligne parente	Usage
`RESTRICT` / `NO ACTION` (défaut)	**Refusé** tant que des lignes dépendent	Produits d'une catégorie
`CASCADE`	Les lignes dépendantes sont **supprimées**	Tables de liaison
`SET NULL`	La clé étrangère devient `NULL`	Colonne facultative

4.12 Les jointures : JOIN

Une **jointure** assemble des lignes de plusieurs tables dans un seul résultat, en suivant les clés étrangères.

07_joins.sql SQL

```sql
-- Chaque produit avec le nom de sa catégorie
SELECT p.name AS produit, c.name AS categorie, p.price
FROM products p
JOIN categories c ON c.id = p.category_id
WHERE p.deleted_at IS NULL
ORDER BY c.name, p.name;

-- Toutes les catégories, même sans produit
SELECT c.name AS categorie, p.name AS produit
FROM categories c
LEFT JOIN products p ON p.category_id = c.id AND p.deleted_at IS NULL;
```

Élément	Explication
`products p`	`p` est un **alias** (surnom) pour écrire moins.
`JOIN categories c ON c.id = p.category_id`	Relie chaque produit à sa catégorie. `ON` indique la condition de jointure.
`AS produit`	Renomme la colonne dans le résultat.
`LEFT JOIN`	Garde **toutes** les lignes de la table de gauche, même sans correspondance (les colonnes de droite valent `NULL`).

Jointure	Garde
`INNER JOIN` (ou `JOIN`)	Seulement les lignes avec correspondance des deux côtés
`LEFT JOIN`	Toutes les lignes de gauche, avec ou sans correspondance

4.13 Agrégats : compter, additionner, regrouper

08_aggregates.sql SQL

```sql
-- Nombre de produits par catégorie
SELECT c.name, COUNT(p.id) AS nb_produits, AVG(p.price)::INTEGER AS prix_moyen
FROM categories c
LEFT JOIN products p ON p.category_id = c.id AND p.deleted_at IS NULL
GROUP BY c.name
HAVING COUNT(p.id) >= 0
ORDER BY nb_produits DESC;
```

Élément	Explication
`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`	Fonctions d'**agrégat** : elles résument plusieurs lignes en une valeur.
`COUNT(p.id)`	Compte les produits (ignore les `NULL`, donc une catégorie vide donne 0).
`::INTEGER`	Convertit (cast) le résultat en entier.
`GROUP BY c.name`	Regroupe les lignes par catégorie ; chaque groupe donne une ligne de résultat.
`HAVING`	Filtre **après** le regroupement (le `WHERE` filtre **avant**).

C'est exactement le type de requête qui alimentera ton tableau de bord (chiffre d'affaires, nombre de commandes, produits par catégorie).

4.14 Les index

DÉFINITION SIMPLE

Un **index** est une « table des matières » d'une colonne : il permet à PostgreSQL de retrouver une ligne sans lire toute la table.

09_indexes.sql SQL

```sql
CREATE INDEX idx_products_category ON products (category_id);
CREATE INDEX idx_products_active   ON products (created_at DESC) WHERE deleted_at IS NULL;
```

Les colonnes `PRIMARY KEY` et `UNIQUE` ont déjà un index automatique. Ajoute-en un sur chaque **clé étrangère** et sur les colonnes que tu filtres ou tries souvent. Le second exemple est un index **partiel** : il ne couvre que les produits non supprimés.

ATTENTION — Trop d'index nuit

Chaque index ralentit légèrement les écritures et occupe de l'espace. Pour une boutique de 9 produits, il ne change rien ; il devient utile à partir de milliers de lignes. N'en ajoute que là où un besoin réel existe.

4.15 Les transactions

DÉFINITION SIMPLE

Une **transaction** regroupe plusieurs instructions en un bloc **indivisible** : soit toutes réussissent (`COMMIT`), soit aucune n'est appliquée (`ROLLBACK`).

Exemple concret : un client commande 2 T-shirts. Il faut **créer la commande ET diminuer le stock**. Si le serveur plante entre les deux, tu ne veux ni commande sans stock retiré, ni stock retiré sans commande.

10_transaction.sql SQL

```sql
BEGIN;

UPDATE products SET updated_at = now() WHERE slug = 'tshirt-noir-intense';
-- ... d'autres instructions liées ...

COMMIT;
-- ou, si quelque chose s'est mal passé :
-- ROLLBACK;
```

Instruction	Rôle
`BEGIN`	Ouvre la transaction. Les changements ne sont pas encore visibles par les autres.
`COMMIT`	**Valide** : tout devient définitif.
`ROLLBACK`	**Annule** tout ce qui a été fait depuis `BEGIN`.

C'est la protection n°1 contre les données incohérentes. Prisma (partie 6) fournit `prisma.$transaction(...)` pour faire la même chose depuis JavaScript.

4.16 Sauvegarder et restaurer

ATTENTION — Une base sans sauvegarde est une base qui sera perdue

Un disque qui lâche, une mauvaise manipulation ou un `DROP TABLE` malheureux : sans copie, les commandes de tes clients disparaissent. La partie 13 explique comment automatiser les sauvegardes.

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> pg_dump -U lawpiking_user -d lawpiking -F c -f lawpiking_2026-10-31.backup
Mot de passe :
PS C:\Users\mohamed> dir lawpiking_2026-10-31.backup
```

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed> createdb -U postgres -O lawpiking_user lawpiking_test
PS C:\Users\mohamed> pg_restore -U lawpiking_user -d lawpiking_test `
    --no-owner lawpiking_2026-10-31.backup
```

Élément	Explication
`pg_dump`	Programme de sauvegarde.
`-F c`	Format « custom » (compressé, restaurable avec `pg_restore`).
`-f`	Nom du fichier de sortie.
`pg_restore -d lawpiking_test`	Recharge la sauvegarde **dans une autre base**. Ici, une base de test, pour vérifier qu'elle est exploitable sans toucher aux vraies données.

BONNE PRATIQUE — Une sauvegarde non testée n'est pas une sauvegarde

Restaure-la au moins une fois dans une base de test et vérifie que les tables et les données sont là.

4.17 Les migrations

DÉFINITION SIMPLE

Une **migration** est un fichier qui décrit **un changement de structure** de la base (créer une table, ajouter une colonne). On les applique **dans l'ordre**, et on les conserve avec le code.

Au lieu de modifier la base « à la main » (impossible à reproduire sur un autre ordinateur ou en production), on écrit chaque changement dans un fichier daté : `2026-11-01_add_low_stock_threshold.sql`. Ainsi, n'importe quel développeur ou serveur peut recréer la base identique en rejouant les migrations dans l'ordre.

Dans ce guide, **Prisma génère et applique les migrations pour toi** (partie 6) : tu modifies `schema.prisma`, Prisma écrit le SQL. Tu dois cependant savoir le **lire**, puisque c'est celui de cette partie.

4.18 Récapitulatif

Je veux…	Je tape…
Créer une table	`CREATE TABLE ... ( ... );`
Ajouter des lignes	`INSERT INTO ... VALUES ...;`
Lire	`SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT ...;`
Modifier	`UPDATE ... SET ... WHERE ...;`
Supprimer	`DELETE FROM ... WHERE ...;`
Assembler deux tables	`... JOIN ... ON ...`
Compter / résumer	`COUNT`, `SUM`, `GROUP BY`
Tout ou rien	`BEGIN; ... COMMIT;`
Sauvegarder	`pg_dump`

À RETENIR — En résumé — Partie 4

- Une **base relationnelle** range les données dans des tables reliées par des clés primaires et étrangères.
- Les **contraintes** (`NOT NULL`, `UNIQUE`, `CHECK`, `REFERENCES`) protègent la base même si le code a un bug.
- `INSERT`, `SELECT`, `UPDATE`, `DELETE` : **toujours** un `WHERE` pour modifier ou supprimer ; `IS NULL` et non `= NULL`.
- Les **jointures** assemblent les tables ; `GROUP BY` et les agrégats résument ; les **index** accélèrent les recherches.
- Une **transaction** est tout ou rien ; une sauvegarde `pg_dump` non testée n'est pas une vraie sauvegarde.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Installer PostgreSQL et se connecter avec psql	Installer PostgreSQL
Créer une table avec ses contraintes	Créer des tables
Écrire INSERT, SELECT, UPDATE, DELETE	Insérer, lire, modifier
Faire une jointure et un regroupement	Jointures, agrégats
Sauvegarder et restaurer une base	Sauvegarder et restaurer

4.19 Exercices de la partie 4

QUESTIONS DE COMPRÉHENSION

1. Quelle est la différence entre une clé primaire et une clé étrangère ?
2. Pourquoi stocke-t-on le prix en entier pour le franc CFA ?
3. Pourquoi crée-t-on `promotion_products` au lieu d'une colonne `product_ids` dans `promotions` ?
4. Quelle est la différence entre `WHERE` et `HAVING` ?

EXERCICES SQL

5. Écris la requête qui affiche les produits dont le nom contient « rouge », triés par prix décroissant.
6. Écris la requête qui renvoie, pour chaque catégorie, le prix du produit le plus cher.
7. Écris l'`INSERT` d'une promotion « Soldes de fin d'année » de 20 % valable du 1er au 31 décembre 2026, puis l'`INSERT` qui y rattache tous les T-shirts.
8. Écris la requête qui liste les catégories qui n'ont **aucun** produit actif.

DÉBOGAGE

9. `UPDATE products SET price = 15000;` a été exécuté sans `WHERE`. Que s'est-il passé, et que fallait-il faire avant ? Comment se protéger (transaction) ?
10. `WHERE deleted_at = NULL` ne renvoie aucune ligne alors que des produits ne sont pas supprimés. Pourquoi ? Corrige.

MINI-PROJET

11. Crée la table `settings` (`key` texte clé primaire, `value` JSONB non nul, `updated_at`), insère le réglage `whatsapp_number` et le réglage `shop_name`, puis écris la requête qui lit un réglage par sa clé.

PARTIE 5

Conception de la base LAWPIKING

Transformer la boutique actuelle en un modèle de données solide : 11 tables, leurs relations et leurs règles.

5.1 Partir de ce que le projet fait vraiment

Une bonne base se conçoit **à partir des besoins**, pas à partir des tables imaginées. Voici ce que ton frontend gère déjà, et qui devra donc être stocké :

Fonction du frontend	Données à conserver	Table(s)
Connexion de l'administrateur	Email, mot de passe haché, rôle	`users`
Catalogue : T-shirts, Pulls, Bonnets	Nom, slug, description, état actif	`categories`
Fiche produit	Nom, description, prix, catégorie, image	`products`, `product_images`
Tailles et couleurs	Combinaisons taille/couleur, stock par combinaison, seuil d'alerte	`product_variants`
Commande par WhatsApp	Client, articles, total, état, canal	`orders`, `order_items`
Avis clients	Note, commentaire, état de modération	`reviews`
Promotions	Pourcentage, dates, produits concernés	`promotions`, `promotion_products`
Paramètres	Numéro WhatsApp, nom de la boutique…	`settings`

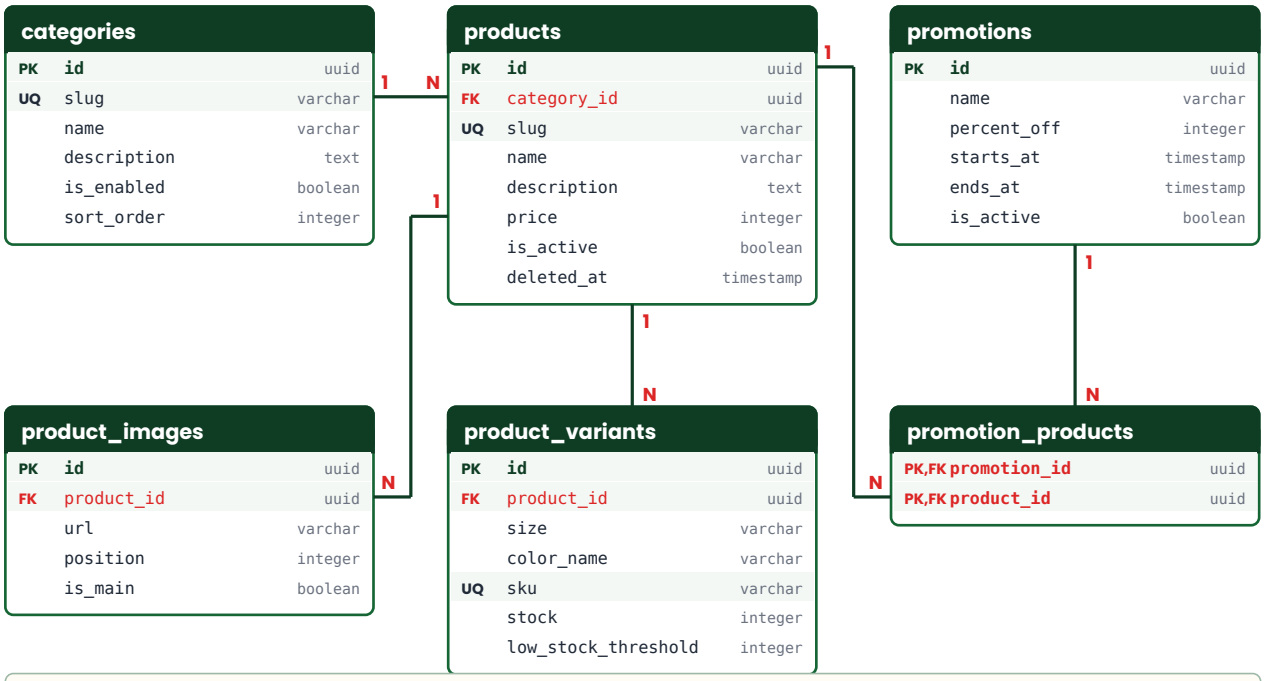
5.2 Les principes de conception

1. **Une information, un seul endroit.** Le nom d'une catégorie n'est écrit que dans `categories`. Les produits y renvoient par une clé étrangère. Changer « T-shirts » en « Tees » se fait en **une** ligne.
2. **Pas de doublons évitables.** Contraintes `UNIQUE` sur les `slug`, l'`email`, la combinaison taille/couleur d'un produit.
3. **Ce qui est dérivé n'est pas stocké.** Ton frontend le fait déjà : le statut « rupture / stock bas / en stock » se **calcule** depuis le stock et le seuil. On ne stocke jamais `status` : il pourrait contredire le stock.
4. **L'historique est figé.** Une commande conserve le **nom** et le **prix** payés au moment de l'achat (voir plus bas).
5. **On ne détruit pas ce qui est référencé.** Produits retirés : suppression douce.
6. **La base se défend elle-même** : `NOT NULL`, `CHECK`, `UNIQUE`, clés étrangères.

5.3 Les deux diagrammes de la base

Pour rester lisible, le modèle est présenté en deux diagrammes. Le **catalogue** (ce que l'on vend) :

ERD 1 — LE CATALOGUE : CATÉGORIES, PRODUITS, IMAGES, VARIANTES ET PROMOTIONS



LÉGENDE

PK	clé primaire : identifiant unique de la ligne
FK	clé étrangère : renvoie vers une autre table
FK?	clé étrangère facultative (peut être vide)
UQ	valeur unique : aucun doublon possible
1 / N	côté « un » / côté « plusieurs » de la relation

Les sizes et colors des variantes ne sont pas dans products : une variante = une combinaison vendable.
promotion_products est la table de liaison (relation plusieurs-à-plusieurs) entre promotions et products.

> **NOTE — Lire un diagramme**
> Chaque boîte est une table. Une ligne entre deux boîtes est une relation. Les symboles de bout de ligne indiquent « un » ou « plusieurs » : une catégorie a **plusieurs** produits ; un produit a **plusieurs** variantes et **plusieurs** images. Les petites boîtes en pointillés rappellent une table déjà dessinée dans l'autre diagramme.

Les relations du catalogue, en toutes lettres :

Relation	Type	Signification	Si on supprime le parent
`categories` → `products`	1-N	Une catégorie contient plusieurs produits	**Refusé** tant qu'il reste des produits
`products` → `product_images`	1-N	Un produit a plusieurs images, dont une principale	Les images sont supprimées (`CASCADE`)
`products` → `product_variants`	1-N	Un produit a une variante par taille et couleur	Les variantes sont supprimées (`CASCADE`)
`promotions` ↔ `products`	N-N	Via `promotion_products`	Seules les lignes de liaison disparaissent

Les **ventes et la gestion** (qui achète, ce qui est acheté, avis, réglages) :

ERD 2 — LES VENTES ET L'ADMINISTRATION : UTILISATEURS, COMMANDES, AVIS, RÉGLAGES

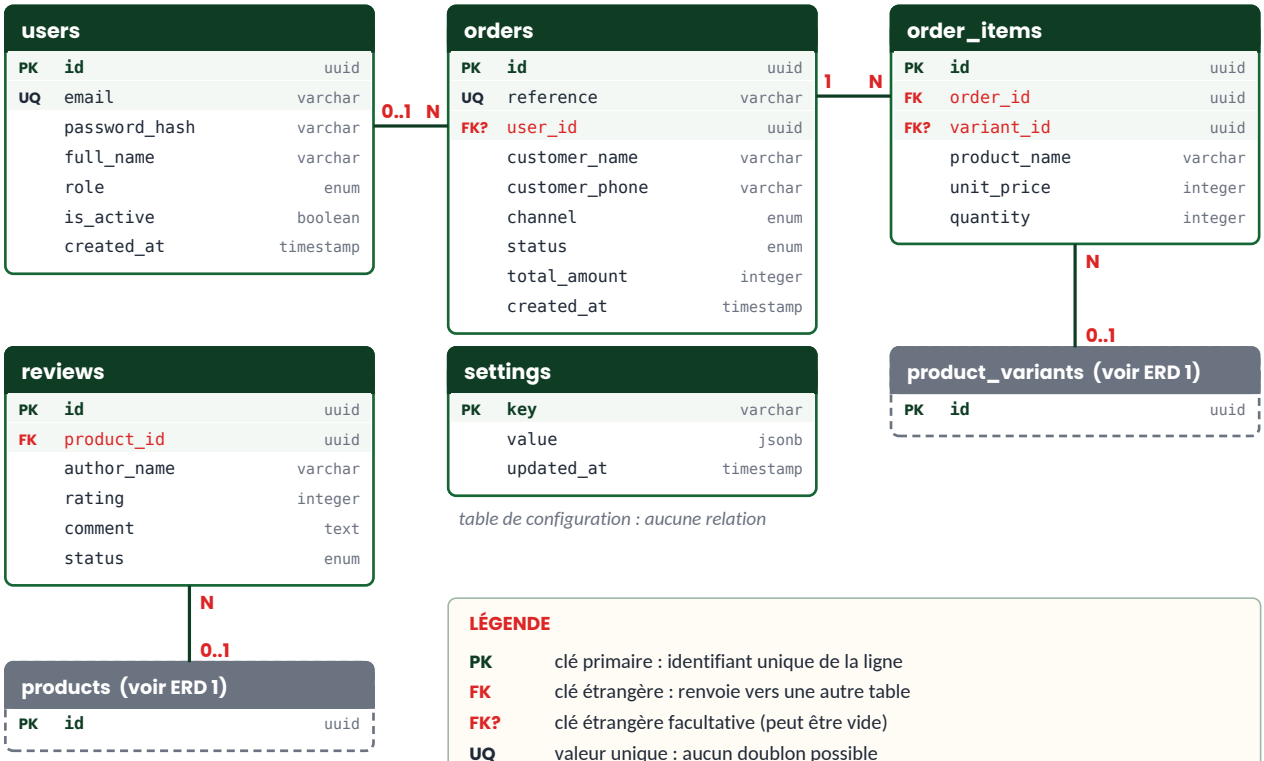


table de configuration : aucune relation

LÉGENDE

PK	clé primaire : identifiant unique de la ligne
FK	clé étrangère : renvoie vers une autre table
FK?	clé étrangère facultative (peut être vide)
UQ	valeur unique : aucun doublon possible
1 / N	côté « un » / côté « plusieurs » de la relation

Une commande garde son prix et son nom de produit : elles ne changent pas si le catalogue évolue.

5.4 Tailles, couleurs et stock : les variantes

C'est la décision de conception la plus importante. Aujourd'hui, ton formulaire produit enregistre **un seul stock par produit** et une liste de tailles et de couleurs. Mais un client commande « *NOIR INTENSE*, *taille M* » : si le stock global est 15, combien en reste-t-il en M ? On ne peut pas le savoir.

SCHÉMA — UN PRODUIT, PLUSIEURS VARIANTES (TAILLE × COULEUR)

products

```
id = 5
name = NOIR INTENSE
price = 10000
```

1 — N

sku	taille	couleur	stock	seuil
LWP-NOIR-M	M	Noir	15	10
LWP-NOIR-L	L	Noir	8	10
LWP-NOIR-XL	XL	Noir	0	10

Une ligne = une combinaison vendable avec SON propre stock.

Pourquoi pas une colonne « tailles » et une colonne « stock » dans products ?

Parce que le stock n'est pas celui du produit, mais celui de chaque combinaison : le noir en M peut être disponible alors que le noir en XL est en rupture. Les variantes portent le stock, le produit porte le reste.

La solution est la table `product_variants` : une ligne par combinaison taille + couleur, avec **son** stock.

product_id	size	color	stock
NOIR INTENSE	M	Noir	6
NOIR INTENSE	L	Noir	9
NOIR INTENSE	XL	Noir	0

Le stock total du produit est alors la **somme** (`SUM(stock)`) de ses variantes, calculée à la demande.

ATTENTION — Choix d'évolution

Le frontend actuel n'a qu'un stock par produit. Le modèle proposé permet d'aller plus loin sans rien casser : tant que tu ne gères pas les tailles individuellement, tu peux créer **une seule variante par produit** (taille « UNIQUE ») portant tout le stock. Tu choisis le niveau de détail quand tu connecteras la page Stock.

5.5 La suppression douce et les produits désactivés

Deux notions distinctes, qu'il ne faut pas confondre :

Notion	Colonne	Effet	Exemple
Désactivé	`is_active = FALSE`	Masqué de la boutique, visible dans l'admin, réactivable d'un clic	Produit en rupture de saison
Supprimé (doux)	`deleted_at` renseigné	Masqué partout, sauf dans l'historique des commandes	Produit retiré du catalogue

Une commande passée garde un lien vers son produit via `order_items.product_id`. Si on supprimait réellement le produit, la commande perdrait son sens. D'où `deleted_at`.

5.6 Les commandes figent le prix et le nom

> **EXEMPLE LAWPIKING — Pourquoi order_items copie le prix**
> Le 1er novembre, NOIR INTENSE coûte 10 000 FCFA et Awa en commande 2. Le 15 novembre, tu passes le prix à 12 000 FCFA. Si `order_items` ne contenait que `product_id`, le total de la commande d'Awa **changerait rétroactivement** pour 24 000 FCFA, et ta comptabilité serait fausse. En stockant `unit_price` et `product_name` **au moment de l'achat**, la commande d'Awa vaudra toujours 20 000 FCFA.

Cette copie volontaire s'appelle un **instantané** (*snapshot*). C'est l'une des rares duplications d'information que l'on **veut**.

5.7 Les rôles : ADMIN et CUSTOMER

La table `users` possède une colonne `role`. Dans LAWPIKING, **seul l'administrateur se connecte**, puisque les clients commandent par WhatsApp sans compte. Le rôle `CUSTOMER` est prévu pour l'avenir (comptes clients) ; il n'y a pas d'inscription publique : l'administrateur est créé par un **script de démarrage** (partie 8).

5.8 Les commandes WhatsApp

Ton site génère un message WhatsApp et ouvre la conversation ; il **n'enregistre rien** aujourd'hui. Avec un backend, on enregistre la commande **avant** d'ouvrir WhatsApp. La table `orders` contient donc le nom et le téléphone du client, un **canal** (`WHATSAPP` ; d'autres comme `WEBSITE` pourront venir) et un **statut** (`PENDING`, `CONFIRMED`, `DELIVERED`, `CANCELLED`). Ainsi, l'administrateur retrouve toutes les commandes dans la page Commandes.

5.9 Le SQL complet : les 11 tables, étape par étape

Le script s'exécute **dans l'ordre** : une table doit exister avant que d'autres s'y réfèrent. Nous le découpons en 10 étapes, chacune expliquée ligne par ligne.

Étape 1 : extensions et types

etape01_types.sql SQL

```sql
CREATE TYPE user_role    AS ENUM ('ADMIN', 'CUSTOMER');
CREATE TYPE order_status AS ENUM ('PENDING', 'CONFIRMED', 'DELIVERED', 'CANCELLED');
CREATE TYPE order_channel AS ENUM ('WHATSAPP', 'WEBSITE');
CREATE TYPE review_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED');
```

Un **type ENUM** est une liste fermée de valeurs autorisées. La base **refuse** `status = 'expediee'` si cette valeur n'est pas dans la liste. C'est plus sûr qu'un simple texte.

Étape 2 : users

etape02_users.sql SQL

```sql
CREATE TABLE users (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         VARCHAR(160) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  full_name     VARCHAR(80)  NOT NULL,
  role          user_role    NOT NULL DEFAULT 'CUSTOMER',
```

```
  is_active     BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

Ligne	Explication
`email ... UNIQUE`	Deux comptes ne partagent pas le même email.
`password_hash`	Le mot de passe **haché** (bcrypt). Il n'existe **aucune** colonne `password` : on ne stocke jamais un mot de passe en clair.
`role user_role`	Utilise le type ENUM de l'étape 1.
`is_active`	Permet de bloquer un compte sans le supprimer.

Étape 3 : categories

etape03_categories.sql SQL

```sql
CREATE TABLE categories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        VARCHAR(40) NOT NULL UNIQUE,
  name        VARCHAR(60) NOT NULL,
  description TEXT        NOT NULL DEFAULT '',
  is_enabled  BOOLEAN     NOT NULL DEFAULT TRUE,
  sort_order  INTEGER     NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_categories_name ON categories (lower(name));
```

La dernière ligne crée un **index unique sur le nom en minuscules** : « T-shirts » et « t-SHIRTS » sont considérés comme le même nom, comme le fait déjà la validation de ton frontend (qui ignore aussi les majuscules).

Étape 4 : products

etape04_products.sql SQL

```sql
CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id UUID         NOT NULL REFERENCES categories(id),
  slug        VARCHAR(100) NOT NULL UNIQUE,
  name        VARCHAR(80)  NOT NULL,
  description TEXT         NOT NULL DEFAULT '',
  price       INTEGER      NOT NULL CHECK (price > 0 AND price <= 10000000),
  is_active   BOOLEAN      NOT NULL DEFAULT TRUE,
  deleted_at  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX idx_products_category ON products (category_id);
CREATE UNIQUE INDEX uq_products_name ON products (lower(name)) WHERE deleted_at IS NULL;
```

Les limites (80 caractères, prix maximum 10 000 000) reprennent **exactement** celles de `productValidation.js` dans ton frontend. L'index unique partiel garantit qu'**aucun produit actif n'a le même nom** qu'un autre, tout en autorisant à réutiliser le nom d'un produit supprimé.

Étape 5 : product_images

etape05_product_images.sql SQL

```sql
CREATE TABLE product_images (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID         NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  url        VARCHAR(300) NOT NULL,
  alt_text   VARCHAR(160) NOT NULL DEFAULT '',
  position   INTEGER      NOT NULL DEFAULT 0,
  is_main    BOOLEAN      NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_product_images_product ON product_images (product_id);
CREATE UNIQUE INDEX uq_product_main_image ON product_images (product_id) WHERE is_main;
```

La table stocke **le chemin** du fichier (`/uploads/products/noir-intense-1.webp`), jamais l'image elle-même. L'index unique partiel garantit **une seule image principale** par produit.

Étape 6 : product_variants

etape06_product_variants.sql SQL

```sql
CREATE TABLE product_variants (
  id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id         UUID        NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  size               VARCHAR(10) NOT NULL,
  color_name         VARCHAR(40) NOT NULL,
  color_hex          CHAR(7)     NOT NULL CHECK (color_hex ~ '^#[0-9A-Fa-f]{6}$'),
  sku                VARCHAR(40) UNIQUE,
  stock              INTEGER     NOT NULL DEFAULT 0 CHECK (stock >= 0),
  low_stock_threshold INTEGER    NOT NULL DEFAULT 5 CHECK (low_stock_threshold >= 0),
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (product_id, size, color_name)
);
```

Ligne	Explication
`color_hex ... CHECK (color_hex ~ '^#...$')`	`~` teste une **expression régulière** : la couleur doit ressembler à `#1A2B3C`.
`sku`	Référence interne facultative, unique.
`stock >= 0`	Le stock **ne peut pas** devenir négatif : la base refuse une vente impossible.
`UNIQUE (product_id, size, color_name)`	Interdit de créer deux fois « NOIR INTENSE, M, Noir ».

Étape 7 : promotions et promotion_products

etape07_promotions.sql SQL

```sql
CREATE TABLE promotions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(80)  NOT NULL,
  percent_off INTEGER      NOT NULL CHECK (percent_off BETWEEN 1 AND 90),
  starts_at   TIMESTAMPTZ  NOT NULL,
  ends_at     TIMESTAMPTZ  NOT NULL,
  is_active   BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  CHECK (ends_at > starts_at)
);
CREATE TABLE promotion_products (
```

```
  promotion_id UUID NOT NULL REFERENCES promotions(id) ON DELETE CASCADE,
  product_id   UUID NOT NULL REFERENCES products(id)   ON DELETE CASCADE,
  PRIMARY KEY (promotion_id, product_id)
);
```

Le statut « à venir / en cours / terminée » d'une promotion est **calculé** à partir de `starts_at`, `ends_at` et de l'heure actuelle, jamais stocké.

Étape 8 : orders et order_items

etape08_orders.sql SQL

```sql
CREATE TABLE orders (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reference      VARCHAR(20)   NOT NULL UNIQUE,
  user_id        UUID REFERENCES users(id) ON DELETE SET NULL,
  customer_name  VARCHAR(80)   NOT NULL,
  customer_phone VARCHAR(30)   NOT NULL,
  customer_note  TEXT          NOT NULL DEFAULT '',
  channel        order_channel NOT NULL DEFAULT 'WHATSAPP',
  status         order_status  NOT NULL DEFAULT 'PENDING',
  total_amount   INTEGER       NOT NULL CHECK (total_amount >= 0),
  created_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at     TIMESTAMPTZ   NOT NULL DEFAULT now()
);
CREATE INDEX idx_orders_status_date ON orders (status, created_at DESC);

CREATE TABLE order_items (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id     UUID         NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id   UUID         REFERENCES products(id) ON DELETE SET NULL,
  variant_id   UUID         REFERENCES product_variants(id) ON DELETE SET NULL,
  product_name VARCHAR(80)  NOT NULL,
  size         VARCHAR(10)  NOT NULL,
  color_name   VARCHAR(40)  NOT NULL,
  unit_price   INTEGER      NOT NULL CHECK (unit_price >= 0),
  quantity     INTEGER      NOT NULL CHECK (quantity > 0)
);
CREATE INDEX idx_order_items_order ON order_items (order_id);
```

Élément	Explication
`reference`	Un numéro lisible, par exemple `LP-20261031-0042`, à communiquer au client.
`user_id ... ON DELETE SET NULL`	Un client sans compte (cas actuel) laisse `user_id` vide. Si le compte est supprimé, la commande reste.
`total_amount`	Total **figé** de la commande.
`product_name`, `size`, `color_name`, `unit_price`	L'**instantané** : valeurs copiées au moment de l'achat.
`ON DELETE CASCADE` sur `order_id`	Supprimer une commande supprime ses lignes.
`ON DELETE SET NULL` sur `product_id`	L'historique survit même si le produit est supprimé.

Étape 9 : reviews

etape09_reviews.sql SQL

```sql
CREATE TABLE reviews (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id  UUID          NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  author_name VARCHAR(60)   NOT NULL,
  rating      INTEGER       NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment     TEXT          NOT NULL DEFAULT '',
  status      review_status NOT NULL DEFAULT 'PENDING',
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT now()
);
CREATE INDEX idx_reviews_product_status ON reviews (product_id, status);
```

Un avis arrive en `PENDING` : l'administrateur l'approuve ou le rejette depuis la page Avis. Seuls les avis `APPROVED` sont visibles dans la boutique.

Étape 10 : settings

etape10_settings.sql SQL

```sql
CREATE TABLE settings (
  key        VARCHAR(60) PRIMARY KEY,
  value      JSONB       NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
INSERT INTO settings (key, value) VALUES
  ('shop_name', '"LAWPIKING"'),
  ('whatsapp_number', '"221700000000"'),
  ('currency', '"XOF"');
```

Une table **clé / valeur** convient aux réglages : ajouter un nouveau réglage n'oblige pas à modifier la structure. Le type `JSONB` accepte un texte, un nombre ou un objet. Le numéro ci-dessus est un exemple : mets le tien.

5.10 Vérifier la cohérence

Après la création, ces commandes psql doivent donner la liste suivante :

TERMINAL Windows PowerShell

```
lawpiking=> \dt
```

RÉSULTAT résultat attendu

```
 Schéma |        Nom         | Type  |    Propriétaire
--------+--------------------+-------+--------------------
 public | categories         | table | lawpiking_user
 public | order_items        | table | lawpiking_user
 public | orders             | table | lawpiking_user
 public | product_images     | table | lawpiking_user
 public | product_variants   | table | lawpiking_user
 public | products           | table | lawpiking_user
 public | promotion_products | table | lawpiking_user
 public | promotions         | table | lawpiking_user
 public | reviews            | table | lawpiking_user
 public | settings           | table | lawpiking_user
 public | users              | table | lawpiking_user
(11 lignes)
```

Voici une requête qui vérifie que le modèle répond à une vraie question : « stock total et statut de chaque produit » (le statut dérivé de ton frontend) :

stock_par_produit.sql SQL

```sql
SELECT p.name,
       COALESCE(SUM(v.stock), 0) AS stock_total,
       CASE
         WHEN COALESCE(SUM(v.stock), 0) = 0 THEN 'rupture'
         WHEN COALESCE(SUM(v.stock), 0) <= MIN(v.low_stock_threshold) THEN 'stock bas'
         ELSE 'en stock'
       END AS statut
FROM products p
LEFT JOIN product_variants v ON v.product_id = p.id
WHERE p.deleted_at IS NULL
GROUP BY p.id, p.name;
```

NOTE — Dans la pratique

Dans le backend, ce calcul sera fait par le code JavaScript (comme `getStockStatus` dans ton frontend), et non en SQL. La requête sert à vérifier que le modèle **permet** de répondre.

5.11 Ce que le modèle ne prévoit pas (volontairement)

- **Pas de table** `carts` : le panier reste dans le navigateur jusqu'à la commande.
- **Pas de paiement en ligne** : le paiement se règle via WhatsApp.
- **Pas de table** `favorites` : les favoris restent dans le navigateur.

Ces choix suivent l'état réel du projet ; on pourra ajouter ces tables plus tard par migration.

À RETENIR — En résumé — Partie 5

- On conçoit la base **à partir des besoins** réels du projet : 11 tables, une information à un seul endroit.
- Le statut de stock ou de promotion est **calculé**, jamais stocké.
- `product_variants` porte taille, couleur et stock : le stock total est la somme des variantes.
- Une commande garde un **instantané** du nom et du prix payés ; un produit se retire par **suppression douce**.
- Le mot de passe n'existe qu'en **hachage** ; il n'y a pas d'inscription publique.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Justifier la présence de chaque table	Les principes de conception
Expliquer le rôle des variantes	Tailles, couleurs et stock
Expliquer l'instantané des commandes	Les commandes figent le prix
Lire le SQL complet des 11 tables	Le SQL complet
Dessiner les relations de la base	Les deux diagrammes

5.12 Exercices de la partie 5

QUESTIONS DE COMPRÉHENSION

1. Pourquoi ne stocke-t-on pas le statut « rupture / en stock » dans une colonne ?
2. Pourquoi `order_items` contient-il `unit_price` alors que `products.price` existe ?
3. Différence entre `is_active = FALSE` et `deleted_at` renseigné ?
4. Quelle relation (1-N, N-N) existe entre promotions et produits ? Comment est-elle représentée ?

EXERCICES SQL

5. Écris la requête qui renvoie les variantes en stock bas (stock > 0 et ≤ seuil), avec le nom du produit.
6. Écris la requête qui renvoie le chiffre d'affaires total des commandes `DELIVERED`.
7. Écris la requête qui renvoie la note moyenne des avis `APPROVED` de chaque produit.

DÉBOGAGE

8. Un `INSERT` dans `order_items` échoue avec « viole la contrainte de vérification ». Quelles valeurs peuvent en être la cause ?
9. Tu veux supprimer une catégorie qui contient des produits et PostgreSQL refuse. Pourquoi, et que proposes-tu ?

MINI-PROJET

10. Ajoute au modèle une table `contact_messages` (messages du formulaire de contact : nom, email, message, lu/non lu, date) en respectant les conventions de la partie.

PARTIE 6

Prisma ORM

Parler à PostgreSQL en JavaScript : schéma, migrations, requêtes, relations, transactions et erreurs.

6.1 Qu'est-ce que Prisma ?

DÉFINITION SIMPLE
Un **ORM** (*Object-Relational Mapper*) est un outil qui traduit les tables de la base en **objets JavaScript** et tes instructions JavaScript en **requêtes SQL**. **Prisma** est l'ORM que nous utilisons : il lit un fichier de description (`schema.prisma`), génère un **client** pratique et gère les **migrations** de la base.

Comparons : « les produits d'une catégorie, avec leurs variantes » en SQL, puis avec Prisma.

```sql
SELECT p.*, v.*
FROM products p
LEFT JOIN product_variants v ON v.product_id = p.id
WHERE p.category_id = '...' AND p.deleted_at IS NULL;
```

```js
const products = await prisma.product.findMany({
  where: { categoryId, deletedAt: null },
  include: { variants: true },
});
```

Prisma ne remplace pas le SQL : il le génère à ta place, avec trois avantages. **(1)** Ton éditeur te **suggère** les noms de champs. **(2)** Les requêtes sont **paramétrées** (protection contre les injections SQL, partie 8). **(3)** La structure de la base est décrite **à un seul endroit**, versionnée avec le code.

CHANGEMENT DE VERSION — Prisma 7 et Prisma 8 : lis ceci avant d'installer
Ce guide utilise Prisma 7, la version stable à la date de rédaction (octobre 2026). Prisma 8 est annoncé comme version majeure ; au moment de rédiger, il est encore en **candidat de sortie** (*release candidate*) et change profondément l'écriture des requêtes et des migrations. Pour que les commandes de ce guide fonctionnent, **installe explicitement la version 7** : `npm install prisma@7 --save-dev` et `npm install @prisma/client@7`. N'écris pas `@latest` tant que tu apprends. Prisma 7, par rapport aux tutoriels plus anciens (version 5 ou 6) : le **fichier** `prisma.config` remplace la configuration de la base dans `package.json`, la **connexion** se fait par un *driver adapter* (`@prisma/adapter-pg`), la **génération** du client (`prisma generate`) n'est plus lancée automatiquement, et `$use` est remplacé par `$extends`. Selon la sous-version 7.x, le CLI peut chercher un fichier nommé `prisma.config.js` **ou** `prisma7.config.js` : si Prisma affiche qu'il ne trouve pas sa configuration, renomme le fichier comme il l'indique. Dans tous les cas, la page de démarrage officielle (prisma.io/docs) fait foi.

6.2 Installation

Dans `lawpiking-backend` :

TERMINAL — Windows PowerShell

```
PS ...\lawpiking-backend> npm install @prisma/client@7 @prisma/adapter-pg pg dotenv
PS ...\lawpiking-backend> npm install prisma@7 --save-dev
PS ...\lawpiking-backend> npx prisma init --datasource-provider postgresql
```

Paquet	Rôle	Type
`prisma`	**L'outil en ligne de commande** : migrations, génération, Studio	Développement (`--save-dev`)
`@prisma/client`	**Le client** que ton code importe	Production
`@prisma/adapter-pg`	Le **pilote** qui relie Prisma à PostgreSQL	Production
`pg`	Le pilote PostgreSQL de Node.js utilisé par l'adapter	Production
`dotenv`	Lit le fichier `.env`	Production

`prisma init` crée le dossier `prisma/` avec un fichier `schema.prisma`, un fichier de configuration et complète `.env`. Assure-toi que `.env` contient bien ta `DATABASE_URL` (partie 4) et que `.env` figure dans `.gitignore`.

Le fichier de configuration indique à Prisma où trouver le schéma, les migrations et la base :

prisma.config.js — JavaScript

```
1  import "dotenv/config";
2  import { defineConfig, env } from "prisma/config";
3
4  export default defineConfig({
5    schema: "prisma/schema.prisma",
6    migrations: {
7      path: "prisma/migrations",
8      seed: "node prisma/seed.js",
9    },
10   datasource: {
11     url: env("DATABASE_URL"),
12   },
13 });
```

Ligne	Explication
1	Charge `.env` pour que `DATABASE_URL` soit disponible.
2	`defineConfig` aide l'éditeur à compléter les options ; `env()` lit une variable d'environnement.
5	Emplacement du schéma.
6-9	Dossier des migrations, et commande de **peuplement** initial (*seed*) lancée par `prisma db seed`.
10-12	L'adresse de la base : **ici**, plus dans le schéma.

6.3 Le fichier schema.prisma

C'est le cœur : il décrit la base **dans le langage de Prisma**. Voici d'abord la structure, puis un exemple.

prisma/schema.prisma (début) Prisma

```prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

enum UserRole {
  ADMIN
  CUSTOMER

  @@map("user_role")
}

enum OrderStatus {
  PENDING
  CONFIRMED
  DELIVERED
  CANCELLED

  @@map("order_status")
}
```

Bloc	Rôle
`generator client`	Indique **quoi générer** : le client JavaScript. `prisma-client-js` produit un client utilisable en JavaScript pur, importé depuis `@prisma/client`.
`datasource db`	Dit que la base est PostgreSQL (l'adresse vient de `prisma.config.js`).
`enum`	Une liste fermée de valeurs, qui correspond aux types `ENUM` de la partie 5.

> **NOTE — JavaScript ou TypeScript ?**
> Les tutoriels récents de Prisma utilisent TypeScript et un générateur nommé `prisma-client` qui écrit les fichiers dans ton projet. Comme LAWPIKING est en JavaScript, nous gardons le générateur `prisma-client-js`, qui reste compatible. Les **modèles** et les **requêtes** sont identiques dans les deux cas.

Un modèle, ligne par ligne

```prisma
model Product {
  id          String    @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  categoryId  String    @map("category_id") @db.Uuid
  slug        String    @unique @db.VarChar(100)
  name        String    @db.VarChar(80)
  description String    @default("")
  price       Int
  isActive    Boolean   @default(true) @map("is_active")
  deletedAt   DateTime? @map("deleted_at") @db.Timestamptz
  createdAt   DateTime  @default(now()) @map("created_at") @db.Timestamptz
  updatedAt   DateTime  @updatedAt @map("updated_at") @db.Timestamptz

  category    Category        @relation(fields: [categoryId], references: [id])
  images      ProductImage[]
```

```
  variants   ProductVariant[]
  promotions PromotionProduct[]

  @@index([categoryId])
  @@map("products")
}
```

Élément	Signification
`model Product`	Un modèle = une table. Nom en **PascalCase singulier** (`Product`).
`id String @id`	`@id` : clé primaire.
`@default(dbgenerated("gen_random_uuid()"))`	Valeur par défaut **calculée par PostgreSQL**, comme dans notre SQL.
`@db.Uuid`	Type exact dans la base (sinon Prisma créerait un `TEXT`).
`@map("category_id")`	Le champ s'appelle `categoryId` en JavaScript mais la colonne `category_id` en SQL.
`@@map("products")`	Le modèle `Product` correspond à la table `products`.
`String?` / `DateTime?`	Le `?` rend le champ **facultatif** (la colonne accepte `NULL`).
`@unique`	Contrainte d'unicité.
`@updatedAt`	Prisma met la date à jour **à chaque modification**.
`@@index([categoryId])`	Crée un index.
`category Category @relation(...)`	Champ de **relation** : n'existe pas comme colonne ; il permet d'écrire `include: { category: true }`. `fields` est la clé étrangère locale, `references` la clé de l'autre table.
`images ProductImage[]`	Le côté « plusieurs » : une liste de l'autre modèle.

Les types Prisma les plus courants :

Prisma	PostgreSQL	JavaScript
`String`	`TEXT` ou `VARCHAR(n)` (avec `@db.VarChar(n)`)	`string`
`Int`	`INTEGER`	`number`
`Boolean`	`BOOLEAN`	`boolean`
`DateTime`	`TIMESTAMP` (`@db.Timestamptz` pour le fuseau)	`Date`
`Json`	`JSONB`	objet
`enum`	`ENUM`	chaîne

Le schéma complet de LAWPIKING

Les 11 modèles correspondent exactement aux tables de la partie 5.

prisma/schema.prisma (modèles du catalogue) Prisma

```prisma
model Category {
  id          String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  slug        String   @unique @db.VarChar(40)
  name        String   @db.VarChar(60)
  description String   @default("")
  isEnabled   Boolean  @default(true) @map("is_enabled")
  sortOrder   Int      @default(0) @map("sort_order")
  createdAt   DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt   DateTime @updatedAt @map("updated_at") @db.Timestamptz

  products Product[]

  @@map("categories")
}

model ProductImage {
  id        String  @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  productId String  @map("product_id") @db.Uuid
  url       String  @db.VarChar(300)
  altText   String  @default("") @map("alt_text") @db.VarChar(160)
  position  Int     @default(0)
  isMain    Boolean @default(false) @map("is_main")

  product Product @relation(fields: [productId], references: [id], onDelete: Cascade)

  @@index([productId])
  @@map("product_images")
}

model ProductVariant {
  id                String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  productId         String   @map("product_id") @db.Uuid
  size              String   @db.VarChar(10)
  colorName         String   @map("color_name") @db.VarChar(40)
  colorHex          String   @map("color_hex") @db.Char(7)
  sku               String?  @unique @db.VarChar(40)
  stock             Int      @default(0)
  lowStockThreshold Int      @default(5) @map("low_stock_threshold")
  createdAt         DateTime @default(now()) @map("created_at") @db.Timestamptz
  updatedAt         DateTime @updatedAt @map("updated_at") @db.Timestamptz

  product    Product     @relation(fields: [productId], references: [id], onDelete: Cascade)
  orderItems OrderItem[]

  @@unique([productId, size, colorName])
  @@map("product_variants")
}

model Promotion {
  id         String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  name       String   @db.VarChar(80)
  percentOff Int      @map("percent_off")
  startsAt   DateTime @map("starts_at") @db.Timestamptz
  endsAt     DateTime @map("ends_at") @db.Timestamptz
  isActive   Boolean  @default(true) @map("is_active")
  createdAt  DateTime @default(now()) @map("created_at") @db.Timestamptz

  products PromotionProduct[]
```

```
59
60    @@map("promotions")
61  }
62
63  model PromotionProduct {
64    promotionId String @map("promotion_id") @db.Uuid
65    productId   String @map("product_id") @db.Uuid
66
67    promotion Promotion @relation(fields: [promotionId], references: [id], onDelete: Cascade)
68    product   Product   @relation(fields: [productId], references: [id], onDelete: Cascade)
69
70    @@id([promotionId, productId])
71    @@map("promotion_products")
72  }
```

prisma/schema.prisma (ventes et gestion) Prisma

```
1   model User {
2     id           String   @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
3     email        String   @unique @db.VarChar(160)
4     passwordHash String   @map("password_hash") @db.VarChar(255)
5     fullName     String   @map("full_name") @db.VarChar(80)
6     role         UserRole @default(CUSTOMER)
7     isActive     Boolean  @default(true) @map("is_active")
8     createdAt    DateTime @default(now()) @map("created_at") @db.Timestamptz
9     updatedAt    DateTime @updatedAt @map("updated_at") @db.Timestamptz
10
11    orders Order[]
12
13    @@map("users")
14  }
15
16  model Order {
17    id            String       @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
18    reference     String       @unique @db.VarChar(20)
19    userId        String?      @map("user_id") @db.Uuid
20    customerName  String       @map("customer_name") @db.VarChar(80)
21    customerPhone String       @map("customer_phone") @db.VarChar(30)
22    customerNote  String       @default("") @map("customer_note")
23    channel       OrderChannel @default(WHATSAPP)
24    status        OrderStatus  @default(PENDING)
25    totalAmount   Int          @map("total_amount")
26    createdAt     DateTime     @default(now()) @map("created_at") @db.Timestamptz
27    updatedAt     DateTime     @updatedAt @map("updated_at") @db.Timestamptz
28
29    user  User?       @relation(fields: [userId], references: [id], onDelete: SetNull)
30    items OrderItem[]
31
32    @@index([status, createdAt(sort: Desc)])
33    @@map("orders")
34  }
35
36  model OrderItem {
37    id          String  @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
38    orderId     String  @map("order_id") @db.Uuid
39    productId   String? @map("product_id") @db.Uuid
40    variantId   String? @map("variant_id") @db.Uuid
41    productName String  @map("product_name") @db.VarChar(80)
42    size        String  @db.VarChar(10)
43    colorName   String  @map("color_name") @db.VarChar(40)
44    unitPrice   Int     @map("unit_price")
45    quantity    Int
46
```

```
  order   Order          @relation(fields: [orderId], references: [id], onDelete: Cascade)
  product Product?       @relation(fields: [productId], references: [id], onDelete: SetNull)
  variant ProductVariant? @relation(fields: [variantId], references: [id], onDelete: SetNull)

  @@index([orderId])
  @@map("order_items")
}

model Review {
  id         String       @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  productId  String       @map("product_id") @db.Uuid
  authorName String       @map("author_name") @db.VarChar(60)
  rating     Int
  comment    String       @default("")
  status     ReviewStatus @default(PENDING)
  createdAt  DateTime     @default(now()) @map("created_at") @db.Timestamptz
  updatedAt  DateTime     @updatedAt @map("updated_at") @db.Timestamptz

  product Product @relation(fields: [productId], references: [id], onDelete: Cascade)

  @@index([productId, status])
  @@map("reviews")
}

model Setting {
  key       String   @id @db.VarChar(60)
  value     Json
  updatedAt DateTime @updatedAt @map("updated_at") @db.Timestamptz

  @@map("settings")
}

enum OrderChannel {
  WHATSAPP
  WEBSITE

  @@map("order_channel")
}

enum ReviewStatus {
  PENDING
  APPROVED
  REJECTED

  @@map("review_status")
}
```

Pour que ce schéma soit complet, le modèle `Product` doit aussi déclarer les relations inverses vers `OrderItem` et `Review` :

```
orderItems OrderItem[]
reviews    Review[]
```

ATTENTION — Les relations vont toujours par deux

Prisma exige que **chaque relation soit déclarée des deux côtés** : `Product.variants` d'un côté, `ProductVariant.product` de l'autre. Si tu en oublies un, `prisma validate` (voir plus bas) t'indique laquelle.

> **NOTE — Ce que Prisma ne sait pas écrire**
> Certaines règles de la partie 5 (`CHECK (price > 0)`, index uniques partiels, `lower(name)`) n'ont pas d'équivalent dans `schema.prisma`. On les **ajoute à la main** dans le fichier SQL de la migration (voir ci-dessous) : Prisma permet de modifier une migration avant de l'appliquer. Le backend valide de toute façon ces règles avec Zod (partie 3).

6.4 Commandes essentielles de Prisma

Commande	Rôle	Quand
`npx prisma validate`	Vérifie que le schéma est correct	Après toute modification
`npx prisma format`	Met en forme le schéma et complète les relations	Avant de valider
`npx prisma migrate dev --name nom`	**Crée** une migration à partir du schéma et l'**applique** à la base locale	En développement
`npx prisma migrate deploy`	Applique les migrations **existantes** sans en créer	En production (partie 13)
`npx prisma generate`	Génère le **client** dans `node_modules`	Après chaque changement de schéma
`npx prisma db seed`	Lance le script de peuplement	Après une nouvelle base
`npx prisma studio`	Ouvre une interface web pour consulter les données	Pour explorer
`npx prisma migrate reset`	**Détruit** la base de développement, rejoue tout, relance le seed	Repartir de zéro (jamais en production)

> **CHANGEMENT DE VERSION — Depuis Prisma 7, `generate` n'est plus automatique**
> Avec les anciennes versions, `migrate dev` régénérait le client sans rien dire. Avec Prisma 7, tu lances explicitement `npx prisma generate` après chaque changement de schéma, et le seed avec `npx prisma db seed`. Si ton code ne « voit » pas un nouveau champ, c'est presque toujours que tu as oublié `generate`.

Créer la première migration

TERMINAL — Windows PowerShell

```
PS ...\lawpiking-backend> npx prisma migrate dev --name init
Applying migration `20261101090000_init`

The following migration(s) have been created and applied from new schema changes:

prisma/migrations/
  └─ 20261101090000_init/
    └─ migration.sql

Your database is now in sync with your schema.
```

Prisma a comparé ton schéma à la base (vide), a **écrit** le fichier `migration.sql` (le SQL de la partie 5, en version générée) et l'a **exécuté**. Ouvre ce fichier : tu y retrouveras les `CREATE TABLE` que tu sais lire. Garde-le dans Git : c'est l'historique de ta base.

Pour ajouter les règles que Prisma ne sait pas écrire, utilise `--create-only` : Prisma crée le fichier **sans l'appliquer**, tu le complètes, puis tu lances `npx prisma migrate dev`.

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npx prisma migrate dev --create-only --name add_checks
```

prisma/migrations/..._add_checks/migration.sql SQL

```sql
ALTER TABLE products
  ADD CONSTRAINT products_price_check CHECK (price > 0 AND price <= 10000000);
ALTER TABLE product_variants
  ADD CONSTRAINT variants_stock_check CHECK (stock >= 0);
ALTER TABLE reviews
  ADD CONSTRAINT reviews_rating_check CHECK (rating BETWEEN 1 AND 5);
CREATE UNIQUE INDEX uq_categories_name ON categories (lower(name));
CREATE UNIQUE INDEX uq_products_name ON products (lower(name)) WHERE deleted_at IS NULL;
```

ERREUR FRÉQUENTE — « Drift detected » ou « The migration is failed »

Tu as modifié la base **à la main** (avec pgAdmin ou psql) en dehors des migrations : elle ne correspond plus à l'historique. En développement, `npx prisma migrate reset` règle le problème (au prix de **toutes** les données locales). En production, on ne modifie jamais la base à la main.

6.5 Le client Prisma

Le client est créé **une seule fois**, puis réutilisé partout. Créons `src/config/prisma.js` :

src/config/prisma.js JavaScript

```js
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";
import { env } from "./env.js";

const adapter = new PrismaPg({ connectionString: env.databaseUrl });

export const prisma = new PrismaClient({
  adapter,
  log: env.nodeEnv === "development" ? ["warn", "error"] : ["error"],
});
```

Ligne	Explication
4	L'**adapter** ouvre les connexions à PostgreSQL avec l'adresse de `.env`.
6-9	Le client reçoit l'adapter. Il écrit dans la console les avertissements et les erreurs, et plus de détails en développement.
6	`export const prisma` : **un seul** objet partagé dans toute l'application. Créer un client par requête épuiserait les connexions de la base.

ERREUR FRÉQUENTE — « Cannot find module '.prisma/client' » ou « @prisma/client did not initialize yet »

Le client n'a pas été généré. Lance `npx prisma generate`, puis relance le serveur.

Fermer proprement la connexion

src/server.js (ajout) JavaScript

```
const shutdown = async () => {
  await prisma.$disconnect();
  process.exit(0);
};
process.on("SIGINT", shutdown);
process.on("SIGTERM", shutdown);
```

`SIGINT` correspond à `Ctrl + C` ; `SIGTERM` à l'arrêt demandé par un hébergeur. Ainsi, les connexions sont fermées avant l'arrêt.

6.6 Peupler la base : le script seed

Pour ne pas partir d'une base vide, un script crée l'administrateur, les catégories, quelques produits et les réglages. Ce script tourne **à la demande**.

prisma/seed.js JavaScript

```
import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";

const prisma = new PrismaClient({
  adapter: new PrismaPg({ connectionString: process.env.DATABASE_URL }),
});

async function main() {
  const tshirts = await prisma.category.upsert({
    where: { slug: "tshirts" },
    update: {},
    create: { slug: "tshirts", name: "T-shirts", description: "Les T-shirts LAWPIKING" },
  });
  await prisma.category.upsert({
    where: { slug: "pulls" },
    update: {},
    create: { slug: "pulls", name: "Pulls" },
  });
  await prisma.category.upsert({
    where: { slug: "bonnets" },
    update: {},
    create: { slug: "bonnets", name: "Bonnets" },
  });

  await prisma.product.upsert({
    where: { slug: "tshirt-noir-intense" },
    update: {},
    create: {
      slug: "tshirt-noir-intense",
      name: "NOIR INTENSE",
      price: 10000,
      categoryId: tshirts.id,
      variants: {
        create: [
          { size: "M", colorName: "Noir", colorHex: "#111111", stock: 6 },
          { size: "L", colorName: "Noir", colorHex: "#111111", stock: 9 },
        ],
      },
    },
  });
```

```
42
43   await prisma.setting.upsert({
44     where: { key: "shop_name" },
45     update: {},
46     create: { key: "shop_name", value: "LAWPIKING" },
47   });
48   console.log("Seed terminé.");
49 }
50
51 main()
52   .catch((error) => {
53     console.error(error);
54     process.exitCode = 1;
55   })
56   .finally(() => prisma.$disconnect());
```

`upsert` signifie « **mettre à jour ou créer** » : si le `slug` existe déjà, `update: {}` ne change rien ; sinon, `create` crée la ligne. On peut donc relancer le seed sans créer de doublons. L'administrateur est créé en partie 8, car il demande un mot de passe haché.

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npx prisma db seed
Seed terminé.
```

6.7 Les opérations CRUD avec Prisma

Chaque modèle devient une propriété du client, en **camelCase** : `prisma.product`, `prisma.category`, `prisma.orderItem`. Toutes les méthodes sont **asynchrones** : on utilise `await`.

Opération	Méthode	Résultat si rien n'est trouvé
Lire une ligne	`findUnique({ where: { id } })`	`null`
Lire la première correspondance	`findFirst({ where })`	`null`
Lire une liste	`findMany({ where, orderBy, skip, take })`	`[]`
Créer	`create({ data })`	—
Modifier	`update({ where, data })`	**Exception** `P2025`
Supprimer	`delete({ where })`	**Exception** `P2025`
Créer ou modifier	`upsert({ where, create, update })`	—
Compter	`count({ where })`	`0`
Modifier / supprimer plusieurs	`updateMany`, `deleteMany`	`{ count: 0 }`
Résumer	`aggregate`, `groupBy`	—

Créer

```
const product = await prisma.product.create({
  data: {
    slug: "tshirt-vert-menthe",
    name: "VERT MENTHE",
    price: 10000,
    category: { connect: { slug: "tshirts" } },
  },
});
```

`connect` relie le produit à une catégorie **existante** (au lieu de fournir `categoryId` à la main). Prisma renvoie l'objet créé, avec son `id` généré.

Lire : where, select, include, orderBy

```
const products = await prisma.product.findMany({
  where: {
    deletedAt: null,
    isActive: true,
    name: { contains: "noir", mode: "insensitive" },
    price: { gte: 5000, lte: 20000 },
    category: { slug: "tshirts" },
  },
  orderBy: [{ createdAt: "desc" }, { name: "asc" }],
  include: { variants: true, images: { orderBy: { position: "asc" } } },
});
```

Option	Rôle	Équivalent SQL
`where`	Filtre	`WHERE`
`contains` + `mode: "insensitive"`	Texte contenu, sans tenir compte des majuscules	`ILIKE '%noir%'`
`gte`, `lte`, `gt`, `lt`, `equals`, `in`, `not`	Comparaisons	`>=`, `<=`, `>`, `<`, `=`, `IN`, `<>`
`AND`, `OR`, `NOT`	Combinaisons	`AND`, `OR`, `NOT`
`category: { slug: "tshirts" }`	Filtre **sur la relation**	`JOIN ... WHERE`
`orderBy`	Tri	`ORDER BY`
`include`	Ajoute les **relations** à la réponse	`JOIN`
`select`	Choisit les **champs** renvoyés	liste de colonnes

ATTENTION — include ou select, pas les deux au même niveau

On ne peut pas mélanger `select` et `include` dans la même requête au même niveau. Utilise `select` pour limiter les champs (utile pour ne **jamais renvoyer** `passwordHash`), et `include` pour ajouter des relations complètes. Dans `select`, tu peux choisir des relations : `select: { name: true, variants: { select: { size: true, stock: true } } }`.

Modifier et supprimer

```
const updated = await prisma.product.update({
  where: { id },
  data: { price: 12000 },
});

await prisma.productVariant.update({
  where: { id: variantId },
  data: { stock: { decrement: 2 } },
});

await prisma.product.update({
  where: { id },
  data: { deletedAt: new Date(), isActive: false },
});
```

`{ decrement: 2 }` (ou `increment`) modifie la valeur **directement dans la base** (`stock = stock - 2`) : c'est sûr même si deux commandes arrivent en même temps, contrairement à lire le stock en JavaScript, soustraire, puis réécrire. La dernière requête est la **suppression douce** de la partie 5.

Pagination et comptage

```
const [total, products] = await prisma.$transaction([
  prisma.product.count({ where }),
  prisma.product.findMany({ where, skip, take: limit, orderBy: { createdAt: "desc" } }),
]);
```

`skip` et `take` sont ceux de `getPagination` (partie 3). La liste et le total sont lus dans **la même transaction** pour être cohérents.

Agrégats

```
const stockByProduct = await prisma.productVariant.groupBy({
  by: ["productId"],
  _sum: { stock: true },
});

const revenue = await prisma.order.aggregate({
  where: { status: "DELIVERED" },
  _sum: { totalAmount: true },
  _count: true,
});
```

Ce sont les équivalents de `GROUP BY` et de `SUM`/`COUNT` : exactement ce qu'il faut pour le tableau de bord.

6.8 Les transactions Prisma

Passer une commande demande trois actions **indivisibles** : vérifier le stock, créer la commande et ses lignes, retirer le stock.

src/modules/orders/orders.repository.js (extrait) JavaScript

```
export async function createOrderWithStock(order, items) {
  return prisma.$transaction(async (tx) => {
    for (const item of items) {
      const result = await tx.productVariant.updateMany({
        where: { id: item.variantId, stock: { gte: item.quantity } },
```

```
        data: { stock: { decrement: item.quantity } },
      });
      if (result.count === 0) {
        throw new AppError("Stock insuffisant", 409, "OUT_OF_STOCK");
      }
    }
    return tx.order.create({
      data: { ...order, items: { create: items.map(toOrderItemData) } },
      include: { items: true },
    });
  });
}
```

Ligne	Explication
2	`prisma.$transaction(async (tx) => {...})` : tout ce qui utilise `tx` fait partie de la **même transaction**.
4-7	`updateMany` avec `stock: { gte: quantité }` : le retrait **n'a lieu que s'il reste assez de stock**. C'est atomique : deux clients qui commandent le dernier article ne peuvent pas tous deux réussir.
8-10	Aucune ligne modifiée : stock insuffisant. On **lance** une erreur : cela **annule tout** (`ROLLBACK`) et rien n'est enregistré.
12-15	Création de la commande **et** de ses lignes en une instruction (`create` imbriqué).

> **À RETENIR**
> Dans une transaction interactive, **utilise `tx`, jamais `prisma`**. Si tu appelles `prisma...` à l'intérieur, la requête sort de la transaction et ne sera pas annulée en cas d'erreur. Une exception lancée à l'intérieur **annule** tout.

6.9 Les erreurs Prisma

Prisma lance des exceptions avec un **code** commençant par `P`. Les trois à connaître :

Code	Signification	Réponse HTTP adaptée
`P2002`	**Violation d'unicité** (slug, email déjà pris)	`409 Conflict`
`P2003`	**Clé étrangère** invalide (catégorie inexistante, ou suppression d'une catégorie qui a des produits)	`409` ou `422`
`P2025`	**Enregistrement introuvable** pour un `update` ou `delete`	`404 Not Found`

On les traduit **au même endroit** que le reste, dans le gestionnaire d'erreurs central de la partie 3. Ajoute ces lignes **avant** le message générique :

src/middlewares/errorHandler.js (ajout) JavaScript

```
if (err.code === "P2002") {
  const fields = Array.isArray(err.meta?.target) ? err.meta.target.join(", ") : "valeur";
  return res.status(409).json({
    error: { code: "ALREADY_EXISTS", message: `Cette valeur existe déjà (${fields})` },
  });
}
if (err.code === "P2025") {
  return res.status(404).json({
    error: { code: "NOT_FOUND", message: "Élément introuvable" },
  });
}
```

```
if (err.code === "P2003") {
  return res.status(409).json({
    error: { code: "RELATION_CONSTRAINT", message: "Élément lié à d'autres données" },
  });
}
```

NOTE — Le nom exact du champ dans err.meta

Selon la version de Prisma et de l'adapter, `err.meta.target` peut être une liste de colonnes ou une autre forme. C'est pourquoi le code vérifie `Array.isArray` avant de l'utiliser. En cas de doute, affiche `console.log(err.code, err.meta)` pendant le développement.

6.10 Prisma Studio

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npx prisma studio
Prisma Studio is running at http://localhost:5555
```

Dans le navigateur, tu vois tous tes modèles, tu peux filtrer, créer ou modifier des lignes. C'est l'outil idéal pour **vérifier** qu'une route de l'API a bien écrit ce qu'elle devait. **N'expose jamais Studio sur Internet** : il donne un accès complet à la base ; utilise-le uniquement en local.

6.11 Étendre le client : $extends

Pour appliquer une règle partout (par exemple ne jamais lire les produits supprimés), on peut **étendre** le client, sans réécrire chaque requête.

src/config/prisma.js (version étendue, facultative) JavaScript

```
export const prisma = base.$extends({
  query: {
    product: {
      async findMany({ args, query }) {
        args.where = { deletedAt: null, ...args.where };
        return query(args);
      },
    },
  },
});
```

La fonction intercepte chaque `findMany` de `product` et ajoute `deletedAt: null`, sauf si l'appel précise autre chose. C'est le remplaçant de l'ancien `prisma.$use`, supprimé dans Prisma 7.

FACULTATIF — Cette extension n'est pas nécessaire

Dans le projet pratique, nous écrivons simplement `deletedAt: null` dans les requêtes du repository. Retiens que `$extends` existe pour le jour où tu en auras besoin.

6.12 SQL ou Prisma : que choisir ?

Besoin	Meilleur choix
CRUD classique, relations, pagination	**Prisma**
Requête de statistiques très complexe	`prisma.$queryRaw` (SQL paramétré)
Créer ou modifier la structure	**Migrations** Prisma
Explorer, corriger une donnée en urgence	pgAdmin / psql / Studio

```
const rows = await prisma.$queryRaw`
  SELECT date_trunc('day', created_at) AS jour, SUM(total_amount)::int AS total
  FROM orders WHERE status = 'DELIVERED' GROUP BY 1 ORDER BY 1`;
```

ATTENTION — Jamais de concaténation dans $queryRaw

Avec la forme « gabarit de texte » (la requête est écrite entre accents graves, comme dans l'exemple ci-dessus), Prisma transforme chaque `${valeur}` en paramètre sûr. N'utilise **jamais** `$queryRawUnsafe` avec des valeurs venant d'un utilisateur : c'est la porte ouverte aux injections SQL.

6.13 Les bonnes pratiques

1. **Un seul** `PrismaClient` partagé, créé dans `src/config/prisma.js`.
2. Les requêtes Prisma n'apparaissent que dans les **repositories** (partie 7), jamais dans les routes.
3. `select` pour limiter les champs sensibles (`passwordHash` ne sort jamais).
4. `generate` après chaque changement de schéma ; **migrations versionnées** dans Git.
5. **Transaction** dès que plusieurs écritures doivent réussir ensemble.
6. `findMany` toujours **paginé** (`take` plafonné).
7. Gérer `P2002`, `P2003`, `P2025` dans le gestionnaire central.
8. Jamais `migrate reset` ni `db push` en production.

À RETENIR — En résumé — Partie 6

- **Prisma** décrit la base dans `schema.prisma`, génère le client et gère les migrations ; installe la version 7 explicitement.
- `migrate dev` crée et applique en développement ; `migrate deploy` applique en production ; `generate` après chaque changement de schéma.
- Un seul `PrismaClient`, créé avec l'adapter PostgreSQL, partagé dans toute l'application.
- `include`, `select`, `where`, `orderBy`, `skip`/`take` couvrent la lecture ; `increment`/`decrement` et `$transaction` sécurisent les écritures.
- Traduis `P2002` en 409, `P2003` en 409 et `P2025` en 404 dans le gestionnaire central.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais...	Où revoir
Installer Prisma 7 et configurer la connexion	Installation
Écrire un modèle avec ses relations	Le fichier schema.prisma
Créer et appliquer une migration	Commandes essentielles
Écrire un CRUD avec Prisma	Les opérations CRUD
Gérer les erreurs P2002, P2003 et P2025	Les erreurs Prisma

6.14 Exercices de la partie 6

QUESTIONS DE COMPRÉHENSION

1. Quelle est la différence entre `migrate dev` et `migrate deploy` ?
2. Pourquoi crée-t-on un seul `PrismaClient` ?
3. Que signifie `@map("created_at")` ? Pourquoi l'utilise-t-on ?
4. Pourquoi `stock: { decrement: 2 }` est-il plus sûr que lire puis réécrire le stock ?

EXERCICES PRISMA

5. Écris la requête qui renvoie les 5 produits actifs les plus chers de la catégorie `pulls`, avec leurs variantes.
6. Écris la requête qui renvoie les variantes dont le stock est inférieur ou égal au seuil, avec le nom du produit.
7. Écris la suppression douce d'une catégorie, mais **refuse** (en lançant une `AppError`) si elle contient encore des produits non supprimés.
8. Écris la requête qui calcule la note moyenne des avis `APPROVED` d'un produit.

DÉBOGAGE

9. `prisma.product.update({ where: { id: "inconnu" }, data: {...} })` plante avec le code `P2025`. Quelle réponse HTTP doit recevoir le client ?
10. Après avoir ajouté `isFeatured Boolean @default(false)` dans le schéma, `product.isFeatured` vaut `undefined`. Quelles commandes as-tu oublié de lancer ?

MINI-PROJET

11. Ajoute au schéma le modèle `ContactMessage` (exercice 10 de la partie 5), crée sa migration avec `--name add_contact_messages`, puis écris le repository avec `create`, `findMany` paginé et `markAsRead`.

PARTIE 7

Architecture en couches

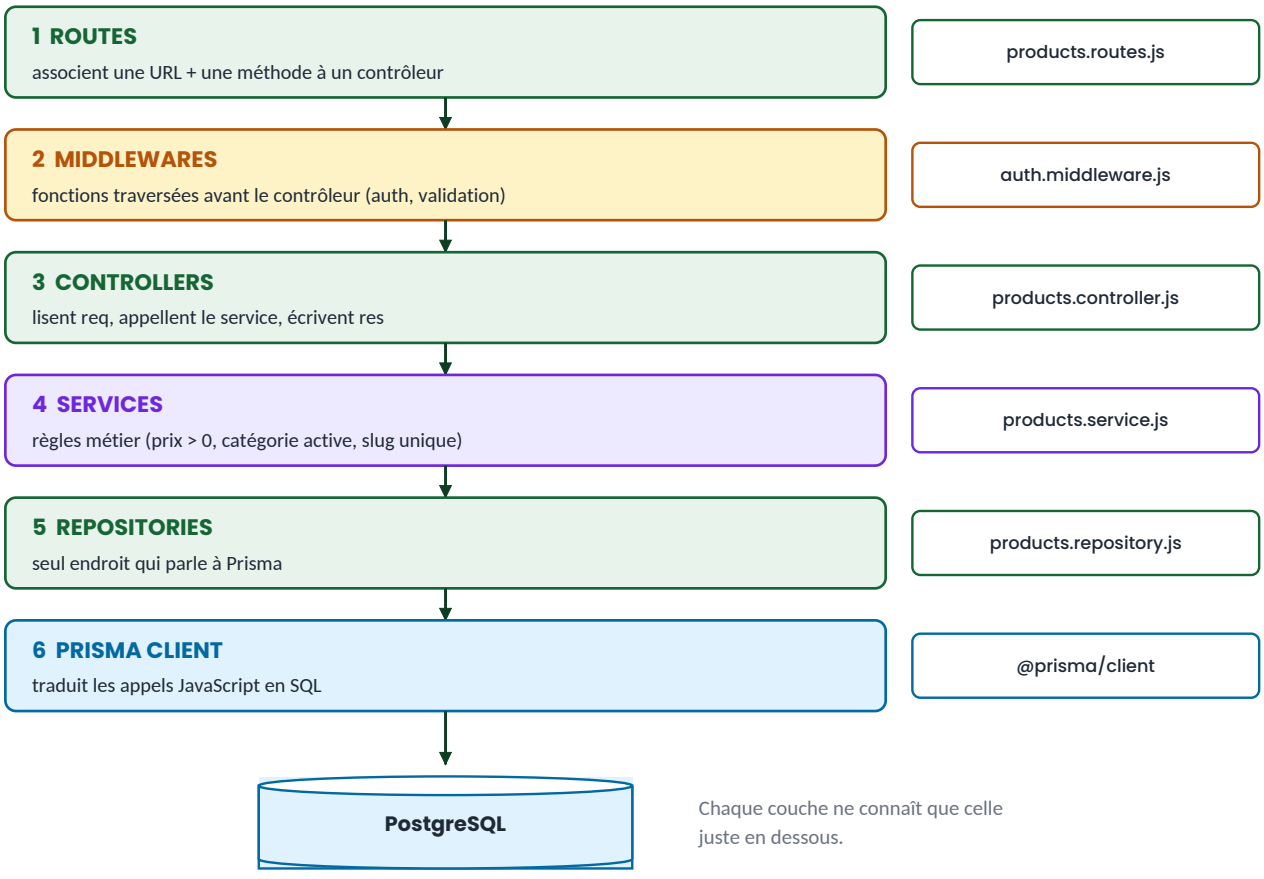
Organiser le code pour qu'il reste lisible, testable et évolutif : routes, middlewares, contrôleurs, services, repositories.

7.1 Pourquoi organiser le code ?

Tant que tu écris 5 routes, tout dans un seul fichier fonctionne. Avec les 10 modules de LAWPIKING (produits, commandes, avis, promotions...), ce même fichier ferait des milliers de lignes, mélangeant lecture de la requête, règles de la boutique, SQL et réponses. Un changement à un endroit casserait ailleurs.

La solution est l'**architecture en couches** : on découpe le code en **couches**, chacune avec **une seule responsabilité**. Une couche ne parle qu'à la couche voisine.

SCHÉMA — ARCHITECTURE EN COUCHES DU BACKEND LAWPIKING



Chaque couche ne connaît que celle juste en dessous.

Couche	Responsabilité	Elle sait	Elle ignore
Route	Associer méthode + chemin à des middlewares et un contrôleur	Les URLs	La logique métier
Middleware	Vérifications transversales : connexion, rôle, validation, upload	La requête, l'utilisateur	Les règles de la boutique
Contrôleur	Lire `req`, appeler **un** service, envoyer `res`	HTTP (statuts, JSON)	La base de données
Service	**Règles métier** : slug unique, stock suffisant, prix figé, calculs	Les règles de la boutique	HTTP (`req`, `res`)
Repository	**Seul** endroit qui parle à Prisma	Les requêtes Prisma	HTTP et règles métier
Base de données	Stocker	Tables et contraintes	Tout le reste

> **À RETENIR**
> La règle d'or : **les dépendances descendent**. Une route appelle un contrôleur, qui appelle un service, qui appelle un repository. Jamais l'inverse. Un service ne doit jamais contenir `req` ni `res`, et un contrôleur ne doit jamais contenir `prisma`.

Pourquoi c'est utile concrètement

- **Changer d'ORM ou de base** ne touche que les repositories.
- **Tester** une règle (« stock insuffisant ») ne demande ni serveur HTTP ni vraie base : on teste le service seul.
- **Réutiliser** : la commande WhatsApp et une future commande en ligne appellent le même service.
- **Lire** : quand un bug concerne le stock, tu sais **où** chercher.

7.2 La structure des dossiers

Chaque fonctionnalité a son **module** avec ses cinq fichiers :

```
lawpiking-backend/
├─ prisma/
│  ├─ schema.prisma
│  ├─ seed.js
│  └─ migrations/
├─ src/
│  ├─ app.js
│  ├─ server.js
│  ├─ config/
│  │  ├─ env.js
│  │  └─ prisma.js
│  ├─ middlewares/
│  │  ├─ requestLogger.js
│  │  ├─ validate.js
│  │  ├─ authenticate.js
│  │  ├─ requireAdmin.js
│  │  ├─ upload.js
│  │  └─ errorHandler.js
│  ├─ utils/
│  │  ├─ AppError.js
│  │  ├─ pagination.js
│  │  └─ slugify.js
│  └─ modules/
│     ├─ auth/
│     ├─ users/
```

```
│     ├─ categories/
│     ├─ products/
│     │  ├─ products.routes.js
│     │  ├─ products.controller.js
│     │  ├─ products.service.js
│     │  ├─ products.repository.js
│     │  └─ products.schema.js
│     ├─ orders/
│     ├─ reviews/
│     ├─ promotions/
│     ├─ stock/
│     └─ settings/
├─ uploads/
├─ .env
├─ .env.example
├─ .gitignore
└─ package.json
```

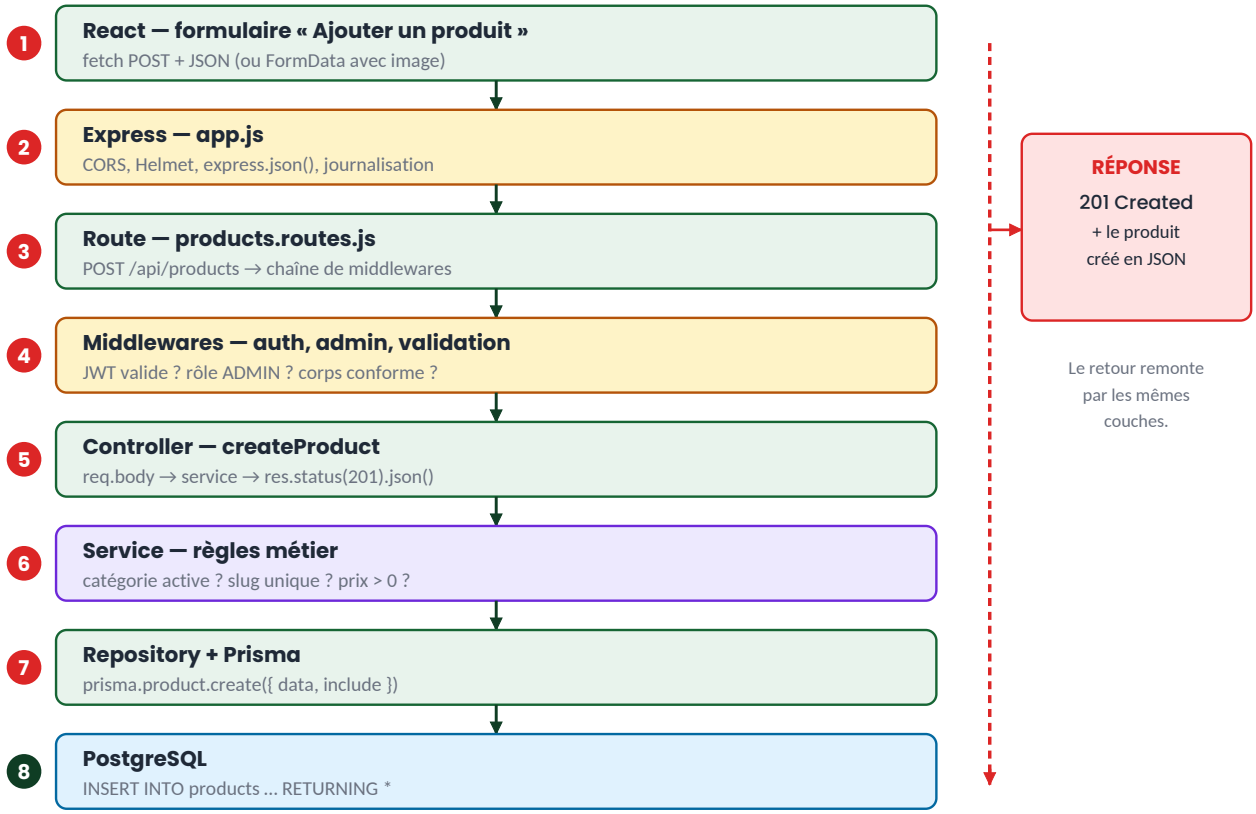
Dossier	Contenu
`config/`	Lecture des variables d'environnement, client Prisma
`middlewares/`	Fonctions partagées par plusieurs modules
`utils/`	Petits outils sans dépendance métier
`modules/<nom>/`	Tout ce qui concerne **une** ressource, regroupé
`uploads/`	Images envoyées en développement (partie 9), **ignorées** par Git

BONNE PRATIQUE — Groupe par fonctionnalité, pas par type
Une autre organisation consisterait à avoir un dossier `controllers/` avec tous les contrôleurs, un dossier `services/`, etc. Elle fonctionne, mais pour modifier les produits tu ouvres cinq dossiers. En regroupant **par module**, tout ce qui concerne les produits est au même endroit.

7.3 Le trajet complet d'une requête : créer un produit

Reprenons ta page **Ajouter un produit** de l'administration. Aujourd'hui, `createAdminProduct(values)` écrit dans le `localStorage`. Demain, la même fonction enverra une requête au backend. Voici **tout** le voyage de cette requête :

SCHÉMA — LE VOYAGE DE POST /API/PRODUCTS, DE REACT À POSTGRESQL



1. **React** : l'administrateur remplit le formulaire et clique sur *Enregistrer*. `createAdminProduct(values)` envoie `POST /api/products` avec le jeton de connexion.
2. **Express** reçoit la requête. `requestLogger` note l'heure ; `express.json()` lit le JSON.
3. **Route** : `POST /api/products` correspond. Les middlewares de la route s'exécutent dans l'ordre : `authenticate` (qui es-tu ?), `requireAdmin` (as-tu le droit ?), `validate(createProductSchema)` (les données sont-elles correctes ?).
4. **Contrôleur** : lit `req.body`, appelle `productsService.create(data)`, puis répond `201`.
5. **Service** : applique les règles : la catégorie existe et est active ? Le nom est-il libre ? Génère un `slug` unique.
6. **Repository** : exécute `prisma.product.create(...)`.
7. **PostgreSQL** : insère la ligne, vérifie les contraintes, renvoie la ligne créée.
8. **Retour** : repository → service → contrôleur → `res.status(201).json(product)` → React affiche le message de succès.

Le code, couche par couche

1. La route

src/modules/products/products.routes.js JavaScript

```
import { Router } from "express";
import * as controller from "./products.controller.js";
import { authenticate } from "../../middlewares/authenticate.js";
import { requireAdmin } from "../../middlewares/requireAdmin.js";
import { validate } from "../../middlewares/validate.js";
import {
  createProductSchema,
  updateProductSchema,
```

```
  listProductsSchema,
} from "./products.schema.js";

const router = Router();

router.get("/", validate(listProductsSchema, "query"), controller.list);
router.get("/:id", controller.getById);

const adminOnly = [authenticate, requireAdmin];

router.post("/", ...adminOnly, validate(createProductSchema), controller.create);
router.put("/:id", ...adminOnly, validate(updateProductSchema), controller.update);
router.delete("/:id", ...adminOnly, controller.remove);

export default router;
```

Lecture : les routes **de lecture** (`GET`) sont publiques, car la boutique doit afficher le catalogue ; les routes **d'écriture** exigent un administrateur connecté. Les middlewares sont listés **dans l'ordre d'exécution**.

2. Le schéma de validation

src/modules/products/products.schema.js JavaScript

```
import { z } from "zod";

const variant = z.object({
  size: z.string().trim().min(1).max(10),
  colorName: z.string().trim().min(1).max(40),
  colorHex: z.string().regex(/^#[0-9A-Fa-f]{6}$/, "Couleur invalide"),
  stock: z.coerce.number().int().min(0).max(100000).default(0),
  lowStockThreshold: z.coerce.number().int().min(0).default(5),
});

export const createProductSchema = z.object({
  name: z.string().trim().min(1, "Le nom est obligatoire").max(80),
  description: z.string().max(1000).default(""),
  price: z.coerce.number().int("Entier requis")
    .positive("Doit être supérieur à 0").max(10_000_000),
  categoryId: z.string().uuid("Catégorie invalide"),
  variants: z.array(variant).max(60).default([]),
});

export const updateProductSchema = createProductSchema.partial();

export const listProductsSchema = z.object({
  search: z.string().trim().max(80).optional(),
  category: z.string().trim().max(40).optional(),
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(50).default(10),
});
```

Les limites (80 caractères, 1000 de description, prix maximum, 12 couleurs…) sont **celles de ton formulaire actuel** (`productValidation.js`). `.partial()` rend tous les champs facultatifs pour la modification.

3. Le contrôleur : fin et sans logique

src/modules/products/products.controller.js JavaScript

```
import * as productsService from "./products.service.js";

export async function create(req, res) {
  const product = await productsService.create(req.body);
```

```
  res.status(201).json(product);
}

export async function list(req, res) {
  const result = await productsService.list(req.query);
  res.status(200).json(result);
}

export async function getById(req, res) {
  const product = await productsService.getById(req.params.id);
  res.status(200).json(product);
}

export async function update(req, res) {
  const product = await productsService.update(req.params.id, req.body);
  res.status(200).json(product);
}

export async function remove(req, res) {
  await productsService.remove(req.params.id);
  res.sendStatus(204);
}
```

À RETENIR

Un bon contrôleur tient en **2 ou 3 lignes** : il lit la requête, appelle le service, envoie la réponse. Pas de `try/catch` : Express 5 transmet les erreurs au gestionnaire central.

4. Le service : les règles de la boutique

src/modules/products/products.service.js JavaScript

```
import { AppError } from "../../utils/AppError.js";
import { slugify } from "../../utils/slugify.js";
import * as productsRepository from "./products.repository.js";
import * as categoriesRepository from "../categories/categories.repository.js";

export async function create(data) {
  const category = await categoriesRepository.findById(data.categoryId);
  if (!category) throw new AppError("Catégorie introuvable", 404, "CATEGORY_NOT_FOUND");
  if (!category.isEnabled) {
    throw new AppError("Cette catégorie est désactivée", 409, "CATEGORY_DISABLED");
  }

  const nameTaken = await productsRepository.findByName(data.name);
  if (nameTaken) {
    throw new AppError("Un produit porte déjà ce nom", 409, "PRODUCT_NAME_TAKEN");
  }

  const slug = await buildUniqueSlug(category.slug, data.name);
  return productsRepository.create({ ...data, slug });
}

export async function getById(id) {
  const product = await productsRepository.findById(id);
  if (!product) throw new AppError("Produit introuvable", 404, "PRODUCT_NOT_FOUND");
  return product;
}

async function buildUniqueSlug(categorySlug, name) {
  const prefix = categorySlug.replace(/s$/, "");
  const base = `${prefix}-${slugify(name)}`;
```

```
  let slug = base;
  for (let n = 2; await productsRepository.slugExists(slug); n += 1) {
    slug = `${base}-${n}`;
  }
  return slug;
}
```

Ce service reprend **les mêmes règles que ton** `adminProductService.js` **actuel** (catégorie désactivée refusée, nom unique, slug unique avec suffixe `-2`, `-3`), mais cette fois côté serveur, donc impossibles à contourner.

Le petit outil `slugify`, utilisé par le service (il fabrique `noir-intense` à partir de « NOIR INTENSE », en retirant les accents) :

src/utils/slugify.js JavaScript

```
export function slugify(text) {
  return text
    .normalize("NFD")
    .replace(/\p{M}/gu, "")
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, "-")
    .replace(/(^-|-$)/g, "");
}
```

C'est la même logique que la fonction `slugify` de ton `adminProductService.js` : `normalize("NFD")` sépare la lettre de son accent, `\p{M}` retire les accents, le reste remplace tout caractère non alphanumérique par un tiret.

5. Le repository : Prisma uniquement

src/modules/products/products.repository.js JavaScript

```
import { prisma } from "../../config/prisma.js";

const withRelations = {
  category: { select: { id: true, slug: true, name: true } },
  images: { orderBy: { position: "asc" } },
  variants: true,
};

export const findById = (id) =>
  prisma.product.findFirst({ where: { id, deletedAt: null }, include: withRelations });

export const findByName = (name) =>
  prisma.product.findFirst({
    where: { name: { equals: name, mode: "insensitive" }, deletedAt: null },
    select: { id: true },
  });

export const slugExists = async (slug) =>
  (await prisma.product.count({ where: { slug } })) > 0;

export function create({ variants, ...data }) {
  return prisma.product.create({
    data: { ...data, variants: { create: variants } },
    include: withRelations,
  });
}
```

Le repository **ne décide de rien** : il pose une question à la base et renvoie la réponse. Si demain on change de base, seul ce fichier change.

ATTENTION — Une erreur Prisma ne doit pas fuir
Même avec les vérifications du service, deux administrateurs peuvent créer le même produit **au même instant**. La contrainte `UNIQUE` de la base refuse alors la seconde insertion : Prisma lance `P2002`, que le gestionnaire central (partie 6) convertit en `409`. La **base** est le dernier rempart.

7.4 Brancher les modules dans app.js

src/app.js (routes de l'API) JavaScript

```
import productsRouter from "./modules/products/products.routes.js";
import categoriesRouter from "./modules/categories/categories.routes.js";
import authRouter from "./modules/auth/auth.routes.js";

app.use("/api/auth", authRouter);
app.use("/api/categories", categoriesRouter);
app.use("/api/products", productsRouter);
```

Les autres modules (`orders`, `reviews`, `promotions`, `stock`, `settings`, `users`) se branchent de la même façon (partie 11).

7.5 Services entre modules

Le service de commandes a besoin des produits (prix, stock). La règle : **un service appelle le service ou le repository d'un autre module, jamais son contrôleur ni ses routes.** Évite les dépendances circulaires (A appelle B qui appelle A) : si deux modules s'appellent mutuellement, extrais la partie commune dans un troisième fichier.

7.6 Les erreurs traversent toutes les couches

```
Repository : P2002 (Prisma) ─┐
Service    : AppError 409   ─┼─→ gestionnaire central → JSON { error: { code, message } }
Middleware : AppError 401   ─┘
```

Chaque couche peut **lancer** ; une seule couche (le gestionnaire central) **répond**. C'est ce qui garantit des réponses d'erreur cohérentes pour ton frontend.

7.7 Les erreurs d'architecture les plus fréquentes

Erreur	Pourquoi c'est un problème	Correction
`prisma.…` dans un contrôleur	Le contrôleur connaît la base ; impossible à tester seul	Déplacer dans le repository
`req.body` passé tel quel à Prisma	Un client peut envoyer des champs interdits (`role`, `deletedAt`…)	Valider avec Zod, ne garder que les champs prévus
Règle métier dans la route	Dupliquée et oubliée ailleurs	La mettre dans le service
`res.json()` dans un service	Le service dépend d'HTTP	Retourner la donnée, laisser le contrôleur répondre
Un seul gros fichier `index.js`	Illisible dès 10 routes	Un module par ressource

7.8 Quand est-ce trop ?

Pour le module `settings` (lire et écrire quelques réglages), un repository + un service minuscule suffisent. L'important est **la cohérence** : tous les modules ont la même forme, donc tu sais toujours où chercher. Ne crée pas de couche supplémentaire « au cas où ».

> **À RETENIR — En résumé — Partie 7**
>
> - Cinq couches : **route → middleware → contrôleur → service → repository**, qui descendent toujours dans le même sens.
> - Le contrôleur est mince, le service porte les règles de la boutique, seul le repository parle à Prisma.
> - Un dossier **par module** regroupe les cinq fichiers ; le code reste lisible à 10 modules.
> - Les erreurs sont lancées par toutes les couches et converties en réponse par **une seule** : le gestionnaire central.
> - La base reste le dernier rempart : les contraintes `UNIQUE` couvrent les requêtes simultanées.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Nommer les cinq couches et leur rôle	Pourquoi organiser le code
Organiser un module en fichiers	La structure des dossiers
Suivre une requête de React à PostgreSQL	Le trajet complet d'une requête
Écrire un contrôleur mince	Le code, couche par couche
Repérer les erreurs d'architecture	Les erreurs d'architecture

7.9 Exercices de la partie 7

QUESTIONS DE COMPRÉHENSION

1. Quelles sont les cinq couches d'un module et le rôle de chacune ?
2. Pourquoi un service ne doit-il pas contenir `req` ni `res` ?
3. Pourquoi les routes `GET /api/products` sont-elles publiques, alors que `POST` est protégée ?
4. Pourquoi vérifier l'unicité du nom dans le service **et** avec une contrainte `UNIQUE` en base ?

EXERCICES DE CODE

5. Écris `categories.routes.js`, `categories.controller.js` et `categories.service.js` pour **lister** et **créer** une catégorie (nom unique, insensible à la casse).
6. Écris le service `productsService.remove(id)` qui fait une suppression douce, et renvoie 404 si le produit n'existe pas.
7. Écris `productsService.list(query)` qui renvoie `{ total, page, limit, totalPages, products }` à partir de `getPagination` et du repository.

DÉBOGAGE

8. Un contrôleur contient `await prisma.product.create({ data: req.body })`. Cite trois problèmes (architecture et sécurité) et corrige.
9. Le service de commandes importe le service de produits, qui importe le service de commandes : Node affiche une erreur de dépendance circulaire. Comment réorganiser ?

MINI-PROJET

10. Dessine (sur papier ou avec un outil) le trajet complet de la requête `PATCH /api/products/:id/stock` : de la page Stock React jusqu'à PostgreSQL, en nommant chaque fichier traversé.

POUR RÉFLÉCHIR

11. Pourquoi ne place-t-on pas la vérification « le stock est-il suffisant ? » dans le contrôleur de commandes ? Cite deux conséquences concrètes si elle y était.

PARTIE 8

Authentification et sécurité

Connecter uniquement l'administrateur, protéger les routes, et fermer les portes classiques aux attaquants.

8.1 Authentification et autorisation

Deux mots proches, deux questions différentes :

Notion	Question	Exemple LAWPIKING	Échec
Authentification	**Qui es-tu ?**	L'administrateur saisit son email et son mot de passe	`401 Unauthorized`
Autorisation	**As-tu le droit de faire ça ?**	Un compte `CUSTOMER` essaie de supprimer un produit	`403 Forbidden`

Aujourd'hui, la connexion de ton administration est une **simulation** dans le navigateur : n'importe qui qui sait ouvrir les outils de développement peut contourner une vérification faite côté frontend. Une vraie protection est **toujours côté serveur**.

> **ATTENTION — Le frontend ne protège rien**
> Cacher le bouton « Supprimer » ou la page `/admin` dans React améliore le confort, jamais la sécurité. Un attaquant peut envoyer directement `DELETE /api/products/…` avec `curl`. **Chaque route sensible doit vérifier l'identité et le rôle côté backend.**

8.2 Ne jamais stocker un mot de passe en clair

Si ta base fuite (sauvegarde volée, erreur de configuration), les mots de passe en clair sont lisibles par tous, et comme beaucoup de gens réutilisent le même mot de passe ailleurs, le vol se propage. On stocke donc un **hachage** :

> **DÉFINITION SIMPLE**
> Un **hachage** est une transformation à **sens unique** : à partir du mot de passe on calcule une empreinte, mais on ne peut pas retrouver le mot de passe depuis l'empreinte. À la connexion, on hache le mot de passe saisi et on **compare les deux empreintes**.

bcrypt est un algorithme conçu pour cela. Il est volontairement **lent** (ce qui décourage les attaques par essais massifs) et ajoute un **sel** aléatoire : deux comptes avec le même mot de passe ont des empreintes différentes.

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install bcryptjs jsonwebtoken cookie-parser `
    helmet cors express-rate-limit
```

Paquet	Rôle
`bcryptjs`	bcrypt écrit en JavaScript : s'installe partout sans compilation (idéal sous Windows)
`jsonwebtoken`	Crée et vérifie les jetons JWT
`cookie-parser`	Lit les cookies dans `req.cookies`
`helmet`	Ajoute des en-têtes HTTP de sécurité
`cors`	Autorise (ou non) ton frontend à appeler l'API
`express-rate-limit`	Limite le nombre de requêtes

```
import bcrypt from "bcryptjs";

const hash = await bcrypt.hash("MonMotDePasseSolide!2026", 12);
console.log(hash);
const ok = await bcrypt.compare("MonMotDePasseSolide!2026", hash);
console.log(ok);
```

RÉSULTAT résultat attendu

```
$2b$12$kq3...(60 caractères au total, différents à chaque exécution)
true
```

Élément	Explication
`bcrypt.hash(motDePasse, 12)`	Calcule l'empreinte. **12** est le « coût » : chaque point supplémentaire double le temps de calcul. Entre 10 et 12 convient aujourd'hui.
`$2b$12$...`	L'empreinte contient l'algorithme, le coût et le **sel**. Pas besoin de stocker le sel à part.
`bcrypt.compare(saisi, hash)`	Compare un mot de passe saisi à une empreinte. Renvoie `true` ou `false`.

8.3 Les jetons JWT

DÉFINITION SIMPLE

Un **JWT** (*JSON Web Token*) est un texte signé par le serveur qui contient quelques informations (l'identité de l'utilisateur, son rôle, une date d'expiration). Le serveur le remet à l'utilisateur après la connexion ; l'utilisateur le renvoie à chaque requête ; le serveur vérifie la **signature** pour s'assurer que personne ne l'a modifié.

Un JWT ressemble à `xxxxx.yyyyy.zzzzz` : trois parties séparées par des points.

Partie	Contenu	Exemple décodé
En-tête	Algorithme de signature	`{ "alg": "HS256", "typ": "JWT" }`
Charge utile (*payload*)	Les informations	`{ "sub": "0b9c…", "role": "ADMIN", "exp": 1788000000 }`
Signature	Garantie d'intégrité, calculée avec le **secret**	(illisible)

ATTENTION — Un JWT est signé, pas chiffré

Tout le monde peut **lire** la charge utile (c'est du Base64). N'y mets **jamais** de mot de passe ni de données sensibles. La signature empêche seulement de la **modifier** : si quelqu'un change `"role": "ADMIN"`, la vérification échoue.

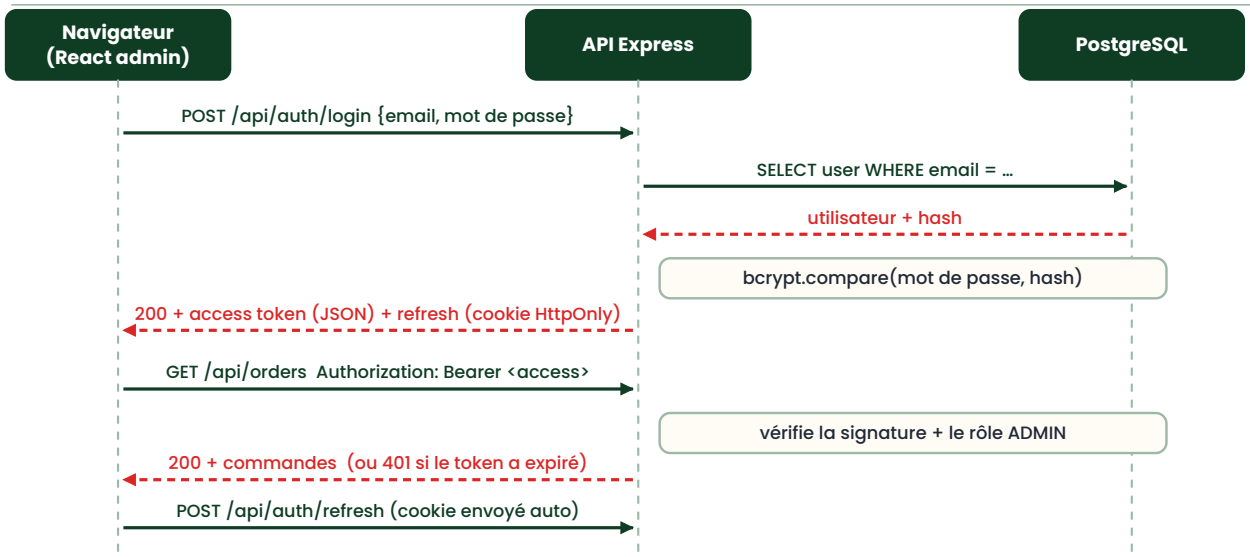
```
import jwt from "jsonwebtoken";

const token = jwt.sign({ role: "ADMIN" }, process.env.JWT_ACCESS_SECRET, {
  subject: user.id,
  expiresIn: "15m",
});

const payload = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
```

`jwt.sign` crée le jeton ; `expiresIn: "15m"` le fait expirer après 15 minutes ; `jwt.verify` vérifie la signature **et** l'expiration (il lance une erreur sinon). Le secret ne doit jamais quitter le serveur.

SCHÉMA — CONNEXION ADMINISTRATEUR AVEC JWT ET COOKIE HTTPONLY



8.4 Comparer les approches

Où et comment garder la preuve de connexion ? Voici les solutions courantes :

Approche	Fonctionnement	Avantages	Inconvénients
Session côté serveur	Le serveur garde une session ; le navigateur reçoit un cookie d'identifiant	Révocation immédiate	Demande un stockage de sessions (table ou Redis)
JWT dans `localStorage`	Le frontend stocke le jeton et l'envoie dans `Authorization`	Très simple	Un script malveillant injecté dans la page (**XSS**) peut **voler** le jeton
JWT unique dans un cookie HttpOnly	Le jeton vit dans un cookie illisible par JavaScript	Résiste au vol par XSS	Exposé aux attaques **CSRF** s'il est mal configuré ; durée de vie longue = risque
Access token court + refresh token en cookie HttpOnly	Un jeton de 15 minutes en mémoire (en-tête `Authorization`), plus un jeton de renouvellement de 7 jours dans un cookie HttpOnly	Le jeton volé expire vite ; le refresh n'est pas lisible par JavaScript	Un peu plus de code

EXEMPLE LAWPIKING — Notre recommandation pour LAWPIKING

Access token court (15 minutes) conservé en mémoire côté React, plus refresh token (7 jours) dans un cookie HttpOnly. C'est le meilleur compromis pour une boutique avec un seul administrateur : pas de stockage de sessions à gérer, un jeton volé devient inutile en 15 minutes, et le jeton de longue durée n'est jamais visible par le JavaScript de la page. Nous gardons le modèle de 11 tables de la partie 5.

Limite honnête de ce choix : le refresh token est ici **sans état** (stateless). On ne peut donc pas le révoquer individuellement avant son expiration. Pour une boutique à un administrateur, c'est acceptable. Si l'équipe grandit, la suite naturelle est une table `refresh_tokens` (empreinte du jeton, date d'expiration, révocation) : il suffit d'une migration.

8.5 Variables d'environnement à ajouter

.env

```env
JWT_ACCESS_SECRET="un-long-secret-aleatoire-64-caracteres-minimum"
JWT_REFRESH_SECRET="un-autre-secret-different-64-caracteres-minimum"
FRONTEND_URL="http://localhost:5173"
ADMIN_EMAIL="admin@lawpiking.sn"
ADMIN_PASSWORD="change-moi-immediatement-Aa1!"
```

Pour **générer** un bon secret :

TERMINAL — Windows PowerShell

```
PS ...\lawpiking-backend> node -p "require('crypto').randomBytes(48).toString('hex')"
9f2c…(96 caractères hexadécimaux)
```

Utilise deux valeurs **différentes** pour les deux secrets. Étends `src/config/env.js` :

src/config/env.js (version complète)

```javascript
import "dotenv/config";

function required(name) {
  const value = process.env[name];
  if (!value) {
    console.error(`Variable d'environnement manquante : ${name}`);
```

```
    process.exit(1);
  }
  return value;
}

export const env = {
  port: Number(process.env.PORT) || 3000,
  nodeEnv: process.env.NODE_ENV || "development",
  databaseUrl: required("DATABASE_URL"),
  frontendUrl: required("FRONTEND_URL"),
  jwtAccessSecret: required("JWT_ACCESS_SECRET"),
  jwtRefreshSecret: required("JWT_REFRESH_SECRET"),
};
```

La fonction `required("NOM")` **arrête le démarrage** avec un message clair si une variable manque : mieux vaut un échec immédiat qu'un bug mystérieux plus tard. Mets à jour `.env.example` avec les **noms** de variables (sans les vraies valeurs) : c'est lui qui est versionné, jamais `.env`.

8.6 Créer l'administrateur sans inscription publique

Il n'existe **aucune route d'inscription** : n'importe quel visiteur pourrait sinon se créer un compte. L'administrateur est créé **une seule fois**, par un script lancé par toi, sur ta machine ou sur le serveur.

prisma/createAdmin.js JavaScript

```
import "dotenv/config";
import bcrypt from "bcryptjs";
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";

const prisma = new PrismaClient({
  adapter: new PrismaPg({ connectionString: process.env.DATABASE_URL }),
});

const email = process.env.ADMIN_EMAIL?.trim().toLowerCase();
const password = process.env.ADMIN_PASSWORD;

if (!email || !password || password.length < 12) {
  console.error("ADMIN_EMAIL et ADMIN_PASSWORD (12 caractères minimum) sont requis.");
  process.exit(1);
}

const passwordHash = await bcrypt.hash(password, 12);

const admin = await prisma.user.upsert({
  where: { email },
  update: { passwordHash, role: "ADMIN", isActive: true },
  create: { email, passwordHash, fullName: "Administrateur LAWPIKING", role: "ADMIN" },
});

console.log(`Administrateur prêt : ${admin.email}`);
await prisma.$disconnect();
```

Ajoute dans `package.json` : `"admin:create": "node prisma/createAdmin.js"`, puis :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm run admin:create
Administrateur prêt : admin@lawpiking.sn
```

Le mot de passe est haché **avant** d'entrer dans la base. Après la création, **retire** `ADMIN_PASSWORD` de ton `.env`. Le même script sert à **réinitialiser** le mot de passe si tu l'oublies (l'`upsert` met à jour le compte).

8.7 Le module auth

Le schéma de connexion

src/modules/auth/auth.schema.js JavaScript

```
import { z } from "zod";

export const loginSchema = z.object({
  email: z.string().trim().toLowerCase().email("Email invalide").max(160),
  password: z.string().min(1, "Le mot de passe est obligatoire").max(200),
});
```

Le service

src/modules/auth/auth.service.js JavaScript

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { env } from "../../config/env.js";
import { AppError } from "../../utils/AppError.js";
import * as usersRepository from "../users/users.repository.js";

const FAKE_HASH = bcrypt.hashSync("mot-de-passe-factice", 12);

export function signAccessToken(user) {
  return jwt.sign({ role: user.role }, env.jwtAccessSecret, {
    subject: user.id,
    expiresIn: "15m",
  });
}

export function signRefreshToken(user) {
  return jwt.sign({}, env.jwtRefreshSecret, { subject: user.id, expiresIn: "7d" });
}

export async function login(email, password) {
  const user = await usersRepository.findByEmail(email);
  const hash = user?.passwordHash ?? FAKE_HASH;
  const passwordOk = await bcrypt.compare(password, hash);

  if (!user || !passwordOk || !user.isActive || user.role !== "ADMIN") {
    throw new AppError("Email ou mot de passe incorrect", 401, "INVALID_CREDENTIALS");
  }
  return {
    user: { id: user.id, email: user.email, fullName: user.fullName, role: user.role },
    accessToken: signAccessToken(user),
    refreshToken: signRefreshToken(user),
  };
}

export async function refresh(refreshToken) {
  let payload;
  try {
    payload = jwt.verify(refreshToken, env.jwtRefreshSecret);
  } catch {
    throw new AppError("Session expirée", 401, "SESSION_EXPIRED");
  }
```

```
  const user = await usersRepository.findById(payload.sub);
  if (!user || !user.isActive) {
    throw new AppError("Session invalide", 401, "SESSION_INVALID");
  }
  return { accessToken: signAccessToken(user), refreshToken: signRefreshToken(user) };
}
```

Point	Explication
Message unique « Email ou mot de passe incorrect »	Si on répondait « email inconnu », un attaquant saurait quels emails existent.
`FAKE_HASH`	Si l'email n'existe pas, on compare quand même avec une fausse empreinte : le temps de réponse reste le même, ce qui masque l'existence du compte.
`user.role !== "ADMIN"`	Seul un administrateur peut se connecter à l'espace d'administration.
`isActive`	Un compte désactivé ne peut plus ni se connecter ni renouveler sa session.
`refresh`	Vérifie le refresh token et **relit l'utilisateur** en base : s'il a été désactivé, la session s'arrête.

Le contrôleur et les cookies

src/modules/auth/auth.controller.js JavaScript

```
import { env } from "../../config/env.js";
import * as authService from "./auth.service.js";

const REFRESH_COOKIE = "lp_refresh";

const cookieOptions = {
  httpOnly: true,
  secure: env.nodeEnv === "production",
  sameSite: env.nodeEnv === "production" ? "none" : "lax",
  path: "/api/auth",
  maxAge: 7 * 24 * 60 * 60 * 1000,
};

export async function login(req, res) {
  const { user, accessToken, refreshToken } = await authService.login(
    req.body.email,
    req.body.password,
  );
  res.cookie(REFRESH_COOKIE, refreshToken, cookieOptions);
  res.status(200).json({ user, accessToken });
}

export async function refresh(req, res) {
  const token = req.cookies[REFRESH_COOKIE];
  if (!token) {
    throw new AppError("Non connecté", 401, "NO_SESSION");
  }
  const { accessToken, refreshToken } = await authService.refresh(token);
  res.cookie(REFRESH_COOKIE, refreshToken, cookieOptions);
  res.status(200).json({ accessToken });
}

export function logout(req, res) {
  res.clearCookie(REFRESH_COOKIE, { ...cookieOptions, maxAge: undefined });
```

```
  res.sendStatus(204);
}

export function me(req, res) {
  res.status(200).json({ user: req.user });
}
```

Option du cookie	Rôle
`httpOnly: true`	Le JavaScript de la page **ne peut pas lire** le cookie : un script injecté ne peut pas le voler.
`secure`	En production, le cookie ne voyage qu'en **HTTPS**.
`sameSite`	`"lax"` en local (même site). En production, si le frontend et l'API sont sur des domaines **différents** (par exemple deux hébergeurs), il faut `"none"` (qui impose `secure`).
`path: "/api/auth"`	Le navigateur n'envoie ce cookie qu'aux routes d'authentification, pas à toute l'API.
`maxAge`	7 jours en millisecondes.

Les routes

src/modules/auth/auth.routes.js — JavaScript

```
import { Router } from "express";
import rateLimit from "express-rate-limit";
import * as controller from "./auth.controller.js";
import { validate } from "../../middlewares/validate.js";
import { authenticate } from "../../middlewares/authenticate.js";
import { loginSchema } from "./auth.schema.js";

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 5,
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    error: { code: "TOO_MANY_ATTEMPTS", message: "Trop de tentatives, réessayez" },
  },
});

const router = Router();

router.post("/login", loginLimiter, validate(loginSchema), controller.login);
router.post("/refresh", controller.refresh);
router.post("/logout", controller.logout);
router.get("/me", authenticate, controller.me);

export default router;
```

`loginLimiter` autorise **5 tentatives de connexion par adresse IP et par 15 minutes** : une attaque qui essaie des milliers de mots de passe devient impraticable.

8.8 Les middlewares authenticate et requireAdmin

src/middlewares/authenticate.js JavaScript

```
import jwt from "jsonwebtoken";
import { env } from "../config/env.js";
import { AppError } from "../utils/AppError.js";

export function authenticate(req, res, next) {
  const header = req.headers.authorization ?? "";
  const [scheme, token] = header.split(" ");

  if (scheme !== "Bearer" || !token) {
    throw new AppError("Connexion requise", 401, "UNAUTHENTICATED");
  }
  try {
    const payload = jwt.verify(token, env.jwtAccessSecret);
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    throw new AppError("Session expirée, reconnectez-vous", 401, "TOKEN_EXPIRED");
  }
}
```

src/middlewares/requireAdmin.js JavaScript

```
import { AppError } from "../utils/AppError.js";

export function requireAdmin(req, res, next) {
  if (req.user?.role !== "ADMIN") {
    throw new AppError("Accès réservé à l'administrateur", 403, "FORBIDDEN");
  }
  next();
}
```

Point	Explication
`Authorization: Bearer <jeton>`	La convention pour transmettre un jeton d'accès dans une requête HTTP.
`req.user = {...}`	On **attache** l'identité à la requête : les contrôleurs suivants savent qui appelle.
401 puis 403	`authenticate` répond 401 (« qui es-tu ? »), `requireAdmin` répond 403 (« tu n'as pas le droit »).
`throw` dans un middleware synchrone	Express capture l'exception et la transmet au gestionnaire d'erreurs central.

8.9 Les protections de base dans app.js

src/app.js (ordre recommandé) JavaScript

```
import helmet from "helmet";
import cors from "cors";
import cookieParser from "cookie-parser";
import rateLimit from "express-rate-limit";

app.use(helmet());
app.use(cors({ origin: env.frontendUrl, credentials: true }));
app.use(
  rateLimit({ windowMs: 15 * 60 * 1000, limit: 300, standardHeaders: true }),
```

```
);
app.use(express.json({ limit: "100kb" }));
app.use(cookieParser());
app.use(requestLogger);
```

Protection	Ce qu'elle fait	Pourquoi
`helmet()`	Ajoute des en-têtes de sécurité (anti-clickjacking, anti-détection de type de contenu…)	Réduit des familles d'attaques en une ligne
`cors({ origin, credentials })`	N'autorise **que** l'adresse de ton frontend à appeler l'API depuis un navigateur, et autorise l'envoi du cookie	Sans cela, le navigateur bloque les appels du frontend ; avec `origin: "*"`, n'importe quel site pourrait appeler ton API
Limite globale (300 requêtes / 15 min)	Freine les abus et les robots	Protège la bande passante et la base
`express.json({ limit: "100kb" })`	Refuse les corps JSON énormes	Évite de saturer la mémoire
`cookieParser()`	Lit `req.cookies`	Nécessaire pour le refresh token

ERREUR FRÉQUENTE — « Access to fetch has been blocked by CORS policy »

Le navigateur bloque l'appel du frontend vers l'API. Causes courantes : `FRONTEND_URL` ne correspond pas **exactement** à l'adresse du frontend (`http://localhost:5173`, sans `/` final) ; `cors` est déclaré **après** les routes ; il manque `credentials: true` alors que le frontend envoie les cookies. Le blocage CORS est une règle du **navigateur** : `curl` et Postman ne le subissent pas.

8.10 Les injections SQL

DÉFINITION SIMPLE

Une **injection SQL** consiste à glisser du SQL dans une donnée saisie (un champ de formulaire, un paramètre d'URL) pour que le serveur l'exécute. Exemple : si on construit la requête par concaténation, `"SELECT * FROM users WHERE email = '" + email + "'"`, et que l'attaquant saisit `' OR '1'='1`, la condition devient toujours vraie.

Prisma protège par défaut : toutes ses méthodes (`findMany`, `create`…) envoient les valeurs comme **paramètres**, séparés du SQL, donc jamais interprétées comme du code. Les dangers viennent de ce qu'on écrit **à côté** de Prisma :

```
// DANGEREUX : concaténation dans du SQL brut
await prisma.$queryRawUnsafe(`SELECT * FROM products WHERE name = '${req.query.name}'`);

// SÛR : gabarit paramétré
await prisma.$queryRaw`SELECT * FROM products WHERE name = ${req.query.name}`;
```

Règles : pas de `$queryRawUnsafe` avec des données utilisateur, **validation** de chaque entrée avec Zod, et un utilisateur PostgreSQL aux droits limités (partie 4).

8.11 Les autres attaques à connaître

Attaque	Principe	Défense dans LAWPIKING
XSS (injection de script)	Un texte malveillant (dans un avis, un nom) s'exécute dans la page d'un autre visiteur	React **échappe** le texte par défaut : n'utilise jamais `dangerouslySetInnerHTML` avec des données utilisateur ; Helmet ; refresh token en cookie HttpOnly
CSRF	Un site tiers déclenche une action à l'insu de l'utilisateur connecté, en profitant du cookie	Le cookie ne part que vers `/api/auth` ; les actions sensibles exigent l'en-tête `Authorization` (que le site tiers ne peut pas fournir) ; `sameSite`
Force brute	Essais massifs de mots de passe	Limite de 5 tentatives, bcrypt lent
Énumération d'utilisateurs	Deviner quels emails existent	Message d'erreur unique, `FAKE_HASH`
Mass assignment	L'attaquant ajoute `role: "ADMIN"` dans le JSON	Zod ne garde que les champs prévus ; ne passe jamais `req.body` brut à Prisma
Fuite d'informations	Une trace d'erreur révèle des détails internes	Message générique en 500 (partie 3)
Fichiers malveillants	Envoi d'un script déguisé en image	Contrôle du type, de la taille, renommage (partie 9)

8.12 Les secrets

1. **Jamais** dans le code, jamais dans Git. Dans `.env`, listé dans `.gitignore`.
2. Un **secret différent** par environnement (développement, production).
3. En production, ils se configurent dans le **tableau de bord de l'hébergeur** (partie 13).
4. Si un secret fuite (poussé sur GitHub par erreur) : **change-le immédiatement**, ne te contente pas de supprimer le fichier : l'historique Git le conserve.

.gitignore Texte

```
node_modules/
.env
uploads/
*.log
```

8.13 Tester l'authentification

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> curl.exe -i -X POST http://localhost:3000/api/auth/login `
    -H "Content-Type: application/json" `
    -d "{\"email\":\"admin@lawpiking.sn\",\"password\":\"mauvais\"}"
HTTP/1.1 401 Unauthorized
{"error":{"code":"INVALID_CREDENTIALS",
"message":"Email ou mot de passe incorrect","details":null}}
```

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> curl.exe -i -X POST http://localhost:3000/api/products `
    -H "Content-Type: application/json" -d "{}"
HTTP/1.1 401 Unauthorized
{"error":{"code":"UNAUTHENTICATED","message":"Connexion requise","details":null}}
```

Avec le bon mot de passe, la réponse est `200` avec `accessToken` ; l'en-tête `Set-Cookie: lp_refresh=...; HttpOnly` apparaît. Test de bout en bout dans Postman : partie 12.

8.14 Liste de contrôle sécurité

- [ ] Aucun mot de passe en clair, nulle part (code, base, journaux)
- [ ] `.env` dans `.gitignore`, secrets longs et aléatoires
- [ ] Pas d'inscription publique ; administrateur créé par script
- [ ] Routes d'écriture : `authenticate` + `requireAdmin`
- [ ] Limite de tentatives sur `/login`
- [ ] CORS limité à ton frontend ; Helmet actif
- [ ] Toute entrée validée avec Zod
- [ ] Pas de `$queryRawUnsafe` avec des données utilisateur
- [ ] Messages d'erreur génériques en production
- [ ] HTTPS en production, cookie `secure`

À RETENIR — En résumé — Partie 8

- **Authentification** = qui es-tu (401) ; **autorisation** = as-tu le droit (403) ; la vraie protection est toujours côté serveur.
- Mots de passe **hachés avec bcrypt**, jamais en clair ; administrateur créé par script, sans inscription publique.
- Recommandation : **access token** de 15 minutes (en mémoire côté React) + **refresh token** de 7 jours dans un cookie **HttpOnly**.
- Helmet, CORS limité, limitation de fréquence, validation Zod, jamais de SQL brut concaténé.
- Les secrets vivent dans `.env` (et chez l'hébergeur), jamais dans Git ; un secret qui fuit se change.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Hacher et vérifier un mot de passe avec bcrypt	Ne jamais stocker un mot de passe
Expliquer un JWT et ses limites	Les jetons JWT
Justifier le choix access plus refresh token	Comparer les approches
Protéger une route par rôle	authenticate et requireAdmin
Citer les attaques courantes et leurs défenses	Les autres attaques

8.15 Exercices de la partie 8

QUESTIONS DE COMPRÉHENSION

1. Quelle est la différence entre authentification et autorisation ? Donne le code HTTP associé à chacune.
2. Pourquoi un hachage est-il préférable à un chiffrement pour un mot de passe ?
3. Pourquoi un access token dure-t-il 15 minutes alors que le refresh dure 7 jours ?
4. Pourquoi le message d'erreur de connexion est-il identique que l'email existe ou non ?

EXERCICES DE CODE

5. Écris le middleware `requireRole(...roles)` générique, qui accepte plusieurs rôles.
6. Écris une route `PATCH /api/auth/password` qui permet à l'administrateur connecté de changer son mot de passe (ancien mot de passe requis, nouveau d'au moins 12 caractères).
7. Écris le schéma Zod `changePasswordSchema` avec un champ de confirmation qui doit être identique.

DÉBOGAGE

8. Après la connexion, `POST /api/products` renvoie toujours 401 alors que le jeton est envoyé. Le frontend envoie `Authorization: <jeton>`. Qu'est-ce qui manque ?
9. Le cookie `lp_refresh` n'apparaît jamais dans le navigateur en production (frontend et API sur deux domaines). Quelles options de cookie et de CORS vérifier ?

MINI-PROJET

10. Écris un script `npm run admin:disable` qui désactive un compte par email, puis démontre qu'il ne peut plus renouveler sa session.

PARTIE 9

Images et fichiers

Envoyer, contrôler, enregistrer, remplacer et supprimer les photos des produits.

9.1 Le problème à résoudre

Aujourd'hui, dans ton formulaire produit, l'image choisie est convertie en texte (une *data URL*) et rangée dans le `localStorage`. C'est la raison du message « Le stockage du navigateur est plein » : une photo de téléphone pèse plusieurs mégaoctets. Avec un backend, la photo devient un **vrai fichier** sur un disque ou un service dédié, et seule son **adresse** est notée dans la base.

9.2 Pourquoi ne pas mettre les images dans la base ?

PostgreSQL *peut* stocker des fichiers (type `BYTEA`), mais ce n'est presque jamais une bonne idée :

Image dans la base	Fichier + chemin dans la base
Table énorme, sauvegardes lourdes et lentes	Table légère : un texte de 40 caractères
Chaque affichage de page lit la base	Le serveur web ou un CDN envoie le fichier directement
Difficile à mettre en cache	Mise en cache naturelle du navigateur
Redimensionner/déplacer = requêtes SQL	Copier des fichiers

> **À RETENIR**
> La base stocke **le chemin** (`/uploads/products/3f2c….webp`), jamais l'image. C'est le rôle de la colonne `product_images.url` de la partie 5.

9.3 Comment un fichier voyage : multipart/form-data

Jusqu'ici, tes requêtes contenaient du JSON. Un fichier est du binaire, qu'on ne peut pas mettre dans du JSON. Le navigateur utilise donc un autre format : `multipart/form-data`, qui découpe la requête en plusieurs **parties**, chacune avec son nom, comme les champs d'un formulaire :

```
POST /api/products/9f2c.../images HTTP/1.1
Content-Type: multipart/form-data; boundary=----XYZ

------XYZ
Content-Disposition: form-data; name="alt"

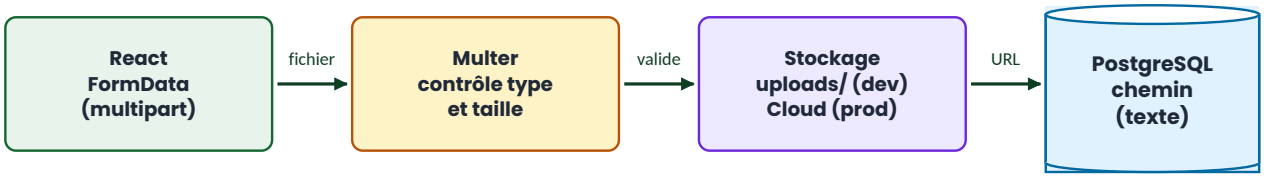
T-shirt noir intense, vu de face
------XYZ
Content-Disposition: form-data; name="image"; filename="photo.jpg"
```

```
Content-Type: image/jpeg

(octets binaires de l'image)
------XYZ--
```

`express.json()` **ne sait pas** lire ce format. On utilise **Multer**, un middleware spécialisé.

SCHÉMA — ENVOI D'UNE IMAGE PRODUIT



Le fichier va sur un disque (ou un service de stockage). La base ne garde que son chemin :
`/uploads/products/tshirt-noir-intense-1730000000.webp`
C'est léger, rapide à lire, et les images restent servies par un serveur de fichiers.

9.4 Installer Multer

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install multer
```

CHANGEMENT DE VERSION — Multer 2
Utilise la version 2 ou supérieure de Multer : les versions 1.x ont des failles de sécurité connues et ne sont plus maintenues.

9.5 Local ou distant ?

Stockage	Où	Avantages	Limite
Disque local (dossier `uploads/`)	Ton ordinateur / le serveur	Simple, gratuit, parfait pour **développer**	Sur beaucoup d'hébergeurs, le disque est **éphémère** : les fichiers disparaissent à chaque redéploiement. Pas adapté à plusieurs serveurs.
Stockage distant (service d'objets : Cloudinary, stockage compatible S3, etc.)	Un service dédié	Fichiers **durables**, CDN rapide, sauvegardés	Compte à créer, configuration, parfois payant

La stratégie de ce guide : **développer en local, publier en distant**, en isolant la logique dans **un seul fichier** (`storage.js`). Le reste du code ne sait pas où l'image est rangée : il demande « enregistre ceci » et obtient une adresse. Passer du local au distant ne change qu'un fichier.

9.6 Le contrôle des fichiers

Un fichier envoyé est une **donnée non fiable** : le nom et le type annoncés par l'utilisateur peuvent mentir. Nous appliquons quatre protections :

Protection	Pourquoi
Taille maximale (5 Mo)	Évite de saturer la mémoire et le disque
Types autorisés (JPEG, PNG, WebP)	Refuse les scripts, PDF, exécutables
Vérification de la signature réelle	Le type annoncé (`Content-Type`) peut être falsifié ; les premiers octets du fichier ne mentent pas
Renommage aléatoire	Le nom d'origine peut contenir `../` (accès à d'autres dossiers) ou des caractères dangereux ; deux fichiers du même nom s'écraseraient

Le middleware d'envoi

src/middlewares/upload.js JavaScript

```
import multer from "multer";
import { AppError } from "../utils/AppError.js";

const ALLOWED = new Set(["image/jpeg", "image/png", "image/webp"]);

export const uploadImage = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 5 * 1024 * 1024, files: 1 },
  fileFilter(req, file, cb) {
    if (!ALLOWED.has(file.mimetype)) {
      const message = "Formats acceptés : JPEG, PNG, WebP";
      return cb(new AppError(message, 415, "UNSUPPORTED_MEDIA"));
    }
    cb(null, true);
  },
}).single("image");
```

Élément	Explication
`memoryStorage()`	Le fichier reste **en mémoire** (dans `req.file.buffer`) le temps de le contrôler ; c'est notre `storage.js` qui décide où l'écrire.
`limits.fileSize`	5 Mo maximum. Dépassement : Multer lance une erreur de code `LIMIT_FILE_SIZE`.
`limits.files: 1`	Un seul fichier par requête.
`fileFilter`	Premier contrôle sur le type **annoncé**.
`.single("image")`	Attend **un** fichier dans le champ nommé `image` et le place dans `req.file`.

Multer a ses propres erreurs ; on les traduit dans le gestionnaire central :

src/middlewares/errorHandler.js (ajout) JavaScript

```
if (err.name === "MulterError") {
  const tooBig = err.code === "LIMIT_FILE_SIZE";
  const message = tooBig ? "Image trop lourde (5 Mo max.)" : "Envoi de fichier invalide";
  return res.status(413).json({ error: { code: err.code, message } });
}
```

Vérifier la signature du fichier

Chaque format commence par des octets caractéristiques (sa « signature »). On les lit nous-mêmes :

src/utils/imageSignature.js JavaScript

```
export function detectImageType(buffer) {
  if (buffer.length < 12) return null;

  const isJpeg = buffer[0] === 0xff && buffer[1] === 0xd8 && buffer[2] === 0xff;
  if (isJpeg) return { mime: "image/jpeg", ext: "jpg" };

  const PNG_SIGNATURE = Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a]);
  const isPng = buffer.subarray(0, 8).equals(PNG_SIGNATURE);
  if (isPng) return { mime: "image/png", ext: "png" };

  const isWebp =
    buffer.subarray(0, 4).toString("ascii") === "RIFF" &&
    buffer.subarray(8, 12).toString("ascii") === "WEBP";
  if (isWebp) return { mime: "image/webp", ext: "webp" };

  return null;
}
```

Un fichier `virus.exe` renommé `photo.jpg` ne commence pas par `FF D8 FF` : il est **refusé**, même si le navigateur annonce `image/jpeg`.

Le module de stockage

src/utils/storage.js JavaScript

```
import { mkdir, writeFile, unlink } from "node:fs/promises";
import path from "node:path";
import { randomUUID } from "node:crypto";
import { AppError } from "./AppError.js";
import { detectImageType } from "./imageSignature.js";

const UPLOAD_DIR = path.resolve("uploads", "products");
const PUBLIC_PREFIX = "/uploads/products/";

export async function saveProductImage(buffer) {
  const type = detectImageType(buffer);
  if (!type) throw new AppError("Fichier image invalide", 415, "INVALID_IMAGE");

  const fileName = `${randomUUID()}.${type.ext}`;
  await mkdir(UPLOAD_DIR, { recursive: true });
  await writeFile(path.join(UPLOAD_DIR, fileName), buffer);
  return PUBLIC_PREFIX + fileName;
}

export async function deleteProductImage(url) {
  if (!url?.startsWith(PUBLIC_PREFIX)) return;
  const fileName = path.basename(url);
  try {
    await unlink(path.join(UPLOAD_DIR, fileName));
  } catch (error) {
    if (error.code !== "ENOENT") throw error;
  }
}
```

Point	Explication
`randomUUID()`	Nom **imprévisible et unique** : jamais d'écrasement, pas d'accès deviné.
Extension déduite de la **signature**	On ne fait jamais confiance au nom d'origine.
`mkdir({ recursive: true })`	Crée `uploads/products/` s'il n'existe pas.
`path.basename(url)`	Ne garde que le nom : même si l'adresse contenait `../`, on reste dans le dossier.
`ENOENT` ignoré	Si le fichier n'existe déjà plus, ce n'est pas une erreur.
Valeur renvoyée	`/uploads/products/3f2c….webp` : ce que l'on écrira dans `product_images.url`.

> **NOTE — Version distante**
> En production avec un service distant, **seul ce fichier change** : `saveProductImage` envoie le buffer au service (par son SDK) et renvoie l'adresse complète (`https://…`) ; `deleteProductImage` demande la suppression au service. Les routes, services et la base ne changent pas.

Servir les fichiers

src/app.js (ajout) JavaScript

```
app.use("/uploads", express.static("uploads", { maxAge: "7d", fallthrough: false }));
```

`express.static` envoie les fichiers du dossier `uploads/` : `http://localhost:3000/uploads/products/3f2c….webp` affiche l'image. `maxAge` autorise le navigateur à garder l'image 7 jours. Ajoute aussi `app.use(helmet({ crossOriginResourcePolicy: { policy: "cross-origin" } }))` : par défaut Helmet empêche un autre site (ton frontend, sur une autre adresse) d'afficher ces images.

9.7 Les routes d'images

src/modules/products/products.routes.js (ajout) JavaScript

```
router.post("/:id/images", ...adminOnly, uploadImage, controller.addImage);
router.patch("/:id/images/:imageId/main", ...adminOnly, controller.setMainImage);
router.delete("/:id/images/:imageId", ...adminOnly, controller.removeImage);
```

Ordre des middlewares : **d'abord** `authenticate` et `requireAdmin` (on ne reçoit pas un fichier d'un inconnu), **ensuite** Multer.

src/modules/products/products.service.js (ajout) JavaScript

```
import { saveProductImage, deleteProductImage } from "../../utils/storage.js";

export async function addImage(productId, file, altText = "") {
  if (!file) throw new AppError("Aucune image envoyée", 400, "NO_FILE");
  await getById(productId);

  const url = await saveProductImage(file.buffer);
  try {
    return await productsRepository.addImage(productId, { url, altText });
  } catch (error) {
    await deleteProductImage(url);
    throw error;
```

```
  }
}

export async function removeImage(productId, imageId) {
  const image = await productsRepository.findImage(productId, imageId);
  if (!image) throw new AppError("Image introuvable", 404, "IMAGE_NOT_FOUND");
  await productsRepository.deleteImage(imageId);
  await deleteProductImage(image.url);
}
```

Point	Explication
`await getById(productId)`	Vérifie que le produit existe **avant** d'écrire un fichier.
`try/catch` autour de l'écriture en base	Si la base échoue, on **supprime le fichier** déjà écrit : pas de fichier orphelin.
`removeImage` : base **puis** fichier	Si la suppression du fichier échouait, la base serait déjà propre ; au pire, un fichier inutile reste sur le disque (moins grave qu'une ligne pointant vers un fichier absent).

Pour **remplacer** une image : ajouter la nouvelle, puis supprimer l'ancienne (ou, côté API, un `PUT` qui enchaîne les deux). L'image **principale** est celle où `is_main = true` ; la contrainte `UNIQUE` partielle de la partie 5 garantit qu'il n'y en a qu'une par produit. Le repository, pour `setMainImage`, passe l'ancienne à `false` et la nouvelle à `true` **dans une transaction**.

9.8 Côté React : envoyer le fichier

```
async function uploadProductImage(productId, file, altText) {
  const form = new FormData();
  form.append("image", file);
  form.append("alt", altText);

  const response = await fetch(`${API_URL}/api/products/${productId}/images`, {
    method: "POST",
    headers: { Authorization: `Bearer ${accessToken}` },
    body: form,
  });
  if (!response.ok) throw await response.json();
  return response.json();
}
```

> **ERREUR FRÉQUENTE — N'écris jamais Content-Type à la main avec FormData**
> Si tu ajoutes `"Content-Type": "multipart/form-data"`, la requête échoue : il manque le `boundary` (le séparateur). Laisse le navigateur écrire l'en-tête **tout seul** quand le corps est un `FormData`.

Dans la liste des produits, `<img src={API_URL + product.mainImage.url}>` affiche l'image. Garde ton **aperçu local** (`URL.createObjectURL(file)`) avant l'envoi : l'utilisateur voit son image tout de suite, sans attendre le serveur.

9.9 Les bonnes pratiques

1. Taille maximale **et** types limités, vérifiés côté serveur.
2. **Renommer** chaque fichier ; ne jamais réutiliser le nom d'origine.
3. Vérifier la **signature**, pas seulement le type annoncé.
4. Un seul module `storage.js` : local ou distant sans toucher au reste.
5. Supprimer le fichier **et** la ligne en base ensemble ; pas d'orphelins.
6. `uploads/` dans `.gitignore`.
7. Redimensionner les grandes photos (ex. 1200 px, WebP) avant stockage pour accélérer la boutique (facultatif ; la bibliothèque `sharp` s'en charge).

> **À RETENIR — En résumé — Partie 9**
> - Les images sont des **fichiers** ; la base ne garde que leur **chemin**.
> - Un envoi de fichier utilise `multipart/form-data` et **Multer**, avec taille limitée et types autorisés.
> - On vérifie la **signature réelle** du fichier et on le renomme avec un UUID.
> - Tout passe par un module `storage.js` : local en développement, stockage distant en production.
> - Supprime fichier **et** ligne ensemble ; jamais d'orphelins.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Expliquer multipart/form-data	Comment un fichier voyage
Configurer Multer avec limites	Le middleware d'envoi
Vérifier la signature d'une image	Vérifier la signature
Distinguer stockage local et distant	Local ou distant
Supprimer image et ligne ensemble	Les routes d'images

9.10 Exercices de la partie 9

QUESTIONS DE COMPRÉHENSION

1. Pourquoi ne stocke-t-on pas les images dans PostgreSQL ?
2. Pourquoi renomme-t-on les fichiers avec un UUID ?
3. Pourquoi vérifier la signature des fichiers en plus du `Content-Type` ?
4. Pourquoi le stockage local ne convient-il pas à un hébergeur à disque éphémère ?

EXERCICES DE CODE

5. Ajoute la prise en charge du format AVIF dans `detectImageType` (cherche sa signature : en-tête « ftyp » avec la marque « avif »).
6. Écris `productsRepository.setMainImage(productId, imageId)` avec une transaction Prisma.
7. Écris la route et le service qui **remplacent** l'image principale d'un produit.

DÉBOGAGE

8. `req.file` vaut toujours `undefined`. Le frontend envoie `form.append("photo", file)` et Multer attend `.single("image")`. Que corriger ?
9. L'upload fonctionne mais l'image ne s'affiche pas dans le frontend (erreur `ERR_BLOCKED_BY_RESPONSE`). Quelle option Helmet est en cause ?

MINI-PROJET

10. Ajoute l'envoi d'une **image de catégorie** : colonne `image_url` dans `categories` (migration), route, service et suppression propre lors du remplacement.

POUR RÉFLÉCHIR

11. Une cliente envoie une photo de 4 Mo (6000 × 4000 pixels). Quels problèmes cela pose-t-il pour la boutique, et quelle amélioration proposes-tu ?

PARTIE 10

Connecter React au backend

Remplacer les données locales par de vraies requêtes, sans réécrire les pages.

ATTENTION — Cette partie décrit la méthode, elle ne modifie pas ton projet
Le guide ne touche à aucun fichier du frontend. Les exemples ci-dessous montrent **ce que tu changeras plus tard**, quand le backend existera et que tu décideras de connecter le frontend, étape par étape.

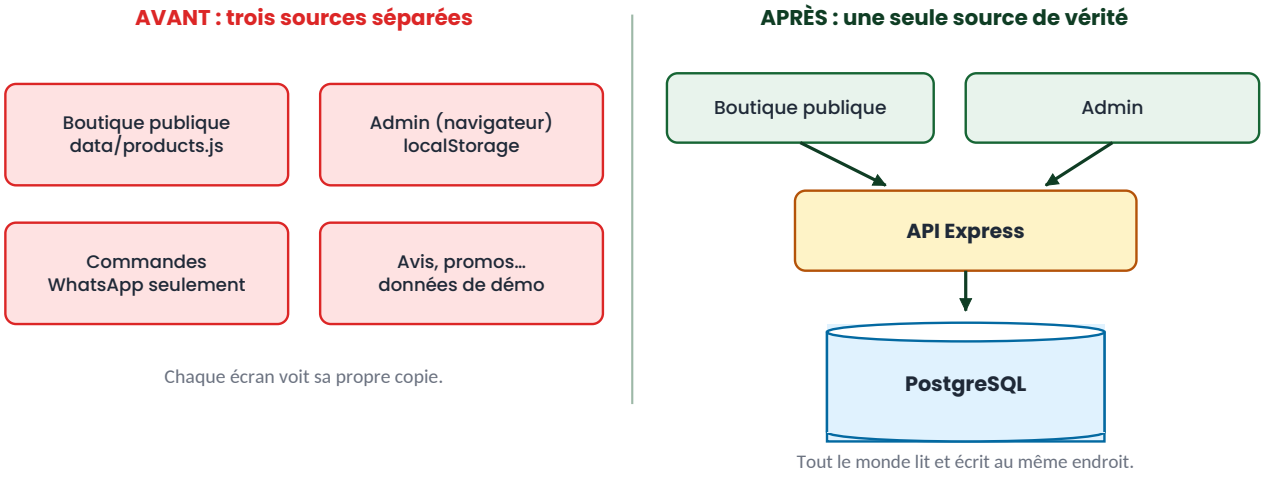
10.1 La bonne nouvelle : le frontend est déjà prêt

Quand tu as construit l'administration, chaque service a été écrit pour être remplacé. Par exemple, l'en-tête de `adminProductService.js` dit : « *pour passer à l'API Node.js, il suffira de remplacer le contenu de ces fonctions par des appels fetch* », avec la correspondance :

Fonction frontend actuelle	Route API
`listAdminProducts()`	`GET /api/products`
`getAdminProduct(id)`	`GET /api/products/:id`
`createAdminProduct(data)`	`POST /api/products`
`updateAdminProduct(id, data)`	`PUT /api/products/:id`
`updateAdminProductStock(id, data)`	`PATCH /api/products/:id/stock`
`deleteAdminProduct(id)`	`DELETE /api/products/:id`

Les **pages** n'appellent que ces fonctions asynchrones (déjà munies d'états de chargement et d'erreur). Il suffit de changer **l'intérieur** des services : **les signatures restent identiques**, les pages ne bougent pas.

SCHÉMA — AVANT (AUJOURD'HUI) ET APRÈS (AVEC LE BACKEND)



10.2 Ce qui change selon la page

Partie du frontend	Aujourd'hui	Difficulté du branchement
Services d'administration (`admin*Service.js`)	Déjà **asynchrones** (`async`, `await`)	**Facile** : remplacer le contenu
`authService.js` (connexion)	**Synchrone** : `isAuthenticated()` lit le navigateur	**Moyenne** : demande une session restaurée au démarrage (voir plus bas)
Boutique publique (`productService.js`)	**Synchrone** : `getProducts()` renvoie un tableau statique	**Plus de travail** : les composants doivent attendre les données (chargement, erreur)
Commande WhatsApp	Génère un message et ouvre WhatsApp	**Moyenne** : enregistrer la commande **avant** d'ouvrir WhatsApp

10.3 Configurer l'adresse de l'API

Dans le code, n'écris jamais `http://localhost:3000` en dur : l'adresse change entre le développement et la production. Vite permet d'utiliser des **variables d'environnement**.

.env.development (à la racine du frontend) .env

```
1 VITE_API_URL=http://localhost:3000
```

.env.production .env

```
1 VITE_API_URL=https://api.lawpiking.sn
```

src/services/apiConfig.js JavaScript

```
1 export const API_URL = import.meta.env.VITE_API_URL;
```

Point	Explication
Préfixe `VITE_`	**Obligatoire** : Vite n'expose au navigateur que les variables qui commencent par `VITE_`.
`import.meta.env`	L'équivalent de `process.env` dans un projet Vite.
Les valeurs sont **publiques**	Elles sont écrites dans le code envoyé au visiteur. N'y mets **jamais** de secret (clé JWT, mot de passe de base).
Redémarrer `npm run dev`	Après avoir modifié un `.env`, Vite doit être relancé.

ATTENTION — L'adresse seule n'est pas un secret, mais les clés le sont

Une adresse d'API est publique par nature : tout le monde la voit dans l'onglet *Réseau* du navigateur. Ce qui protège l'API, c'est l'authentification et la validation côté serveur (partie 8).

10.4 fetch ou Axios ?

	`fetch`	**Axios**
Installation	Aucune (intégré au navigateur)	`npm install axios`
JSON	`await response.json()` manuel	Automatique
Erreurs HTTP (404, 500)	**Ne lance pas** d'exception : à tester avec `response.ok`	Lance une exception
Intercepteurs (ajouter le jeton partout)	À écrire soi-même	Intégrés
Poids	0	Une dépendance de plus

Notre choix : `fetch`, enveloppé dans un petit client maison. Le frontend n'a ainsi aucune dépendance de plus, et tu comprends exactement ce qui se passe.

10.5 Le client API

Un seul fichier gère l'adresse, le jeton, le JSON, les erreurs et le renouvellement de session.

src/services/apiClient.js JavaScript

```
import { API_URL } from "./apiConfig";

let accessToken = null;
let refreshing = null;

export const setAccessToken = (token) => { accessToken = token; };

export class ApiError extends Error {
  constructor(status, body) {
    super(body?.error?.message ?? "Une erreur est survenue.");
    this.name = "ApiError";
    this.status = status;
    this.code = body?.error?.code;
    this.fields = body?.error?.details ?? null;
  }
}

async function refreshAccessToken() {
  refreshing ??= fetch(`${API_URL}/api/auth/refresh`, {
    method: "POST",
    credentials: "include",
  })
    .then(async (response) => {
      if (!response.ok) {
        throw new ApiError(response.status, await response.json().catch(() => null));
      }
      accessToken = (await response.json()).accessToken;
    })
    .finally(() => { refreshing = null; });
  return refreshing;
}

export async function api(path, { method = "GET", body, retry = true } = {}) {
  const isForm = body instanceof FormData;
  const response = await fetch(`${API_URL}${path}`, {
    method,
    credentials: "include",
    headers: {
```

```
      ...(body && !isForm ? { "Content-Type": "application/json" } : {}),
      ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}),
    },
    body: body ? (isForm ? body : JSON.stringify(body)) : undefined,
  });

  if (response.status === 401 && retry && !path.startsWith("/api/auth/")) {
    await refreshAccessToken();
    return api(path, { method, body, retry: false });
  }
  if (response.status === 204) return null;

  const data = await response.json().catch(() => null);
  if (!response.ok) throw new ApiError(response.status, data);
  return data;
}
```

Élément	Explication
`let accessToken = null`	Le jeton d'accès vit **en mémoire** (variable du module), pas dans le `localStorage` : un script malveillant ne peut pas le lire en fouillant le stockage.
`ApiError`	Une erreur typée : porte le **statut HTTP**, le **code** (`PRODUCT_NAME_TAKEN`) et les erreurs **par champ** (`fields`) pour les formulaires.
`credentials: "include"`	Demande au navigateur d'envoyer et de recevoir le **cookie** du refresh token. Doit correspondre à `credentials: true` côté CORS (partie 8).
`body instanceof FormData`	Pour l'envoi d'images, on ne fixe pas `Content-Type` (partie 9).
Statut 401	Le jeton de 15 minutes a expiré : on appelle `/api/auth/refresh`, puis on **rejoue une fois** la requête d'origine.
`refreshing ??=`	Si plusieurs requêtes échouent en même temps, **un seul** renouvellement est lancé.
Statut 204	Pas de corps (suppression réussie) : on renvoie `null`.

10.6 Le service d'authentification

On conserve **exactement** les fonctions de `authService.js`, mais elles appellent le backend. Pour que `isAuthenticated()` reste synchrone (utilisé tel quel par `AdminLayout`), on garde la session **en mémoire du module** :

src/services/authService.js (version connectée) JavaScript

```
import { api, setAccessToken } from "./apiClient";

let session = null;

export const getSession = () => session;
export const isAuthenticated = () => Boolean(session);

export async function login({ email, password }) {
  const { user, accessToken } = await api("/api/auth/login", {
    method: "POST",
    body: { email, password },
  });
  setAccessToken(accessToken);
  session = { email: user.email, name: user.fullName };
```

```
  return session;
}

export async function logout() {
  try {
    await api("/api/auth/logout", { method: "POST" });
  } finally {
    setAccessToken(null);
    session = null;
  }
}

export async function restoreSession() {
  try {
    const { accessToken } = await api("/api/auth/refresh", { method: "POST" });
    setAccessToken(accessToken);
    const { user } = await api("/api/auth/me");
    session = { email: user.email, name: user.fullName };
  } catch {
    session = null;
  }
}
```

Problème : en mémoire, la session **disparaît** quand on recharge la page (F5). C'est pourquoi `restoreSession()` existe : elle redemande un jeton grâce au **cookie** HttpOnly, qui, lui, survit. On l'appelle **une fois, avant d'afficher l'application** :

src/main.jsx (principe) JavaScript

```
import { restoreSession } from "./services/authService";

await restoreSession();
ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

Si le cookie est valide, l'administrateur reste connecté après un rechargement ; sinon `session` reste `null` et `AdminLayout` redirige vers `/admin/login`, comme aujourd'hui.

> **NOTE — La case « Se souvenir de moi »**
> Ton formulaire actuel a une option `remember`. Avec le cookie de 7 jours, la session se maintient déjà d'une visite à l'autre. Tu peux supprimer la case, ou réserver la durée longue aux utilisateurs qui la cochent (le backend ajuste alors `maxAge`).

Protéger les pages d'administration

`AdminLayout` redirige déjà vers la page de connexion si `isAuthenticated()` est faux : **conserve-le**, c'est un confort. Mais rappelle-toi la règle de la partie 8 : la **vraie** protection est dans le backend. Si quelqu'un contourne la redirection, les routes d'API répondent `401`/`403`. Ajoute un cas : si l'API renvoie 401 **après** le renouvellement, la session est perdue ; on redirige vers la connexion.

10.7 Les états : chargement, erreur, vide

Aucun appel réseau n'est instantané. Une page qui lit des données connaît **quatre** situations :

État	Quand	Ce que l'utilisateur voit
Chargement	En attente de la réponse	Un indicateur (squelette, « Chargement… »)
Succès	Données reçues	La liste
Vide	Succès mais zéro élément	« Aucun produit pour le moment »
Erreur	Réseau coupé, 500, 401…	Un message + bouton « Réessayer »

Tes pages d'administration le font déjà. Pour la boutique publique, un petit **hook** réutilisable évite de répéter le code :

src/hooks/useApi.js JavaScript

```
import { useCallback, useEffect, useState } from "react";

export function useApi(loader, deps = []) {
  const [state, setState] = useState({ data: null, loading: true, error: null });

  const load = useCallback(() => {
    setState((s) => ({ ...s, loading: true, error: null }));
    loader()
      .then((data) => setState({ data, loading: false, error: null }))
      .catch((error) => setState({ data: null, loading: false, error }));
  }, deps);

  useEffect(() => { load(); }, [load]);
  return { ...state, reload: load };
}
```

```
const { data, loading, error, reload } = useApi(() => api("/api/products?limit=50"), []);

if (loading) return <p>Chargement des produits...</p>;
if (error) return <button onClick={reload}>Réessayer</button>;
if (!data.products.length) return <p>Aucun produit pour le moment.</p>;
```

10.8 Avant / après : créer un produit

Avant (version actuelle, simplifiée) : validation, simulation de délai, écriture dans le `localStorage`.

```
export async function createAdminProduct(values) {
  const { products, nextId } = load();
  const errors = validateProduct(values, { existingNames: products.map((p) => p.name) });
  if (Object.keys(errors).length) throw new ProductValidationError(errors);
  await simulateLatency();
  const product = { ...toProductPayload(values), id: nextId, slug: uniqueSlug(...) };
  save([product, ...products], nextId + 1);
  return product;
}
```

Après : la validation métier et le slug passent côté serveur ; le frontend garde la validation de confort, envoie, et **traduit les erreurs du serveur** en erreurs de champs.

src/services/adminProductService.js (version connectée) JavaScript

```
import { api, ApiError } from "./apiClient";
import { validateProduct, ProductValidationError } from "./productValidation";
import { toApiProduct, fromApiProduct } from "./productMapper";

export async function createAdminProduct(values) {
  const errors = validateProduct(values);
  if (Object.keys(errors).length) throw new ProductValidationError(errors);

  try {
    const body = toApiProduct(values);
    const created = await api("/api/products", { method: "POST", body });
    return fromApiProduct(created);
  } catch (error) {
    const known = error instanceof ApiError;
    if (known && error.fields) throw new ProductValidationError(error.fields);
    if (known && error.code === "PRODUCT_NAME_TAKEN") {
      throw new ProductValidationError({ name: "Un produit porte déjà ce nom." });
    }
    throw error;
  }
}
```

La page d'ajout, qui attend déjà une `ProductValidationError` avec `fields`, **n'a rien à changer** : elle affiche les messages sous les champs, venant désormais du serveur.

Le mapper : faire parler deux formats

Ton frontend nomme les champs `desc`, `productType`, `img`, `sizes`, `colors`, `stock`. L'API renvoie `description`, `category`, `images`, `variants`. Plutôt que de modifier toutes les pages, **un seul fichier** traduit dans les deux sens :

src/services/productMapper.js JavaScript

```
export function fromApiProduct(product) {
  const variants = product.variants ?? [];
  const main = product.images?.find((image) => image.isMain) ?? product.images?.[0];
  return {
    id: product.id,
    slug: product.slug,
    name: product.name,
    desc: product.description,
    price: product.price,
    productType: product.category?.slug,
    img: main ? `${import.meta.env.VITE_API_URL}${main.url}` : "",
    sizes: [...new Set(variants.map((v) => v.size))],
    colors: [
      ...new Map(
        variants.map((v) => [v.colorHex, { name: v.colorName, hex: v.colorHex }]),
      ).values(),
    ],
    stock: variants.reduce((sum, v) => sum + v.stock, 0),
    lowStockThreshold: Math.min(...variants.map((v) => v.lowStockThreshold), Infinity),
  };
}
```

Champ frontend	Calculé à partir de
`desc`	`description`
`productType`	`category.slug` (`tshirts`, `pulls`, `bonnets`)
`img`	L'image **principale** (`isMain`), préfixée par l'adresse de l'API
`sizes`, `colors`	Les **variantes** (valeurs uniques)
`stock`	La **somme** des stocks des variantes

`toApiProduct(values)` fait l'opération inverse : construit `categoryId` et les `variants` à partir du formulaire. Tant que tu n'as pas de gestion fine des tailles, tu peux envoyer une variante par combinaison taille/couleur cochée, avec le stock réparti.

> **BONNE PRATIQUE — Le mapper protège le frontend des évolutions de l'API**
> Si demain l'API renomme `description` en `details`, tu changes **une ligne** dans le mapper, pas vingt composants.

10.9 Synchroniser produits, catégories et stock

Aujourd'hui, la boutique lit un catalogue **statique**, tandis que l'administration utilise un autre stock (`localStorage`) : ce sont deux sources séparées (ton propre commentaire dans `adminProductService.js` le souligne). Avec le backend, il n'y a **plus qu'une source de vérité** : PostgreSQL.

Action dans l'administration	Appel API	Effet sur la boutique
Ajouter un produit	`POST /api/products`	Apparaît dans le catalogue (si actif)
Désactiver un produit	`PUT /api/products/:id` (`isActive`)	Disparaît de la boutique
Changer le stock	`PATCH /api/products/:id/stock`	« Rupture » / « Stock bas » mis à jour
Désactiver une catégorie	`PUT /api/categories/:id`	Les nouveaux produits ne peuvent plus la choisir

Côté React, après une modification, **recharge** la donnée concernée (`reload()`) plutôt que de la recalculer à la main : la liste affichée correspond ainsi toujours à ce que la base contient.

10.10 Les commandes réelles : enregistrer, puis ouvrir WhatsApp

Aujourd'hui, valider le panier génère un message et ouvre WhatsApp. Avec le backend, on **enregistre d'abord** la commande ; l'administrateur la retrouve dans la page Commandes.

```
async function placeOrder(cartItems, customer) {
  const order = await api("/api/orders", {
    method: "POST",
    body: {
      customerName: customer.name,
      customerPhone: customer.phone,
      items: cartItems.map((item) => ({
        variantId: item.variantId,
        quantity: item.qty,
```

```
      })),
    },
  });

  const message = buildWhatsAppMessage(order);
  window.open(`https://wa.me/${whatsappNumber}?text=${encodeURIComponent(message)}`, "_blank");
  return order;
}
```

Point	Explication
On envoie des **identifiants de variante** et des **quantités**, pas les prix	Le backend lit les **vrais prix** dans la base. Un client malveillant ne peut pas commander à 1 FCFA en modifiant la requête.
La réponse contient `reference` et `totalAmount`	Le message WhatsApp affiche le **numéro de commande officiel** et le total calculé par le serveur.
Si l'enregistrement échoue (stock insuffisant, 409)	On n'ouvre **pas** WhatsApp et on affiche l'erreur au client.

> **À RETENIR**
> Règle de confiance : le frontend envoie **ce que le client veut** (quels articles, quelle quantité) ; le backend décide **de tout le reste** (prix, total, stock, statut).

Dans la page Commandes, `formatOrderNumber(number)` (qui fabrique `#LWP001`) est à adapter : le backend renvoie une `reference` déjà prête (`LP-20261031-0042`) ; on l'affiche telle quelle.

10.11 Les erreurs de connexion les plus fréquentes

Symptôme	Cause probable	Solution
`Failed to fetch` dans la console	Backend éteint ou mauvaise adresse	Vérifie `npm run dev` du backend et `VITE_API_URL`
`blocked by CORS policy`	`FRONTEND_URL` du backend différent de l'adresse de Vite	Corrige `FRONTEND_URL` (sans `/` final) et relance
401 en boucle	Le cookie de refresh n'est pas envoyé	`credentials: "include"` côté frontend **et** `credentials: true` côté CORS
Rechargement (F5) = déconnexion	`restoreSession()` pas appelée au démarrage	L'ajouter avant le rendu de l'application
`undefined` à la place du prix	Le mapper lit un mauvais nom de champ	Affiche la réponse brute (`console.log`) et compare avec le mapper
Images cassées	Chemin relatif sans l'adresse de l'API	Préfixer par `VITE_API_URL` (voir mapper)

10.12 Ordre de branchement conseillé

1. Santé : `GET /api/health` depuis React.
2. Connexion de l'administrateur (`authService`).
3. Catégories (module le plus simple).
4. Produits (liste, ajout, modification, suppression).
5. Images.
6. Stock.

7. Commandes (côté admin, puis côté boutique).
8. Avis, promotions, paramètres.
9. Enfin, la **boutique publique** (lecture des produits depuis l'API).

Un module à la fois, en testant chaque étape avant de passer à la suivante : si quelque chose casse, tu sais **où** chercher.

> **À RETENIR — En résumé — Partie 10**
> - Les services d'administration sont déjà asynchrones : on remplace leur **contenu**, pas leurs signatures.
> - `VITE_API_URL` configure l'adresse de l'API ; une variable `VITE_` est **publique**.
> - Un **client API** unique gère jeton, erreurs, renouvellement de session et JSON.
> - Un **mapper** traduit le format de l'API vers celui des pages, qui n'ont donc pas à changer.
> - Le client envoie quoi et combien ; le serveur décide des prix, du total et du stock.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Configurer VITE_API_URL	Configurer l'adresse de l'API
Écrire un client API avec jeton	Le client API
Restaurer la session au démarrage	Le service d'authentification
Écrire un mapper API vers frontend	Le mapper
Enregistrer une commande avant WhatsApp	Les commandes réelles

10.13 Exercices de la partie 10

QUESTIONS DE COMPRÉHENSION

1. Pourquoi une variable Vite doit-elle commencer par `VITE_` ? Peut-on y mettre un secret ?
2. Pourquoi garde-t-on l'access token en mémoire plutôt que dans le `localStorage` ?
3. À quoi sert `restoreSession()` et pourquoi doit-elle être appelée avant le rendu ?
4. Pourquoi le frontend n'envoie-t-il pas les prix dans la commande ?

EXERCICES DE CODE

5. Écris `adminProductService.listAdminProducts()` version connectée (avec `fromApiProduct`).
6. Écris `updateAdminProductStock(id, { stock, threshold })` version connectée, avec traduction des erreurs de champs.
7. Écris `toApiProduct(values)` qui produit `{ name, description, price, categoryId, variants }`.

DÉBOGAGE

8. Après une modification de `VITE_API_URL` dans `.env.development`, l'ancienne adresse est toujours utilisée. Pourquoi ?
9. La page Produits affiche « Chargement… » sans fin et la console ne montre aucune erreur. Cite deux causes possibles dans `useApi`.

MINI-PROJET

10. Fais la liste de **tous** les fichiers du frontend qui changeraient pour connecter la page Catégories (service, mapper, composants), et de ceux qui ne changeraient pas.

POUR RÉFLÉCHIR

11. Pourquoi garde-t-on la validation du formulaire côté frontend alors que le backend valide déjà ?
12. Écris `deleteAdminProduct(id)` version connectée. L'ancienne version renvoyait la liste restante : conserve cette signature pour que les pages n'aient pas à changer.
13. Pourquoi est-il avantageux que les pages d'administration n'appellent que les fonctions d'un service plutôt que `fetch` directement ? Donne un exemple concret tiré de LAWPIKING.

PARTIE 11

Les modules LAWPIKING

Les dix modules du backend, un par un : objectif, routes, règles, requêtes Prisma, tests et lien avec le frontend.

11.1 Comment lire cette partie

Chaque module suit **la même fiche**, puis les extraits de code qui comptent. Les fichiers respectent la structure de la partie 7 : `routes`, `controller`, `service`, `repository` et `schema` (par exemple `products.service.js`). Quand un fichier ressemble à ce que tu as déjà vu, il n'est pas répété.

Dans les tableaux de routes, l'accès est noté **PUBLIC** (sans connexion) ou **ADMIN** (`authenticate` + `requireAdmin`).

Module	Route de base	Tables	Page du frontend concernée
auth	`/api/auth`	`users`	Connexion
users	`/api/users`	`users`	Paramètres (compte)
categories	`/api/categories`	`categories`	Catégories
products	`/api/products`	`products`	Produits, boutique
images	`/api/products/:id/images`	`product_images`	Formulaire produit
stock et variantes	`/api/stock`, `/api/variants`	`product_variants`	Stock
orders	`/api/orders`	`orders`, `order_items`	Commandes, panier
reviews	`/api/reviews`	`reviews`	Avis
promotions	`/api/promotions`	`promotions`, `promotion_products`	Promotions
settings	`/api/settings`	`settings`	Paramètres

11.2 Le module auth

Fiche	
Objectif	Connecter l'administrateur, renouveler sa session, le déconnecter.
Données	`users` (email, `passwordHash`, rôle, `isActive`).
Validations	`loginSchema` (email valide, mot de passe non vide).
Règles du service	Message d'erreur unique ; compte actif et `ADMIN` obligatoires ; refresh relit l'utilisateur.
Prisma	`user.findUnique({ where: { email } })`, `user.findUnique({ where: { id } })`.
Tests	Bon mot de passe = 200 + cookie ; mauvais = 401 ; 6e tentative = 429 ; jeton expiré = 401 ; compte désactivé = 401 au refresh.
Frontend	`authService.js` : `login`, `logout`, `restoreSession` (partie 10).

Méthode	Route	Accès
POST	`/api/auth/login`	PUBLIC (limité à 5 essais / 15 min)
POST	`/api/auth/refresh`	PUBLIC (cookie)
POST	`/api/auth/logout`	PUBLIC
GET	`/api/auth/me`	Connecté

Le code complet est en partie 8.

11.3 Le module users

Fiche	
Objectif	Lire les comptes et permettre à l'administrateur de changer son mot de passe. Aucune inscription publique.
Données	`users`.
Validations	`changePasswordSchema` : ancien mot de passe, nouveau de 12 caractères minimum, confirmation identique.
Règles du service	Vérifier l'ancien mot de passe ; ne **jamais** renvoyer `passwordHash`.
Prisma	`findUnique`, `update`, `select` explicite.
Tests	Ancien mot de passe faux = 401 ; nouveau trop court = 422 ; la réponse ne contient jamais le hachage.
Frontend	Page Paramètres (changement de mot de passe, à ajouter).

Méthode	Route	Accès
GET	`/api/users`	ADMIN
PATCH	`/api/users/me/password`	ADMIN

src/modules/users/users.repository.js JavaScript

```
1 import { prisma } from "../../config/prisma.js";
2
3 export const publicFields = {
4   id: true, email: true, fullName: true, role: true, isActive: true, createdAt: true,
```

```
};

export const findByEmail = (email) => prisma.user.findUnique({ where: { email } });
export const findById = (id) => prisma.user.findUnique({ where: { id } });

export const list = () =>
  prisma.user.findMany({ select: publicFields, orderBy: { createdAt: "desc" } });

export const updatePassword = (id, passwordHash) =>
  prisma.user.update({ where: { id }, data: { passwordHash }, select: publicFields });
```

`findByEmail` et `findById` renvoient l'utilisateur **complet** (le service d'authentification a besoin du hachage). Toutes les routes qui sortent vers le client passent par `select: publicFields`.

11.4 Le module categories

Fiche	
Objectif	Gérer T-shirts, Pulls, Bonnets et toute nouvelle catégorie.
Données	`categories` (slug, nom, description, `isEnabled`, `sortOrder`).
Validations	Nom obligatoire (60 caractères max.), description (200 max.), `isEnabled` booléen.
Règles du service	Nom unique sans tenir compte de la casse ni des accents ; **slug unique généré** ; une catégorie qui contient encore des produits ne peut pas être supprimée (409).
Prisma	`findMany` + `_count`, `create`, `update`, `delete`, `count`.
Tests	Doublon « t-shirts » = 409 ; suppression d'une catégorie non vide = 409 ; liste publique sans catégories désactivées.
Frontend	`adminCategoryService.js`, `categoryStore.js`, page Catégories.

Méthode	Route	Accès
GET	`/api/categories`	PUBLIC (catégories actives) ; ADMIN avec `?all=true`
POST	`/api/categories`	ADMIN
PUT	`/api/categories/:id`	ADMIN
DELETE	`/api/categories/:id`	ADMIN

src/modules/categories/categories.service.js JavaScript

```
import { AppError } from "../../utils/AppError.js";
import { slugify } from "../../utils/slugify.js";
import * as repo from "./categories.repository.js";

export async function create(data) {
  if (await repo.nameExists(data.name)) {
    throw new AppError("Une catégorie porte déjà ce nom", 409, "CATEGORY_NAME_TAKEN");
  }
  return repo.create({ ...data, slug: slugify(data.name) });
}

export async function remove(id) {
  const category = await repo.findById(id);
  if (!category) throw new AppError("Catégorie introuvable", 404, "CATEGORY_NOT_FOUND");

  const productCount = await repo.countActiveProducts(id);
```

```
  if (productCount > 0) {
    throw new AppError(
      `Cette catégorie contient ${productCount} produit(s)`, 409, "CATEGORY_NOT_EMPTY",
    );
  }
  await repo.remove(id);
}
```

src/modules/categories/categories.repository.js (extraits) JavaScript

```
import { prisma } from "../../config/prisma.js";

export const nameExists = async (name, excludeId) =>
  (await prisma.category.count({
    where: {
      name: { equals: name, mode: "insensitive" },
      NOT: excludeId ? { id: excludeId } : undefined,
    },
  })) > 0;

export const countActiveProducts = (categoryId) =>
  prisma.product.count({ where: { categoryId, deletedAt: null } });

export const list = ({ onlyEnabled }) =>
  prisma.category.findMany({
    where: onlyEnabled ? { isEnabled: true } : {},
    orderBy: [{ sortOrder: "asc" }, { name: "asc" }],
    include: { _count: { select: { products: { where: { deletedAt: null } } } } },
  });
```

`_count` ajoute **le nombre de produits** de chaque catégorie, ce que ta page Catégories affiche déjà.

11.5 Le module products

La création est détaillée en partie 7 ; les images en partie 9.

Fiche	
Objectif	Catalogue complet : lister avec filtres, voir, créer, modifier, activer/désactiver, supprimer (doucement).
Données	`products`, relations `category`, `images`, `variants`.
Validations	Celles de ton formulaire : nom 80, description 1000, prix entier > 0 et ≤ 10 000 000, catégorie valide, au plus 12 couleurs.
Règles du service	Catégorie active ; nom unique ; slug unique (`-2`, `-3`) ; suppression douce (`deletedAt`) ; la boutique ne voit que `isActive` et non supprimé.
Prisma	`findMany` (+ `count`, pagination), `findFirst`, `create` imbriqué, `update`.
Tests	Création valide = 201 ; prix négatif = 422 ; nom en double = 409 ; produit supprimé absent de la liste publique ; non connecté = 401.
Frontend	`adminProductService.js`, `productService.js`, pages Produits, ajout, modification.

Méthode	Route	Accès
GET	`/api/products`	PUBLIC (actifs) ; ADMIN avec `?all=true`
GET	`/api/products/:id`	PUBLIC
POST	`/api/products`	ADMIN
PUT	`/api/products/:id`	ADMIN
PATCH	`/api/products/:id/status`	ADMIN (activer / désactiver)
DELETE	`/api/products/:id`	ADMIN (suppression douce)

src/modules/products/products.repository.js (liste filtrée) JavaScript

```
export async function list({ search, category, onlyActive, skip, take }) {
  const where = {
    deletedAt: null,
    ...(onlyActive ? { isActive: true, category: { isEnabled: true } } : {}),
    ...(category ? { category: { slug: category } } : {}),
    ...(search ? { name: { contains: search, mode: "insensitive" } } : {}),
  };
  const [total, products] = await prisma.$transaction([
    prisma.product.count({ where }),
    prisma.product.findMany({
      where, skip, take, orderBy: { createdAt: "desc" }, include: withRelations,
    }),
  ]);
  return { total, products };
}
```

Le `where` est **construit morceau par morceau** : chaque filtre n'est ajouté que s'il est demandé. Le contrôleur décide `onlyActive` : vrai pour un visiteur, faux pour l'administrateur qui demande `?all=true`.

11.6 Le module images

Voir la partie 9 pour le code complet.

Fiche	
Objectif	Ajouter, remplacer, définir comme principale et supprimer les photos d'un produit.
Données	`product_images` (`url`, `altText`, `position`, `isMain`).
Validations	Fichier JPEG/PNG/WebP, 5 Mo maximum, signature vérifiée.
Règles du service	Fichier **et** ligne supprimés ensemble ; une seule image principale ; première image = principale par défaut.
Prisma	`productImage.create`, `delete`, `updateMany` dans une transaction.
Tests	`.exe` renommé `.jpg` = 415 ; image de 8 Mo = 413 ; suppression retire fichier et ligne.
Frontend	`imageService.js`, composant `ProductImageField`.

Méthode	Route	Accès
POST	`/api/products/:id/images`	ADMIN (`multipart/form-data`)
PATCH	`/api/products/:id/images/:imageId/main`	ADMIN
DELETE	`/api/products/:id/images/:imageId`	ADMIN

11.7 Le module stock et variantes

Fiche	
Objectif	Gérer les quantités par taille et couleur, et calculer le statut (en stock / bas / rupture).
Données	`product_variants` (taille, couleur, `stock`, `lowStockThreshold`).
Validations	Stock entier entre 0 et 100 000 ; seuil entier ≥ 0.
Règles du service	Le statut est **calculé**, jamais stocké ; combinaison taille/couleur unique par produit ; le stock ne devient jamais négatif.
Prisma	`productVariant.update`, `findMany` avec `include: { product }`, `groupBy`.
Tests	Stock -1 = 422 ; mise à jour du stock reflétée dans `GET /api/stock` ; variante en double = 409.
Frontend	`adminProductService.updateAdminProductStock`, page Stock.

Méthode	Route	Accès
GET	`/api/stock`	ADMIN (variantes avec leur statut)
POST	`/api/products/:id/variants`	ADMIN
PATCH	`/api/variants/:id/stock`	ADMIN (`{ stock, lowStockThreshold }`)
DELETE	`/api/variants/:id`	ADMIN

La règle de statut est la **même** que celle de ton frontend (`getStockStatus`), exprimée en JavaScript côté serveur :

src/modules/stock/stockStatus.js JavaScript

```
export function getStockStatus({ stock, lowStockThreshold }) {
  if (!(stock > 0)) return "out";
  return stock <= lowStockThreshold ? "low" : "in";
}
```

src/modules/stock/stock.service.js JavaScript

```
import * as repo from "./stock.repository.js";
import { getStockStatus } from "./stockStatus.js";

export async function listStock() {
  const variants = await repo.listVariants();
  return variants.map((variant) => ({ ...variant, status: getStockStatus(variant) }));
}

export async function updateStock(id, { stock, lowStockThreshold }) {
  const updated = await repo.updateVariant(id, { stock, lowStockThreshold });
  return { ...updated, status: getStockStatus(updated) };
}
```

`repo.updateVariant` appelle `prisma.productVariant.update(...)`. Si l'`id` n'existe pas, Prisma lance `P2025`, traduit automatiquement en 404 par le gestionnaire central : pas de `if` à écrire.

11.8 Le module orders

C'est le module le plus délicat : il touche aux **prix**, au **stock** et à l'**historique**.

Fiche	
Objectif	Enregistrer les commandes des clients, les suivre et changer leur statut.
Données	`orders`, `order_items` (instantané du nom et du prix), `product_variants` (stock).
Validations	Nom (80), téléphone, 1 à 30 articles ; chaque article : `variantId` valide, quantité 1 à 20.
Règles du service	**Prix lus en base** (jamais envoyés par le client) ; promotion active appliquée ; stock vérifié **et** retiré dans une transaction ; transitions de statut contrôlées ; l'annulation **rend** le stock.
Prisma	`$transaction`, `productVariant.updateMany`, `order.create` imbriqué, `findMany` paginé.
Tests	Stock insuffisant = 409 et aucun stock retiré ; prix envoyé par le client ignoré ; passage `DELIVERED` vers `PENDING` refusé ; annulation rend le stock.
Frontend	`adminOrderService.js`, page Commandes, panier / commande WhatsApp.

Méthode	Route	Accès
POST	`/api/orders`	PUBLIC (limité : 10 par heure et par adresse IP)
GET	`/api/orders`	ADMIN (filtres `status`, `search`, pagination)
GET	`/api/orders/:id`	ADMIN
PATCH	`/api/orders/:id/status`	ADMIN
DELETE	`/api/orders/:id`	ADMIN (réservé aux commandes `CANCELLED`)

src/modules/orders/orders.schema.js JavaScript

```
import { z } from "zod";

export const createOrderSchema = z.object({
  customerName: z.string().trim().min(2, "Nom obligatoire").max(80),
  customerPhone: z.string().trim().regex(/^[+0-9 ]{8,20}$/, "Téléphone invalide"),
  customerNote: z.string().trim().max(500).default(""),
  items: z
    .array(
      z.object({
        variantId: z.string().uuid(),
        quantity: z.coerce.number().int().min(1).max(20),
      }),
    )
    .min(1, "Le panier est vide")
    .max(30),
});

export const updateOrderStatusSchema = z.object({
  status: z.enum(["PENDING", "CONFIRMED", "DELIVERED", "CANCELLED"]),
});
```

Les transitions de statut autorisées :

De	Vers	Remarque
PENDING	CONFIRMED, CANCELLED	
CONFIRMED	DELIVERED, CANCELLED	
DELIVERED	(aucune)	Une commande livrée est **définitive**
CANCELLED	(aucune)	Le stock a déjà été rendu

src/modules/orders/orders.service.js JavaScript

```
import { randomInt } from "node:crypto";
import { AppError } from "../../utils/AppError.js";
import * as repo from "./orders.repository.js";
import * as promotions from "../promotions/promotions.service.js";

const TRANSITIONS = {
  PENDING: ["CONFIRMED", "CANCELLED"],
  CONFIRMED: ["DELIVERED", "CANCELLED"],
  DELIVERED: [],
  CANCELLED: [],
};

function makeReference(now = new Date()) {
  const day = now.toISOString().slice(0, 10).replaceAll("-", "");
  return `LP-${day}-${String(randomInt(0, 10000)).padStart(4, "0")}`;
}

export async function create(input) {
  const ids = input.items.map((item) => item.variantId);
  const variants = await repo.findVariantsWithProduct(ids);
  if (variants.length !== new Set(ids).size) {
    throw new AppError("Un article n'existe plus", 404, "VARIANT_NOT_FOUND");
  }

  const items = [];
  let totalAmount = 0;
  for (const wanted of input.items) {
    const variant = variants.find((v) => v.id === wanted.variantId);
    const { product } = variant;
    if (!product.isActive || product.deletedAt) {
      const message = `${product.name} n'est plus disponible`;
      throw new AppError(message, 409, "PRODUCT_UNAVAILABLE");
    }
    const percent = await promotions.activePercentFor(product.id);
    const unitPrice = Math.round(product.price * (100 - percent) / 100);
    totalAmount += unitPrice * wanted.quantity;
    items.push({
      productId: product.id, variantId: variant.id, productName: product.name,
      size: variant.size, colorName: variant.colorName, unitPrice, quantity: wanted.quantity,
    });
  }

  const order = {
    reference: makeReference(),
    customerName: input.customerName,
    customerPhone: input.customerPhone,
    customerNote: input.customerNote,
    totalAmount,
  };
  return repo.createWithStock(order, items);
}

```

```
export async function changeStatus(id, nextStatus) {
  const order = await repo.findById(id);
  if (!order) throw new AppError("Commande introuvable", 404, "ORDER_NOT_FOUND");
  if (!TRANSITIONS[order.status].includes(nextStatus)) {
    throw new AppError(
      `Passage impossible : ${order.status} vers ${nextStatus}`, 409, "INVALID_TRANSITION",
    );
  }
  return repo.updateStatus(order, nextStatus);
}
```

Point	Explication
`findVariantsWithProduct(ids)`	Lit **en base** variantes et produits : les prix viennent d'ici.
`new Set(ids).size`	Détecte un identifiant inconnu (et ignore les doublons du panier).
`promotions.activePercentFor`	Pourcentage de la promotion en cours pour ce produit (0 s'il n'y en a pas).
Instantané `items`	Nom, taille, couleur et **prix payé** figés : la partie 5 expliquait pourquoi.
`makeReference()`	Une référence lisible (`LP-20261031-0042`). La contrainte `UNIQUE` garantit l'unicité ; en cas de collision très rare, le gestionnaire renvoie un 409 et le client réessaie.
`TRANSITIONS`	Le **plan** des changements autorisés, lisible d'un coup d'œil.

Côté repository, deux opérations utilisent une transaction : `createWithStock` (vue en partie 6 : vérification et retrait atomiques du stock puis création) et `updateStatus`, qui **rend le stock** si la commande passe à `CANCELLED` :

src/modules/orders/orders.repository.js (extrait) JavaScript

```
export function updateStatus(order, status) {
  return prisma.$transaction(async (tx) => {
    if (status === "CANCELLED") {
      for (const item of order.items) {
        if (!item.variantId) continue;
        await tx.productVariant.update({
          where: { id: item.variantId },
          data: { stock: { increment: item.quantity } },
        });
      }
    }
    return tx.order.update({
      where: { id: order.id },
      data: { status },
      include: { items: true },
    });
  });
}
```

Le helper `toOrderItemData` de la partie 6 est simplement l'identité (`(item) => item`) ici, puisque `items` est déjà au bon format.

11.9 Le module reviews

Fiche	
Objectif	Recevoir les avis des clients, les **modérer**, n'afficher que ceux qui sont approuvés.
Données	`reviews` (`rating` 1 à 5, `comment`, `status`).
Validations	Nom (60), note entière 1 à 5, commentaire (1000 max.).
Règles du service	Tout nouvel avis est `PENDING` ; seuls les `APPROVED` sont publics ; limite d'envoi par IP (anti-spam) ; le texte est **affiché échappé** par React (jamais en HTML brut).
Prisma	`review.create`, `findMany` filtré, `update`, `aggregate` (`_avg`, `_count`).
Tests	Note 6 = 422 ; avis publié absent de la liste publique tant qu'il est `PENDING` ; approuvé = visible ; modération sans connexion = 401.
Frontend	`adminReviewService.js`, page Avis, fiche produit.

Méthode	Route	Accès
POST	`/api/products/:id/reviews`	PUBLIC (limité : 5 par heure et par adresse IP)
GET	`/api/products/:id/reviews`	PUBLIC (`APPROVED` seulement)
GET	`/api/reviews`	ADMIN (filtre `status`)
PATCH	`/api/reviews/:id/status`	ADMIN (`APPROVED` ou `REJECTED`)
DELETE	`/api/reviews/:id`	ADMIN

src/modules/reviews/reviews.repository.js (extraits) JavaScript

```
export const create = (productId, data) =>
  prisma.review.create({ data: { ...data, productId, status: "PENDING" } });

export const listApproved = (productId) =>
  prisma.review.findMany({
    where: { productId, status: "APPROVED" },
    orderBy: { createdAt: "desc" },
    select: { id: true, authorName: true, rating: true, comment: true, createdAt: true },
  });

export const summary = (productId) =>
  prisma.review.aggregate({
    where: { productId, status: "APPROVED" },
    _avg: { rating: true },
    _count: true,
  });
```

Remarque : `create` impose `status: "PENDING"` **après** `...data` : même si un client ajoutait `status: "APPROVED"` dans sa requête, la valeur serait écrasée. C'est la défense contre le *mass assignment* (partie 8).

11.10 Le module promotions

Fiche	
Objectif	Créer des promotions en pourcentage sur certains produits, pour une période donnée.
Données	`promotions`, `promotion_products` (table de liaison).
Validations	Nom (80), pourcentage entre 1 et 90, `endsAt` strictement après `startsAt`, au moins un produit.
Règles du service	Statut **calculé** (à venir, en cours, terminée) ; si plusieurs promotions s'appliquent à un produit, **la plus avantageuse** gagne ; produits supprimés ignorés.
Prisma	`create` avec `products: { create: [...] }`, `findMany` avec `include`, `deleteMany` + `createMany` à la modification.
Tests	Fin avant début = 422 ; prix en commande réduit pendant la promotion ; après `endsAt`, plein tarif ; produit inexistant = 404.
Frontend	`adminPromotionService.js`, `promotionValidation.js`, page Promotions.

Méthode	Route	Accès
GET	`/api/promotions`	ADMIN
GET	`/api/promotions/active`	PUBLIC (promotions en cours avec leurs produits)
POST	`/api/promotions`	ADMIN
PUT	`/api/promotions/:id`	ADMIN
DELETE	`/api/promotions/:id`	ADMIN

src/modules/promotions/promotions.schema.js JavaScript

```
import { z } from "zod";

export const promotionSchema = z
  .object({
    name: z.string().trim().min(1, "Le nom est obligatoire").max(80),
    percentOff: z.coerce.number().int().min(1).max(90),
    startsAt: z.coerce.date(),
    endsAt: z.coerce.date(),
    isActive: z.boolean().default(true),
    productIds: z.array(z.string().uuid()).min(1, "Choisissez au moins un produit"),
  })
  .refine((p) => p.endsAt > p.startsAt, {
    message: "La fin doit être après le début",
    path: ["endsAt"],
  });
```

`.refine` ajoute une règle qui compare **plusieurs champs** ; `path` indique sous quel champ afficher l'erreur.

src/modules/promotions/promotions.service.js (extrait) JavaScript

```
export function getPromotionStatus({ isActive, startsAt, endsAt }, now = new Date()) {
  if (!isActive) return "disabled";
  if (now < startsAt) return "upcoming";
  if (now > endsAt) return "ended";
  return "running";
}

```

```
export async function activePercentFor(productId, now = new Date()) {
  const rows = await repo.findRunningForProduct(productId, now);
  return rows.reduce((best, promo) => Math.max(best, promo.percentOff), 0);
}
```

src/modules/promotions/promotions.repository.js (extrait) JavaScript

```
export const findRunningForProduct = (productId, now) =>
  prisma.promotion.findMany({
    where: {
      isActive: true,
      startsAt: { lte: now },
      endsAt: { gte: now },
      products: { some: { productId } },
    },
    select: { percentOff: true },
  });
```

`products: { some: { productId } }` signifie « au moins une ligne de liaison concerne ce produit ». Le service des commandes utilise `activePercentFor` : **l'unique source de vérité** du prix remisé est ici.

11.11 Le module settings

Fiche	
Objectif	Gérer les réglages de la boutique (nom, numéro WhatsApp, devise…).
Données	`settings` (clé / valeur JSON).
Validations	**Liste blanche** de clés autorisées, chacune avec sa règle (le numéro WhatsApp : chiffres seulement, 8 à 15).
Règles du service	Une clé inconnue est refusée ; seules les clés **publiques** sortent sans connexion.
Prisma	`setting.findMany`, `upsert`.
Tests	Clé inconnue = 422 ; numéro invalide = 422 ; `GET /api/settings/public` ne renvoie pas les clés privées.
Frontend	`adminSettingsService.js`, page Paramètres, numéro WhatsApp de la boutique.

Méthode	Route	Accès
GET	`/api/settings/public`	PUBLIC (nom, numéro WhatsApp, devise)
GET	`/api/settings`	ADMIN
PUT	`/api/settings`	ADMIN

src/modules/settings/settings.schema.js JavaScript

```
import { z } from "zod";

export const settingsSchema = z
  .object({
    shop_name: z.string().trim().min(1).max(60),
    whatsapp_number: z.string().regex(/^[0-9]{8,15}$/, "Chiffres uniquement, 8 à 15"),
    currency: z.enum(["XOF"]),
  })
```

```
  .partial()
  .strict();

export const PUBLIC_KEYS = ["shop_name", "whatsapp_number", "currency"];
```

`.strict()` **refuse** toute clé non prévue (impossible d'écrire n'importe quoi dans la table). Le service écrit chaque clé reçue avec un `upsert` dans une transaction, et `GET /api/settings/public` ne renvoie que `PUBLIC_KEYS`, au format `{ shop_name: "LAWPIKING", ... }`.

11.12 Brancher tous les modules

src/app.js (routes finales) JavaScript

```
app.use("/api/auth", authRouter);
app.use("/api/users", usersRouter);
app.use("/api/categories", categoriesRouter);
app.use("/api/products", productsRouter);
app.use("/api/stock", stockRouter);
app.use("/api/variants", variantsRouter);
app.use("/api/orders", ordersRouter);
app.use("/api/reviews", reviewsRouter);
app.use("/api/promotions", promotionsRouter);
app.use("/api/settings", settingsRouter);
```

Les routes d'avis liées à un produit (`/api/products/:id/reviews`) et d'images se déclarent dans le routeur des produits.

À RETENIR — En résumé — Partie 11

- Chaque module suit la même fiche : objectif, données, routes, validations, règles, Prisma, tests, lien frontend.
- Les prix, le stock et les statuts sont décidés par le **service**, jamais par le client.
- Une commande est **atomique** : vérification du stock, retrait et création dans une transaction.
- Avis et promotions utilisent des statuts et des dates ; les réglages passent par une **liste blanche** de clés.
- Chaque route se teste avec les cas nominal, invalide, non connecté, sans droit et introuvable.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Décrire les routes de chaque module	Le tableau d'ensemble
Écrire une commande atomique	Le module orders
Modérer des avis	Le module reviews
Calculer le prix remisé côté serveur	Le module promotions
Protéger les réglages par liste blanche	Le module settings

11.13 Exercices de la partie 11

QUESTIONS DE COMPRÉHENSION

1. Pourquoi le client envoie-t-il seulement `variantId` et `quantity` dans une commande ?
2. Pourquoi le statut d'une promotion est-il calculé et non stocké ?
3. Que protège `.strict()` dans le schéma des réglages ?
4. Pourquoi une commande `DELIVERED` ne peut-elle plus changer de statut ?

EXERCICES DE CODE

5. Écris `ordersRepository.createWithStock(order, items)` complet, avec `toOrderItemData`.
6. Écris le service `reviews.moderate(id, status)` qui n'accepte que `APPROVED` ou `REJECTED`.
7. Écris la route `GET /api/dashboard` (ADMIN) qui renvoie : nombre de commandes par statut, chiffre d'affaires des commandes livrées, nombre de produits en rupture.

DÉBOGAGE

8. Une commande de 2 articles est créée, mais le stock n'a pas diminué. Quelle ligne manque dans `createWithStock` ?
9. `GET /api/products` renvoie des produits dont la catégorie est désactivée. Quel filtre manque dans le repository ?

MINI-PROJET

10. Ajoute le module **contact** (messages du formulaire de contact) : table, migration, route publique limitée en fréquence, route admin de lecture et de marquage « lu ».

PARTIE 12

Tests et débogage

Vérifier que l'API fait ce qu'elle doit, trouver les bugs vite, et ne plus avoir peur de modifier le code.

12.1 Pourquoi tester ?

Chaque fois que tu modifies le code, tu risques de casser autre chose sans t'en apercevoir. Les tests sont un **filet de sécurité** : quelques secondes pour vérifier que les règles de la boutique fonctionnent encore. Il existe trois niveaux :

Niveau	Ce qu'on teste	Vitesse	Exemple LAWPIKING
Manuel	Les routes, avec un outil graphique	Lent, à refaire à la main	Créer un produit dans Postman
Unitaire	Une **fonction** seule, sans base ni serveur	Très rapide	`getStockStatus`, `slugify`, `getPagination`
Intégration	Plusieurs couches ensemble : route, service, base	Rapide à moyen	`POST /api/orders` avec une vraie base de test

12.2 Tester à la main avec Postman ou Insomnia

Postman et **Insomnia** sont des applications gratuites pour envoyer des requêtes HTTP sans écrire de code. Installe l'une des deux (postman.com ou insomnia.rest).

Organiser une collection

Crée une **collection** « LAWPIKING API » avec des **dossiers** par module (Auth, Products, Orders…) et une **variable d'environnement** :

Variable	Valeur
`baseUrl`	`http://localhost:3000`
`accessToken`	(vide au départ ; remplie après la connexion)

Les requêtes utilisent `{{baseUrl}}/api/products`. Pour les routes protégées, onglet **Authorization**, type **Bearer Token**, valeur `{{accessToken}}`.

Mémoriser automatiquement le jeton

Dans la requête **Login**, onglet **Scripts** (ou *Tests* selon la version), ajoute :

```
const body = pm.response.json();
pm.environment.set("accessToken", body.accessToken);
```

Après chaque connexion réussie, `accessToken` est mis à jour tout seul : toutes les autres requêtes en profitent. (Insomnia propose la même chose avec une référence « Réponse » dans le champ du jeton.)

Une variante sans outil : un fichier .http

Avec l'extension VS Code **REST Client**, un fichier texte versionné avec le code remplace Postman pour les cas simples :

requests.http HTTP

```http
@baseUrl = http://localhost:3000

### Santé de l'API
GET {{baseUrl}}/api/health

### Connexion
# @name login
POST {{baseUrl}}/api/auth/login
Content-Type: application/json

{ "email": "admin@lawpiking.sn", "password": "{{$dotenv ADMIN_PASSWORD}}" }

### Créer un produit
POST {{baseUrl}}/api/products
Authorization: Bearer {{login.response.body.accessToken}}
Content-Type: application/json

{ "name": "BORDEAUX", "price": 10000, "categoryId": "REMPLACE-PAR-UN-UUID" }
```

12.3 Que faut-il tester sur chaque route ?

Pour **chaque** route, vérifie systématiquement cette grille :

Cas	Ce qu'on envoie	Réponse attendue
Nominal	Des données valides, connecté en administrateur	200 / 201 / 204
Validation	Champ manquant, prix négatif, texte trop long	422 avec `details` par champ
JSON cassé	`{ "name":`	400 `INVALID_JSON`
Non connecté	Pas d'en-tête `Authorization`	401
Jeton expiré ou falsifié	Un jeton modifié d'un caractère	401
Sans permission	Compte non administrateur	403
Introuvable	Un `id` qui n'existe pas	404
Conflit	Nom ou slug déjà utilisé, stock insuffisant	409
Abus	6 connexions ratées d'affilée	429
Pas de fuite	Lire les erreurs 500 et les réponses utilisateur	Aucun `passwordHash`, aucune trace de pile

> **BONNE PRATIQUE — Teste aussi ce que le frontend ne fera jamais**
> Envoie un `role: "ADMIN"` dans une requête de création, un prix sous forme de texte, un `id` qui n'est pas un UUID, un champ de 1 Mo. Un attaquant, lui, ne passe pas par ton formulaire.

12.4 Les tests unitaires avec le test runner de Node.js

Depuis Node.js 20, Node propose son propre outil de test : `node:test`, sans rien installer. Les fichiers `*.test.js` sont découverts automatiquement.

tests/utils.test.js JavaScript

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { slugify } from "../src/utils/slugify.js";
import { getPagination } from "../src/utils/pagination.js";
import { getStockStatus } from "../src/modules/stock/stockStatus.js";

test("slugify retire les accents et les espaces", () => {
  assert.equal(slugify("ROUGE CARDINAL"), "rouge-cardinal");
  assert.equal(slugify("  Bleu Nuit Élégant! "), "bleu-nuit-elegant");
});

test("getPagination plafonne et corrige les valeurs", () => {
  assert.deepEqual(getPagination({}), { page: 1, limit: 10, skip: 0 });
  const third = getPagination({ page: "3", limit: "10" });
  assert.deepEqual(third, { page: 3, limit: 10, skip: 20 });
  assert.equal(getPagination({ limit: "100000" }).limit, 50);
  assert.equal(getPagination({ page: "-4" }).page, 1);
});

test("getStockStatus", async (t) => {
  await t.test("rupture", () => {
    assert.equal(getStockStatus({ stock: 0, lowStockThreshold: 5 }), "out");
  });
  await t.test("stock bas (stock égal au seuil)", () => {
    assert.equal(getStockStatus({ stock: 5, lowStockThreshold: 5 }), "low");
  });
  await t.test("en stock", () => {
    assert.equal(getStockStatus({ stock: 6, lowStockThreshold: 5 }), "in");
  });
});
```

Élément	Explication
`test("nom", fonction)`	Définit un test : un nom lisible et une fonction qui vérifie.
`assert.equal(réel, attendu)`	Échoue si les deux valeurs diffèrent. Avec `node:assert/strict`, la comparaison est stricte (`===`).
`assert.deepEqual`	Compare le **contenu** d'objets ou de tableaux.
`t.test(...)`	Sous-tests regroupés sous un même titre.

Ajoute dans `package.json` : `"test": "node --test"`, puis :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm test
✔ slugify retire les accents et les espaces (1.2ms)
✔ getPagination plafonne et corrige les valeurs (0.5ms)
▶ getStockStatus
  ✔ rupture (0.2ms)
  ✔ stock bas (stock égal au seuil) (0.2ms)
  ✔ en stock (0.2ms)
```

```
▶ getStockStatus (3.1ms)
ℹ tests 6
ℹ pass 6
ℹ fail 0
```

La durée affichée varie d'une machine à l'autre ; ce qui compte : `fail 0`. Ces fonctions pures sont les plus faciles à tester : pas de base, pas de serveur.

> **BONNE PRATIQUE — Garde les fonctions de calcul dans des fichiers « purs »**
> Un fichier de test unitaire ne doit pas démarrer toute l'application. C'est pourquoi `getStockStatus` vit dans son propre fichier, `stockStatus.js`, qui n'importe **rien**. Si on l'avait laissée dans `stock.service.js`, l'import de ce fichier aurait chargé le repository, donc Prisma et `env.js`, qui exige `DATABASE_URL` et arrête le programme s'il manque. Règle : **les règles de calcul dans des fichiers sans dépendance**, les fichiers qui parlent à la base ailleurs.

12.5 Les tests d'intégration avec Supertest

Rappelle-toi : `app.js` **exporte** l'application **sans l'écouter sur un port** (partie 3). **Supertest** envoie des requêtes directement à cette application, sans serveur réel.

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install --save-dev supertest
```

Une base de test dédiée

Un test ne doit **jamais** utiliser ta vraie base de développement : il y crée et efface des données. Crée une base séparée :

```
CREATE DATABASE lawpiking_test OWNER lawpiking_user;
```

.env.test .env

```
1 DATABASE_URL="postgresql://lawpiking_user:ton_mot_de_passe@localhost:5432/lawpiking_test"
2 NODE_ENV=test
3 FRONTEND_URL="http://localhost:5173"
4 JWT_ACCESS_SECRET="secret-de-test-uniquement-aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
5 JWT_REFRESH_SECRET="secret-de-test-uniquement-bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb"
```

Applique les migrations sur cette base et lance les tests avec ce fichier :

TERMINAL Windows PowerShell

```
PS> $env:DATABASE_URL = "postgresql://lawpiking_user:MDP@localhost/lawpiking_test"
PS ...\lawpiking-backend> npx prisma migrate deploy
PS ...\lawpiking-backend> node --env-file=.env.test --test tests/
```

Pour les tests, `node --env-file=.env.test` charge les variables **avant** tout le reste : elles ont priorité sur `.env` (que `dotenv` ne remplace pas).

> **ATTENTION — Sécurité : ne lance jamais les tests sur la base de production**
> Les tests vident des tables. Vérifie toujours le nom de la base dans `DATABASE_URL` avant de lancer. Un garde-fou simple : refuser de démarrer si l'URL ne contient pas `_test`.

Un premier test d'intégration

tests/products.test.js JavaScript

```javascript
import { test, before, after, beforeEach } from "node:test";
import assert from "node:assert/strict";
import request from "supertest";
import bcrypt from "bcryptjs";
import app from "../src/app.js";
import { prisma } from "../src/config/prisma.js";

let token;
let categoryId;

before(async () => {
  assert.match(process.env.DATABASE_URL, /_test/, "Les tests exigent une base _test");
  await prisma.user.deleteMany();
  await prisma.user.create({
    data: {
      email: "admin@test.sn",
      passwordHash: await bcrypt.hash("MotDePasseDeTest-12", 4),
      fullName: "Admin Test",
      role: "ADMIN",
    },
  });
  const login = await request(app)
    .post("/api/auth/login")
    .send({ email: "admin@test.sn", password: "MotDePasseDeTest-12" });
  token = login.body.accessToken;
});

beforeEach(async () => {
  await prisma.product.deleteMany();
  await prisma.category.deleteMany();
  const category = await prisma.category.create({
    data: { slug: "tshirts", name: "T-shirts" },
  });
  categoryId = category.id;
});

after(async () => {
  await prisma.$disconnect();
});

test("crée un produit valide", async () => {
  const response = await request(app)
    .post("/api/products")
    .set("Authorization", `Bearer ${token}`)
    .send({ name: "BORDEAUX", price: 10000, categoryId });

  assert.equal(response.status, 201);
  assert.equal(response.body.slug, "tshirt-bordeaux");
});

test("refuse un prix négatif (422)", async () => {
  const response = await request(app)
    .post("/api/products")
    .set("Authorization", `Bearer ${token}`)
    .send({ name: "BORDEAUX", price: -5, categoryId });

  assert.equal(response.status, 422);
  assert.ok(response.body.error.details.price);
});
```

```
60
61 test("refuse sans connexion (401)", async () => {
62   const response = await request(app).post("/api/products").send({});
63   assert.equal(response.status, 401);
64 });
65
66 test("refuse un nom en double (409)", async () => {
67   const body = { name: "BORDEAUX", price: 10000, categoryId };
68   await request(app)
69     .post("/api/products")
70     .set("Authorization", `Bearer ${token}`)
71     .send(body);
72   const again = await request(app)
73     .post("/api/products")
74     .set("Authorization", `Bearer ${token}`)
75     .send({ ...body, name: "bordeaux" });
76   assert.equal(again.status, 409);
77 });
```

Élément	Explication
`before` / `after`	Une fois avant / après **tous** les tests de ce fichier : préparer l'administrateur, fermer la connexion.
`beforeEach`	Avant **chaque** test : repartir d'une base propre, pour que les tests ne dépendent pas les uns des autres.
`bcrypt.hash(..., 4)`	Coût **4** (rapide) pour accélérer les tests ; le coût réel est 12.
`request(app).post(...)`	Envoie la requête à l'application, **sans** réseau.
`.set("Authorization", ...)`	Ajoute l'en-tête de connexion.
`assert.equal(response.status, 201)`	Vérifie le code HTTP.

> **NOTE — Limite honnête**
> Dans ce guide, les exemples de tests d'intégration illustrent la méthode : ils s'exécutent avec **ton** base de test, qui n'est pas installée dans l'environnement où le guide a été préparé. Si un test échoue au premier lancement, lis le message : il indique presque toujours une variable manquante, une migration non appliquée ou un nom de champ différent.

Écrire de bons tests

1. **Un test = une idée.** Le nom dit ce qu'on vérifie (`refuse un prix négatif (422)`).
2. **Indépendants.** Chaque test prépare ses données et ne dépend pas de l'ordre.
3. **Arrange, Act, Assert** : préparer, agir, vérifier.
4. **Teste les règles importantes d'abord** : prix et stock des commandes, droits d'accès.
5. **Un bug trouvé = un test ajouté** : il ne reviendra plus.

12.6 Déboguer

1. Lire le message d'erreur

```
Error: Cannot find module 'C:\...\src\modules\products\products.service.js' imported from ...
    at finalizeResolution (node:internal/modules/esm/resolve:...)
```

Lis **la première ligne** (le type et le message), puis la **première ligne de la pile qui concerne ton code** (un chemin dans `src/`). Les lignes `node:internal/...` sont celles de Node : ignore-les.

Erreur fréquente	Signification	Solution
`Cannot find module '...'`	Mauvais chemin, ou **extension `.js` oubliée** dans un `import` (obligatoire avec les ES Modules)	Vérifier le chemin et ajouter `.js`
`SyntaxError: Cannot use import statement outside a module`	`"type": "module"` manque dans `package.json`	Ajouter `"type": "module"`
`ReferenceError: x is not defined`	Variable non déclarée ou mal orthographiée	Corriger le nom
`TypeError: Cannot read properties of undefined (reading 'id')`	Tu lis `.id` sur `undefined` (par exemple `req.user` absent)	Vérifier d'où vient la valeur (`authenticate` oublié ?)
`EADDRINUSE: address already in use :::3000`	Le port est déjà pris par une autre instance	Fermer l'autre terminal ou changer `PORT`
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL n'est pas démarré	Démarrer le service (`Get-Service postgres`)
`P1000` / `password authentication failed`	Mauvais utilisateur ou mot de passe dans `DATABASE_URL`	Corriger l'URL
`P1001: Can't reach database server`	Mauvais hôte ou port, serveur arrêté	Vérifier hôte, port, service
`P2021: table does not exist`	Migrations non appliquées	`npx prisma migrate dev`

2. Afficher les valeurs : console.log

Le plus simple. Affiche ce que tu soupçonnes :

```
console.log("body reçu :", req.body);
console.log("utilisateur :", req.user);
console.dir(order, { depth: null });
```

`console.dir(objet, { depth: null })` déroule les objets imbriqués. **Retire** ces lignes avant de publier ; et n'affiche jamais de mots de passe, jetons ou secrets.

3. Voir le SQL que Prisma envoie

```
export const prisma = new PrismaClient({ adapter, log: ["query", "warn", "error"] });
```

Chaque requête SQL s'affiche dans le terminal, avec ses paramètres. Pratique pour repérer une requête manquante, ou **trop de requêtes** pour une seule page.

4. Le débogueur de VS Code

Un débogueur te permet de **mettre le programme en pause** sur une ligne et d'inspecter chaque variable.

.vscode/launch.json JSON

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "API LAWPIKING",
      "program": "${workspaceFolder}/src/server.js",
      "console": "integratedTerminal"
    }
  ]
}
```

Clique à gauche d'un numéro de ligne pour poser un **point d'arrêt** (un point rouge), puis lance la configuration (touche F5). Quand une requête atteint cette ligne, le programme s'arrête : tu vois les variables, tu avances ligne par ligne (F10), tu entres dans une fonction (F11).

5. Prisma Studio et pgAdmin

Pour savoir **ce que contient réellement la base** : `npx prisma studio`, ou pgAdmin. Beaucoup de bugs viennent d'une donnée que l'on croyait différente.

6. Isoler le problème

Quand rien n'est clair : réduis. Appelle la route avec `curl.exe`, depuis Postman, puis depuis React. Si Postman marche mais pas React : le problème est côté frontend (CORS, jeton, URL). Si les deux échouent : il est côté backend. Puis descends les couches : la route répond-elle ? Le service est-il appelé ? Le repository renvoie-t-il quelque chose ?

> **À RETENIR**
> Méthode de débogage : **reproduire** le bug, **réduire** le cas, **observer** (message, journaux, variables), **formuler une hypothèse**, **corriger**, puis **ajouter un test** qui garantit qu'il ne revient pas.

12.7 Les journaux (logs)

En développement, ton `requestLogger` suffit. En production, on veut des journaux **structurés** (une ligne JSON par événement), horodatés, avec un niveau (`info`, `warn`, `error`). La bibliothèque **pino** est un bon choix ; en attendant, retiens la règle : on journalise les **erreurs inattendues** et les **actions sensibles** (connexions ratées, changements de statut de commande), jamais les mots de passe, jetons ou numéros complets.

12.8 Liste de contrôle avant de dire « ça marche »

- [ ] `npm test` : zéro échec
- [ ] La grille de la section « Que faut-il tester » passée pour les routes modifiées
- [ ] Aucun `console.log` de débogage oublié
- [ ] `git status` : pas de `.env` ni de dossier `uploads/` prêt à être envoyé
- [ ] Migrations créées **et** commitées

> **À RETENIR — En résumé — Partie 12**
> - Trois niveaux de tests : manuel (Postman), unitaire (fonctions pures), intégration (Supertest + base `_test`).
> - Une base de test **séparée** : jamais les tests sur les vraies données.
> - Débogage : lire le message, afficher les valeurs, voir le SQL de Prisma, poser des points d'arrêt, isoler le problème.
> - Un bug corrigé = un test ajouté pour qu'il ne revienne pas.
> - Garde les fonctions de calcul dans des fichiers sans dépendance pour les tester facilement.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Tester une route avec Postman	Tester à la main
Écrire un test unitaire	Les tests unitaires
Écrire un test d'intégration	Les tests d'intégration
Lire une erreur Node ou Prisma	Déboguer
Utiliser le débogueur de VS Code	Le débogueur

12.9 Exercices de la partie 12

QUESTIONS DE COMPRÉHENSION

1. Quelle est la différence entre un test unitaire et un test d'intégration ?
2. Pourquoi les tests utilisent-ils une base `lawpiking_test` ?
3. Pourquoi `app.js` ne doit-il pas appeler `listen` pour que Supertest fonctionne ?
4. Que signifie `ECONNREFUSED 127.0.0.1:5432` et comment le résoudre ?

EXERCICES DE CODE

5. Écris les tests unitaires de la fonction `makeReference` (format `LP-AAAAMMJJ-NNNN`) en passant une date fixe.
6. Écris un test d'intégration qui vérifie qu'une commande dont le stock est insuffisant renvoie 409 et ne modifie pas le stock.
7. Écris le test qui vérifie que `GET /api/products` ne renvoie pas un produit supprimé (`deletedAt` renseigné).

DÉBOGAGE

8. Le test `refuse sans connexion (401)` renvoie 404. Quelles causes probables ?
9. `req.user` est `undefined` dans `requireAdmin`. Quel middleware manque, ou est dans le mauvais ordre ?

MINI-PROJET

10. Construis la collection Postman complète de l'API (auth, catégories, produits, commandes), avec le jeton mémorisé automatiquement et au moins trois tests (`pm.test`) par requête.

POUR RÉFLÉCHIR

11. Si tu n'avais le temps d'écrire que cinq tests pour LAWPIKING, lesquels choisirais-tu en priorité ? Justifie.
12. Décris le test qui vérifie que la limitation de fréquence de la connexion fonctionne.
13. Pourquoi un test qui dépend de l'ordre d'exécution des autres tests est-il dangereux ?

PARTIE 13

Déploiement

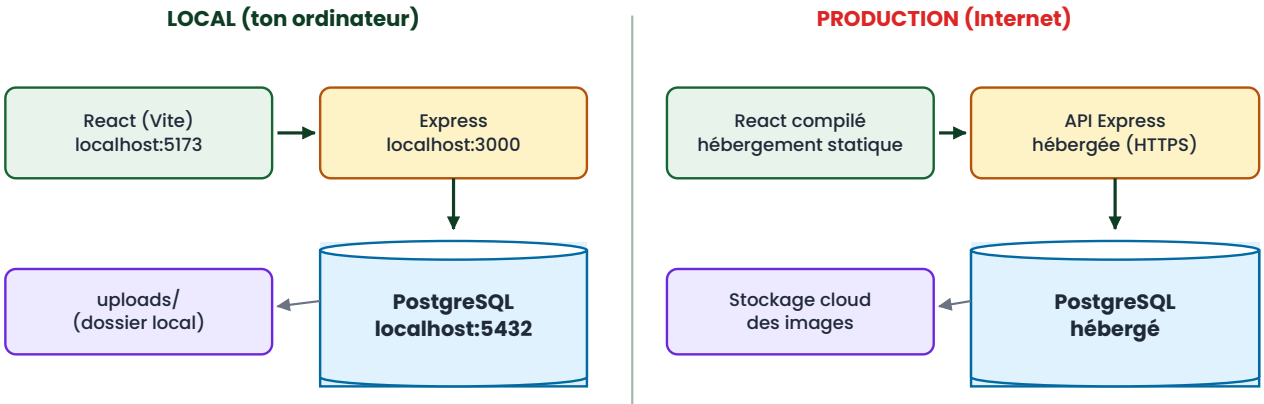
Mettre LAWPIKING en ligne : base distante, hébergement de l'API, migrations, images, sécurité, journaux et sauvegardes.

13.1 Local et production : deux mondes différents

Jusqu'ici, tout tourne sur **ton ordinateur** (« local », ou *développement*). Le **déploiement** consiste à faire tourner le backend et la base sur des machines accessibles depuis Internet (« production »), pour de vrais clients.

	Développement (local)	Production
Adresse de l'API	`http://localhost:3000`	`https://api.lawpiking.sn`
Base de données	PostgreSQL installé chez toi	Base **distante**, gérée, sauvegardée
Images	Dossier `uploads/`	Stockage distant durable
Secrets	`.env` sur ton disque	Variables configurées chez l'hébergeur
HTTPS	Non	**Obligatoire**
Erreurs	Détaillées dans le terminal	Journalisées, message générique au client
Données	Fausses (seed)	**Réelles** : on ne les efface jamais
`NODE_ENV`	`development`	`production`
Commande de migration	`prisma migrate dev`	`prisma migrate deploy`

SCHÉMA — DU DÉVELOPPEMENT LOCAL À LA PRODUCTION



Même code, autres valeurs : seules les variables d'environnement (.env) changent entre les deux mondes.

> **ATTENTION — Les noms et les offres des hébergeurs changent vite**
> Ce guide décrit **la démarche**, qui reste valable d'un hébergeur à l'autre, et cite des exemples. Les prix, les offres gratuites et les interfaces évoluent : vérifie la documentation actuelle du service que tu choisis avant de t'engager.

13.2 Les trois éléments à héberger

Élément	Contenu	Exemples de solutions
Frontend	Les fichiers produits par `npm run build` (dossier `dist/`)	Hébergeurs de sites statiques (Netlify, Vercel, Cloudflare Pages…)
Backend	L'API Node.js	Plateformes d'applications (Render, Railway, Fly.io…) ou un **VPS** (un serveur à administrer soi-même)
Base PostgreSQL	Les données	PostgreSQL managé (fourni par l'hébergeur, ou des services dédiés comme Neon, Supabase…)

Un **PostgreSQL managé** est recommandé pour débuter : sauvegardes, mises à jour et sécurité sont pris en charge, tu te concentres sur ton application. Un **VPS** donne plus de liberté mais **tu** es responsable de tout (mises à jour, pare-feu, sauvegardes).

13.3 Préparer le backend

1. Les scripts de package.json

package.json (scripts) JSON

```json
{
  "scripts": {
    "dev": "node --watch src/server.js",
    "start": "node src/server.js",
    "build": "prisma generate",
    "migrate:deploy": "prisma migrate deploy",
    "admin:create": "node prisma/createAdmin.js",
    "test": "node --test"
  }
}
```

Script	Qui l'utilise
`start`	**L'hébergeur**, pour lancer l'API (pas de `--watch` en production)
`build`	L'hébergeur, avant le démarrage : génère le client Prisma (indispensable avec Prisma 7)
`migrate:deploy`	Applique les migrations existantes sur la base distante

2. Prisma en production

`prisma` est une dépendance de **développement**. Sur certains hébergeurs, les dépendances de développement ne sont pas installées en production, et `npx prisma ...` échoue alors. Deux solutions : **installer aussi les dépendances de développement pendant la phase de build** (réglage de la plateforme), ou déplacer `prisma` dans `dependencies`. La seconde est la plus simple pour un débutant :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install prisma@7
```

(sans `--save-dev` : il passe dans `dependencies`).

3. Ce que doit faire le serveur au démarrage

Les commandes que la plateforme exécutera, dans cet ordre :

```
npm ci                      installe exactement les versions du package-lock.json
npm run build               prisma generate
npm run migrate:deploy      applique les migrations
npm start                   démarre l'API
```

`npm ci` est préférable à `npm install` en production : il suit **strictement** `package-lock.json`, donc les versions testées chez toi sont celles du serveur.

4. Écouter le bon port

Les plateformes fournissent le port via la variable `PORT`. Ton `env.js` la lit déjà (`Number(process.env.PORT) || 3000`). N'écris jamais un port fixe dans le code.

5. Faire confiance au proxy

Derrière un hébergeur, ton API reçoit les requêtes **par l'intermédiaire d'un proxy**. Sans réglage, Express voit **la même adresse IP** pour tous les visiteurs, et la limitation de fréquence (partie 8) bloquerait tout le monde ensemble, ou personne. Ajoute dans `app.js`, avant les middlewares :

```
app.set("trust proxy", 1);
```

`1` signifie « fais confiance à **un** niveau de proxy » : Express lit alors la vraie adresse du visiteur.

6. Une route de santé

`GET /api/health` (partie 3) sert à l'hébergeur pour vérifier que l'API répond. Version avancée, qui teste aussi la base :

```
app.get("/api/health", async (req, res) => {
  await prisma.$queryRaw`SELECT 1`;
  res.status(200).json({ status: "ok", database: "ok" });
});
```

Si la base est inaccessible, Prisma lève une erreur, Express répond 500 et la plateforme sait qu'il y a un problème.

13.4 Les variables d'environnement de production

À configurer dans le tableau de bord de l'hébergeur (pas dans un fichier) .env

```
NODE_ENV=production
PORT=                                  (fourni par la plateforme)
DATABASE_URL=postgresql://utilisateur:motdepasse@hôte:5432/base?sslmode=require
FRONTEND_URL=https://www.lawpiking.sn
JWT_ACCESS_SECRET=(nouveau secret, 64 caractères aléatoires minimum)
JWT_REFRESH_SECRET=(un autre secret)
```

Point	Explication
Nouveaux secrets	Génère de **nouvelles** valeurs pour la production. Ne réutilise **pas** celles de ton ordinateur.
`sslmode=require`	La plupart des bases distantes **exigent** une connexion chiffrée. La documentation de ton hébergeur donne l'URL exacte à copier : utilise-la telle quelle.
`FRONTEND_URL`	L'adresse **exacte** de ton site (CORS et cookies en dépendent), sans `/` final.
Pas d'`ADMIN_PASSWORD` permanent	Utilise-le le temps de lancer `npm run admin:create`, puis supprime-le.

13.5 La base PostgreSQL distante

1. **Crée** la base chez l'hébergeur de ton choix ; il te donne une **URL de connexion**.
2. **Copie** cette URL dans `DATABASE_URL` (variable de **production** uniquement).
3. **Applique les migrations** : `npm run migrate:deploy`.
4. **Crée l'administrateur** : `npm run admin:create` (une seule fois).
5. **Ne lance jamais** `prisma migrate dev`, `prisma migrate reset` ou `prisma db push` contre la production : ils peuvent **effacer des données**.
6. Pour exécuter ces commandes depuis ton ordinateur, utilise temporairement l'URL de production dans la variable d'environnement de **ce** terminal seulement :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> $env:DATABASE_URL = "URL-de-production"
PS ...\lawpiking-backend> npx prisma migrate deploy
PS ...\lawpiking-backend> Remove-Item Env:DATABASE_URL
```

La dernière ligne supprime la variable de la session, pour éviter d'exécuter une commande dangereuse sur la production plus tard.

Migrer sans casser les données

`migrate deploy` applique **uniquement** les migrations qui n'ont pas encore été appliquées. Certaines modifications sont **dangereuses** sur une base qui contient des données :

Modification	Risque	Méthode sûre
Ajouter une colonne `NOT NULL` sans valeur par défaut	Échec : les lignes existantes n'ont pas de valeur	Ajouter une valeur par défaut, ou ajouter la colonne facultative, la remplir, **puis** la rendre obligatoire
Supprimer ou renommer une colonne	**Perte de données** (un renommage généré peut être compris comme « supprimer + créer »)	Lire le SQL de la migration avant de la lancer ; adapter à la main avec `--create-only`
Changer le type d'une colonne	Conversion impossible ou perte	Créer une nouvelle colonne, copier, basculer, supprimer l'ancienne

Avant toute migration en production : **sauvegarde** (voir plus bas), **lecture** du fichier `migration.sql`, test sur une **copie** de la base.

13.6 Le frontend en production

Le frontend est construit **avec** l'adresse de l'API :

.env.production (frontend) .env

```
1 VITE_API_URL=https://api.lawpiking.sn
```

TERMINAL Windows PowerShell

```
PS ...\Lawpikine> npm run build
```

Les variables `VITE_` sont **inscrites dans le code** au moment du `build` : changer l'adresse de l'API demande de **reconstruire** et republier le frontend.

Cookies et domaines

Le cookie de refresh (partie 8) a deux comportements selon l'architecture :

Architecture	Exemple	Réglage du cookie
Même site (sous-domaines du même domaine)	`www.lawpiking.sn` et `api.lawpiking.sn`	`sameSite: "lax"` possible, `secure: true`
Sites différents	`lawpiking.vercel.app` et `lawpiking-api.onrender.com`	`sameSite: "none"` + `secure: true` obligatoires

Le plus propre à terme : un **nom de domaine à toi** (`lawpiking.sn`) avec un sous-domaine pour l'API. Les cookies fonctionnent mieux, et le site inspire confiance.

> **ERREUR FRÉQUENTE — Le cookie n'est pas enregistré en production**
> Vérifie dans l'ordre : le site est bien en **HTTPS** ; `secure: true` ; `sameSite: "none"` si les sites sont différents ; `FRONTEND_URL` identique à l'adresse réelle du site (CORS `credentials: true`) ; `credentials: "include"` côté frontend ; `trust proxy` activé. Onglet *Application* des outils de développement du navigateur : le cookie `lp_refresh` doit y apparaître.

13.7 Les images en production

Le disque de nombreux hébergeurs est **éphémère** : il est effacé à chaque redéploiement, et les images envoyées disparaissent. En production, il faut un **stockage d'objets distant** (Cloudinary, stockage compatible S3, etc.).

Grâce au module `storage.js` (partie 9), le changement ne touche **qu'un fichier** :

1. Créer un compte et un « bucket » (espace de stockage) chez le fournisseur ; récupérer ses clés.
2. Ajouter les clés aux variables d'environnement de production.
3. Réécrire `saveProductImage` et `deleteProductImage` avec le SDK du fournisseur ; elles renvoient l'**adresse complète** (`https://…`).
4. Pour les images existantes, les envoyer au stockage distant et mettre à jour `product_images.url` (script de migration unique).
5. Dans le frontend, le mapper (partie 10) ne préfixe plus par l'adresse de l'API si l'URL commence par `https://`.

13.8 Durcir le serveur

Mesure	Détail
HTTPS partout	Fourni automatiquement par la plupart des plateformes ; redirige le HTTP vers le HTTPS
Secrets	Uniquement dans les variables de l'hébergeur, rotation si fuite
Helmet, CORS limité, limitation de fréquence	Déjà en place (partie 8)
Messages d'erreur génériques	Pas de trace de pile envoyée au client
Dépendances à jour	`npm audit` régulier, mises à jour de sécurité
Utilisateur PostgreSQL restreint	Pas le super-utilisateur ; uniquement sa base
Base non exposée publiquement	Si l'hébergeur le permet, n'autoriser que l'API à s'y connecter
Prisma Studio jamais en ligne	Local uniquement
Pas de compte administrateur par défaut	Mot de passe fort, unique
Limites de taille	JSON 100 Ko, images 5 Mo

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm audit
found 0 vulnerabilities
```

`npm audit` liste les failles connues dans tes dépendances. `npm audit fix` applique les corrections **compatibles** ; examine toujours ce qu'il modifie avant de publier.

13.9 Les journaux et la surveillance

Les plateformes affichent la sortie de ton application (`console.log`, `console.error`) dans un onglet **Logs**. Pour qu'ils soient utiles :

- journalise les **erreurs 500** avec le contexte (méthode, route, identifiant de l'utilisateur), **sans** données sensibles ;
- journalise les actions importantes : connexions réussies et ratées, changement de statut de commande, suppression de produit ;
- active, si l'hébergeur le propose, une **alerte** quand l'API ne répond plus (via la route `/api/health`).

13.10 Les sauvegardes

Une base de production **sans sauvegarde** est un risque inacceptable : tes commandes clients y sont.

Stratégie	Description
Sauvegardes automatiques de l'hébergeur	Souvent incluses dans l'offre payante ; **vérifie** la fréquence et la durée de conservation, et teste une restauration
Export manuel régulier	`pg_dump -F c -f lawpiking_AAAA-MM-JJ.backup "URL-de-production"` depuis ton ordinateur, au moins chaque semaine, **et avant chaque migration**
Copie hors site	Garde un exemplaire sur un autre support (disque externe, stockage cloud privé)
Images	Le stockage distant les protège ; sinon, sauvegarde aussi `uploads/`

> **BONNE PRATIQUE — La règle du 3-2-1**
> **3** copies de tes données, sur **2** supports différents, dont **1** hors de ton hébergeur principal. Et teste la restauration au moins une fois : une sauvegarde jamais restaurée est une hypothèse, pas une garantie.

13.11 Maintenance

Fréquence	Tâche
Chaque déploiement	Sauvegarde ; lire les migrations ; tests ; vérifier `/api/health` ensuite
Chaque semaine	Regarder les journaux d'erreurs ; sauvegarde manuelle
Chaque mois	`npm audit` ; mise à jour des dépendances mineures **sur une branche**, avec les tests ; vérifier l'espace disque de la base
Chaque année	Changer les secrets JWT ; revoir les accès ; tester une restauration complète

Mises à jour de Prisma : lis les notes de version avant de passer à une version **majeure** (7 vers 8). Applique-la d'abord sur ta base de test, avec la suite de tests.

13.12 Le déploiement pas à pas : résumé

1. Le code est sur **GitHub** (sans `.env`, sans `uploads/`, sans `node_modules/`).
2. Créer la **base PostgreSQL** distante et récupérer son URL.
3. Créer le **service backend** relié au dépôt ; définir commandes de build, de démarrage, variables.
4. Lancer les **migrations** de production et créer l'**administrateur**.
5. Vérifier `https://api…/api/health`.
6. Configurer le **stockage d'images** distant.
7. Construire et publier le **frontend** avec `VITE_API_URL` de production.
8. **Tester** le parcours complet : connexion, création d'un produit avec image, commande, changement de statut.
9. Activer la **surveillance** et planifier les **sauvegardes**.

13.13 Liste de contrôle avant la mise en ligne

- [ ] `NODE_ENV=production`, secrets neufs, `.env` jamais commité
- [ ] HTTPS actif ; cookie `secure`, `sameSite` adapté
- [ ] `trust proxy` activé ; CORS limité à l'adresse du site
- [ ] Migrations appliquées avec `migrate deploy` ; sauvegarde faite avant
- [ ] Administrateur créé, mot de passe fort, `ADMIN_PASSWORD` retiré
- [ ] Images sur un stockage durable
- [ ] `npm audit` sans faille critique
- [ ] Sauvegardes automatiques ou planifiées, restauration testée
- [ ] Route de santé surveillée ; journaux consultables

À RETENIR — En résumé — Partie 13

- Local et production sont deux mondes : base, secrets, HTTPS, images et commandes de migration changent.
- Production : `npm ci`, `prisma generate`, `migrate deploy`, `npm start`, avec `trust proxy` et des secrets neufs.
- Ne lance jamais `migrate dev`, `migrate reset` ni `db push` contre la production.
- Images sur un stockage distant durable ; cookies `secure` et `sameSite` adaptés à l'architecture.
- Sauvegarde avant chaque migration, restauration testée, `npm audit` régulier, journaux surveillés.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Distinguer développement et production	Local et production
Préparer les scripts de déploiement	Préparer le backend
Appliquer des migrations sans risque	La base PostgreSQL distante
Régler cookies et CORS en production	Cookies et domaines
Mettre en place des sauvegardes	Les sauvegardes

13.14 Exercices de la partie 13

QUESTIONS DE COMPRÉHENSION

1. Quelle différence entre `migrate dev` et `migrate deploy` ? Lequel est utilisé en production ?
2. Pourquoi faut-il `trust proxy` derrière un hébergeur ?
3. Pourquoi les variables `VITE_` ne doivent-elles jamais contenir de secret ?
4. Pourquoi le disque local d'un hébergeur ne convient-il pas pour les images ?

EXERCICES PRATIQUES

5. Écris la liste exacte des commandes à lancer pour déployer une nouvelle version du backend contenant une migration (de la sauvegarde à la vérification).
6. Écris le script qui envoie les images du dossier `uploads/products/` vers un stockage distant et met à jour `product_images.url`.
7. Écris la procédure de **restauration** d'une sauvegarde `.backup` dans une base de test, puis les trois requêtes SQL qui vérifient qu'elle est exploitable.

DÉBOGAGE

8. Après le déploiement, toutes les requêtes du frontend échouent avec « blocked by CORS policy ». Quelle variable vérifier en premier ?
9. La limitation de fréquence bloque tous les visiteurs après quelques requêtes seulement. Quelle ligne manque dans `app.js` ?

MINI-PROJET

10. Rédige le **plan de déploiement de LAWPIKING** en une page : choix de chaque hébergeur (frontend, API, base, images) avec une justification, coût mensuel estimé d'après les offres actuelles, et plan de sauvegarde.

PARTIE 14

Projet pratique

Construire le backend LAWPIKING de zéro, étape par étape, avec un contrôle à chaque étape.

14.1 Le principe

Tu as maintenant toutes les notions. Ce projet les assemble : **douze étapes**, chacune avec un **objectif**, les **actions**, un **contrôle** (comment savoir que ça marche) et un **point de repère** (le commit à faire). Ne passe à l'étape suivante que si le contrôle est réussi : c'est le seul moyen de repérer un problème **là où il est né**.

> **ATTENTION — Rappel : ce guide ne développe pas le backend à ta place**
> Le but est que **tu** le construises en comprenant chaque fichier. Les parties précédentes contiennent le code ; ce projet te dit **dans quel ordre** l'assembler. Garde le frontend tel quel pendant tout le projet : tu ne le branches qu'à l'étape finale.

14.2 Vue d'ensemble

Étape	Objectif	Parties à relire	Contrôle
1	Environnement et dossier	2	`node -v`, `psql --version`
2	Serveur Express minimal	3	`GET /api/health` répond
3	Gestion d'erreurs et validation	3	404 et 422 au bon format
4	Base PostgreSQL	4, 5	11 tables visibles
5	Prisma : schéma et migration	6	`prisma migrate dev` réussi
6	Seed	6	Catégories et produits visibles dans Studio
7	Authentification	8	Connexion, 401 et 403 corrects
8	Catégories	7, 11	CRUD complet testé
9	Produits	7, 11	Création, filtres, suppression douce
10	Images et stock	9, 11	Upload contrôlé, statut de stock
11	Commandes, avis, promotions, réglages	11	Commande atomique, prix serveur
12	Tests, sécurité, mise en ligne	12, 13	`npm test` vert, liste de contrôle

14.3 Étape 1 : préparer l'environnement

Actions

TERMINAL Windows PowerShell

```
PS C:\Users\mohamed\Desktop> node -v
v24.22.1
PS C:\Users\mohamed\Desktop> mkdir lawpiking-backend
PS C:\Users\mohamed\Desktop> cd lawpiking-backend
PS ...\lawpiking-backend> git init
PS ...\lawpiking-backend> npm init -y
```

Édite `package.json` : ajoute `"type": "module"`, change `"name"` en `"lawpiking-backend"`. Crée `.gitignore` (partie 8), `.env` et `.env.example`. Crée les dossiers :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> mkdir src\config, src\middlewares, src\utils, `
    src\modules, prisma, uploads
```

Contrôle : `node -v` affiche une version 24 (ou la version LTS actuelle) ; `git status` ne montre pas `.env`.
Repère : `git add . ; git commit -m "Initialisation du projet"`.

14.4 Étape 2 : le serveur minimal

Installe les dépendances de base :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install express dotenv zod
```

Crée `src/config/env.js` (version simple), `src/app.js` avec `express.json()` et la route `GET /api/health`, `src/server.js` (parties 2 et 3). Ajoute les scripts `dev` et `start`. **Contrôle**

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm run dev
API LAWPIKING sur http://localhost:3000
PS ...\lawpiking-backend> curl.exe http://localhost:3000/api/health
{"status":"ok","shop":"LAWPIKING"}
```

Repère : `git commit -m "Serveur Express minimal"`.

14.5 Étape 3 : erreurs, journal, validation

Crée `utils/AppError.js`, `middlewares/errorHandler.js` (404, JSON invalide, 500), `middlewares/requestLogger.js`, `middlewares/validate.js`, `utils/pagination.js`, `utils/slugify.js`. Branche-les dans `app.js` : **journal, JSON, routes, 404, gestionnaire d'erreurs** (dans cet ordre). **Contrôle**

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> curl.exe -i http://localhost:3000/api/inconnu
HTTP/1.1 404 Not Found
{"error":{"code":"ROUTE_NOT_FOUND",
"message":"Route introuvable : GET /api/inconnu","details":null}}
```

Un JSON cassé (`-d "{"`) doit donner 400 `INVALID_JSON`. **Repère** : `git commit -m "Erreurs, validation, outils"`.

14.6 Étape 4 : la base PostgreSQL

Installe PostgreSQL 18 si ce n'est pas fait (partie 4). Crée l'utilisateur et la base :

```
CREATE USER lawpiking_user WITH PASSWORD 'ton_mot_de_passe';
CREATE DATABASE lawpiking OWNER lawpiking_user ENCODING 'UTF8';
```

Renseigne `DATABASE_URL` dans `.env`. À ce stade, **tu peux** soit exécuter les 10 étapes SQL de la partie 5 pour comprendre, soit laisser Prisma créer les tables à l'étape suivante. Pour apprendre : exécute le SQL, regarde `\dt`, puis **supprime la base et recrée-la vide** (`DROP DATABASE`, puis `CREATE DATABASE`) avant l'étape 5, car Prisma doit créer lui-même les tables. **Contrôle** : connexion réussie avec `psql -U lawpiking_user -d lawpiking`.

14.7 Étape 5 : Prisma

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npm install @prisma/client@7 @prisma/adapter-pg pg
PS ...\lawpiking-backend> npm install prisma@7
PS ...\lawpiking-backend> npx prisma init --datasource-provider postgresql
```

Remplace `prisma/schema.prisma` par le schéma complet de la partie 6 ; crée `prisma.config.js`. Puis :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npx prisma validate
PS ...\lawpiking-backend> npx prisma migrate dev --name init
PS ...\lawpiking-backend> npx prisma migrate dev --create-only --name add_checks
```

Complète le fichier SQL de `add_checks` (contraintes `CHECK`, index uniques partiels), puis `npx prisma migrate dev`. Crée `src/config/prisma.js`. **Contrôle** : `npx prisma studio` montre les 11 modèles ; `\dt` dans psql, 11 tables plus `_prisma_migrations`. **Repère** : `git commit -m "Schéma Prisma et migrations"`.

14.8 Étape 6 : le seed

Écris `prisma/seed.js` (catégories, quelques produits avec variantes, réglages) et `prisma/createAdmin.js`. Ajoute les scripts. Lance :

TERMINAL Windows PowerShell

```
PS ...\lawpiking-backend> npx prisma db seed
PS ...\lawpiking-backend> npm run admin:create
```

Pour reproduire la boutique réelle, crée les neuf produits de ton catalogue : même noms, mêmes prix, mêmes catégories. **Contrôle** : Prisma Studio montre les catégories, produits, variantes ; l'utilisateur administrateur a un `passwordHash` qui commence par `$2b$`.

14.9 Étape 7 : l'authentification

Installe les paquets de sécurité (partie 8). Ajoute `JWT_*` dans `.env`. Crée le module `auth` (schéma, service, contrôleur, routes), `users.repository.js`, et les middlewares `authenticate`, `requireAdmin`. Branche `helmet`, `cors`, `cookieParser`, limitation de fréquence. **Contrôles**

Test	Résultat attendu
Connexion avec le bon mot de passe	200, `accessToken`, cookie `lp_refresh` HttpOnly
Mauvais mot de passe	401 `INVALID_CREDENTIALS`
6 essais ratés d'affilée	429
`GET /api/auth/me` sans jeton	401
`GET /api/auth/me` avec jeton	200 + utilisateur, **sans** `passwordHash`
`POST /api/auth/refresh` avec le cookie	200 + nouveau jeton

Repère : `git commit -m "Authentification administrateur"`.

14.10 Étape 8 : les catégories (le premier module complet)

Crée les cinq fichiers du module `categories` (partie 11). C'est le module le plus simple : il sert d'**entraînement** à la structure en couches. **Contrôles** : lister (public), créer (admin), nom en double = 409, supprimer une catégorie qui contient des produits = 409, suppression d'une catégorie vide = 204.

14.11 Étape 9 : les produits

Module `products` : routes publiques de lecture, routes administrateur d'écriture, filtres et pagination, suppression douce, activation/désactivation (parties 7 et 11). **Contrôles**

- `GET /api/products?search=noir&category=tshirts&page=1&limit=5` renvoie `{ total, page, limit, totalPages, products }`.
- Un produit désactivé n'apparaît plus dans la liste publique, mais apparaît pour l'administrateur avec `?all=true`.
- Un produit supprimé n'apparaît plus **nulle part** dans les listes.
- Prix négatif = 422 avec `details.price`.

14.12 Étape 10 : images et stock

Ajoute `multer`, les utilitaires `imageSignature.js` et `storage.js`, les routes d'images (partie 9), puis les modules `stock` et variantes (partie 11). **Contrôles**

- Un vrai JPEG est accepté ; un fichier texte renommé en `.jpg` donne 415 ; une image de 8 Mo donne 413.
- Après suppression d'une image : plus de ligne en base **et** plus de fichier dans `uploads/products/`.
- `PATCH /api/variants/:id/stock` avec `{ "stock": 3, "lowStockThreshold": 5 }` renvoie le statut `low`.

14.13 Étape 11 : commandes, avis, promotions, réglages

Dans cet ordre : **réglages** (le plus simple), **promotions**, **avis**, puis **commandes** (le plus délicat, qui dépend des promotions et du stock). **Contrôles pour les commandes**

Scénario	Résultat attendu
Commande valide de 2 articles	201, `reference`, `totalAmount` calculé par le serveur
Prix modifié dans la requête	Ignoré : le total vient de la base
Quantité supérieure au stock	409 `OUT_OF_STOCK`, **aucun** stock retiré
Pendant une promotion de 20 %	`unitPrice` réduit de 20 %
Passage `PENDING` vers `DELIVERED`	409 `INVALID_TRANSITION`
Annulation	Stock **rendu**

14.14 Étape 12 : tests, sécurité, déploiement

1. Écris les tests unitaires (fonctions pures) et d'intégration (au minimum : auth, produits, commandes), avec la base `lawpiking_test`. Cible : `npm test` vert.
2. Déroule la **liste de contrôle sécurité** (partie 8).
3. Prépare les scripts, `trust proxy`, les variables de production (partie 13), puis déploie la base, l'API, les images et enfin le frontend.
4. Dernière étape : **connecte le frontend**, module par module, dans l'ordre de la partie 10.

14.15 Récapitulatif des fichiers finaux

```
lawpiking-backend/
├─ prisma/  schema.prisma, seed.js, createAdmin.js, migrations/
├─ prisma.config.js
├─ src/
│  ├─ app.js, server.js
│  ├─ config/        env.js, prisma.js
│  ├─ middlewares/   requestLogger, validate, authenticate, requireAdmin, upload, errorHandler
│  ├─ utils/         AppError, pagination, slugify, imageSignature, storage
│  └─ modules/       auth, users, categories, products, stock,
│                    orders, reviews, promotions, settings
├─ tests/            utils.test.js, products.test.js, orders.test.js
├─ uploads/          (ignoré par Git)
├─ .env  .env.example  .env.test  .gitignore
└─ package.json
```

14.16 Les pièges des débutants sur ce projet

Piège	Conséquence	Prévention
Écrire trop de code avant de tester	Impossible de savoir quelle ligne casse	Tester après **chaque** fichier
Modifier la base à la main pendant que Prisma gère les migrations	« Drift detected »	Toujours passer par `schema.prisma`
Oublier `npx prisma generate`	Le client ne voit pas les nouveaux champs	Le lancer après chaque changement de schéma
Commiter `.env`	Secrets exposés	`.gitignore` dès l'étape 1
Brancher le frontend trop tôt	Deux inconnues à la fois	Valider l'API avec Postman d'abord
Copier-coller sans comprendre	Impossible de déboguer	Relire le tableau « ligne par ligne » de chaque extrait

14.17 Après le projet : pour aller plus loin

- **TypeScript** pour détecter les erreurs avant l'exécution.
- Une table `refresh_tokens` pour pouvoir révoquer des sessions.
- **Redimensionnement automatique des images** (WebP, plusieurs tailles).
- **Notifications** : e-mail ou message WhatsApp à chaque nouvelle commande.
- **Paiement mobile** (Wave, Orange Money) via leurs API officielles.
- **Documentation d'API** avec OpenAPI/Swagger.
- Intégration continue (**GitHub Actions**) pour lancer les tests à chaque envoi de code.

> **À RETENIR — En résumé — Partie 14**
> - Douze étapes, un **contrôle** à chaque étape : on ne passe à la suivante que si la précédente marche.
> - L'ordre compte : serveur, erreurs, base, Prisma, authentification, modules du plus simple au plus délicat.
> - Un commit par étape donne des points de retour et une histoire lisible.
> - Le frontend ne se branche qu'**à la fin**, module par module, après validation avec Postman.
> - Les prochaines évolutions : TypeScript, `refresh_tokens`, notifications, paiement mobile, intégration continue.

AUTO-ÉVALUATION

Coche chaque ligne quand tu sais le faire **sans regarder** le guide. Sinon, relis la section indiquée.

Je sais…	Où revoir
Réaliser les étapes dans l'ordre avec leurs contrôles	Vue d'ensemble
Tester l'authentification de bout en bout	Étape 7
Valider une commande atomique	Étape 11
Préparer la mise en ligne	Étape 12
Éviter les pièges du projet	Les pièges des débutants

14.18 Exercices de la partie 14

QUESTIONS DE COMPRÉHENSION

1. Pourquoi crée-t-on le module `categories` avant `products` ? Et `orders` en dernier ?
2. Pourquoi l'étape 4 demande-t-elle de recréer une base vide avant l'étape 5 ?
3. Que contrôle-t-on à la fin de l'étape 7 et pourquoi avant d'écrire les modules suivants ?

EXERCICES PRATIQUES

4. Réalise les étapes 1 à 3 sur ton ordinateur et note, pour chaque étape, la commande qui prouve qu'elle est terminée.
5. Après l'étape 9, crée avec Postman les neuf produits de LAWPIKING en respectant leurs prix et catégories.

DÉBOGAGE

6. À l'étape 5, `prisma migrate dev` répond « Drift detected ». Décris la démarche pour t'en sortir sans perdre de données importantes.

MINI-PROJET FINAL

7. Termine le projet : dossier Git propre, au moins 12 commits (un par étape), `npm test` vert, collection Postman exportée, et un fichier `README.md` qui explique à un nouveau développeur comment installer et lancer le backend en cinq commandes.

ANNEXE A

Aide-mémoire et glossaire

Les commandes, les codes et les mots à garder sous la main.

Les commandes essentielles

Besoin	Commande
Version de Node.js / npm	`node -v` ; `npm -v`
Lancer un fichier	`node src/server.js`
Lancer en rechargeant automatiquement	`node --watch src/server.js`
Installer toutes les dépendances	`npm install` ; en production : `npm ci`
Ajouter une dépendance	`npm install nom`
Ajouter une dépendance de développement	`npm install --save-dev nom`
Lancer un script du `package.json`	`npm run nom` (`npm start`, `npm test` sans `run`)
Lancer les tests	`npm test`
Chercher les failles	`npm audit`
Tester une route (GET)	`curl.exe -i http://localhost:3000/api/health`
Envoyer du JSON	`curl.exe -X POST URL -H "Content-Type: application/json" -d "{...}"`

psql

Commande	Rôle
`psql -U utilisateur -d base`	Se connecter
`\l`	Lister les bases
`\c base`	Changer de base
`\dt`	Lister les tables
`\d table`	Décrire une table
`\x`	Affichage vertical
`\i fichier.sql`	Exécuter un fichier
`\q`	Quitter
`pg_dump -U u -d base -F c -f fichier.backup`	Sauvegarder
`pg_restore -U u -d base fichier.backup`	Restaurer

Prisma

Commande	Rôle
`npx prisma validate`	Vérifier le schéma
`npx prisma format`	Mettre en forme le schéma
`npx prisma migrate dev --name nom`	Créer et appliquer une migration (développement)
`npx prisma migrate dev --create-only --name nom`	Créer la migration sans l'appliquer
`npx prisma migrate deploy`	Appliquer les migrations (production)
`npx prisma migrate reset`	Tout effacer et rejouer (développement uniquement)
`npx prisma generate`	Générer le client
`npx prisma db seed`	Lancer le script de peuplement
`npx prisma studio`	Interface de consultation

Les codes HTTP

Code	Nom	Usage dans LAWPIKING
200	OK	Lecture ou modification réussie
201	Created	Produit, commande, avis créés
204	No Content	Suppression réussie
400	Bad Request	JSON cassé, requête illisible
401	Unauthorized	Non connecté, jeton expiré
403	Forbidden	Connecté mais sans droit
404	Not Found	Produit, route introuvables
409	Conflict	Nom ou slug déjà pris, stock insuffisant, transition interdite
413	Payload Too Large	Image trop lourde
415	Unsupported Media Type	Format de fichier refusé
422	Unprocessable Content	Données invalides (validation)
429	Too Many Requests	Limitation de fréquence
500	Internal Server Error	Bug inattendu

Les codes d'erreur à connaître

Source	Code	Signification
PostgreSQL	`23505`	Doublon (`UNIQUE`)
PostgreSQL	`23503`	Clé étrangère invalide
PostgreSQL	`23502`	Valeur `NULL` interdite
PostgreSQL	`23514`	Contrainte `CHECK` violée
Prisma	`P2002`	Violation d'unicité
Prisma	`P2003`	Violation de clé étrangère
Prisma	`P2025`	Enregistrement introuvable
Prisma	`P1001`	Base inaccessible
Node.js	`EADDRINUSE`	Port déjà utilisé
Node.js	`ENOENT`	Fichier ou dossier introuvable

Glossaire

Terme	Définition
ACID	Garanties d'une base fiable : Atomicité, Cohérence, Isolation, Durabilité
Adapter (pilote)	Composant qui relie Prisma à PostgreSQL (`@prisma/adapter-pg`)
API	Interface qu'un programme offre aux autres programmes (ici : les routes HTTP)
Async / await	Écriture qui permet d'attendre un résultat sans bloquer le programme
Authentification	Prouver qui l'on est
Autorisation	Vérifier ce que l'on a le droit de faire
Backend	La partie serveur, invisible du visiteur
bcrypt	Algorithme de hachage lent, destiné aux mots de passe
Body (corps)	Les données envoyées dans une requête ou une réponse
Client	Le programme qui envoie la requête (le navigateur, React)
Contrainte	Règle que la base fait respecter (`UNIQUE`, `CHECK`…)
Contrôleur	Couche qui lit la requête, appelle le service et répond
CORS	Règle du navigateur qui limite les sites autorisés à appeler une API
CRUD	Create, Read, Update, Delete : les quatre opérations de base
CSRF	Attaque qui déclenche une action à l'insu d'un utilisateur connecté
Endpoint	Une route précise de l'API (méthode + chemin)
ENUM	Type à liste fermée de valeurs
Frontend	La partie visible : l'interface dans le navigateur
Hachage	Empreinte à sens unique d'une donnée
HttpOnly	Cookie illisible par le JavaScript de la page

Terme	Définition
Index	Structure qui accélère la recherche dans une table
Injection SQL	Attaque qui glisse du SQL dans une donnée saisie
JSON	Format texte d'échange de données
JWT	Jeton signé qui prouve l'identité de son porteur
Middleware	Fonction exécutée entre la requête et la réponse
Migration	Fichier qui décrit un changement de structure de la base
ORM	Outil qui traduit objets JavaScript et tables SQL
Pagination	Découper une liste en pages
Promesse (Promise)	Valeur qui sera disponible plus tard
Repository	Couche qui seule parle à la base
REST	Style d'API : ressources, URLs, méthodes HTTP
Route	Association méthode HTTP + chemin + fonction
Schéma	Description des données attendues ou stockées
Seed	Script qui remplit la base avec des données de départ
Service	Couche qui contient les règles métier
Slug	Version d'un nom utilisable dans une URL (`tshirt-noir-intense`)
Snapshot (instantané)	Copie figée d'une valeur à un instant (prix d'une commande)
Soft delete	Masquer sans supprimer (`deleted_at`)
Token d'accès	Jeton de courte durée envoyé à chaque requête
Token de renouvellement	Jeton de longue durée qui permet d'en obtenir un nouveau
Transaction	Ensemble d'opérations tout-ou-rien
UUID	Identifiant universel imprévisible de 36 caractères
Validation	Vérifier que les données respectent les règles
XSS	Injection d'un script malveillant dans une page

ANNEXE B

Corrigés des exercices

Les réponses détaillées, partie par partie. Essaie toujours seul avant de lire.

> **BONNE PRATIQUE — Comment utiliser les corrigés**
> Un exercice n'est utile que si tu as **cherché**. Lis l'énoncé, essaie, teste ton code, puis compare. Ta solution peut être différente de la mienne et **correcte** : l'important est qu'elle respecte les règles de l'énoncé.

Corrigés de la partie 1

1. Tout ce qui est dans le frontend est **envoyé au navigateur** : n'importe qui peut lire le code et les identifiants, ou contourner la vérification. La vérification doit se faire sur le serveur, que le visiteur ne peut pas modifier.

2. (a) `GET` ; (b) `PATCH` (on modifie une seule information, le statut) ou `PUT` ; (c) `DELETE` ; (d) `POST`.

3. (a) `201` ; (b) `422` (donnée invalide) ; (c) `404` ; (d) `401` (non connecté) ; (e) `409` (conflit).

4. Dans `/api/products/5?category=tshirts`, `5` est un **paramètre de chemin** (`req.params.id`) : il identifie **un** produit. `category=tshirts` est un **paramètre de requête** (`req.query.category`) : il filtre une liste.

5. Si le serveur répond `404`, la requête **a abouti** : `fetch` ne lance pas d'exception, mais `response.ok` vaut `false`, donc le code lève `Error("Erreur HTTP 404")`. Si le serveur est éteint, la requête **n'aboutit pas** : `fetch` lui-même lève une exception (`TypeError: Failed to fetch`). Les deux cas aboutissent à une erreur, mais **pas la même** : l'une vient de notre `throw`, l'autre de `fetch`.

6.

```
{
  "id": 3,
  "value": "bonnets",
  "description": "Les bonnets de la collection LAWPIKING.",
  "image": "",
  "enabled": true
}
```

```
await fetch("http://localhost:3000/api/categories", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(category),
});
```

Il faut `JSON.stringify` (le corps doit être du texte) et l'en-tête `Content-Type` pour que le serveur sache le lire.

7. Voyage attendu : la page Stock (React) appelle `updateAdminProductStock` → `fetch PATCH /api/products/5/stock` avec `{ "stock": 20 }` et le jeton → le serveur Express → middlewares (journal, JSON, authentification, droit administrateur, validation) → contrôleur → service → repository → PostgreSQL

(`UPDATE`) → la réponse remonte → `200` avec le produit mis à jour → React affiche le nouveau stock.

Corrigés de la partie 2

1. JavaScript est le **langage**. V8 est le **moteur** (de Chrome) qui exécute ce langage. Node.js est un **programme** qui embarque V8 et lui ajoute ce qu'un navigateur n'a pas : fichiers, réseau, processus.

2. `node_modules/` est énorme et se **recrée** avec `npm install` : le versionner alourdit le dépôt. `package-lock.json` fixe les **versions exactes** installées : l'avoir garantit que toute l'équipe (et le serveur) utilise les mêmes.

3. `^5.1.0` autorise les versions `>= 5.1.0` et `< 6.0.0` : mises à jour mineures et correctifs, mais pas le changement de version majeure (qui peut casser le code).

4. Une fonction « Sync » **bloque tout le programme** pendant qu'elle travaille : dans un serveur, tous les autres visiteurs attendent. Dans un petit script qui fait une seule chose, personne n'attend : c'est acceptable.

5.

src/utils/stock.js JavaScript

```javascript
export function getStockStatus(stock, threshold = 10) {
  if (stock === 0) return "out";
  if (stock <= threshold) return "low";
  return "in";
}
```

test.js JavaScript

```javascript
import { getStockStatus } from "./src/utils/stock.js";

for (const stock of [0, 8, 25]) console.log(stock, getStockStatus(stock));
```

RÉSULTAT résultat attendu

```
0 out
8 low
25 in
```

6.

```javascript
import { readFile } from "node:fs";
import { readFile as readFilePromise } from "node:fs/promises";

readFile("hello.js", "utf8", (error, text) => {
  if (error) return console.error(error.message);
  console.log(text.length);
});

readFilePromise("hello.js", "utf8")
  .then((text) => console.log(text.length))
  .catch((error) => console.error(error.message));

try {
  const text = await readFilePromise("hello.js", "utf8");
  console.log(text.length);
} catch (error) {
  console.error(error.message);
}
```

Comparaison : le callback force l'imbrication ; `.then` enchaîne à plat ; `async/await` se lit comme du code ordinaire et se protège avec un simple `try/catch`.

7.

```
const pause = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function hello() {
  await pause(1000);
  console.log("bonjour");
}
await hello();
```

8. Une fonction `async` **renvoie toujours une promesse**. Il manque `await` : `const produits = await getProducts();` (dans une fonction `async` ou au niveau supérieur d'un module ES).

9. Indication de solution : `const parts = url.pathname.split("/");` donne `["", "api", "products", "5"]` ; `parts[3]` est l'identifiant ; `Number.isInteger(Number(parts[3]))` valide ; sinon `400`. Chercher dans la liste ; absent : `404`. Constat : chaque route demande 15 lignes de découpage et de contrôles : c'est ce qu'Express simplifie.

Corrigés de la partie 3

1. Pour pouvoir tester l'application avec Supertest **sans ouvrir de port**, et pour séparer **la construction** (routes, middlewares) du **démarrage** (port, connexion).

2. `req.params` : un morceau du chemin (`/api/products/:id` → l'identifiant). `req.query` : après le `?` (`?category=tshirts` → filtre). `req.body` : le corps JSON (le nom et le prix d'un produit à créer).

3. `next()` passe la main au middleware ou à la route suivante. S'il est oublié (et qu'aucune réponse n'est envoyée), la requête **reste bloquée** jusqu'à l'expiration.

4. Par son **nombre de paramètres** : quatre `(err, req, res, next)`.

5. Déclarer la route **avant** `/:id` :

```
router.get("/low-stock", (req, res) => {
  const threshold = Number(req.query.threshold) || 10;
  const result = products.filter((p) => p.stock > 0 && p.stock <= threshold);
  res.json({ total: result.length, products: result });
});
router.get("/:id", controller.getProduct);
```

Sinon `low-stock` serait interprété comme un `:id`.

6.

```
export function requireJson(req, res, next) {
  const writes = ["POST", "PUT", "PATCH"];
  if (writes.includes(req.method) && !req.is("application/json")) {
    throw new AppError("Content-Type JSON requis", 415, "UNSUPPORTED_MEDIA");
  }
  next();
}
```

`req.is("application/json")` teste l'en-tête `Content-Type`.

7.

```
export const createCategorySchema = z.object({
  name: z.string().trim().min(1, "Le nom est obligatoire").max(60, "60 caractères maximum"),
  description: z.string().trim().max(200).default(""),
  enabled: z.boolean().default(true),
});
```

Résultat vérifié : `parse({ name: " Bonnets " })` donne `{"name":"Bonnets","description":"","enabled":true}`.

8. `req.params.id` est un **texte** (`"5"`) tandis que `p.id` est un nombre : `===` est faux. Corrige : `p.id === Number(req.params.id)`.

9. Il manque la réponse : après `products.push(product);` ajoute `res.status(201).json(product);`.

10. Plan de solution : routeur, contrôleur, schéma Zod (`name` obligatoire) et gestionnaire d'erreurs comme pour les produits. Pour le 409 : normalise les noms avant de comparer.

```
const normalize = (text) =>
  text.normalize("NFD").replace(/\p{M}/gu, "").trim().toLowerCase();

if (categories.some((c) => normalize(c.name) === normalize(data.name))) {
  throw new AppError("Cette catégorie existe déjà", 409, "CATEGORY_NAME_TAKEN");
}
```

Corrigés de la partie 4

1. La clé **primaire** identifie une ligne de **sa propre** table (unique, jamais vide). La clé **étrangère** est une colonne qui désigne la clé primaire d'**une autre** table (`products.category_id` vers `categories.id`).

2. Le franc CFA n'a pas de centimes : un entier est **exact**. Les nombres à virgule flottante peuvent produire des erreurs d'arrondi (0,1 + 0,2 n'égale pas 0,3).

3. Une colonne `product_ids` contiendrait plusieurs valeurs dans une même case : impossible à contrôler par une clé étrangère, à rechercher ou à garder cohérente. Une table de liaison a **une ligne par couple**, avec clés étrangères et clé primaire composée qui interdit les doublons.

4. `WHERE` filtre les lignes **avant** le regroupement ; `HAVING` filtre les groupes **après** (par exemple, ne garder que les catégories qui ont plus de 3 produits).

5.

```
SELECT name, price FROM products WHERE name ILIKE '%rouge%' ORDER BY price DESC;
```

6.

```
SELECT c.name, MAX(p.price) AS prix_max
FROM categories c
LEFT JOIN products p ON p.category_id = c.id AND p.deleted_at IS NULL
GROUP BY c.name;
```

`LEFT JOIN` garde les catégories sans produit (`prix_max` vaut alors `NULL`).

7.

```sql
INSERT INTO promotions (name, percent_off, starts_at, ends_at)
VALUES ('Soldes de fin d''année', 20, '2026-12-01 00:00:00+00', '2026-12-31 23:59:59+00');

INSERT INTO promotion_products (promotion_id, product_id)
SELECT pr.id, p.id
FROM promotions pr, products p JOIN categories c ON c.id = p.category_id
WHERE pr.name = 'Soldes de fin d''année' AND c.slug = 'tshirts';
```

L'apostrophe dans « d'année » se double (`''`) pour ne pas fermer le texte.

8.

```sql
SELECT c.name FROM categories c
WHERE NOT EXISTS (
  SELECT 1 FROM products p
  WHERE p.category_id = c.id AND p.is_active AND p.deleted_at IS NULL
);
```

9. **Tous** les prix sont passés à 15 000. Avant : écrire d'abord le même `WHERE` dans un `SELECT`, et travailler dans une transaction : `BEGIN; UPDATE ...;` puis `SELECT` pour vérifier, `ROLLBACK;` en cas d'erreur, `COMMIT;` sinon. Rattrapage : restaurer la sauvegarde.

10. `NULL` n'est égal à rien, pas même à `NULL` : la comparaison renvoie « inconnu ». Il faut `WHERE deleted_at IS NULL`.

11.

```sql
CREATE TABLE settings (
  key        VARCHAR(60) PRIMARY KEY,
  value      JSONB       NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
INSERT INTO settings (key, value) VALUES
  ('whatsapp_number', '"221700000000"'),
  ('shop_name', '"LAWPIKING"');
SELECT value FROM settings WHERE key = 'whatsapp_number';
```

Corrigés de la partie 5

1. Le statut dépend du stock et du seuil : le stocker créerait **deux vérités** qui peuvent se contredire (stock à 0 mais statut « en stock »). On le calcule à chaque lecture.

2. `order_items.unit_price` est l'**instantané** du prix payé. Si `products.price` change plus tard, les commandes passées doivent garder leur montant d'origine.

3. `is_active = FALSE` : produit **masqué mais conservé**, que l'administrateur peut réactiver. `deleted_at` renseigné : produit **retiré** du catalogue ; il n'existe plus que pour l'historique des commandes.

4. Plusieurs-à-plusieurs. Représentée par la table de liaison `promotion_products` (une ligne par couple promotion/produit).

5.

```sql
SELECT p.name, v.size, v.color_name, v.stock, v.low_stock_threshold
FROM product_variants v JOIN products p ON p.id = v.product_id
WHERE v.stock > 0 AND v.stock <= v.low_stock_threshold;
```

6.

```sql
SELECT COALESCE(SUM(total_amount), 0) AS chiffre_affaires
FROM orders WHERE status = 'DELIVERED';
```

`COALESCE` renvoie 0 quand il n'y a aucune commande livrée (`SUM` donnerait `NULL`).

7.

```sql
SELECT p.name, ROUND(AVG(r.rating), 1) AS note_moyenne, COUNT(r.id) AS nb_avis
FROM products p
JOIN reviews r ON r.product_id = p.id AND r.status = 'APPROVED'
GROUP BY p.id, p.name;
```

8. Causes possibles : `quantity` à 0 ou négative (`CHECK (quantity > 0)`), `unit_price` négatif. (Un `total_amount` négatif concernerait `orders`.)

9. La clé étrangère `products.category_id` n'a pas `ON DELETE CASCADE` : PostgreSQL **refuse** de supprimer une catégorie qui contient des produits (erreur `23503`). Proposition : désactiver la catégorie (`is_enabled = FALSE`), ou déplacer ses produits vers une autre avant de la supprimer ; c'est la règle de l'application (409).

10.

```sql
CREATE TABLE contact_messages (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name  VARCHAR(80)  NOT NULL,
  email      VARCHAR(160) NOT NULL,
  message    TEXT         NOT NULL,
  is_read    BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX idx_contact_unread ON contact_messages (created_at DESC) WHERE NOT is_read;
```

Corrigés de la partie 6

1. `migrate dev` **crée** les migrations à partir du schéma et les applique, en **développement** (il peut proposer un `reset`). `migrate deploy` applique seulement les migrations **existantes**, sans en créer et sans rien effacer : c'est celui de la production.

2. Chaque client ouvre ses propres connexions à la base ; en créer un par requête épuiserait rapidement les connexions autorisées par PostgreSQL.

3. Le champ s'appelle `createdAt` en JavaScript, mais la colonne s'appelle `created_at` dans la base : `@map` fait le lien, ce qui garde le style JavaScript (camelCase) d'un côté et le style SQL (snake_case) de l'autre.

4. `decrement` est exécuté **dans la base** en une opération (`stock = stock - 2`). Lire, soustraire en JavaScript puis réécrire laisse un intervalle pendant lequel une autre commande peut modifier le stock : une des deux soustractions serait perdue.

5.

```
const products = await prisma.product.findMany({
  where: { deletedAt: null, isActive: true, category: { slug: "pulls" } },
  orderBy: { price: "desc" },
  take: 5,
  include: { variants: true },
});
```

6. Prisma ne compare pas deux colonnes dans un `where` : on passe par le SQL ou on filtre en JavaScript.

```
const variants = await prisma.$queryRaw`
  SELECT p.name, v.size, v.color_name, v.stock, v.low_stock_threshold
  FROM product_variants v JOIN products p ON p.id = v.product_id
  WHERE v.stock > 0 AND v.stock <= v.low_stock_threshold`;
```

7.

```
export async function softDeleteCategory(id) {
  const count = await prisma.product.count({ where: { categoryId: id, deletedAt: null } });
  if (count > 0) {
    throw new AppError("La catégorie contient encore des produits", 409, "CATEGORY_NOT_EMPTY");
  }
  return prisma.category.update({ where: { id }, data: { isEnabled: false } });
}
```

Dans ce schéma, la catégorie n'a pas de `deletedAt` : la « suppression douce » consiste à la désactiver. Pour en avoir une vraie, il faudrait ajouter la colonne par migration.

8.

```
const stats = await prisma.review.aggregate({
  where: { productId, status: "APPROVED" },
  _avg: { rating: true },
  _count: true,
});
```

`stats._avg.rating` est la note moyenne (`null` s'il n'y a aucun avis).

9. `404 Not Found` : l'élément à modifier n'existe pas. Le gestionnaire central traduit `P2025` en 404.

10. Tu as oublié `npx prisma migrate dev --name add_is_featured` (qui crée la colonne) et `npx prisma generate` (qui met à jour le client).

11. Ajoute dans `schema.prisma` un modèle `ContactMessage` (`@@map("contact_messages")`), puis `npx prisma migrate dev --name add_contact_messages`. Repository :

```
export const create = (data) => prisma.contactMessage.create({ data });
export const list = ({ skip, take }) =>
  prisma.contactMessage.findMany({ skip, take, orderBy: { createdAt: "desc" } });
export const markAsRead = (id) =>
  prisma.contactMessage.update({ where: { id }, data: { isRead: true } });
```

Corrigés de la partie 7

1. Route (associe URL et fonctions) ; middleware (contrôles transversaux) ; contrôleur (lit la requête, répond) ; service (règles métier) ; repository (seul accès à la base).

2. Pour rester indépendant d'HTTP : le même service pourra servir une route, un script ou un test, et se teste sans serveur.

3. La boutique doit afficher le catalogue à tout visiteur (lecture publique). Créer, modifier ou supprimer change les données : réservé à l'administrateur authentifié.

4. Le service donne un **message clair** (409) dans le cas normal. Mais deux requêtes simultanées peuvent passer toutes deux la vérification : la contrainte `UNIQUE` de la base est le **dernier rempart** qui garantit l'unicité.

5.

categories.service.js (extrait) JavaScript

```javascript
export async function create(data) {
  if (await repo.nameExists(data.name)) {
    throw new AppError("Une catégorie porte déjà ce nom", 409, "CATEGORY_NAME_TAKEN");
  }
  return repo.create({ ...data, slug: slugify(data.name) });
}
export const list = (onlyEnabled) => repo.list({ onlyEnabled });
```

categories.controller.js (extrait) JavaScript

```javascript
export async function list(req, res) {
  res.json(await service.list(req.query.all !== "true"));
}
export async function create(req, res) {
  res.status(201).json(await service.create(req.body));
}
```

Les routes : `router.get("/", controller.list)` et `router.post("/", ...adminOnly, validate(schema), controller.create)`.

6.

```javascript
export async function remove(id) {
  const product = await productsRepository.findById(id);
  if (!product) throw new AppError("Produit introuvable", 404, "PRODUCT_NOT_FOUND");
  await productsRepository.softDelete(id);
}
// repository
export const softDelete = (id) =>
  prisma.product.update({ where: { id }, data: { deletedAt: new Date(), isActive: false } });
```

7.

```javascript
export async function list(query) {
  const { page, limit, skip } = getPagination(query);
  const { total, products } = await productsRepository.list({
    search: query.search,
    category: query.category,
    onlyActive: query.all !== "true",
    skip,
```

```
    take: limit,
  });
  return { total, page, limit, totalPages: Math.ceil(total / limit), products };
}
```

8. (1) **Aucune validation** : prix texte, champs vides. (2) **Mass assignment** : le client peut envoyer `deletedAt`, `isActive`… (3) **Aucune règle métier** (catégorie, doublons) et le contrôleur connaît Prisma. Correction : `validate(createProductSchema)` dans la route, appel de `productsService.create(req.body)`, Prisma uniquement dans le repository.

9. Extraire la partie commune (par exemple la lecture d'un prix) dans un troisième fichier, ou faire dépendre le service de commandes du **repository** de produits plutôt que de son service. Une dépendance doit aller dans **un seul sens**.

10. Parcours : page Stock (React) → `updateAdminProductStock` → `api()` (`apiClient.js`) → Express (`app.js`) → `products.routes.js` (ou `stock` : `variants.routes.js`) → `authenticate` → `requireAdmin` → `validate(updateStockSchema)` → contrôleur → service → repository → Prisma → PostgreSQL, puis le retour en sens inverse.

11. Le contrôleur dépend d'HTTP : la règle serait (1) **impossible à tester** sans démarrer le serveur, et (2) **dupliquée** le jour où une autre entrée (script d'import, nouvelle route, autre application) créerait des commandes, avec un risque d'oubli donc de stock négatif. Dans le service, la règle existe **une seule fois**.

Corrigés de la partie 8

1. Authentification : « qui es-tu ? » (erreur : **401**). Autorisation : « as-tu le droit ? » (erreur : **403**).

2. Un chiffrement est **réversible** : qui vole la clé retrouve tous les mots de passe. Un hachage ne l'est pas : on compare des empreintes, on ne retrouve jamais le mot de passe.

3. Si un jeton d'accès est volé, il devient inutile après 15 minutes. Le jeton de renouvellement, plus précieux, n'est jamais lisible par JavaScript (cookie HttpOnly) et sert à en obtenir un nouveau.

4. Pour ne pas révéler à un attaquant quels emails existent dans la base (énumération).

5.

```
export const requireRole = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user?.role)) {
    throw new AppError("Accès refusé", 403, "FORBIDDEN");
  }
  next();
};
```

Utilisation : `router.delete("/:id", authenticate, requireRole("ADMIN"), controller.remove)`.

6.

```
// route
const check = validate(changePasswordSchema);
router.patch("/password", authenticate, check, controller.changePassword);

// service
export async function changePassword(userId, currentPassword, newPassword) {
  const user = await usersRepository.findById(userId);
```

```
  const ok = user && (await bcrypt.compare(currentPassword, user.passwordHash));
  if (!ok) throw new AppError("Mot de passe actuel incorrect", 401, "INVALID_CREDENTIALS");
  await usersRepository.updatePassword(userId, await bcrypt.hash(newPassword, 12));
}
```

7.

```
export const changePasswordSchema = z
  .object({
    currentPassword: z.string().min(1),
    newPassword: z.string().min(12, "12 caractères minimum").max(200),
    confirmPassword: z.string(),
  })
  .refine((v) => v.newPassword === v.confirmPassword, {
    message: "Les mots de passe ne correspondent pas",
    path: ["confirmPassword"],
  });
```

8. L'en-tête doit être `Authorization: Bearer <jeton>`. Sans le mot `Bearer`, `authenticate` ne reconnaît pas le format.

9. Cookie : `secure: true`, `sameSite: "none"`, HTTPS actif. CORS : `origin` égal à l'adresse **exacte** du site, `credentials: true`. Frontend : `credentials: "include"`. Et `trust proxy` activé.

10. Script `prisma/disableUser.js` : lit `process.argv[2]` (l'email), exécute `prisma.user.update({ where: { email }, data: { isActive: false } })`, avec le script npm `"admin:disable": "node prisma/disableUser.js"`. Démonstration : `POST /api/auth/refresh` avec le cookie de ce compte renvoie `401 SESSION_INVALID`, puisque `refresh` relit l'utilisateur et vérifie `isActive`. (Son jeton d'accès actuel reste valable jusqu'à 15 minutes : c'est la raison de cette durée courte.)

Corrigés de la partie 9

1. Table énorme, sauvegardes lourdes, lecture de la base à chaque affichage d'image, mise en cache difficile. Un fichier + son chemin est plus léger et plus rapide.

2. Nom imprévisible, **unique** (pas d'écrasement), sans caractère dangereux, sans possibilité de deviner l'adresse d'un autre fichier.

3. Le `Content-Type` est écrit par le client : il peut mentir. Les premiers octets du fichier sont sa vraie nature.

4. Le disque est effacé à chaque redéploiement : les images envoyées disparaîtraient alors que leurs lignes existent encore en base.

5. Un fichier AVIF est un conteneur « ISO base media » : octets 4 à 7 = `ftyp`, puis la marque (`avif` ou `avis`) aux octets 8 à 11.

```
const isAvif =
  buffer.subarray(4, 8).toString("ascii") === "ftyp" &&
  ["avif", "avis"].includes(buffer.subarray(8, 12).toString("ascii"));
if (isAvif) return { mime: "image/avif", ext: "avif" };
```

N'oublie pas d'ajouter `image/avif` à la liste `ALLOWED` de Multer.

6.

```
export function setMainImage(productId, imageId) {
  return prisma.$transaction([
    prisma.productImage.updateMany({ where: { productId }, data: { isMain: false } }),
    prisma.productImage.update({ where: { id: imageId }, data: { isMain: true } }),
  ]);
}
```

Les deux écritures réussissent ensemble ou pas du tout : la contrainte « une seule image principale » n'est jamais violée durablement.

7. Route `PUT /:id/images/main` avec `uploadImage`. Service : trouver l'ancienne principale, `saveProductImage`, créer la ligne avec `isMain: true` (après avoir mis les autres à `false` dans une transaction), puis supprimer l'ancienne ligne et son fichier.

8. Le nom du champ doit être identique des deux côtés : `form.append("image", file)` (ou `.single("photo")` côté Multer).

9. `crossOriginResourcePolicy` de Helmet : par défaut `same-origin`. Il faut `helmet({ crossOriginResourcePolicy: { policy: "cross-origin" } })`.

10. Migration : `ALTER TABLE categories ADD COLUMN image_url VARCHAR(300)` (champ `imageUrl String? @map("image_url")` en Prisma). Route `POST /:id/image` avec `uploadImage`, service qui enregistre le nouveau fichier, met la colonne à jour, puis supprime l'ancien fichier (`deleteProductImage(ancienneUrl)`).

11. Chaque visiteur télécharge 4 Mo par image : pages lentes, forfaits mobiles consommés, mauvais référencement. Amélioration : **redimensionner et compresser** à l'envoi (par exemple 1200 px, format WebP, avec la bibliothèque `sharp`), éventuellement plusieurs tailles, et charger les images en différé dans la liste.

Corrigés de la partie 10

1. Vite n'expose au navigateur que les variables préfixées `VITE_`, pour éviter de publier par erreur des variables privées. On n'y met **jamais** de secret : elles sont écrites dans le code envoyé aux visiteurs.

2. Un script injecté dans la page (XSS) peut lire le `localStorage`, mais pas une variable interne d'un module. Le coût : la session ne survit pas à un rechargement, d'où `restoreSession()`.

3. Elle redemande un jeton d'accès grâce au cookie de renouvellement, pour que l'administrateur reste connecté après F5. Elle doit être appelée avant l'affichage, sinon `AdminLayout` verrait « non connecté » et redirigerait vers la connexion.

4. Un client malveillant pourrait envoyer un prix de 1 FCFA. Le prix doit venir de la base, côté serveur.

5.

```
export async function listAdminProducts() {
  const { products } = await api("/api/products?all=true&limit=50");
  return products.map(fromApiProduct);
}
```

6.

```
export async function updateAdminProductStock(id, { stock, threshold }) {
  const errors = validateStockValues({ stock, threshold });
  if (Object.keys(errors).length) throw new ProductValidationError(errors);
  try {
    return await api(`/api/variants/${id}/stock`, {
```

```
      method: "PATCH",
      body: { stock: Number(stock), lowStockThreshold: Number(threshold) },
    });
  } catch (error) {
    const known = error instanceof ApiError;
    if (known && error.fields) throw new ProductValidationError(error.fields);
    throw error;
  }
}
```

7.

```
export function toApiProduct(values) {
  return {
    name: values.name.trim(),
    description: values.description ?? "",
    price: Number(String(values.price).replace(/\s/g, "")),
    categoryId: values.categoryId,
    variants: values.colors.flatMap((color) =>
      values.sizes.map((size) => ({
        size, colorName: color.name, colorHex: color.hex, stock: 0,
      })),
    ),
  };
}
```

8. Vite lit les fichiers `.env` **au démarrage**. Il faut arrêter et relancer `npm run dev`.

9. (1) La promesse ne se termine jamais (requête bloquée, pas de réponse) : `loading` reste `true`. (2) `deps` provoque un rechargement à chaque affichage (nouvelle fonction ou tableau recréé à chaque rendu) : la page tourne en boucle sans jamais se stabiliser.

10. Changent : `adminCategoryService.js` (appels API), un mapper éventuel, `categoryStore.js` (source des catégories pour les formulaires). Ne changent pas : `CategoriesAdmin.jsx` et les composants d'affichage, car ils n'utilisent que les fonctions du service (c'est tout l'intérêt de cette séparation).

11. La validation du frontend sert le **confort** : réponse immédiate, sans aller-retour réseau, messages sous les champs. Le backend reste **indispensable** pour la sécurité, puisque le frontend se contourne. Les deux sont complémentaires, pas redondants.

12.

```
export async function deleteAdminProduct(id) {
  await api(`/api/products/${id}`, { method: "DELETE" });
  return listAdminProducts();
}
```

L'API répond `204` (le client renvoie `null`), puis on relit la liste pour rendre exactement ce que rendait l'ancienne fonction.

13. Les pages ignorent d'où viennent les données. Aujourd'hui `listAdminProducts()` lit le `localStorage`, demain l'API : seule la fonction change, jamais les pages. Cela permet aussi de brancher les modules un par un et de tester le service seul.

Corrigés de la partie 11

1. Les prix sont une décision du serveur. Le client dit seulement **quoi** et **combien** ; le serveur lit les vrais prix, applique les promotions et calcule le total.

2. Le statut dépend de la date actuelle : stocké, il serait faux dès que le temps passe, sans que personne ne modifie la ligne.

3. `.strict()` refuse toute clé inconnue : impossible de stocker n'importe quel réglage dans la table `settings`.

4. Elle est définitive : modifier une commande livrée fausserait l'historique et le chiffre d'affaires. En cas d'erreur, on crée un avoir ou une nouvelle commande.

5.

```
export function createWithStock(order, items) {
  return prisma.$transaction(async (tx) => {
    for (const item of items) {
      const result = await tx.productVariant.updateMany({
        where: { id: item.variantId, stock: { gte: item.quantity } },
        data: { stock: { decrement: item.quantity } },
      });
      if (result.count === 0) {
        throw new AppError(`Stock insuffisant : ${item.productName}`, 409, "OUT_OF_STOCK");
      }
    }
    return tx.order.create({
      data: { ...order, items: { create: items } },
      include: { items: true },
    });
  });
}
```

6.

```
export async function moderate(id, status) {
  if (!["APPROVED", "REJECTED"].includes(status)) {
    throw new AppError("Statut invalide", 422, "VALIDATION_ERROR");
  }
  return repo.updateStatus(id, status);
}
```

7. Trois requêtes en parallèle :

```
const [byStatus, revenue, outOfStock] = await Promise.all([
  prisma.order.groupBy({ by: ["status"], _count: true }),
  prisma.order.aggregate({ where: { status: "DELIVERED" }, _sum: { totalAmount: true } }),
  prisma.product.count({
    where: { deletedAt: null, variants: { every: { stock: 0 } } },
  }),
]);
```

`every: { stock: 0 }` signifie « toutes les variantes ont un stock nul ».

8. Le décrément du stock (`tx.productVariant.updateMany(... decrement ...)`) est absent, ou il est exécuté avec `prisma` au lieu de `tx` (donc hors transaction).

9. Le filtre `category: { isEnabled: true }` dans `where` quand `onlyActive` est vrai.

10. Même structure que les autres modules : modèle Prisma `ContactMessage` + migration, `contact.routes.js` (`POST /` public avec une limitation de fréquence, `GET /` et `PATCH /:id/read` pour l'administrateur), schéma Zod (nom, email, message de 2000 caractères maximum), service, repository.

Corrigés de la partie 12

1. Un test unitaire vérifie une **fonction isolée**, sans base ni serveur. Un test d'intégration vérifie **plusieurs couches ensemble**, souvent avec une vraie base de test.

2. Les tests créent et effacent des données ; ils ne doivent jamais toucher aux données de développement ou de production.

3. `listen` ouvre un port : plusieurs fichiers de test le feraient en même temps (conflit). Supertest envoie ses requêtes **directement** à l'application.

4. Node n'arrive pas à se connecter au port 5432 : PostgreSQL n'est pas démarré (ou pas sur ce port). `Get-Service postgres`, démarrer le service, vérifier `DATABASE_URL`.

5.

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { makeReference } from "../src/modules/orders/reference.js";

test("makeReference suit le format LP-AAAAMMJJ-NNNN", () => {
  const ref = makeReference(new Date("2026-10-31T12:00:00Z"));
  assert.match(ref, /^LP-20261031-\d{4}$/);
});
```

Pour qu'elle soit testable sans base, place `makeReference` dans un fichier à part (`reference.js`) et exporte-la.

6.

```
test("refuse une commande si le stock est insuffisant (409)", async () => {
  const variant = await createVariant({ stock: 1 });
  const response = await request(app)
    .post("/api/orders")
    .send({
      customerName: "Awa",
      customerPhone: "770000000",
      items: [{ variantId: variant.id, quantity: 2 }],
    });

  assert.equal(response.status, 409);
  const after = await prisma.productVariant.findUnique({ where: { id: variant.id } });
  assert.equal(after.stock, 1);
});
```

`createVariant` est une fonction d'aide du test qui crée catégorie, produit et variante.

7.

```
test("ne renvoie pas un produit supprimé", async () => {
  await prisma.product.create({ data: { ...base, deletedAt: new Date() } });
  const response = await request(app).get("/api/products");
  assert.equal(response.body.total, 0);
});
```

8. Le routeur n'est pas branché (`app.use("/api/products", ...)` manquant), ou le préfixe est faux : Express ne trouve aucune route et le `notFoundHandler` répond 404 avant l'authentification.

9. `authenticate` est absent de la route, ou placé **après** `requireAdmin`. Il doit venir en premier.

10. Fournir dans l'export Postman : variable `baseUrl`, script de connexion qui enregistre `accessToken`, et pour chaque requête des `pm.test("code 201", () => pm.response.to.have.status(201))`, des vérifications du format JSON et du temps de réponse.

11. On teste d'abord ce qui coûte le plus cher en cas d'erreur : (1) connexion avec un mauvais mot de passe = 401 ; (2) route d'écriture sans jeton = 401 ; (3) commande dont le prix envoyé est ignoré ; (4) stock insuffisant = 409 sans modification du stock ; (5) annulation qui rend le stock. Ils protègent l'argent, les droits d'accès et la cohérence du stock.

12. Envoyer 5 connexions avec un mauvais mot de passe (chacune renvoie 401), puis une 6e : elle doit renvoyer **429**. Attention : le compteur du limiteur est partagé entre les tests ; dans les tests, soit on isole ce scénario dans un fichier à part, soit on rend la limite configurable par une variable d'environnement pour ne pas bloquer les autres tests de connexion.

13. Si le test A prépare des données dont le test B a besoin, B échoue dès qu'on l'exécute seul, dans un autre ordre, ou en parallèle, et l'échec ne montre pas la vraie cause. Chaque test doit préparer lui-même ses données (`beforeEach`).

Corrigés de la partie 13

1. `migrate dev` crée et applique, en développement, et peut proposer d'effacer les données. `migrate deploy` applique seulement les migrations existantes, sans rien effacer : production.

2. Derrière un proxy, toutes les requêtes semblent venir de la même adresse IP (celle du proxy). `trust proxy` indique à Express de lire la vraie adresse du visiteur.

3. Elles sont écrites en clair dans les fichiers JavaScript envoyés à chaque visiteur.

4. Il est éphémère : effacé à chaque redéploiement. Les images disparaîtraient alors que les chemins subsistent en base.

5. (1) `pg_dump` de la base de production. (2) Relire `migration.sql`. (3) Tester la migration sur une copie de la base. (4) Pousser le code. (5) `npm ci`, `npm run build`, `npm run migrate:deploy`, `npm start` (par la plateforme). (6) Vérifier `/api/health` et un parcours complet. (7) En cas de problème : revenir à la version précédente du code et restaurer la sauvegarde si la migration a modifié des données.

6. Script à usage unique : lire `prisma.productImage.findMany({ where: { url: { startsWith: "/uploads/" } } })`, envoyer chaque fichier au stockage distant avec le SDK du fournisseur, `update` de l'`url` avec l'adresse reçue, et ne supprimer le fichier local qu'une fois la ligne mise à jour. Journaliser chaque image traitée pour pouvoir reprendre en cas d'interruption.

7.

TERMINAL — Windows PowerShell

```
PS> createdb -U postgres -O lawpiking_user lawpiking_test
PS> pg_restore -U lawpiking_user -d lawpiking_test --no-owner lawpiking_2026-10-31.backup
```

```
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM orders;
SELECT MAX(created_at) FROM orders;
```

Les comptes correspondent à ceux de la production, et la date de la dernière commande est celle attendue.

8. `FRONTEND_URL` (adresse exacte du site, avec `https://` et sans `/` final).

9. `app.set("trust proxy", 1);`.

10. Rédaction libre : un hébergeur de site statique pour le frontend, une plateforme d'applications pour l'API, un PostgreSQL managé, un stockage d'images distant ; coûts relevés **sur les pages de prix actuelles**, avec sauvegarde hebdomadaire hors site.

Corrigés de la partie 14

1. Le module `categories` est le plus simple : il sert d'entraînement à la structure en couches. `products` en dépend (chaque produit a une catégorie). `orders` dépend de presque tout (produits, variantes, stock, promotions) : il vient en dernier.

2. Prisma doit **créer lui-même** les tables et l'historique `_prisma_migrations`. Si les tables existent déjà (créées à la main), il détecte un décalage (*drift*) et refuse de continuer.

3. On vérifie que la connexion, les erreurs 401/403 et le refus d'un mauvais mot de passe fonctionnent. Tous les modules suivants s'appuient dessus : une faille ici se répéterait partout.

4. Exemples de preuves : étape 1 `node -v` ; étape 2 `GET /api/health` renvoie `{"status":"ok"}` ; étape 3 une route inconnue renvoie le JSON 404 uniforme.

5. Crée un dossier « Produits » dans Postman, une requête `POST /api/products` par produit, en réutilisant les noms, prix et catégories de ton catalogue. Vérifie ensuite `GET /api/products` : `total` doit valoir 9.

6. (1) Ne pas paniquer : la base de développement peut se recréer. (2) Lire le message pour savoir quelle table ou migration pose problème. (3) Si les données de développement sont sans importance (cas général), `npx prisma migrate reset`, puis `npx prisma db seed`. (4) Si des données sont précieuses : `pg_dump` d'abord, puis résoudre le décalage en créant une migration qui reproduit la modification faite à la main. (5) Ne jamais faire `reset` en production.

7. Contrôle final : `git log --oneline` montre une dizaine de commits ; `npm test` affiche `fail 0` ; le `README.md` contient : `git clone`, `npm install`, copie de `.env.example` en `.env`, `npx prisma migrate dev`, `npm run dev`.